

8.4.3 Common Receiver Parameters §

§ Table 8-11 lists the common Receiver parameters that are not directly associated with stressed eye tolerancing. Values are separately defined for the four data rates.

§ Table 8-11 Common Receiver Parameters

Symbol	Parameter	2.5 GT/s value	5.0 GT/s value	8.0 GT/s value	16.0 GT/s value	32.0 GT/s value	64.0 GT/s value	Units	Notes
UI (Rx)	Unit Interval	(min) 399.88 (max) 400.12 (300 PPM)	(min) 199.94 (max) 200.06 (300 PPM)	(min) 124.9625 (max) 125.0375 (300 PPM)	(min) 62.48125 (max) 62.51875 (300 PPM)	(min) 31.246875 (max) 31.253125 (100 PPM)	(min) 31.246875 (max) 31.253125 (100 PPM)	ps	UI tolerance does not include SSC effects
BW_{RX-PKG-PLL1}	Rx PLL bandwidth corresponding to PKG _{RX-PLL1}	(max) 22 (min) 1.5	(max) 16.0 (min) 8	(max) 4.0 (min) 0.5	(max) 4.0 (min) 0.5	(max) 1.8 (min) 0.5	(max) 1.0 (min) 0.5	MHz	Second order PLL transfer bounding function. See Note 1.
BW_{RX-PKG-PLL2}	Rx PLL bandwidth corresponding to PKG _{RX-PLL2}	Not Specified	(max) 16.0 (min) 5.0	(max) 5.0 (min) 0.5	(max) 5.0 (min) 0.5	N/A	N/A	MHz	Second order PLL transfer bounding function. See Note 1.
PKG_{RX-PLL1}	Maximum Rx PLL peaking corresponding to BW _{RX-PKG-PLL1}	(max) 3.0	3.0	2.0	2.0	2.0	2.0	dB	Second order PLL transfer bounding function. See Note 1.
PKG_{RX-PLL2}	Maximum Rx PLL peaking corresponding to BW _{RX-PKG-PLL2}	Not specified	1.0	1.0	1.0	N/A	N/A	dB	Second order PLL transfer bounding function. See Note 1.
RL_{RX-DIFF}	Differential receiver return loss	See § Figure 8-22					See § Figure 8-24	dB	Note 2
RL_{RX-CM}	Common mode receiver return loss	See § Figure 8-23					See § Figure 8-25	dB	Note 2
T_{RX-GND-FLOAT}	Rx termination float time			(max) 500	(max) 500	(max) 500	(max) 500	μs	Note 5
V_{RX-CM-AC-P}	Rx AC common Mode Voltage	(max) 150	(max) 150	(max) 75	(max) 75	(max) 75	(max) 37.5	mVP	Measured at Rx pins into a pair of 50Ω terminations to ground
Z_{RX-DC}	Receiver DC single ended impedance	(min) 40 (max) 60	(min) 40 (max) 60	Not specified	Not specified	Not specified	Not specified	Ω	DC impedance limits are needed to guarantee Receiver detect. For 8.0,

Symbol	Parameter	2.5 GT/s value	5.0 GT/s value	8.0 GT/s value	16.0 GT/s value	32.0 GT/s value	64.0 GT/s value	Units	Notes
									16.0, 32.0, and 64.0 GT/s it is bounded by RL _{RX-CM} . See Note 3.
Z_{RX-HIGH-IMP-DC-POS}	DC input CM input impedance for V ≥ 0 during Reset or power-down	≥ 10K (0-200 mV) ≥ 20K (> 200 mV)	≥ 10K (0-200 mV) ≥ 20K (> 200 mV)	≥ 10K (0-200 mV) ≥ 20K (> 200 mV)	≥ 10K (0-200 mV) ≥ 20K (> 200 mV)	≥ 10K (0-200 mV) ≥ 20K (> 200 mV)	≥ 10K (0-200 mV) ≥ 20K (> 200 mV)	Ω	Voltage measured wrt. ground. Parameters may not scale with process technology. See Note 4.
Z_{RX-HIGH-IMP-DC-NEG}	DC input CM input impedance for V < 0 during Reset or power-down	1.0K (min)	1.0K (min)	1.0K (min)	1.0K (min)	1.0K (min)	1.0K (min)	Ω	Voltage measured over the range of -150 mV to 0 mV wrt. ground. Parameters may not scale with process technology. See Note 4.
V_{RX-IDLE-DET-DIFF-PP}	Electrical Idle Detect threshold	(min) 65 (max) 175	(min) 65 (max) 175	(min) 65 (max) 175	(min) 65 (max) 175	(min) 65 (max) 175	(min) 65 (max) 175	mV	V_{RX-IDLE-DET-DIFFp-p} = $2 \times V_{RX-D+} - V_{RX-D-} $
T_{RX-IDLE-DET-DIFF-ENTERTIME}	Unexpected Electrical Idle Enter Detect Threshold Integration Time	(max) 10	(max) 10	(max) 10	(max) 10	(max) 10	(max) 10	ms	An unexpected Electrical Idle (V _{RX-IDLE-DET-DIFF-PP} < V _{RX-IDLE-DET-DIFFp-p}) must be recognized no longer than T _{RX-IDLE-DET-DIFF-ENTERTIME} to signal an unexpected idle condition.
L_{RX-SKEW}	Lane to Lane skew	(max) 20	(max) 8	(max) 6	(max) 5	(max) 5	(max) 5	ns	Across all Lanes on a Port. LRX-SKEW comprehends Lane-Lane variations due to channel and repeater delay differences.

Notes:

- Two combinations of PLL BW and peaking are specified at 5.0 GT/s to permit designers to make tradeoffs between the two parameters. If the PLL's min BW is ≥ 8 MHz, then up to 3.0 dB of peaking is permitted. If the PLL's min BW is relaxed to ≥ 5.0 MHz, then a tighter peaking value of 1.0 dB must be met. Note: a PLL BW extends from zero up to the value(s) defined as the min or max in the above table. For 2.5 GT/s a single PLL bandwidth and peaking value of 1.5-22 MHz and 3.0 dB are defined.
- Measurements must be made for both common mode and differential return loss. In both cases the DUT must be powered up and DC isolated, and its D+/D- inputs must be in the low-Z state.

8.4.4 Lane Margining at the Receiver - Electrical Requirements §

PCI Express components including retimers that support the 16.0 GT/s rate are required to support Lane margining at the Receiver when operating at 16.0, 32.0, or 64.0 GT/s. Lane Margining enables system software to get the margin information of a given Lane while the Link is in L0 state. The margin information includes both voltage and time, in either direction from the current Receiver position. For 64.0 GT/s PAM4 signaling, the margin information includes both voltage and time for all the three eyes. The margin feature is not permitted to require any additional external hardware to function. Support of Lane margining for voltage is optional at 16.0 GT/s and required at 32.0 and 64.0 GT/s. Support of independent timing margin to the left or to the right is optional for all data rates. For simplicity, the margin commands and requirements described in the protocol chapter(s) of this specification are described in terms of moving the data sample location - but the actual margining method is implementation specific. For example - the timing margin could be implemented on the actual data sampler or an independent error sampler. Further the timing margin can be achieved by injecting an appropriate amount of stress/jitter to the data sample location, or by moving the data/error sample location. The parameters in § Table 8-12 are reported for 16.0, 32, and 64.0 GT/s and are allowed to be different for each speed.

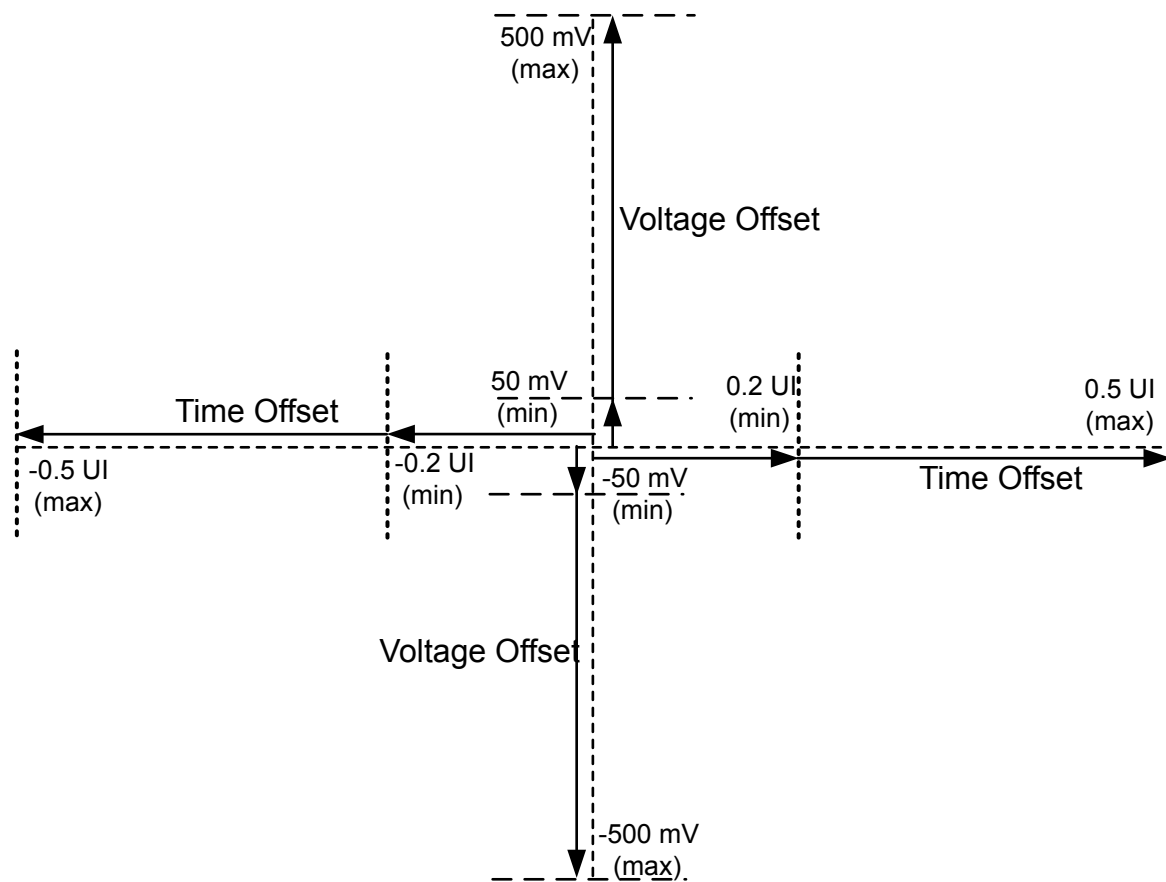


Figure 8-54 Allowed Ranges for Maximum NRZ Timing and Voltage Margin

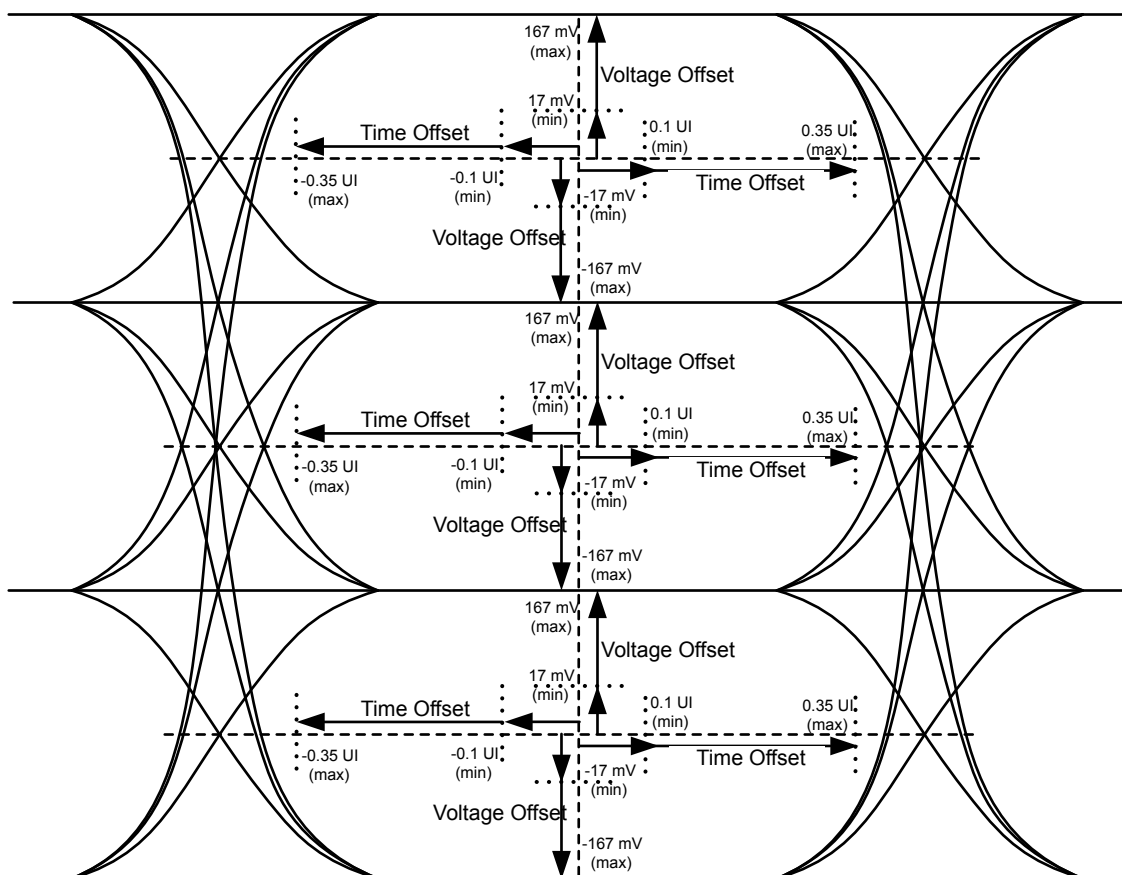


Figure 8-55 Allowed Ranges for Maximum PAM4 Timing and Voltage Margins

§

Table 8-12 Lane Margining

Parameter Name	Min	Max	Description
$M_{NumTimingSteps}$	6	63	Number of time steps from default (to either left or right), range must be at least +/-0.2 UI Timing offset must increase monotonically The number of steps in both positive (toward the end of the unit interval) and negative (toward the beginning of the unit interval) must be identical
$M_{MaxTimingOffset}$	20 %	50 %	Offset from default at maximum step value as percentage of a nominal UI A 0 value may be reported if the vendor chooses not to report the offset
$M_{NumVoltageSteps}$	32	127	Number of voltage steps from default (either up or down), minimum range +/-50 mV as measured by the reference equalizer Voltage offset must increase monotonically The number of steps in both positive and negative direction from the default sample location must be identical

Parameter Name	Min	Max	Description
			This value is undefined if $M_{VoltageSupported}$ is 0b
$M_{MaxVoltageOffset}$	5 %	50 %	Offset from default at maximum step value as percentage of one volt A 0 value may be reported if the vendor chooses not to report the offset when $M_{VoltageSupported}$ is 1b This value is undefined if $M_{VoltageSupported}$ is 0b
$M_{NumTimingStepsPAM4}$	10	63	Number of time steps from default (to either left or right), range must be at least +/-0.1 UI Timing offset must increase monotonically The number of steps in both positive (toward the end of the unit interval) and negative (toward the beginning of the unit interval) must be identical
$M_{MaxTimingOffsetPAM4}$	10 %	35 %	Offset from default at maximum step value as percentage of a nominal UI A 0 value may be reported if the vendor chooses not to report the offset
$M_{NumVoltageStepsPAM4}$	64	127	Number of voltage steps from default (either up or down), minimum range +/-17 mV as measured by the reference equalizer Voltage offset must increase monotonically The number of steps in both positive and negative direction from the default sample location must be identical
$M_{MaxVoltageOffsetPAM4}$	5 %	50 %	Offset from default at maximum step value as percentage of one third volt
$M_{SamplingRateVoltage}$	0	63	The ratio of bits tested to bits received during voltage margining. A value of 0 is a ratio of 1:64 (1 bit of every 64 bits received), and a value of 63 is a ratio of 64:64 (all bits received).
$M_{SamplingRateTiming}$	0	63	The ratio of bits tested to bits received during timing margining. A value of 0 is a ratio of 1:64 (1 bit of every 64 bits received), and a value of 63 is a ratio of 64:64 (all bits received).
$M_{VoltageSupported}$	0	1	1b indicates that voltage margining is supported
$M_{IndLeftRightTiming}$	0	1	1b indicates independent left/right timing margin supported
$M_{IndUpDownVoltage}$	0	1	1b independent up and down voltage margining supported
$M_{IndErrorSampler}$	0	1	1b Margining will not produce errors (change in the error rate) in data stream (ie. error sampler is independent) 0b Margining may produce errors in the data stream
$M_{MaxLanes}$	0	31	Maximum number of Lanes minus 1 that can be margined at the same time. It is recommended that this value be greater than or equal to the number of Lanes in the Link minus 1. Encoding Behavior is undefined if software attempts to margin more than $M_{MaxLanes}+1$ at the same time.

Parameter Name	Min	Max	Description
			Note: This value is permitted to exceed the number of Lanes in the Link minus 1.
<i>M_{SampleReportingMethod}</i>	0	1	Indicates whether sampling rates (<i>M_{SamplingRateVoltage}</i> and <i>M_{SamplingRateTiming}</i>) are supported (1) or a sample count is supported (0). One of the two methods is supported by each device.
<i>M_{ErrorCount}</i>	0	63	If <i>M_{IndErrorSampler}</i> is 1b this is a count of the actual bit errors since margining started. If <i>M_{IndErrorSampler}</i> is 0b this is the actual count of the logical errors since margining started. See the Physical Layer Logical Block chapter for the definition of what errors are counted. The count saturates at 63.
<i>M_{SampleCount}</i>	0	127	Value = $3 \cdot \log_2$ (number of bits margined). Where number of bits margined is a count of the actual number of bits tested during margining. The count stops when margining stops. The count saturates at 127 (after approximately 5.54×10^{12} bits). The count resets to zero when a new margin command is received.

8.4.5 Low Frequency and Miscellaneous Signaling Requirements §

8.4.5.1 ESD Standards §

All PCI Express signal and power supply pins must be tested for ESD protection levels to the Human Body Model (HBM) and the Charged Device Model (CDM) standards in accordance with [ESDA-JEDEC-JS-001-2010] (for HBM) and in accordance with [JEDEC-JESD22-C101] (for CDM). Pins must meet or exceed the minimum levels recommended in [JEDEC-JEP155-JEP157] (HBM/CDM) or JEDEC approved superseding documents.

8.4.5.2 Channel AC Coupling Capacitors §

Each Lane of a PCI Express Link must be AC coupled. The minimum and maximum values for the capacitance are given in § Table 8-7. Capacitors must be placed on only one side of an interface that permits adapters to be plugged and unplugged. Form factor specifications must define the required location of the capacitor. In a topology where everything is located on a single substrate, the capacitors may be located anywhere along the channel. External capacitors are assumed because the values required are too large to feasibly construct on-chip.

8.4.5.3 Short Circuit Requirements §

All Transmitters and Receivers must support surprise hot insertion/removal without damage to the component. The Transmitter and Receiver must be capable of withstanding sustained short circuit to ground of D+ and D-.

8.4.5.4 Transmitter and Receiver Termination §

- The Transmitter is required to meet $RL_{TX-DIFF}$ and RL_{TX-CM} (see § Figure 8-22, § Figure 8-24, § Figure 8-23 and § Figure 8-25) any time functional differential signals are being transmitted.
- The Transmitter is required only to meet $l_{TX-SHORT}$ (see § Table 8-7) any time functional differential signals are not being transmitted.
- Note: The differential impedance during this same time is not defined.
- The Receiver is required to meet $RL_{RX-DIFF}$ and RL_{RX-CM} (see § Table 8-11) during all LTSSM states excluding only times during when the device is powered down, Fundamental Reset is asserted, or when explicitly specified.
- The Receiver is required to meet $Z_{RX-HIGH-IMP-DC-NEG}$ and $Z_{RX-HIGH-IMP-DC-POS}$ (see § Table 8-11) any time adequate power is not provided to the Receiver, Fundamental Reset is asserted, or when explicitly specified.

8.4.5.5 Electrical Idle §

Electrical Idle is a steady state condition where the Transmitter D+ and D- voltages are held constant at the same value. Electrical Idle is primarily used in power saving and inactive states (e.g., Disabled).

Before a Transmitter enters Electrical Idle, it must always send the required number of EIOs except for the LTSSM substates explicitly exempted from this requirement. After sending the last Symbol of the last of the required number of EIOs, the Transmitter must be in a valid Electrical Idle state within the time as specified by $T_{TX-IDLE-SET-TO-IDLE}$ in § Table 8-7.

The successful reception of an EIOS occurs based on the rules defined in the Physical Layer Logical Block chapter. It should be noted that in substates (e.g., Loopback.Active for a Loopback Follower) where multiple consecutive EIOs are expected, the Receiver must receive the appropriate number of EIOS sequences comprising of COM, IDL, IDL, IDL.

The low impedance common mode and differential Receiver termination values (see § Table 8-7 and § Table 8-11) must be met in Electrical Idle. The Transmitter can be in either a low or high impedance mode during Electrical Idle.

Any time a Transmitter enters Electrical Idle it must remain in Electrical Idle for a minimum of $T_{TX-IDLE-MIN}$. The Receiver should expect the last EIOS followed by a minimum amount of time in Electrical Idle ($T_{TX-IDLE-MIN}$) to arm its Electrical Idle Exit detector.

When the Transmitter transitions from Electrical Idle to a valid differential signal level it must meet the output return loss specifications described in § Figure 8-22, § Figure 8-24, § Figure 8-23, and § Figure 8-25.

Electrical Idle Exit shall not occur if a signal smaller than $V_{RX-IDLE-DET-DIFFp-p}$ minimum is detected at a Receiver. Electrical Idle Exit shall occur if a signal larger than $V_{RX-IDLE-DET-DIFFp-p}$ maximum is detected at a Receiver. Electrical Idle may be detected on the received signal regardless of its frequency components, or it may be detected only when the received signal is switching at a frequency of 125 MHz or higher.

8.4.5.6 DC Common Mode Voltage §

The Receiver DC common mode voltage is nominally 0 V when operating at 2.5 GT/s or 5.0 GT/s.

Transmitter DC common mode voltage is held at the same value during all states unless otherwise specified. The range of allowed Transmitter DC common mode values is specified in § Table 8-7 ($V_{TX-DC-CM}$).

8.4.5.7 Receiver Detection §

The Receiver Detection circuit is implemented as part of a Transmitter and must correctly detect whether a load impedance equivalent to a DC impedance implied by the Z_{RX-DC} parameter (40 Ω -60 Ω) is present. Note: Support for Rx detect, which only occurs at 2.5 GT/s, is the reason why 2.5 GT/s Receivers impedance at DC is specified.

The recommended behavior of the Receiver Detection sequence is described below:

Step 1. A Transmitter must start at a stable voltage prior to the detect common mode shift.

Step 2. A Transmitter changes the common mode voltage on D+ and D- consistent with meeting the $V_{TX-RCV-DETECT}$ parameter and consistent with detection of Receiver high impedance which is bounded by parameters $Z_{RX-HIGH-IMP-DC-POS}$, $Z_{RX-HIGH-IMP-DC-NEG}$ in § Table 8-11. Receiver is detected based on the rate that the lines change to the new voltage.

- a. The Receiver is not present if the voltage at the Transmitter charges at a rate dictated only by the Transmitter impedance and the capacitance of the interconnect and series capacitor.
- b. The Receiver is present if the voltage at the Transmitter charges at a rate dictated by the Transmitter impedance, the series capacitor, the interconnect capacitance, and the Receiver termination.

If the Receiver Detection circuit performs the detect sequence on each conductor of the differential pair (both D+ and D-) and detects a load impedance greater than Z_{RX-DC} on either conductor, the Receiver Detection circuit shall interpret this as no termination load present and respond as if neither load were present.

It is required that the detect sequence be performed on both conductors of a differential pair.

8.5 Channel Tolerancing §

8.5.1 Channel Compliance Testing §

This section of the specification is relevant only for those cases where a platform design comprehends the relevant channel between Transmitter device pins and Receiver device pins. These types of platform designs are called “captive channels”. Designs that are not captive channels shall refer to the appropriate form factor (CEM is one example) specification, since in this case the form factor specification takes precedence over this specification and splits the channel between two different types of components.

The key components and processes of channel tolerancing are illustrated in § Figure 8-56 and § Figure 8-57. The major difference lies in the complexity of the Behavioral Tx and Rx equalization, which depends on the data rate. 2.5 and 5.0 GT/s utilize fixed Tx presets and assume no Rx equalization, whereas 8.0, 16.0, 32.0, and 64.0 GT/s assume multi-valued, adjustable Tx presets and a combination of Rx DFE and CTLE.

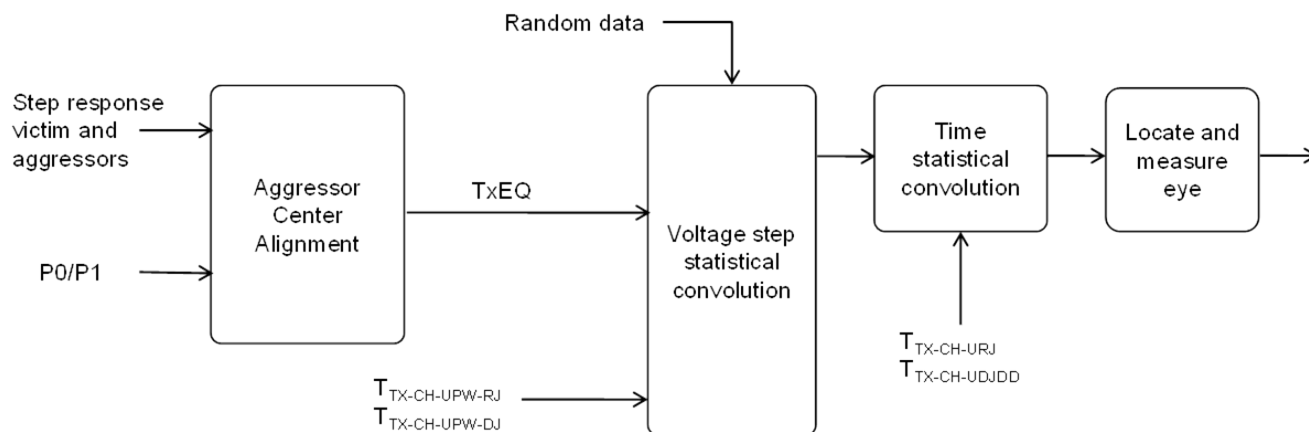


Figure 8-56 Flow Diagram for Channel Tolerancing at 2.5 and 5.0 GT/s

The basic channel compliance approach is to first acquire the channel's characteristics, usually by means of s-parameters or equivalent model. Behavioral Tx and Rx package models are then appended to the channel model to yield a die pad to die pad topology. The model shall include both victim path and a sufficient number of aggressor paths to accurately capture channel crosstalk effects. Using the Tx voltage and jitter limits defined in the Transmitter specification section it is possible to transform these parameters to what would appear at the die pad of a Tx.

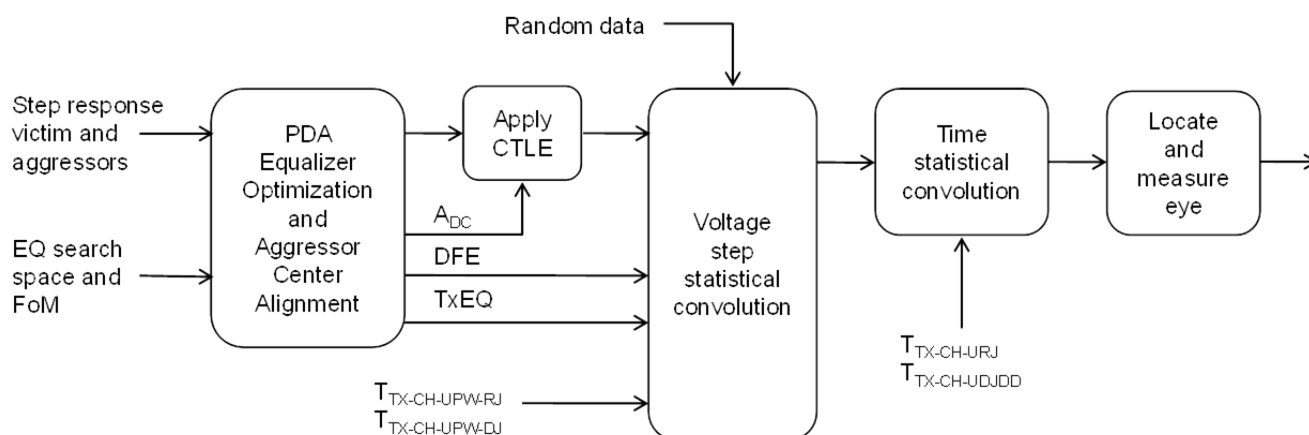


Figure 8-57 Flow Diagram for Channel Tolerancing at 8.0 and 16.0 GT/s

The resulting model is analyzed via simulation, yielding voltage and jitter at a point equivalent to the input latch of the Receiver. The signal observed at the Receiver's latch is referenced to a recovered data clock from which an eye diagram may be constructed.

For 8.0, 16.0, 32.0, and 64.0 GT/s testing the simulation process must properly account for Tx and Rx equalization optimization as must be supported by a minimum capability Tx/Rx pair. This means the simulation process must be able to select the optimum values for the Tx presets or coefficients and Rx equalization settings based upon:

- a 1st order CTLE and a 1-tap DFE for 8.0 GT/s,
- a 1st order CTLE and a 2-tap DFE for 16.0 GT/s,

- a 2nd order CTLE and a 3-tap DFE for 32.0 GT/s, and
- a CTLE transfer function with six poles and three zeros and a 16-tap DFE for 64.0 GT/s.

8.5.1.1 Behavioral Transmitter and Receiver Package Models §

Behavioral package models are defined in this specification to represent the combined die and package loss that is expected to interoperate with the targeted range of channels. Note that at 16.0 GT/s, the behavioral packages represent a high, but not worst-case, loss for many devices. (see § Section 8.3.3.11 and § Section 8.4.1.5)

A separate pair of package models are defined for 8.0, 16.0, 32.0, and 64.0 GT/s. At 8.0 GT/s, separate package models are defined for TX and RX ports to reflect the smaller CPAD capacitance typical in most receiver implementations. At 16.0, 32.0, and 64.0 GT/s, separate package models are defined for devices containing Root Ports and all other devices to reflect the large and socketed nature of most devices containing Root Complexes. Channel testing for all three data rates typically requires testing in both directions.

The package models are included with the specification as design collateral. Each model for 8.0 and 16.0 GT/s comprehends C_{PIN} and C_{PAD} parasitic capacitances plus a differential t-line element as illustrated in § Figure 8-58.

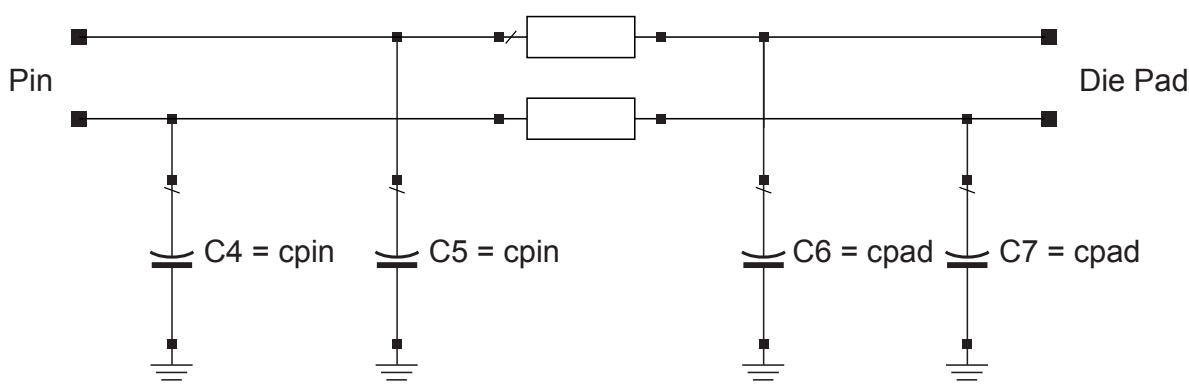


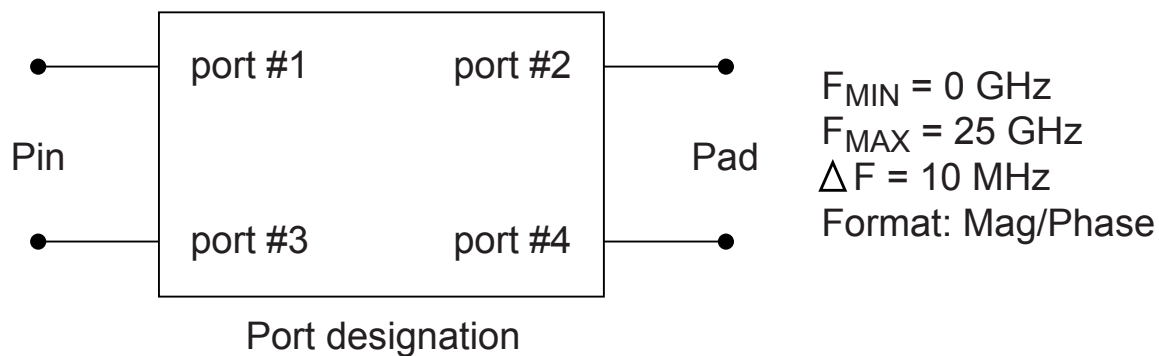
Figure 8-58 Tx/Rx Behavioral Package Models

The C_{PIN} and C_{PAD} values used in the package model generation are provided for informative purposes only.

Table 8-13 Package Model Capacitance Values §

	8.0 GT/s Tx	8.0 GT/s Rx	16.0 GT/s
C_{PIN}	0.25 pF	0.25 pF	0.25 pF
C_{PAD}	1.0 pF	0.8 pF	0.5 pF

For ease of incorporation into the post processing flow the 8.0 and 16.0 behavioral package models are specified as 4-port s-parameter files. The files are specified with port designations, frequency range and granularity as listed below. The reference impedance for the s-parameters is 50 Ω . File format is Touchstone.



A-0839A

Figure 8-59 Behavioral Tx and Rx S-Port Designation for 8.0 and 16.0 GT/s Packages

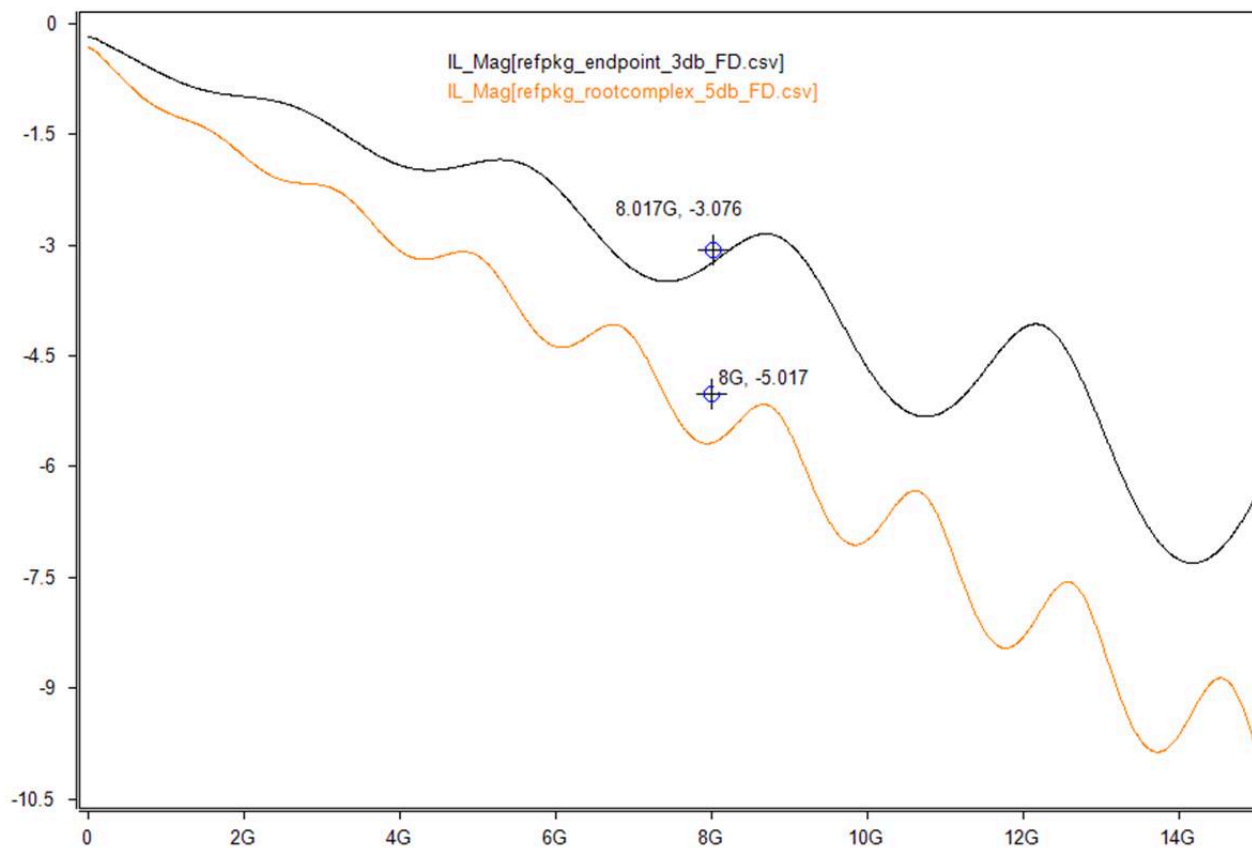


Figure 8-60 SDD21 Plots for Root and Non-Root Packages for 16.0 GT/s

For 32.0 and 64.0 GT/s the package models are based on real package and socket models for the root package and package and BGA models for the non-root package.

For 32.0 GT/s, reference package models the die capacitive loads are included in the models. For 64.0 GT/s, the effective die cap has been replaced by the S-parameter representation of an on-die network that represents the insertion and

return loss characteristics of on-die pad for a typical design. The reference impedance for the 32.0 and 64.0 GT/s reference package model s-parameters is 50 Ω . Figures below show 32.0 and 64.8 GT/s reference package insertion loss, return loss, FEXT, and NEXT S-parameter plots based on 50 Ω impedance. To account for the on-die inductive coil DC loss of about 3 Ω , the Tx and Rx DC termination should be set to 47 Ω in 64.0 GT/s channel compliance simulations.

Insertion loss

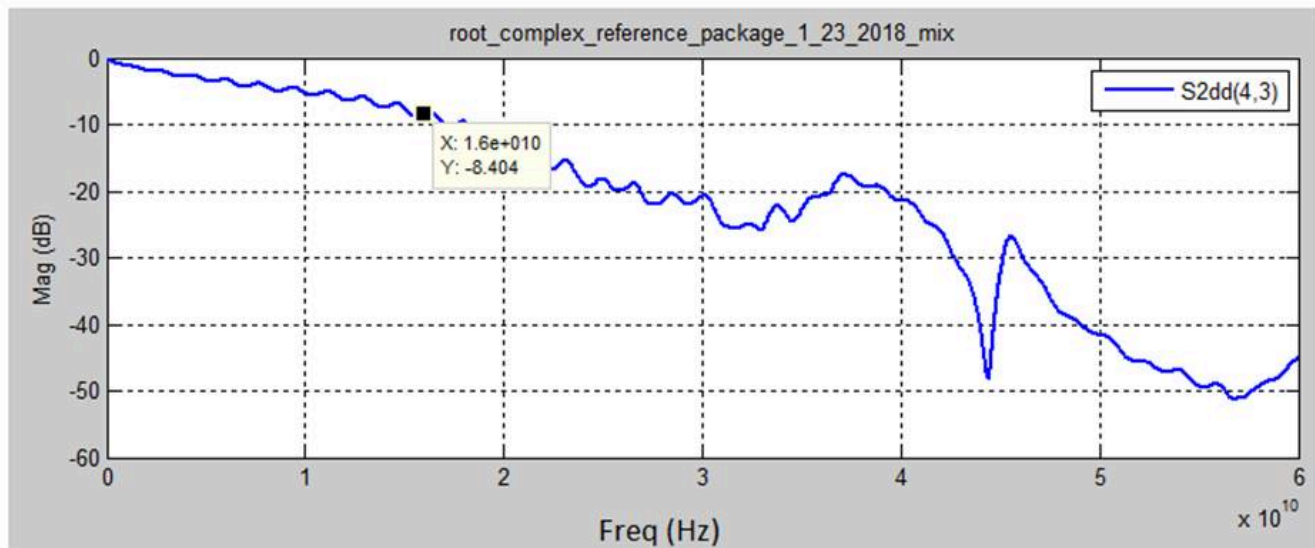


Figure 8-61 Insertion Loss for Root Reference Package for 32.0 GT/s

Return Loss (Board-side)

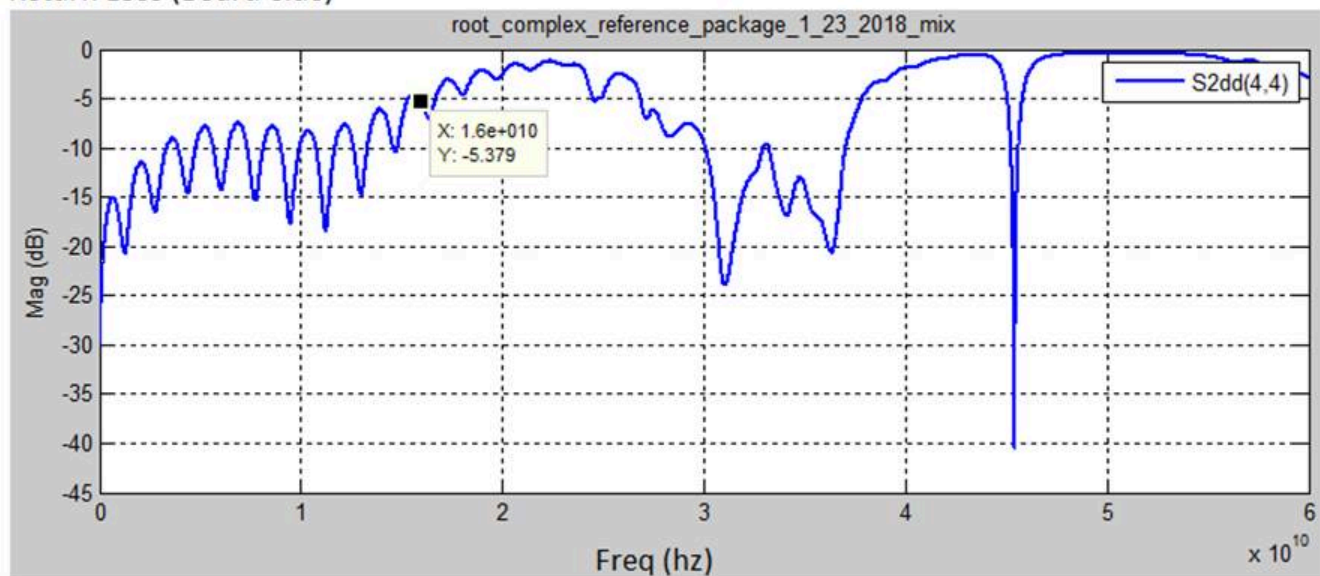


Figure 8-62 Return Loss for Root Reference Package for 32.0 GT/s

NEXT 2

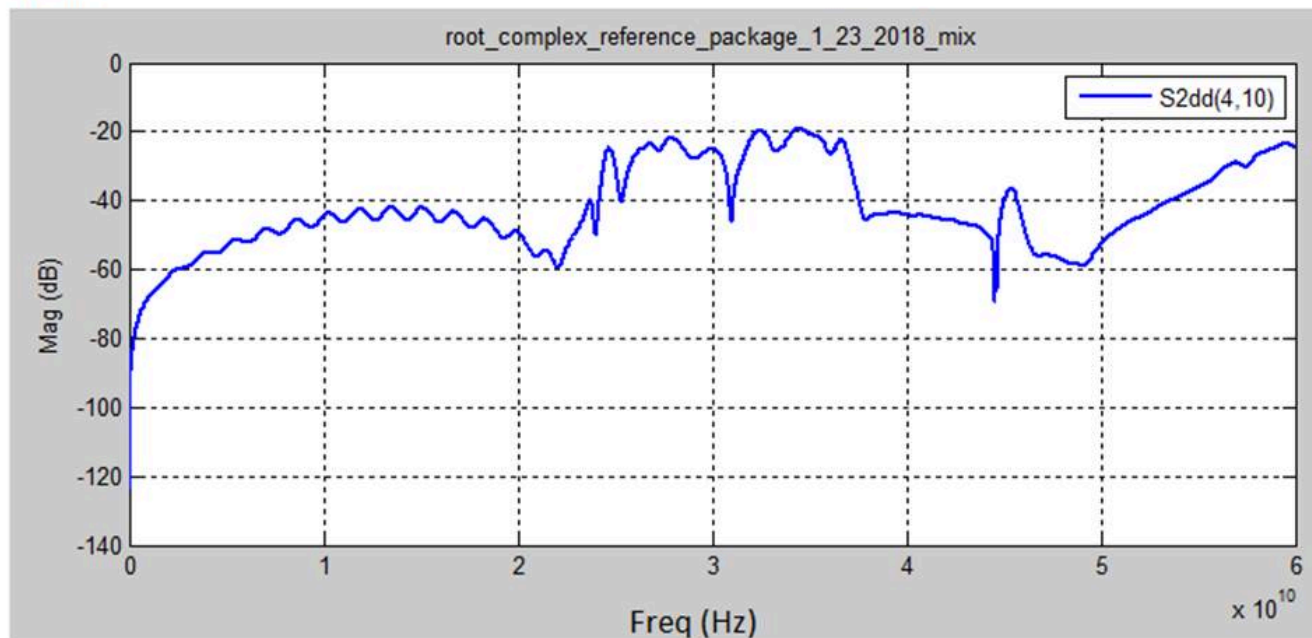


Figure 8-63 NEXT for Root Reference Package (Worst-Case) for 32.0 GT/s

FEXT 1

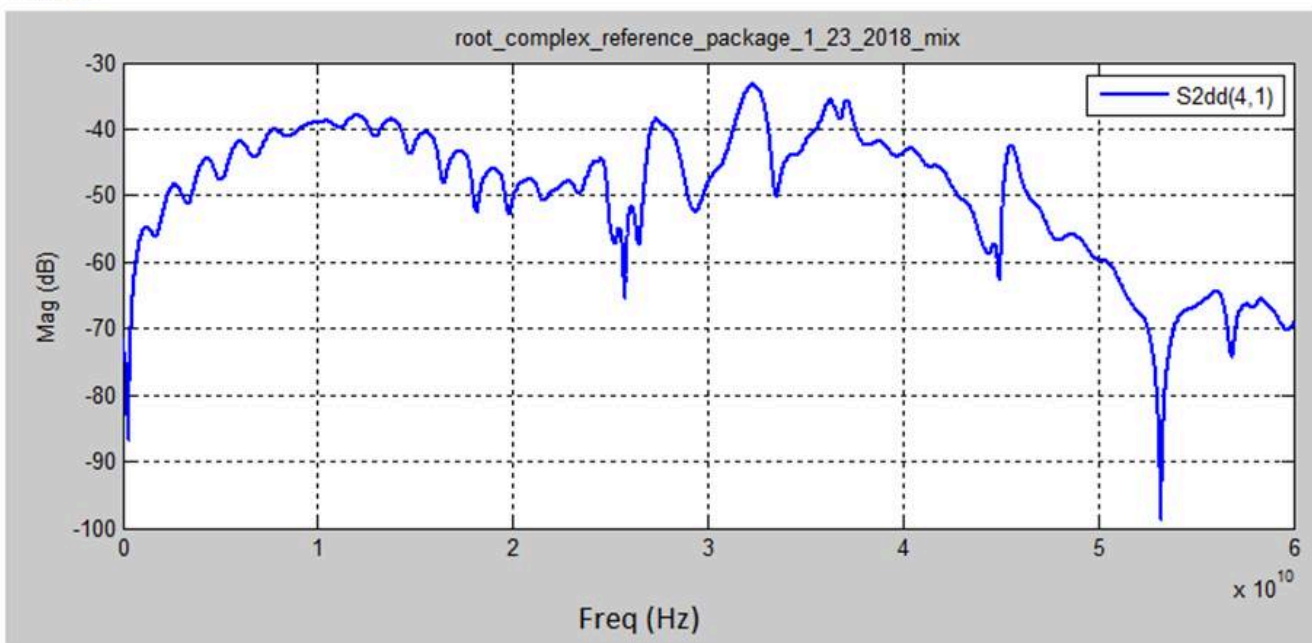


Figure 8-64 FEXT for Root Reference Package (Worst-Case) for 32.0 GT/s

Insertion loss

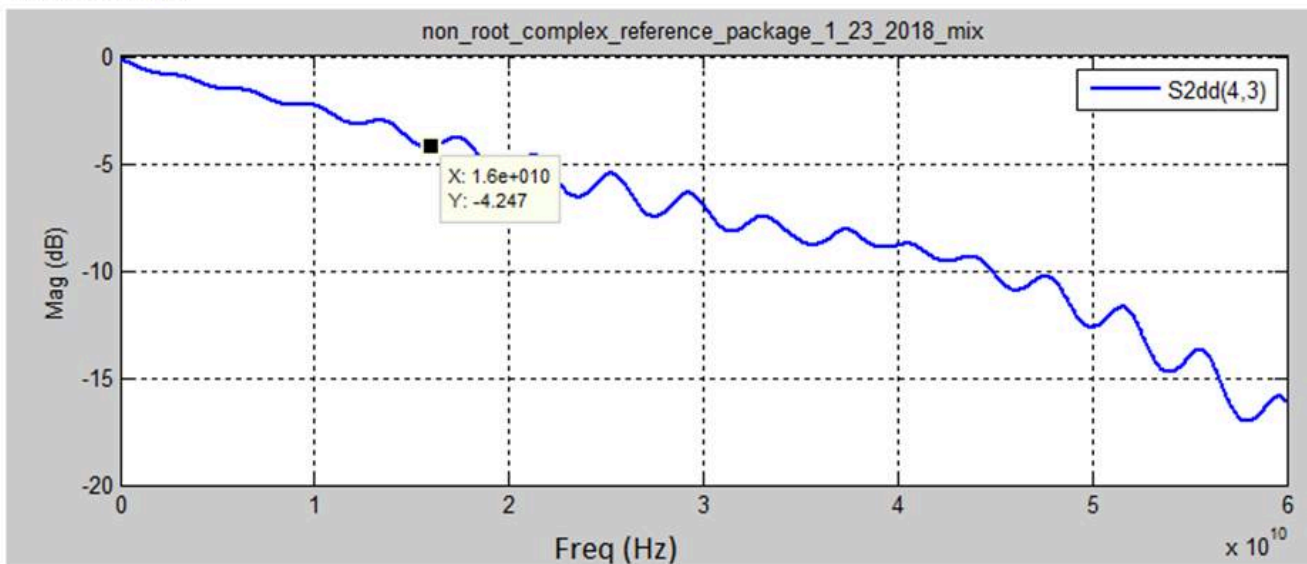


Figure 8-65 Insertion Loss for Non-Root Reference Package for 32.0 GT/s

Return Loss (Board-side)

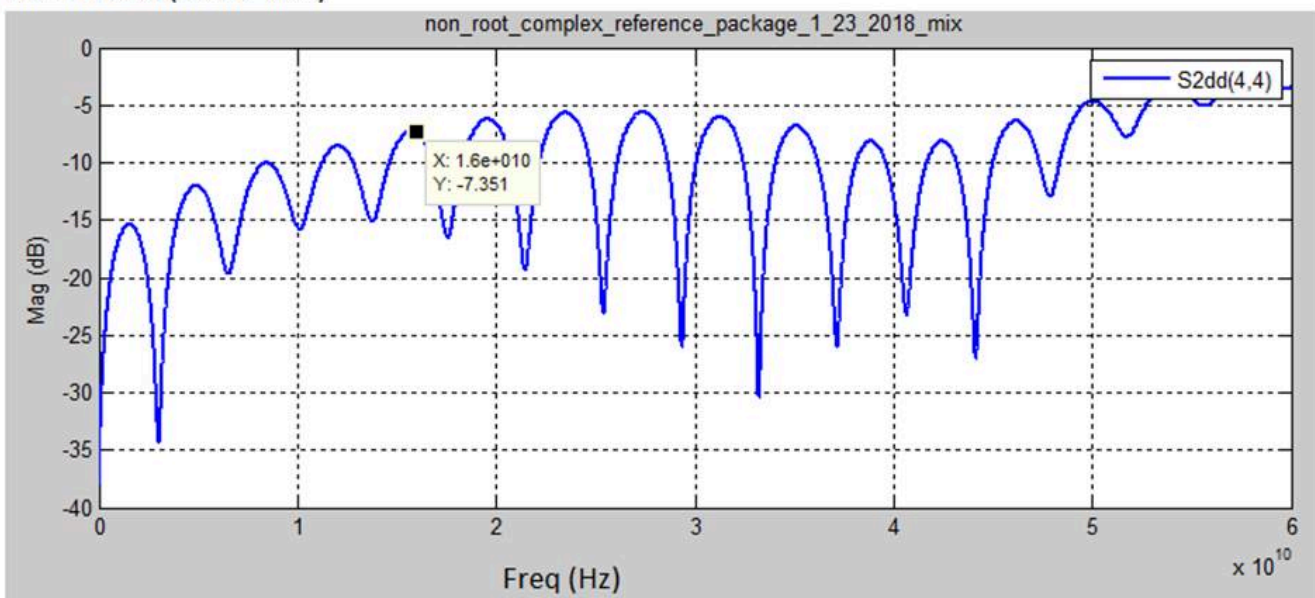


Figure 8-66 Return Loss for Non-Root Reference Package for 32.0 GT/s

NEXT 1

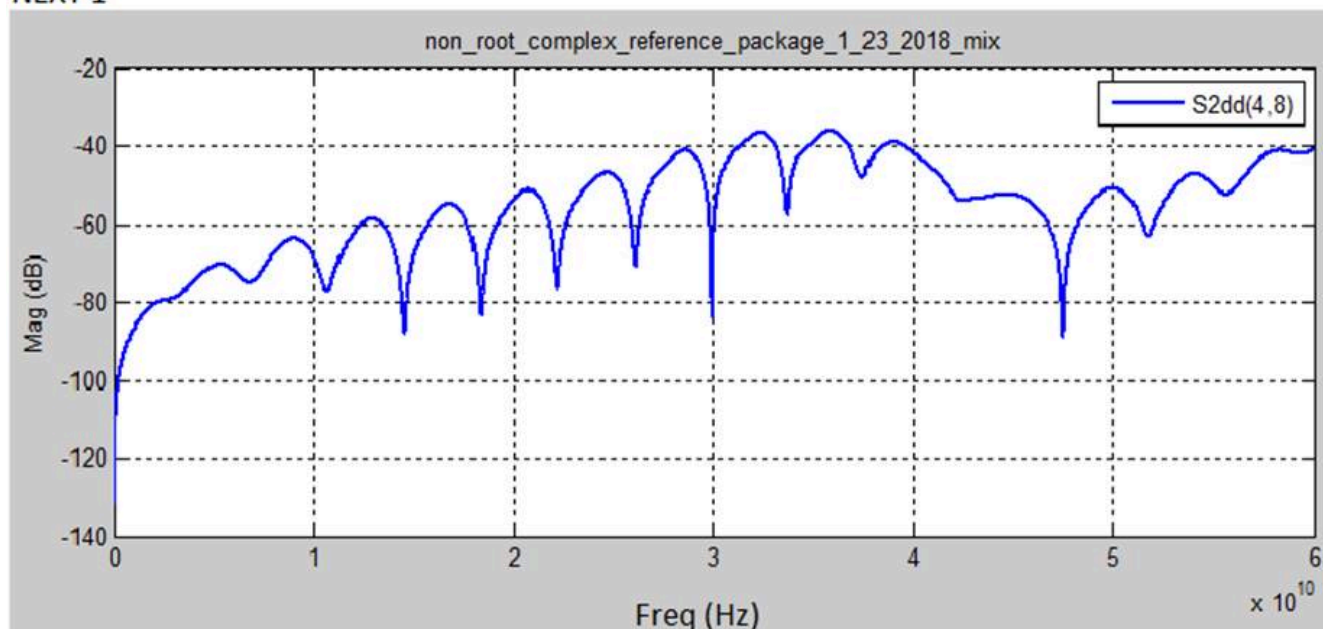


Figure 8-67 NEXT for Non-Root Reference Package (Worst-Case) for 32.0 GT/s

FEXT 2 (Die-side)

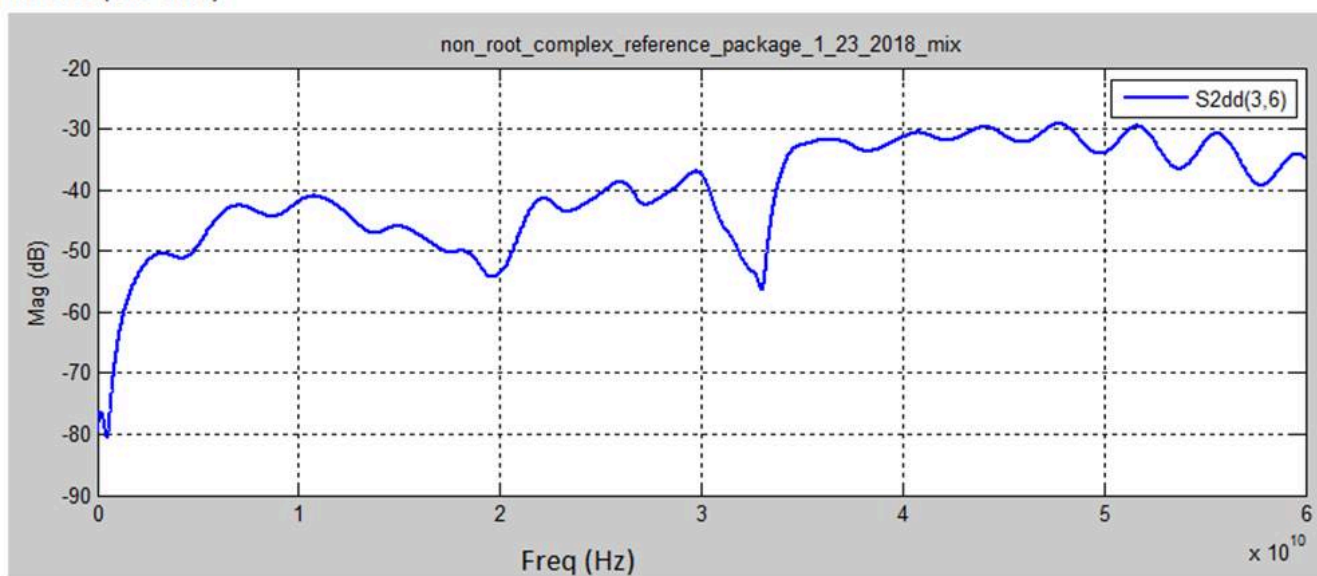


Figure 8-68 FEXT for Non-Root Reference Package (Worst-Case) for 32.0 GT/s

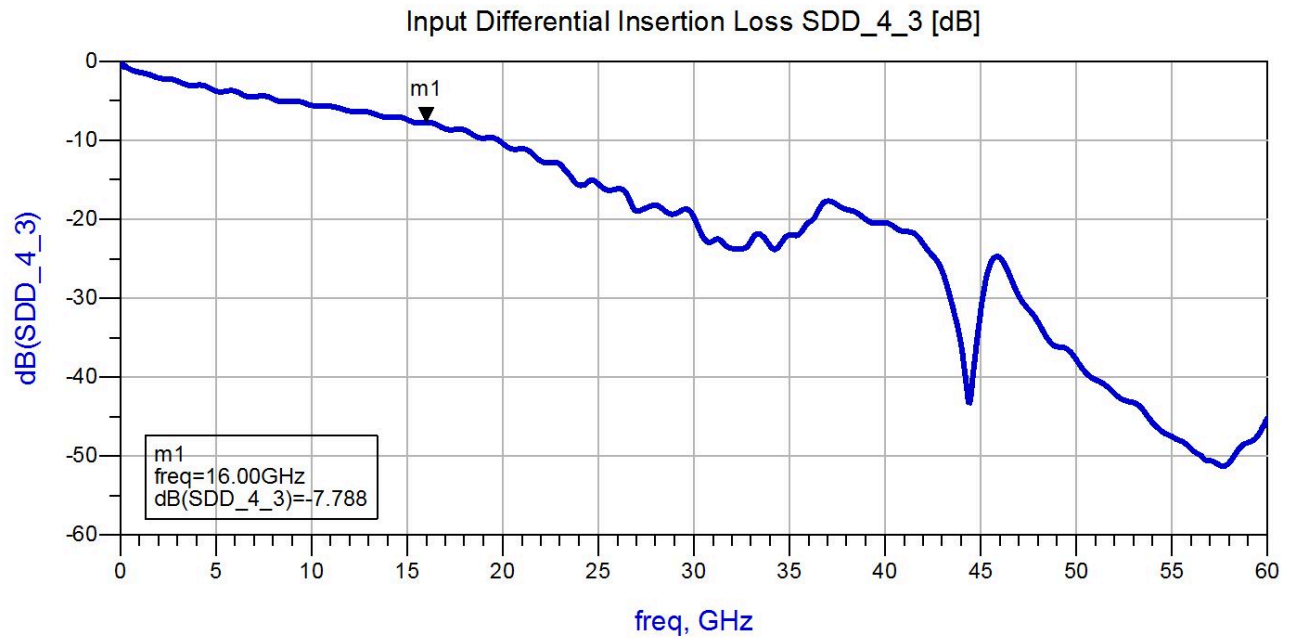


Figure 8-69 Insertion Loss for Root Reference Package for 64.0 GT/s

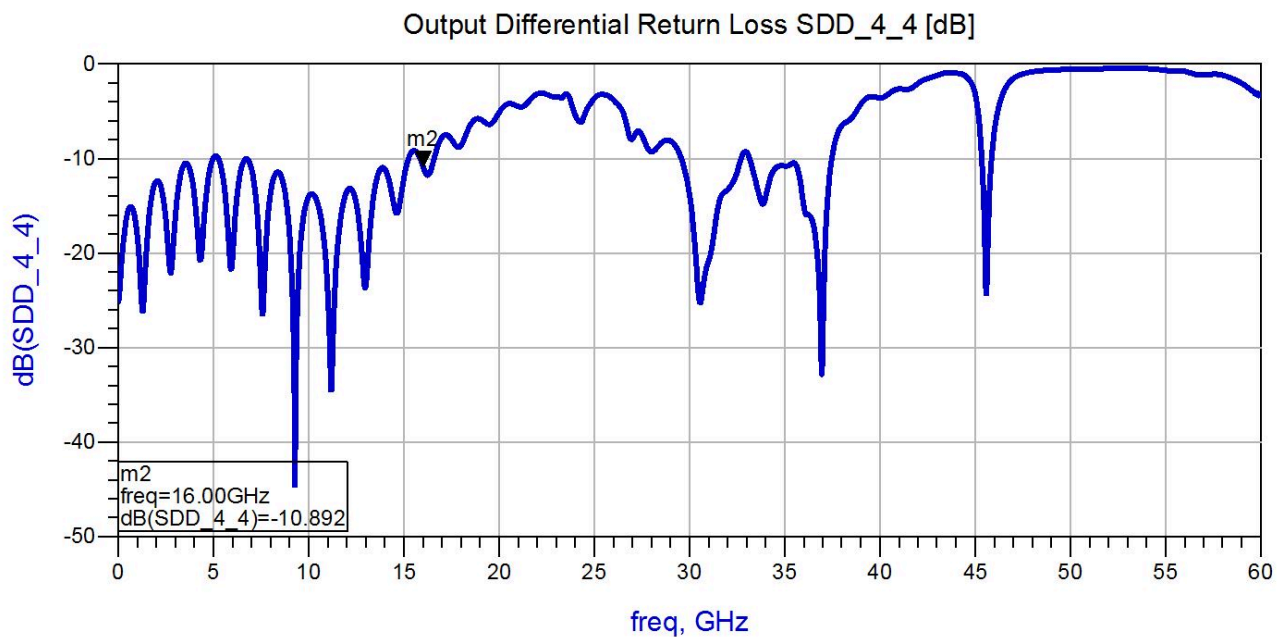


Figure 8-70 Return Loss for Root Reference Package for 64.0 GT/s

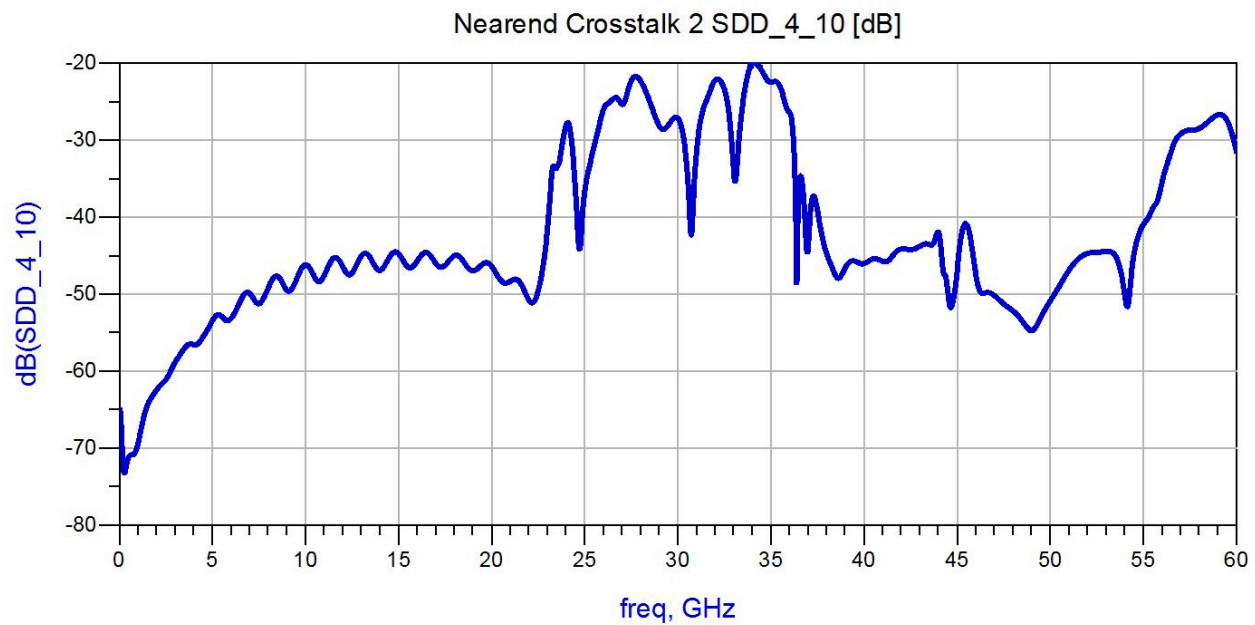


Figure 8-71 NEXT for Root Reference Package (Worst Case) for 64.0 GT/s

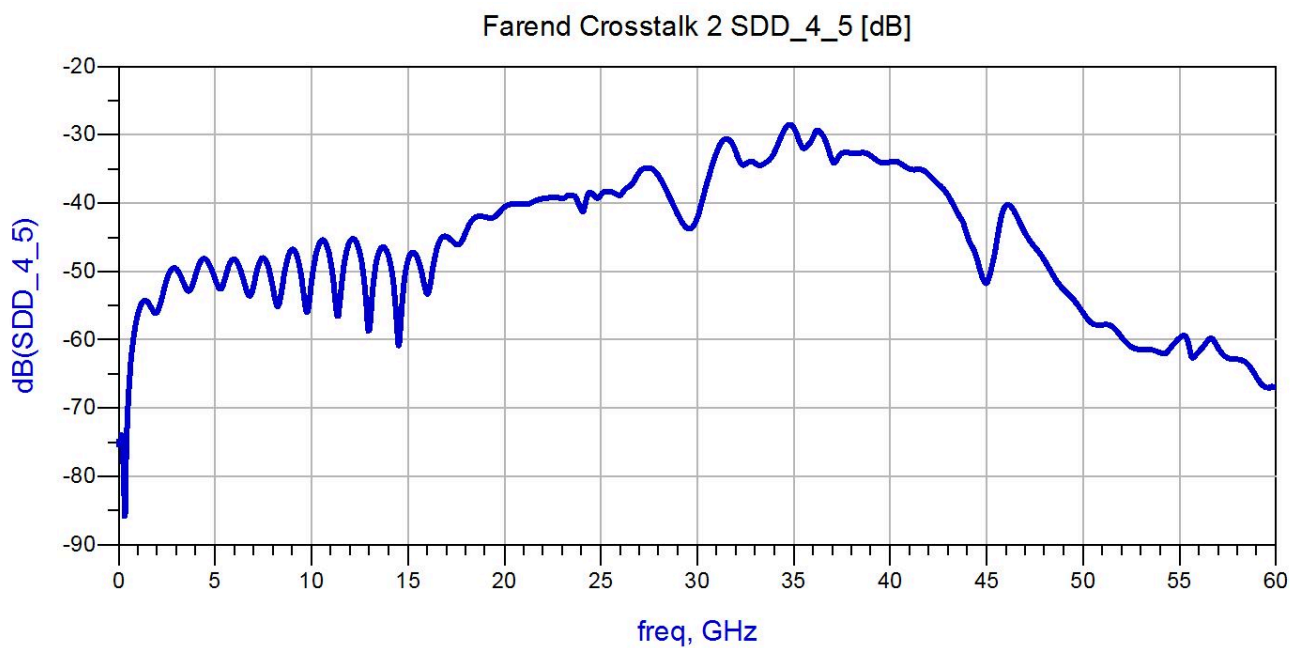


Figure 8-72 FEXT for Root Reference Package (Worst Case) for 64.0 GT/s

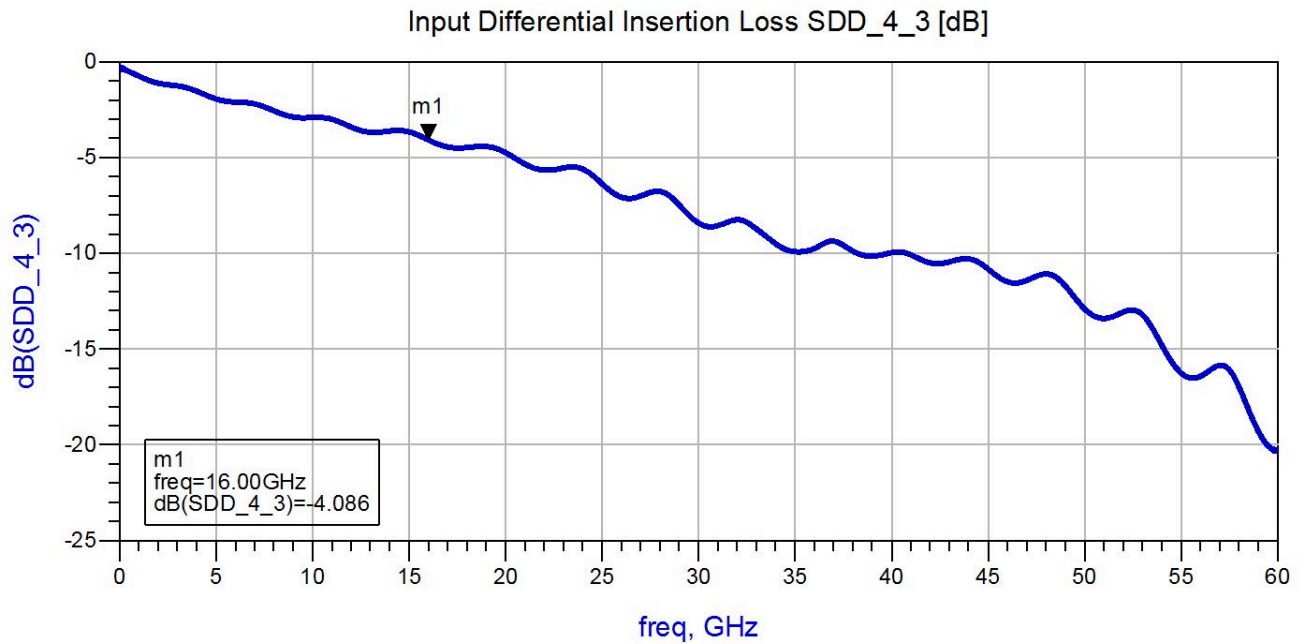


Figure 8-73 Insertion Loss for Non-Root Reference Package for 64.0 GT/s

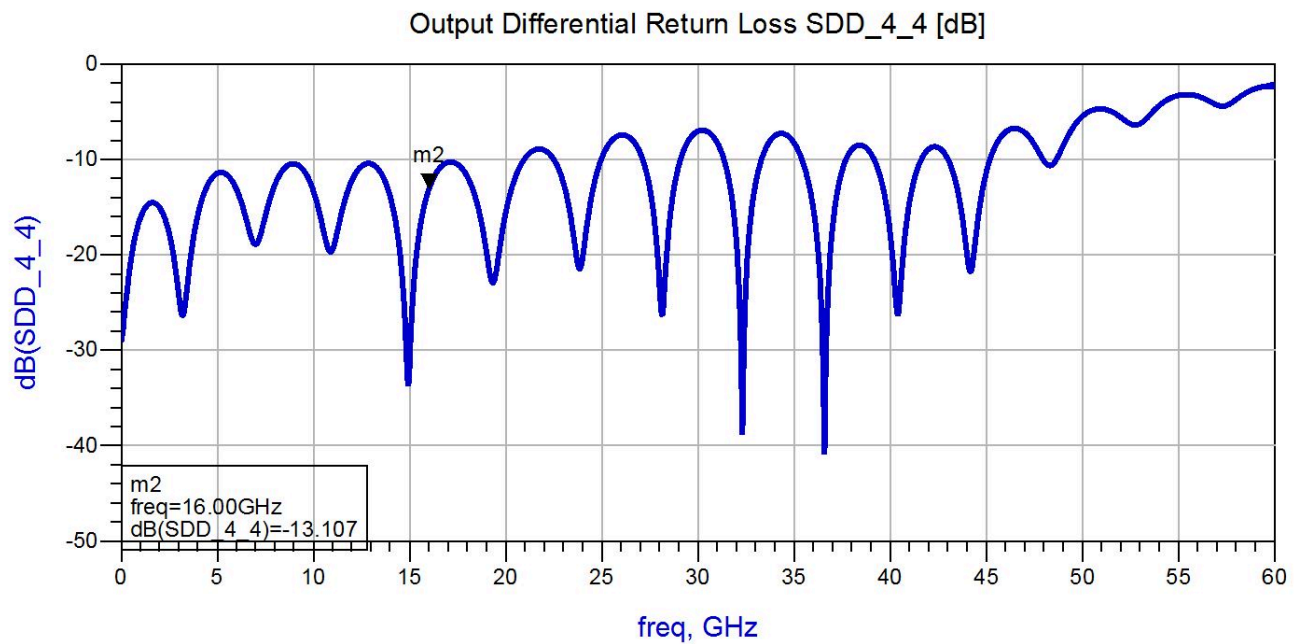


Figure 8-74 Return Loss for Non-Root Reference Package for 64.0 GT/s

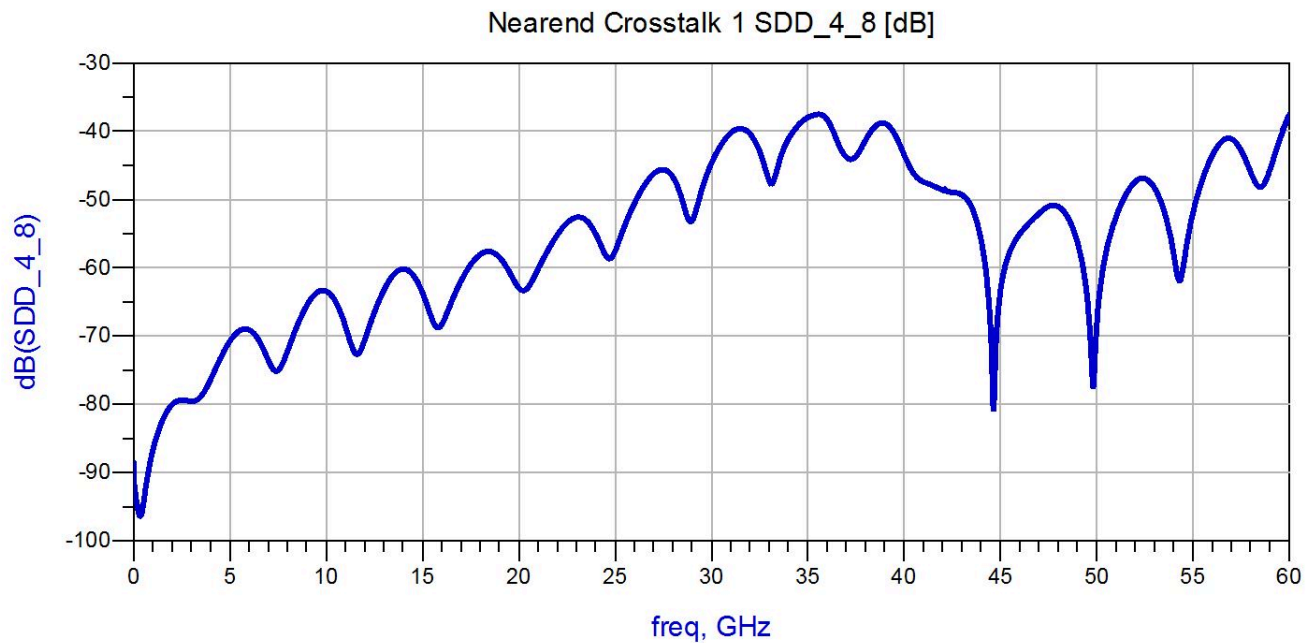


Figure 8-75 NEXT for Non-Root Reference Package (Worst Case) for 64.0 GT/s

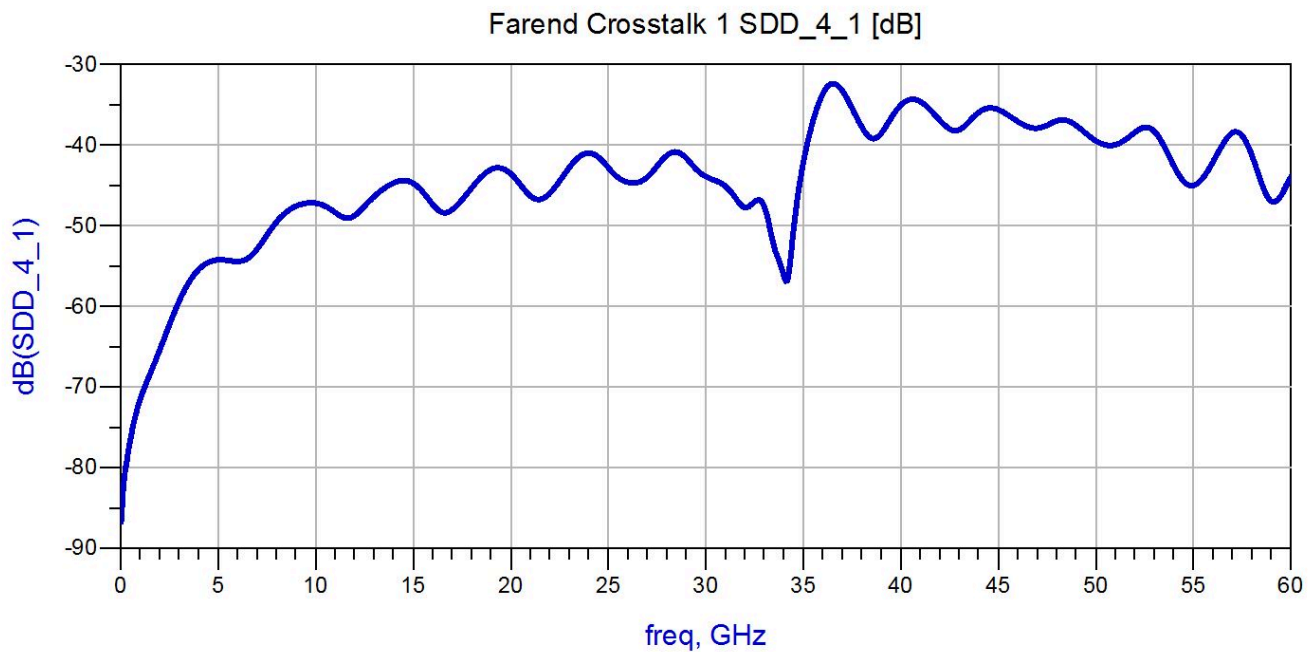
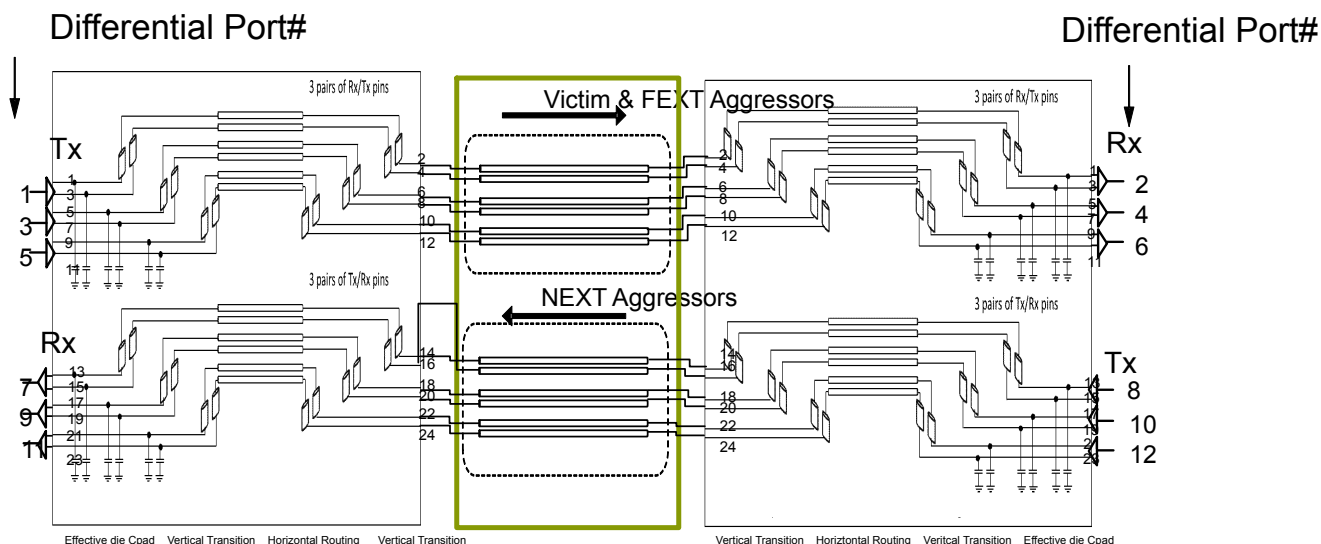


Figure 8-76 FEXT for Non-Root Reference Package (Worst Case) for 64.0 GT/s



Reference Package Channel (Pin to Pin) Reference Package

Figure 8-77 32.0 and 64.0 GT/s Reference Package Port Connections for Pin to Pin Channel Evaluation

§ Figure 8-77 shows the port connections for using the 32.0 and 64.0 GT/s reference packages to evaluate a pin to pin channel using the channel compliance methodology. Both directions (root transmitting and non-root transmitting) must be evaluated. The lane labeled channel 3 to channel 4 is intended as the worst-case victim channel and must be evaluated in both directions. Other channels in the reference package model are intended only for use as cross-talk aggressors (not victim channels).

8.5.1.2 Measuring Package Performance (16.0 GT/s only) §

Package insertion loss at 16.0 GT/s is an informative spec parameter. Some implementations at 16.0 GT/s (see § Section 8.3.3.11) are allowed to have packages that exceed reference packages in insertion loss and/or cross-talk. Actual package performance must be assessed by performing channel compliance with reference channels provided with the specification on the PCI-SIG website. A set of channel compliance simulations are run on the reference channels with one of the reference packages being replaced by the package that is being evaluated. If the eye height or eye width is smaller for any of the channels with the package that is being evaluated, then the package is considered to have worse performance than the reference package. An implementation with a package that has worse performance than the reference package must use the implementation package model in the channel compliance methodology and may optionally use the implementation package in the Receiver stressed eye calibration. Note that form factor and channel compliance (for a captive channel) overall requirements still need to be met regardless of package characteristics.

8.5.1.3 Simulation Tool Requirements §

Channel tolerancing is implemented by means of simulation, where the pass/fail criteria are defined in terms of a time domain eye diagram. The simulation tool must accept a prescribed set of inputs, including the channel under consideration and then simulate based upon a set of post processing requirements. This specification does not stipulate the use of any specific tool for simulating channels. However, any simulation tool must meet the following requirements.

8.5.1.3.1 Simulation Tool Chain Inputs §

- Channel characteristics defined as S-parameters or equivalent model: The model must include the victim differential Lane plus as many aggressors as required to accurately capture crosstalk. In most cases this will be between 2 and 4 additional differential Lanes. Note that 32.0 and 64.0 GT/s are most likely to require additional aggressors to accurately capture worst-case cross-talk and most likely to require several NEXT aggressors to capture crosstalk accurately.
- Behavioral Root and Non-Root package models: The models will be included as part of the specification in the form of s-parameter files (see § Section 8.5.1.1).
- Transmitter Jitter and voltage: The voltage and jitter parameters input to the simulator may be directly obtained from a combination of the Transmitter and Refclk jitter. Since these parameters are fixed the simulation tool may choose to hard code their values.
- Transmitter and Receiver Termination Impedance: The simulator shall use a $2 \times 50 \Omega$ termination for both the Transmitter and Receiver. This value matches the assumptions which are implicit in generating and measuring the stressed eye for Rx tolerancing.

8.5.1.3.2 Processing Steps §

- Time domain representation of the end-to-end connectivity: Included are the behavioral Tx and Rx packages and the channel under test.
- Tx voltage and jitter: Voltage and jitter parameters defined for the Transmitter but have been recalculated to properly comprehend high and low frequency jitter components, and also include Refclk jitter contributions.
- Behavioral Transmitter Equalization: The simulator shall replicate the Transmitter equalization capabilities defined in the Transmitter section.
- Behavioral Rx CTLE: The simulation tool shall implement a behavioral CTLE that replicates the CTLE function employed for Rx tolerancing.
- Behavioral DFE: The simulation tool shall implement a 1-tap (8.0 GT/s), a 2-tap (16.0 GT/s), a 3 tap DFE (32.0 GT/s), and a 16-tap DFE (64.0 GT/s), where the dynamic range for the feedback coefficient is defined in § Section 8.4.1.10 .
- Optimizing Tx equalization and Rx DFE/CTLE settings: The simulation tool shall implement an optimization algorithm that selects the combination of Tx equalization and Rx CTLE and DFE settings that yields a maximum value for the eye height (at the data sample point) multiplied by the eye width at the far end of the channel. For details refer to § Section 8.4.1.8 .
- Statistical Treatment of jitter: In order to avoid overestimating the effect of channel-data and channel-jitter interactions, the tool shall use a statistical analysis of these parameters to generate voltage/jitter eye margins.

8.5.1.3.3 Simulation Tool Outputs §

Output eye parameters: The simulator shall generate a statistically defined output that displays the eye width and eye height. EH will be measured as the peak eye height at the data sample location, while EW shall be measured at the zero-crossing line. Additionally the simulator shall have the capability to adjust the data sample point by ± 0.1 UI from the mean center of the UI for 8.0 and 16.0 GT/s as shown in § Figure 8-79. For 32.0 GT/s the simulator shall adjust the sample point up to 0.30 UI to the left of the mean center of the UI sample position in 0.05 UI increments, computing the DFE coefficients for each sample location and selecting the result producing the maximum value for the eye height (at the data sample point) multiplied by the eye width. For 64.0 GT/s the simulator shall adjust the sample point up to

0.30 UI to the left of the mean center of the UI sample position in 0.015 UI increments, computing the DFE coefficients for each sample location and selecting the result producing the maximum value for the eye height (at the data sample point) multiplied by the eye width.

8.5.1.3.4 Open Source Simulation Tool §

An open source simulation tool shall be provided with the specification as design collateral. The tool will provide a turnkey capability, where the user provides the channel characteristics at the Receiver's die pad as step responses, and the tool calculates a statistical eye showing pass/fail.

8.5.1.4 Behavioral Transmitter Parameters §

8.5.1.4.1 Deriving Voltage and Jitter Parameters §

This section is for informative purposes. The voltage and jitter parameters may be derived from the Transmitter voltage and jitter parameters but are referenced to the die pad. This is necessary to allow the channel simulation to include a behavioral Tx package and drive the package from the die pads. Additionally, the Tj terms must be decomposed into separate Rj and DjDD terms.

- $V_{TX-CH-FS-NO-EQ}$ and $V_{TX-CH-RS-NO-EQ}$: These two parameters define the minimum peak-peak voltage corresponding to Vd in § Figure 8-6.
- The jitter parameters are derived based on the following set of equations. Algebraic manipulation is used to extract the Rj implicitly defined by the combination of Tj and DjDD terms. The following numbers are based on 8.0 GT/s Tx jitter parameters based on values specified in § Table 8-6. The same approach is used to extract jitter parameters for 2.5, 5.0, 16.0, 32.0, and 64.0 GT/s where it is assumed that maximum data rate supported by the PCIe 6.0 device is 64.0 GT/s and the jitter values in § Table 8-6 are used in the extraction process.

$$\begin{aligned} \text{jit_hfrj_nui} &= (T_{TX-UTJ} - T_{TX-UDJ-DD})/14.06 = 1.11\text{ps} \\ T_{TX-CH-UPW-RJ} &= (T_{TX-UPWJ-TJ} - T_{TXUPWJ-DJDD})/14.06 = 1.00\text{ps} \\ T_{TX-CH-UPW-DJ} &= T_{TXUPWJ-DJDD} = 10.0\text{ps} \\ T_{TX-CH-URJ} &= \sqrt{(\text{jit_hfrj_nui})^2 - (T_{TX-CH-UPW-RJ} * 0.707)^2} + T_{REFCLK-RMS} = 1.31\text{ps} \\ T_{TX-CH-UDJDD} &= T_{TX-UDJ-DD} - (T_{TXUPWJ-DJDD})/2 = 7.00\text{ps} \end{aligned}$$

A-0840A

Figure 8-78 Example Derivation of 8.0 GT/s Jitter Parameters for § Table 8-14

A channel must be tested at all data rates, with the corresponding Tx jitter parameters, that it is intended to support during normal operation. For example, a channel intended to support a maximum data rate of 8.0 GT/s must be tested at 2.5, 5.0, and 8.0 GT/s.

Table 8-14 Jitter/Voltage Parameters for Channel Tolerancing

Symbol	Parameter	Value	Units	Notes
$V_{TX-CH-FS-NO-EQ}$	Full swing Tx voltage	804	mVPP	Full swing, No Tx Eq.
$V_{TX-CH-RS-NO-EQ}$	Reduced swing Tx voltage	402	mVPP	Reduced swing, No Tx Eq.

Symbol	Parameter	Value	Units	Notes
2.5 GT/s Jitter Parameters and Voltage Parameters				
$T_{TX-CH-URJ-2.5G}$	Tx uncorrelated Rj	3.45	ps RMS	See Note 1
$T_{TX-CH-UDJDD-2.5G}$	Tx uncorrelated DjDD	20	ps PP	
$T_{TX-CH-UPW-RJ-2.5G}$	Uncorrelated PW Rj	1.42	ps RMS	See Note 2
$T_{TX-CH-UPW-DJ-2.5G}$	PW DDj	80	ps PP	
$T_{TX-DIEPAD-EDGERATE-2.5G}$	Signal edge rate at behavioral Tx die pad	140	ps	Measured 10% to 90% using a gaussian lowpass filter to shape the edge. See Note 3.
5.0 GT/s Jitter Parameters and Voltage Parameters				
$T_{TX-CH-URJ-5G}$	Tx uncorrelated Rj	3.45	ps RMS	See Note 1
$T_{TX-CH-UDJDD-5G}$	Tx uncorrelated DjDD	20-	ps PP	
$T_{TX-CH-UPW-RJ-5G}$	Uncorrelated PW Rj	1.42	ps RMS	
$T_{TX-CH-UPW-DJ-5G}$	PW DDj	40	ps PP	See Note 2.
$T_{TX-DIEPAD-EDGERATE-5G}$	Signal edge rate at behavioral Tx die pad	70	ps	Measured 10% to 90% using a gaussian lowpass filter to shape the edge. See Note 3.
8.0 GT/s Jitter Parameters and Voltage Parameters				
$T_{TX-CH-URJ-8G}$	Tx uncorrelated Rj	1.31	ps RMS	No DDj of HF jitter. See Note 1.
$T_{TX-CH-UDJDD-8G}$	Tx uncorrelated DjDD	7.0	ps PP	No DDj of HF jitter
$T_{TX-CH-UPW-RJ-8G}$	Uncorrelated PW Rj	1.0	ps RMS	
$T_{TX-CH-UPW-DJ-8G}$	PW DDj	10	ps PP	See Note 2.
$T_{TX-DIEPAD-EDGERATE-8G}$	Signal edge rate at behavioral Tx die pad	43.75	ps	Measured 10% to 90% using a gaussian lowpass filter to shape the edge. See Note 3.
16.0 GT/s Jitter Parameters and Voltage Parameters				
$T_{TX-CH-URJ-16G}$	Tx uncorrelated Rj	0.71	ps RMS	See Note 1.
$T_{TX-CH-UDJDD-16G}$	Tx uncorrelated DjDD	3.75	ps PP	
$T_{TX-CH-UPW-RJ-16G}$	Uncorrelated PW Rj	0.54	ps RMS	
$T_{TX-CH-UPW-DJ-16G}$	PW DDj	5.0	ps PP	See Note 2.

Symbol	Parameter	Value	Units	Notes
<i>T_{TX-DIEPAD-EDGERATE-16G}</i>	Signal edge rate at behavioral Tx die pad	21.875	ps	Measured 10% to 90% using a gaussian lowpass filter to shape the edge. See Note 3.

32.0 GT/s Jitter Parameters and Voltage Parameters

<i>T_{TX-CH-URJ-32G}</i>	Tx uncorrelated Rj	0.276	ps RMS	See Note 1.
<i>T_{TX-CH-UDJDD-32G}</i>	Tx uncorrelated DjDD	1.875	ps PP	
<i>T_{TX-CH-UPW-RJ-32G}</i>	Uncorrelated PW Rj	0.27	ps RMS	
<i>T_{TX-CH-UPW-DJ-32G}</i>	PW DDj	2.5	ps PP	See Note 2.
<i>T_{TX-DIEPAD-EDGERATE-32G}</i>	Signal edge rate at behavioral Tx die pad	10.94	ps	Measured 10% to 90% using a gaussian lowpass filter to shape the edge. See Note 3.

64.0 GT/s Jitter Parameters and Voltage Parameters

<i>T_{TX-CH-URJ-64G}</i>	Tx uncorrelated Rj	0.215	ps RMS	See Note 1.
<i>T_{TX-CH-UDJDD-64G}</i>	Tx uncorrelated DjDD	0.938	ps PP	
<i>T_{TX-CH-UPW-RJ-64G}</i>	Uncorrelated PW Rj	0.289	TX-CH-UPW-RJ-32G	
<i>T_{TX-CH-UPW-DJ-64G}</i>	PW DDj	1.25	ps PP	See Note 2.
<i>T_{TX-DIEPAD-EDGERATE-64G}</i>	Signal edge rate at behavioral Tx die pad	10.94	ps	Measured 10% to 90% using a gaussian lowpass filter to shape the edge. See Note 3.

Notes:

1. Includes low frequency (non F/2) Rj components from the Transmitter and Rj from the Refclk.
2. Applied on a per edge basis as a dual Dirac model.
3. Does not include parasitic die pad capacitance. See § Figure 8-58 for details of behavioral package.

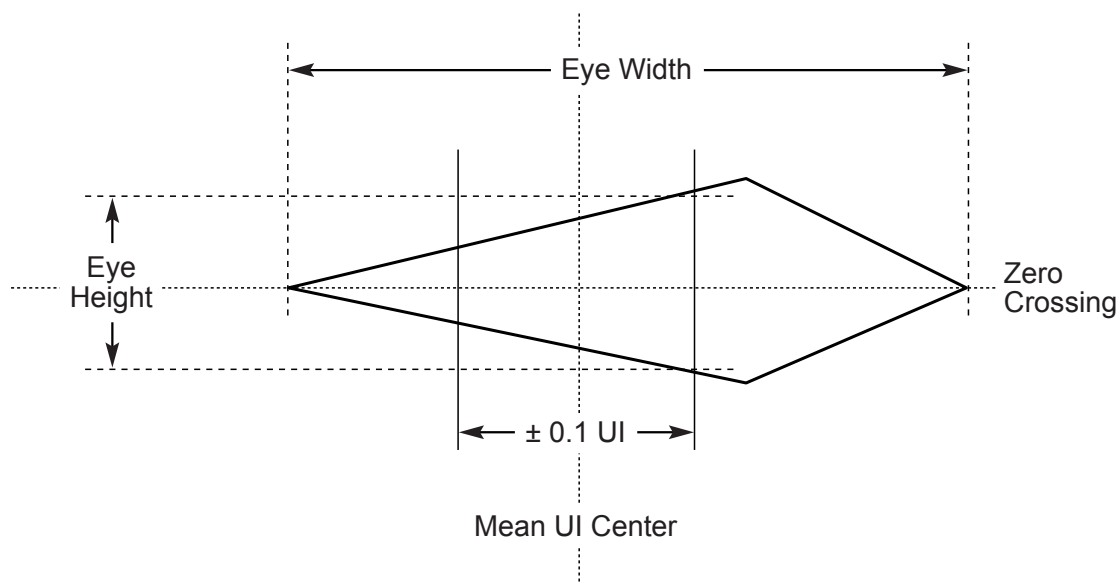
8.5.1.4.2 Optimizing Tx/Rx Equalization (8.0, 16.0, 32.0, and 64.0 GT/s only) §

The behavioral receiver selects the combination of Transmitter Equalization, CTLE, DFE and sample location (32.0 and 64.0 GT/s only) that produces the optimal eye area (eye width multiplied by eye height).

8.5.1.4.3 Pass/Fail Eye Characteristics §

The output of the simulation tool shall be in the form of pass/fail characteristics as defined by an eye mask as shown in § Figure 8-79. EH and EW must meet respectively the voltage and jitter parameters defined in § Table 8-15. Eye margins are defined at the die pad of the Receiver after the appropriate Tx and Rx equalization algorithms have been applied. In

the case where the channel is being designed for a specific pair of silicon devices and the package models of these silicon devices including cross-talk aggressors are known the actual device packages may be used instead of the reference packages in running the pad to pad channel pass/fail compliance simulations.



A-0841A

§

Figure 8-79 EH, EW Mask

Note that the pass/fail EH and EW limits shown in § Figure 8-79 are identical to the limits defined for Rx testing in § Table 8-10 for 8.0 and 16.0 GT/s. For 32.0 and 64.0 GT/s, the pass/fail EH and EW limits are still identical but they are computed at the optimal sample location. For 2.5 and 5.0 GT/s the limits for Rx testing are referenced to the package pins and the limits for channel tolerancing in this table are referenced to the Receiver pad after applying the same reference package used for 8.0 GT/s channel tolerancing.

Table 8-15 Channel Tolerancing Eye Mask Values

Symbol	Parameter	Value	Units	Comments
2.5 GT/s Eye Margins				
$V_{RX-CH-EH-2.5G}$	Eye height	<130 (min)	mVPP	Eye height at BER= 10^{-12} . Note 1
$T_{RX-CH-EW-2.5G}$	Eye width at zero-crossing	<0.35 (min)	UI	Eye width at BER= 10^{-12}
$T_{RX-DS-OFFSET-2.5G}$	Peak EH offset from UI center	± 0.1	UI	See § Figure 8-79 for details.
5.0 GT/s Eye Margins				
$V_{RX-CH-EH-5G}$	Eye height	< 85 (min)	mVPP	Eye height at BER= 10^{-12} . Note 1
$T_{RX-CH-EW-5G}$	Eye width at zero-crossing	<0.30 (min)	UI	Eye width at BER= 10^{-12}

Symbol	Parameter	Value	Units	Comments
<i>T_{RX-DS-OFFSET-5G}</i>	Peak EH offset from UI center	±0.1	UI	See § Figure 8-79 for details.
8.0 GT/s Eye Margins				
<i>V_{RX-CH-EH-8G}</i>	Eye height	25 (min)	mVPP	Eye height at BER=10 ⁻¹² . Note 1.
<i>T_{RX-CH-EW-8G}</i>	Eye width at zero-crossing	0.3 (min)	UI	Eye width at BER=10 ⁻¹²
<i>T_{RX-DS-OFFSET-8G}</i>	Peak EH offset from UI center	±0.1	UI	See § Figure 8-79 for details.
<i>V_{RX-DFE-D1-8G}</i>	Range for DFE d ₁ coefficient	±30	mV	
16.0 GT/s Eye Margins				
<i>V_{RX-CH-EH-16G}</i>	Eye height	15 (min)	mVPP	Eye height at BER=10 ⁻¹² . Note 1.
<i>T_{RX-CH-EW-16G}</i>	Eye width at zero-crossing	0.3 (min)	UI	Eye width at BER=10 ⁻¹²
<i>T_{RX-DS-OFFSET-16G}</i>	Peak EH offset from UI center	±0.1	UI	See § Figure 8-79 for details.
<i>V_{RX-DFE-D1-16G}</i>	Range for DFE d ₁ coefficient	±30	mV	
<i>V_{RX-DFE-D2-16G}</i>	Range for DFE d ₂ coefficient	±20	mV	
32.0 GT/s Eye Margins				
<i>V_{RX-CH-EH-32G}</i>	Eye height	15 (min)	mVPP	Eye height at BER=10 ⁻¹² . Note 1.
<i>T_{RX-CH-EW-32G}</i>	Eye width at zero-crossing	0.3 (min)	UI	Eye width at BER=10 ⁻¹²
<i>T_{RX-DS-OFFSET-32G}</i>	Peak EH offset from UI center	N/A	UI	See § Figure 8-79 for details.
<i>T_{RX-SAMPLE-OFFSET-32G}</i>	Max sample location offset to the left from UI center	0.30	UI	Note 2
<i>T_{RX-SAMPLE-GRANULARITY-32G}</i>	Granularity for sample location offset	0.05	UI	Note 2
<i>V_{RX-DFE-D1-32G}</i>	Range for DFE d ₁ coefficient	d ₁ / d ₀ (cursor amplitude) ≤ 0.8		
<i>V_{RX-DFE-D2-32G}</i>	Range for DFE d ₂ coefficient	±20	mV	
<i>V_{RX-DFE-D3-32G}</i>	Range for DFE d ₃ coefficient	±20	mV	

Symbol	Parameter	Value	Units	Comments
64.0 GT/s Eye Margins				
$V_{RX-CH-TOP-EH-64G}$	Top Eye height	6 (min)	mVPP	Eye height at BER=10 ⁻⁶ . Note 1.
$T_{RX-CH-TOP-EW-64G}$	Top Eye width at zero-crossing	0.1 (min)	UI	Eye width at BER=10 ⁻⁶
$T_{RX-DS-OFFSET-64G}$	Peak EH offset from UI center	N/A	UI	
$T_{RX-SAMPLE-OFFSET-64G}$	Max sample location offset to the left from UI center	0.30	UI	Note 2
$T_{RX-SAMPLE-GRANULARITY-64G}$	Granularity for sample location offset	0.015	UI	Note 2
$V_{RX-DFE-D1-64G}$	Range for DFE d1 coefficient	$ d_1 / d_0$ (cursor amplitude) < 0.55		
$V_{RX-DFE-TAPS-WEIGHTED-SUM-64G}$	Range for weighted sum of absolute values of DFE d1-d16 coefficients	$(d_1 + d_2 + 0.85 \times d_3 + 0.6 \times d_4 + 0.25 \times d_5 + 0.1 \times d_6 + 0.05 \times \{ d_7 + d_8 + d_9 + d_{10} + d_{11} + d_{12} + d_{13} + d_{14} + d_{15} + d_{16} \}) / d_0 < 0.85$	mV	

Notes:

1. **$V_{RX-CH-EH}$** is defined as max EH within an aperture of ± 0.1 UI from mean UI center.
2. The optimal eye area is computed at each offset from mean UI center -0.30 UI to mean UI center with the specified granularity.

8.5.1.4.4 Characterizing Channel Common Mode Noise §

A channel must meet the common mode requirements as they are defined in the receiver specification. In general, it is not possible to accurately simulate all the channel's common mode noise contributions due to the large number of mechanisms that can generate CM noise, including the Transmitter. Typically channel common mode noise is a budgeted parameter, and the limits defined below assume a budgeting process. The channel's CM limit is defined as the amount of CM noise that a channel can add and still meet the Rx CM limits assuming the worst-case Tx CM.

Note that the Tx and channel CM noise parameters cannot simply be added to obtain the Rx CM limit. This is due to the fact that a channel will attenuate some of high frequency Tx CM noise while propagating Tx LF CM noise through with little loss. The channel may also contribute both high and low frequency CM components of its own.

8.5.1.4.5 Verifying $V_{CH-IDLE-DET-DIFF-pp}$ §

$V_{CH-IDLE-DET-DIFF-pp}$ is defined to guarantee that, when a Transmitter issues an EIEOS sequence, the Receiver is guaranteed to detect it. Potentially larger Transmitter equalization boost ratios at 8.0, 16.0, 32.0, and 64.0 GT/s necessitate that this parameter be verified; this procedure was not necessary for 2.5 or 5.0 GT/s, where the max Transmitter equalization boost is smaller. Defining the launch and detect voltages at the Tx/Rx die pad permits $V_{CH-IDLE-DET-DIFF-pp}$ to be verified with the same channel and Tx/Rx package models used to determine eye margins. It is also acceptable to simulate from Tx pin to Rx pin (excluding the Tx and Rx behavioral package models), in which case the EIEOS and idle detect parameters defined in the Tx and Rx sections are applicable.

Long channels, where $V_{TX-EIEOS-FS}$ is applicable, are characterized by driving the channel under test with the EIEOS pattern and -11.0 dB de-emphasis and zero dB preshoot. For short channels, where $V_{TX-EIEOS-RS}$ is applicable, -4.5 dB of de-emphasis and zero dB of preshoot are applied.

Table 8-16 EIEOS Signaling Parameters

Parameter	Description	Value	Units	Comments
$V_{CH-IDLE-EXIT-pp}$	Idle detect voltage seen at the Rx die pad	172	mVPP	Assuming Rx RTERM of $2 \times 50 \Omega$
$V_{CH-EIEOS-FS-Vb}$	PP voltage during Vb interval at behavioral Tx die pad for full swing signaling	255	mVPP	Assuming Tx RS of $2 \times 50 \Omega$
$V_{CH-EIEOS-RS-Vb}$	PP voltage during Vb interval at behavioral Tx die pad for reduced swing signaling	237	mVPP	Assuming Tx RS of $2 \times 50 \Omega$

8.6 Refclk Specifications §

This version of the specification consolidates and streamlines the Refclk requirements. The 2.5 GT/s Refclk parameters are moved from the CEM spec to this spec so that all Refclk parameters for all data rates (2.5, 5.0, 8.0, 16.0, 32.0, and 64.0 GT/s) are now contained in this section.

8.6.1 Refclk Test Setup §

The test setup for the Refclk assumes that only the Refclk generator itself is present. Provision is made in the test setup to account for signal degradation that occurs between the pins of the Refclk generator and the Transmitter or Receiver in an actual system. The above described setup emulates the worst case signal degradation that is likely to occur at the pins of a PCI Express device. Note that the Refclk signal is tested into a load that represents the series (open) termination appearing at the Refclk input pins of a PCIe device for all requirements except 32.0 and 64.0 GT/s reference clock jitter. For 32.0 and 64.0 GT/s, the reference clock jitter is measured with an oscilloscope, and is tested with the reference clock terminated by 50 Ohm terminations without a channel.

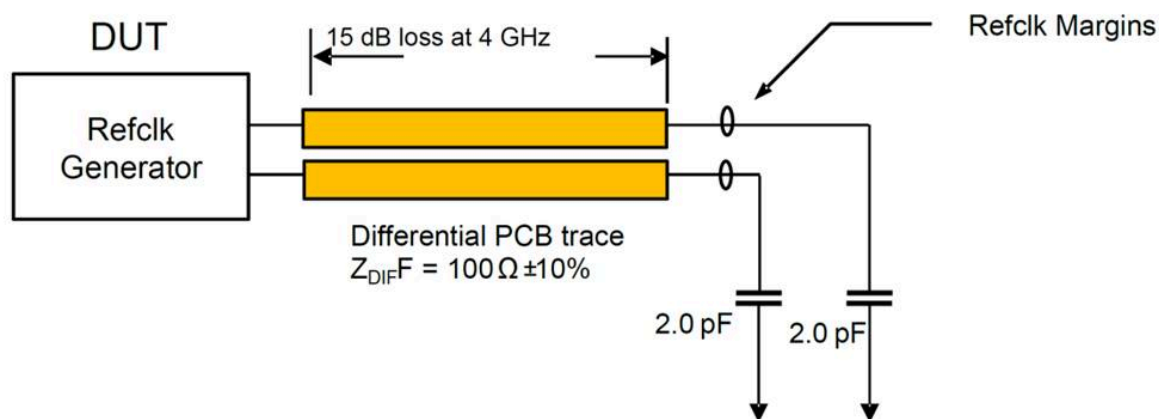


Figure 8-80 Oscilloscope Refclk Test Setup for All Cases Except Jitter at 32.0 and 64.0 GT/s

8.6.2 REFCLK AC Specifications §

All specifications in § Table 8-17 are to be measured using a test configuration as described in Note 11 with a circuit as shown in § Figure 8-80.

Table 8-17 REFCLK DC Specifications and AC Timing Requirements

Symbol	Parameter	100 MHz Input		Unit	Note
		Min	Max		
Rising Edge Rate	Rising Edge Rate	0.6	4.0	V/ns	2, 3
Falling Edge Rate	Falling Edge Rate	0.6	4.0	V/ns	2, 3
V_{IH}	Differential Input High Voltage	+150		mV	2
V_{IL}	Differential Input Low Voltage		-150	mV	2
V_{CROSS}	Absolute crossing point voltage	+250	+550	mV	1, 4, 5
V_{CROSS DELTA}	Variation of V _{CROSS} over all rising clock edges		+140	mV	1, 4, 9
V_{RB}	Ring-back Voltage Margin	-100	+100	mV	2, 12
T_{STABLE}	Time before V _{RB} is allowed	500		ps	2, 12
T_{PERIOD AVG}	Average Clock Period Accuracy	-300	+2800	ppm	2, 10, 13
T_{PERIOD AVG_32G_64G_CC}	Average Clock Period Accuracy for devices that support 32.0 and 64.0 GT/s in CC Mode at any speed	-100	+2600	ppm	2, 10, 13

Symbol	Parameter	100 MHz Input		Unit	Note
		Min	Max		
<i>T_{PERIOD}</i> <i>AVG_32G_64G_SRIS</i>	Average Clock Period Accuracy for devices that support 32.0 and 64.0 GT/s in SRIS Mode at any speed	-100	+1600	ppm	2, 10, 13
<i>T_{PERIOD}</i> <i>ABS</i>	Absolute Period (including Jitter and Spread Spectrum modulation)	9.847	10.203	ns	2, 6
<i>T_{PERIOD}</i> <i>ABS_32G_64G_CC</i>	Absolute Period (including Jitter and Spread Spectrum modulation) for devices that support 32.0 and 64.0 GT/s in CC Mode at any speed	9.849	10.201	ns	2, 6
<i>T_{PERIOD}</i> <i>ABS_32G_64G_SRIS</i>	Absolute Period (including Jitter and Spread Spectrum modulation) for devices that support 32.0 and 64.0 GT/s in SRIS Mode at any speed	9.849	10.181	ns	2, 6
<i>T_{CCJITTER}</i>	Cycle to Cycle jitter		150	ps	2
<i>V_{MAX}</i>	Absolute Max input voltage		+1.15	V	1, 7
<i>V_{MIN}</i>	Absolute Min input voltage		- 0.3	V	1, 8
<i>Duty Cycle</i>	Duty Cycle	40	60	%	2
<i>Rise-Fall Matching</i>	Rising edge rate (REFCLK+) to falling edge rate (REFCLK-) matching		20	%	1, 14
<i>Z_{C-DC}</i>	Clock source DC impedance	40	60	W	1, 11

Notes:

1. Measurement taken from single ended waveform.
2. Measurement taken from differential waveform.
3. Measured from -150 mV to +150 mV on the differential waveform (derived from REFCLK+ minus REFCLK-). The signal must be monotonic through the measurement region for rise and fall time. The 300 mV measurement window is centered on the differential zero-crossing. See § Figure 8-85.
4. Measured at crossing point where the instantaneous voltage value of the rising edge of REFCLK+ equals the falling edge of REFCLK-. See § Figure 8-81.
5. Refers to the total variation from the lowest crossing point to the highest, regardless of which edge is crossing. Refers to all crossing points for this measurement. See § Figure 8-81.
6. Defines as the absolute minimum or maximum instantaneous period. This includes cycle to cycle jitter, relative PPM tolerance, and spread spectrum modulation. See § Figure 8-84.
7. Defined as the maximum instantaneous voltage including overshoot. See § Figure 8-81.
8. Defined as the minimum instantaneous voltage including undershoot. See § Figure 8-81.
9. Defined as the total variation of all crossing voltages of Rising REFCLK+ and Falling REFCLK-. This is the maximum allowed variance in V_{CROSS} for any system. See § Figure 8-82.
10. Note deleted.
11. REFCLK+ and REFCLK- are to be measured at the load capacitors C_L. Single ended probes must be used for measurements requiring single ended measurements. Either single ended probes with math or differential probe can be used for differential measurements. Test load C_L = 2 pF.
12. T_{STABLE} is the time the differential clock must maintain a minimum ±150 mV differential voltage after rising/falling edges before it is allowed to droop back into the V_{RB} ±100 mV differential range. See § Figure 8-86.

Symbol	Parameter	100 MHz Input		Unit	Note
		Min	Max		

13. PPM refers to parts per million and is a DC absolute period accuracy specification. 1 PPM is $1/1,000,000^{\text{th}}$ of 100.000000 MHz exactly or 100 Hz. For example, for 300 PPM, then we have an error budget of 100 Hz/PPM $\times$ 300 PPM = 30 kHz. The period is to be measured with a frequency counter with measurement window set to 100 ms or greater.
14. Matching applies to rising edge rate for REFCLK+ and falling edge rate for REFCLK-. It is measured using a ± 75 mV window centered on the median cross point where REFCLK+ rising meets REFCLK- falling. The median cross point is used to calculate the voltage thresholds the oscilloscope is to use for the edge rate calculations. The Rise Edge Rate of REFCLK+ should be compared to the Fall Edge Rate of REFCLK-; the maximum allowed difference should not exceed 20% of the slowest edge rate. See § Figure 8-83.

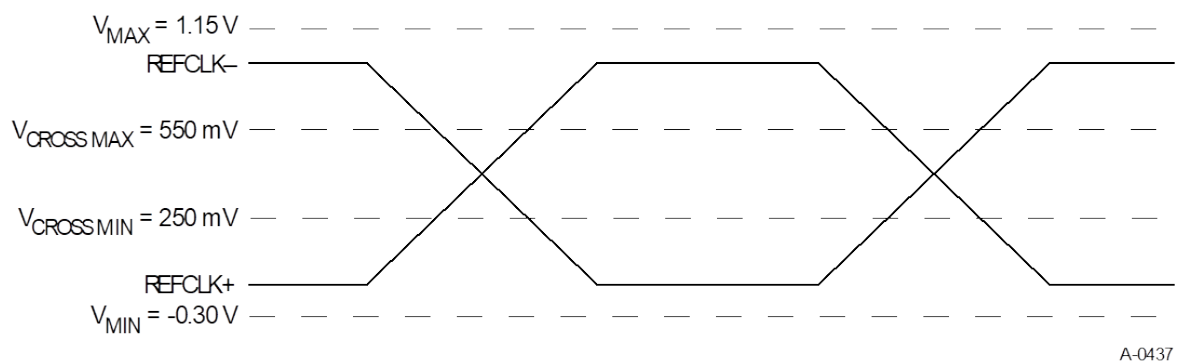


Figure 8-81 Single-Ended Measurement Points for Absolute Cross Point and Swing

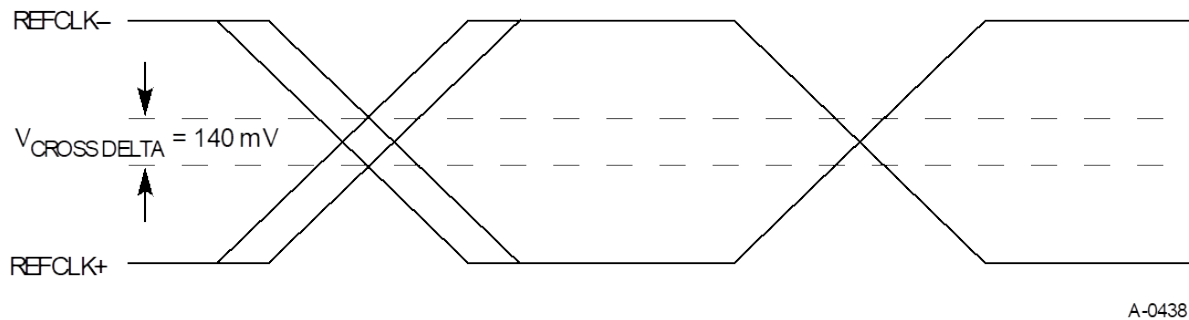


Figure 8-82 Single-Ended Measurement Points for Delta Cross Point

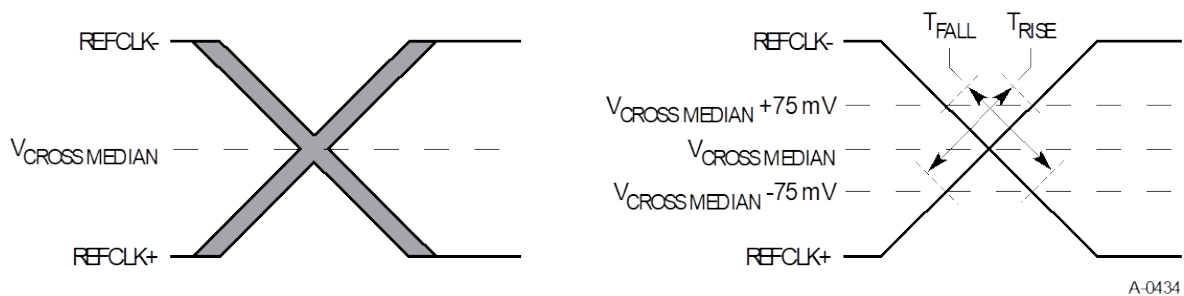


Figure 8-83 Single-Ended Measurement Points for Rise and Fall Time Matching

img alt="A-0435" data-orig="../Word/Chapter-8-0.7-Final_files/image076.png" data-src="../Unknown-Art-File" src="../Word/Chapter-8-0.7-Final_files/image076.png" width="600">

Figure 8-84 Differential Measurement Points for Duty Cycle and Period

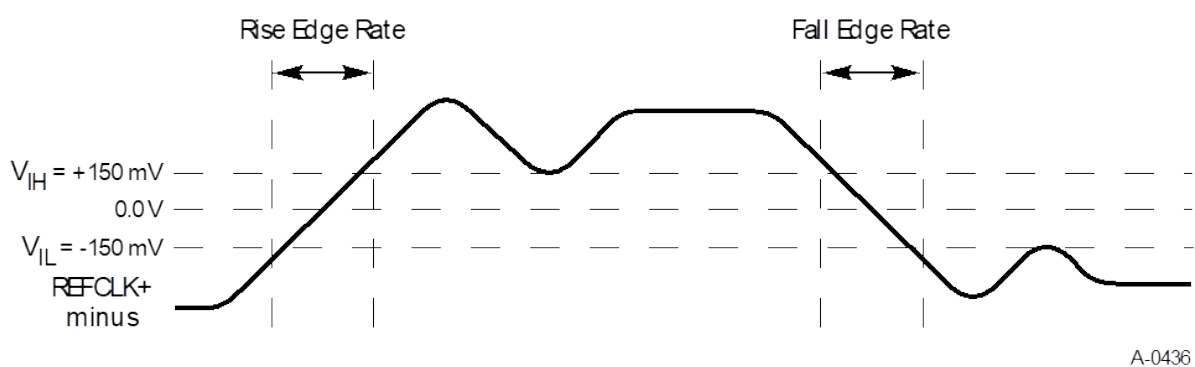


Figure 8-85 Differential Measurement Points for Rise and Fall Time

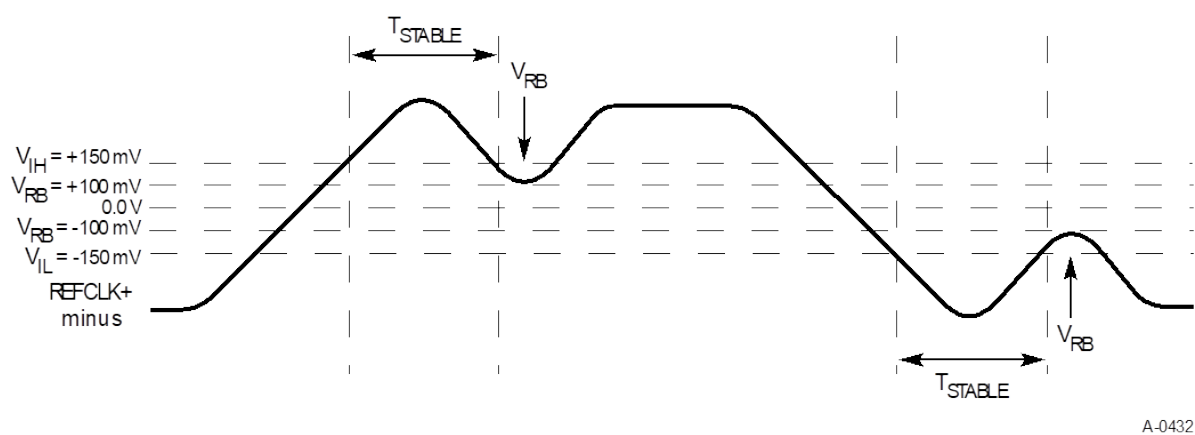


Figure 8-86 Differential Measurement Points for Ringback

8.6.3 Data Rate Independent Refclk Parameters §

A number of Refclk parameters are data rate independent and are listed in the table below. $T_{\text{TRANSPORT-DELAY}}$ is defined in § Section 8.6.6 and illustrated in § Figure 8-89. It is relevant only for the Common Refclk architecture. For the SRIS mode the source of the SSC modulation is implementation dependent.

Table 8-18 Data Rate Independent Refclk Parameters

Symbol	Description	Limits	Units	Notes
F_{REFCLK}	Refclk Frequency	99.97 (min) 100.03 (max)	MHz	
$F_{\text{REFCLK_32G_64G}}$	Refclk Frequency for devices that support 32.0 and 64.0 GT/s	99.99 (min) 100.01 (max)	MHz	
F_{SSC}	SSC frequency range	30 (min) 33 (max)	kHz	3
$T_{\text{SSC-FREQ-DEVIATION}}$	SSC deviation	-0.5 (min) 0.0 (max)	%	3
$T_{\text{SSC-FREQ-DEVIATION_32G_64G_SRIS}}$	SSC deviation for devices that support 32.0 and 64.0 GT/s and SRIS when operating in SRIS mode at all speeds	-0.3 (min) 0.0 (max)	%	3
$T_{\text{TRANSPORT-DELAY}}$	Tx-Rx transport delay	12 (max)	ns	1, 4
$T_{\text{SSC-MAX-FREQ-SLEW}}$	Max SSC df/dt	1250	ppm/ μ s	2, 3

Notes:

1. Parameter is relevant only for Common Refclk architecture.
2. Measurement is made over 0.5 μ s time interval with a 1st order LPF with an f_c of 60x the modulation frequency.
3. When testing the a device configured for the IR reference clock architecture the SSC related parameters must be tested with the Tx output data instead of the reference clock.
4. There are form factors (for example topologies including long cables) that may exceed the transport delay limit. Extra jitter from the large transport delay must be accounted by these form factor specifications.

8.6.3.1 Low Frequency Refclk Jitter Limits §

Refclks supporting SSC must meet an additional jitter limit over a range of low frequencies. Low frequency Refclk jitter limits are defined as a continuous, piece-wise linear graph from 30 kHz to 500 kHz as shown below. Unfiltered Refclk phase jitter must fall below this graph over the frequency range of interest.

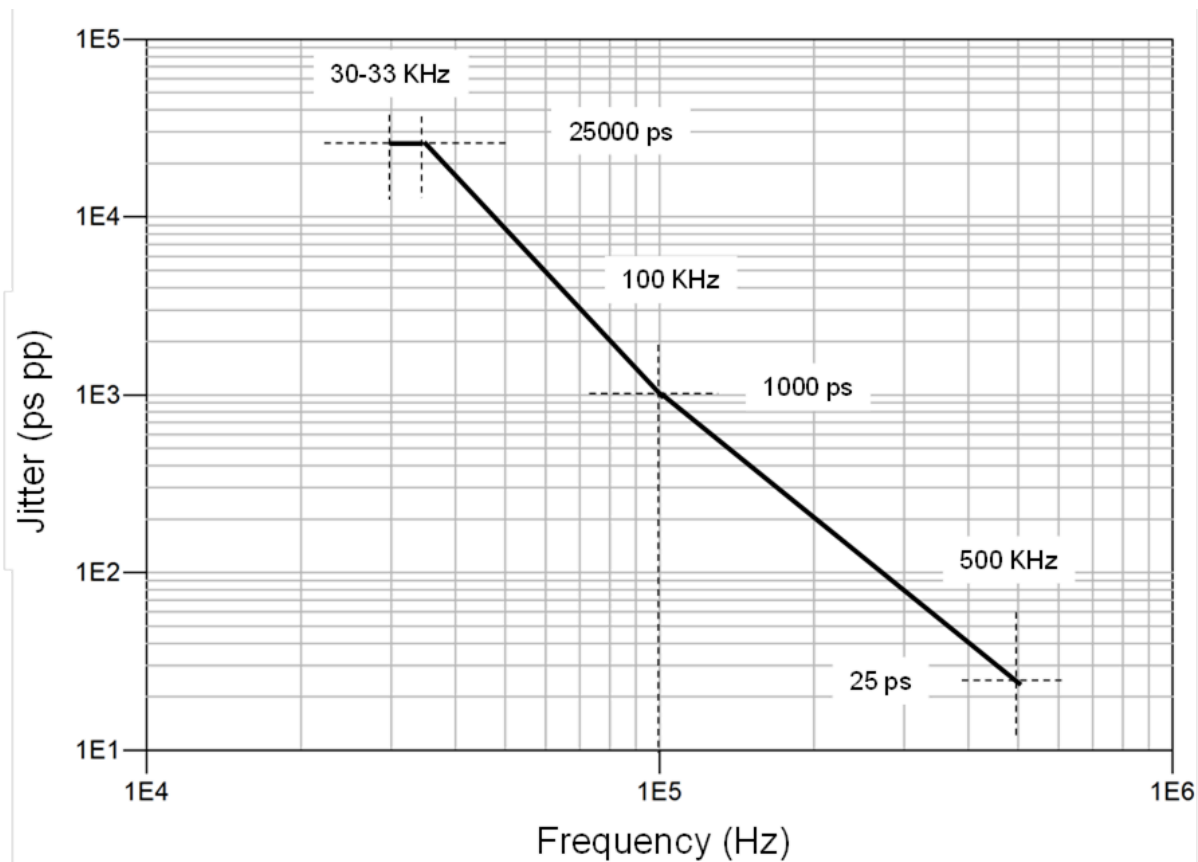


Figure 8-87 Limits for phase jitter from the Reference with 5000 ppm SSC

8.6.4 Refclk Architectures Supported §

Two Refclk architectures are supported: **Common Refclk (CC)** and **Independent Refclk (IR)**. The CC clock architecture is described in terms of its topology and its corresponding jitter transfer function based on PLL and CDR characteristics. Finally, the corresponding Refclk jitter limits are given for each data rate. The jitter transfer function and corresponding jitter limits are not defined for the IR clock architecture. It is up to the implementer to trade off reference clock jitter and PLL characteristics to ensure that Transmitter requirements are met in the IR clocking mode.

8.6.5 Filtering Functions Applied to Raw Data §

Two types of filtering are applied to the raw Refclk data. The first, edge filtering, minimizes the measurement-induced jitter due to the finite sampling rate of the test equipment. The second, replicates the jitter filtering that is inherent in the combination of Tx/Rx PLLs, the Rx CDR and (where applicable) the transport delay. The combination of the preceding filter functions yields the effective Refclk jitter as it appears at the sample latch of the Receiver.

Note that the PLL and CDR filter functions represent minimally capable approximations to actual Receiver implementations and are not intended to define actual PLL or CDR implementations.

8.6.5.1 PLL Filter Transfer Function Example §

All PLLs are behaviorally modeled with a second order transfer function, $H(s)$, as defined in § Figure 8-89. The parameters defining the transfer function include the damping factor ζ and the natural frequency ω_n .

The relation between the 2nd order PLL (lowpass) natural frequency, ω_n and the 3 dB point ω_{3dB} , is given by the following expression:

$$\frac{\omega_{3dB}}{\omega_n} = \sqrt{\sqrt{(2\zeta^2 + 1)^2 + 1} + 2\zeta^2 + 1}$$

Equation 8-16 Relationship between 2nd order PLL natural frequency and 3 dB point

The following plot of a 2nd order PLL illustrates the transfer function with an f_{3dB} of 5.0 MHz and 1.0 dB of peaking. This corresponds to $\zeta = 1.15$, and $\omega_n = 11.55$ Mrad/sec.

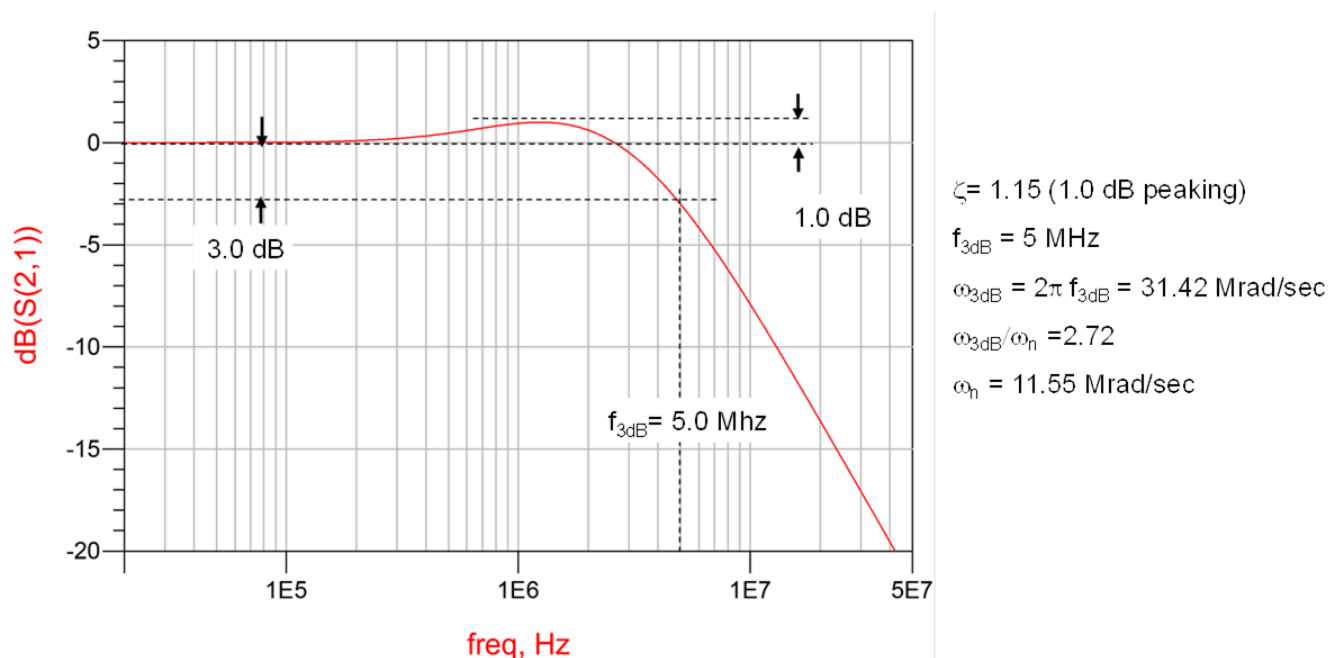


Figure 8-88 5 MHz PLL Transfer Function Example

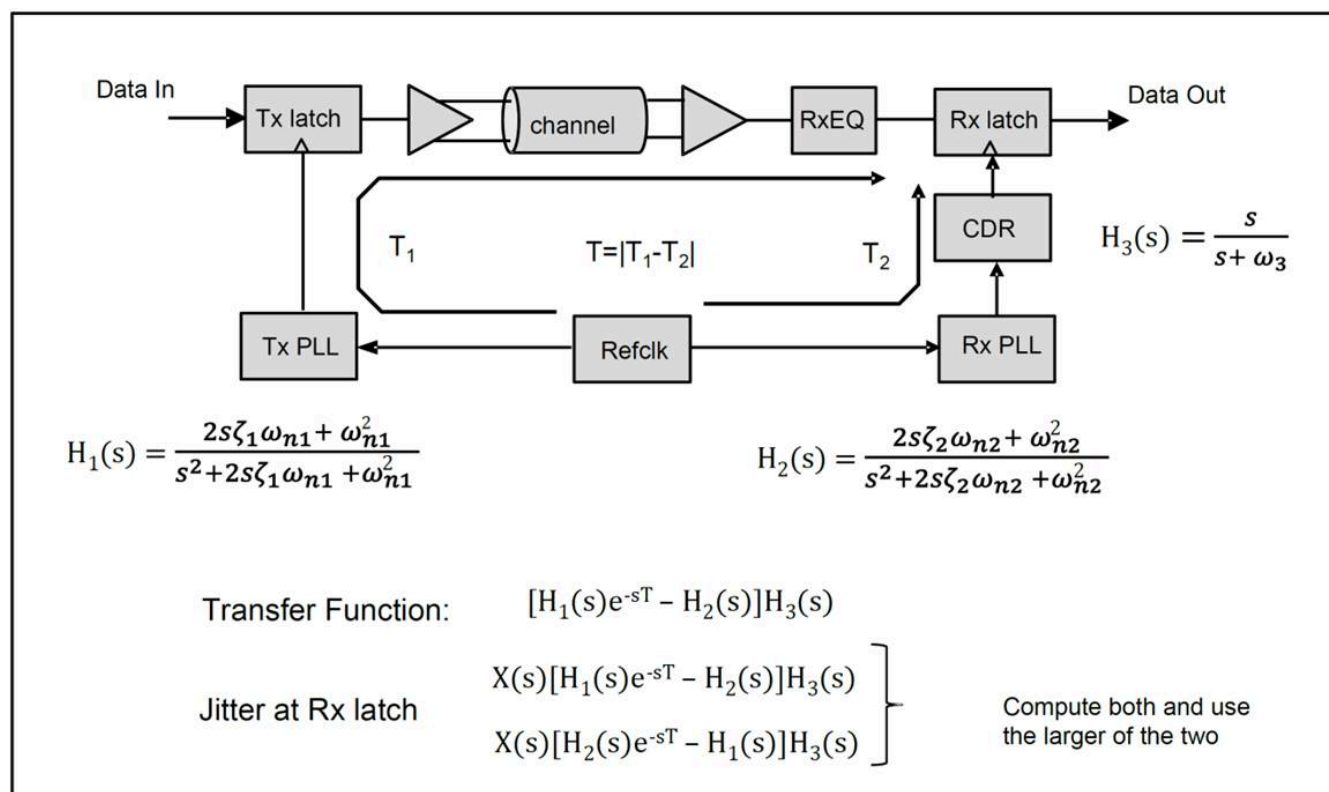
8.6.5.2 CDR Transfer Function Examples §

Depending on the Refclk architecture and data rate, either a first or higher order transfer function shall be used as a behavioral CDR bounding limit.

For behavioral CDR functions refer to § Section 8.3.5.5

8.6.6 Common Refclk Rx Architecture (CC) §

This architecture utilizes a single Refclk source that is distributed to both the Tx and Rx. Most of the SSC jitter sourced by the Refclk is propagated equally through Tx and Rx PLLs, and so intrinsically tracks LF jitter. This is particularly true for SSC which tends to be low frequency. § Figure 8-89 illustrates the Common Refclk Rx architecture, showing key jitter, delay, and PLL and CDR transfer function sources for all data rates except 32.0 and 64.0 GT/s. At 32.0 and 64.0 GT/s the only difference in the figure is Behavioral CDR transfer function as defined in § Section 8.3.5.5. The amount of jitter appearing at the CDR is then defined by the difference function between the Tx and Rx PLLs multiplied by the CDR highpass characteristic.



Based on the above clock architecture, it is possible to define a difference function that corresponds to the worst case mismatch between Tx and Rx PLLs. Second order PLL transfer functions are assumed, (even though most PLL transfer functions are 3rd order or higher), since a 2nd order function tends to yield a slightly conservative difference function vis-a-vis most actual PLL implementations.

In the Common Refclk Rx architecture it is also necessary to comprehend a maximum Transmitter to Receiver transport delay difference. This delay delta is illustrated in § Figure 8-89 and represents the delay difference between the Transmitter data and recovered Receiver clock as seen at the inputs to the receiver's data latch.

8.6.6.1 Determining the Number of PLL BW and peaking Combinations §

A Tx or Rx PLL is defined by the combination of min/max bandwidth and peaking, making for a total of four possible limits. In the CC architecture both the Tx and Rx PLLs contribute to the jitter transfer function. At 2.5 GT/s only one set of BW/peaking limits is defined. If the combinations for the Tx and Rx PLLs limits are defined by the sets (A_{TX}, B_{TX}, C_{TX}, D_{TX}), (A_{RX}, B_{RX}, C_{RX}, D_{RX}) then it's easily demonstrated that there are a total of ten ways of selecting one element from each set. The delay term e^{-sT} can be applied to either H₁(s) or H₂(s), adding another six possibilities. Only six, as opposed to ten, terms are added when the delay term is considered because terms like A_{TX}, A_{RX}, are identical to A_{RX}, A_{TX}.

At 5.0 GT/s and higher data rates, two possible sets of limits of PLL bandwidth and peaking are defined, which may be defined by the sets (A_{TX}, B_{TX}, C_{TX}, D_{TX}) and (E_{RX}, F_{RX}, G_{RX}, H_{RX}). In this case the number of unique 2-element combinations from the above 4-element sets is 36, which increases to 64 when the delay term is considered.

8.6.6.2 CDR and PLL BW and Peaking Limits for Common Refclk §

The Common Refclk architecture filter function is dependent on the difference function defined by the BW and peaking of Tx and the Rx PLLs, plus the CDR high-pass characteristic. It is necessary to consider all corner case combinations of Tx and Rx PLL peaking and bandwidth plus the CDR characteristics at their minimum and maximum peaking values.

This procedure must be applied to all six data rates.

§ Figure 8-90 lists the PLL BW and CDR BW/peaking values that need to be applied as filter functions for 2.5 GT/s data rates. It is necessary to assign a min and a max value for peaking for the 2.5 GT/s PLL. The values of 0.01 dB and 3.0 dB represent best estimates of realistic PLL implementations.

The minimum peaking for 2.5 and 5.0 GT/s has been reduced to 0.01 dB to bring it into line with the 8.0 and 16.0 GT/s cases. Note that the Rx CDR is 1st order, so its natural frequency, ω_n can be directly obtained from its BW, unlike the 2nd order CDRs, where ω_n is a function of both BW and peaking. For 32.0 GT/s, a 2nd-order CDR filter with 20 MHz BW described by § Equation 8-13 must be used.

PLL #1, PLL #2	0.01 dB peaking	3.0 dB peaking	BW_{CDR}(min) = 1.5 MHz, 1st order CDR
BW _{PLL} (min) = 1.5 MHz	$\omega_{n1} = .336 \text{ Mrad/s}$ $\zeta_1 = 14$	$\omega_{n1} = 5.09 \text{ Mrad/s}$ $\zeta_1 = 0.54$	
BW _{PLL} (max) = 22 MHz	$\omega_{n1} = 4.93 \text{ Mrad/s}$ $\zeta_1 = 14$	$\omega_{n1} = 74.68 \text{ Mrad/s}$ $\zeta_1 = 0.54$	
16 combinations			2.5 GT/s

Figure 8-90 Common Refclk PLL and CDR Characteristics for 2.5 GT/s

PLL and CDR jitter and peaking characteristics for 5.0, 8.0, and 16.0 GT/s yield a larger number of possible combinations because two sets of PLL BW and peaking limits are given. This choice to support two sets of BW and peaking was made to give designers as much latitude as possible when designing PLL circuits.

PLL #1	0.01 dB peaking	1.0 dB peaking	PLL #2	0.01 dB peaking	3.0 dB peaking
$BW_{PLL}(\min) = 5.0$ MHz	$\omega_{n1} = 1.12$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 11.01$ Mrad/s $\zeta_1 = 1.16$	$BW_{PLL}(\min) = 8.0$ MHz	$\omega_{n2} = 1.79$ Mrad/s $\zeta_2 = 14$	$\omega_{n2} = 26.86$ Mrad/s $\zeta_2 = 0.54$
$BW_{PLL}(\max) = 16$ MHz	$\omega_{n1} = 3.58$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 35.26$ Mrad/s $\zeta_1 = 1.16$	$BW_{PLL}(\max) = 16$ MHz	$\omega_{n2} = 3.58$ Mrad/s $\zeta_2 = 14$	$\omega_{n2} = 53.73$ Mrad/s $\zeta_2 = 0.54$
$BW_{CDR}(\min) = 5$ MHz, 1 st order			64 combinations		
			5 GT/s		

Figure 8-91 Common Refclk PLL and CDR Characteristics for 5.0 GT/s

PLL #1	0.01 dB peaking	2.0 dB peaking	PLL #2	0.01 dB peaking	1.0 dB peaking
$BW_{PLL}(\min) = 2.0$ MHz	$\omega_{n1} = 0.448$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 6.02$ Mrad/s $\zeta_1 = 0.73$	$BW_{PLL}(\min) = 2.0$ MHz	$\omega_{n2} = 0.448$ Mrad/s $\zeta_2 = 14$	$\omega_{n2} = 4.62$ Mrad/s $\zeta_2 = 1.15$
$BW_{PLL}(\max) = 4.0$ MHz	$\omega_{n1} = 0.896$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 12.04$ Mrad/s $\zeta_1 = 0.73$	$BW_{PLL}(\max) = 5.0$ MHz	$\omega_{n2} = 1.12$ Mrad/s $\zeta_2 = 14$	$\omega_{n2} = 11.53$ Mrad/s $\zeta_2 = 1.15$
$BW_{CDR}(\min) = 10$ MHz, 1 st order			64 combinations		
			8.0, 16.0 GT/s		

Figure 8-92 Common Refclk PLL and CDR Characteristics for 8.0 and 16.0 GT/s

PLL #1, PLL #2	0.01 dB peaking	2.0 dB peaking	32.0 GT/s CC	CDR
$BW_{PLL}(\min) = 0.5$ MHz	$\omega_{n1} = 0.112$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 1.50$ Mrad/s $\zeta_1 = 0.73$		
$BW_{PLL}(\max) = 1.8$ MHz	$\omega_{n1} = 0.403$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 5.42$ Mrad/s $\zeta_1 = 0.73$		
			16 combinations	
			32.0 GT/s	

Figure 8-93 Common Refclk PLL and CDR Characteristics for 32.0 GT/s

PLL #1, PLL #2	0.01 dB peaking	2.0 dB peaking	64.0 GT/s CC	CDR
$BW_{PLL}(\min) = 0.5$ MHz	$\omega_{n1} = 0.112$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 1.50$ Mrad/s $\zeta_1 = 0.73$		
$BW_{PLL}(\max) = 1.0$ MHz	$\omega_{n1} = 0.224$ Mrad/s $\zeta_1 = 14$	$\omega_{n1} = 3.00$ Mrad/s $\zeta_1 = 0.73$		
			16 combinations	
			64.0 GT/s	

Figure 8-94 Common Refclk PLL and CDR Characteristics for 64.0 GT/s

8.6.7 Jitter Limits for Refclk Architectures §

§ Table 8-19 lists the jitter limits for the CC Refclk architecture at each of the four data rates.

Jitter at 2.5 GT/s is measured as a peak to peak jitter value, because a substantial proportion of the jitter is SSC harmonics which appears at the receiver as Dj. The combination of the 2.5 GT/s PLL and CDR bandwidths passes a significant amount of SSC residual, where it appears Dj.

For 5.0, 8.0, and 16.0 GT/s jitter is specified as an RMS (Rj) value. These signaling speeds utilize a lower PLL BW and a higher CDR BW, and the effect is to suppress SSC harmonics such that almost all the jitter appears as Rj.

Table 8-19 Jitter Limits for CC Architecture

Data Rate	CC jitter Limit	Notes
2.5 GT/s	86 ps pp	1, 2
5.0 GT/s	3.1 ps RMS	1, 2
8.0 GT/s	1.0 ps RMS	1, 2
16.0 GT/s	0.5 ps RMS	1, 2, 3, 4
32.0 GT/s	0.15 ps RMS	1, 2, 3, 5
64.0 GT/s	0.1 ps RMS	1, 2, 3, 6

Notes:

1. The Refclk jitter is measured after applying the filter function in § Figure 8-89
2. Jitter measurements shall be made with a capture of at least 100,000 clock cycles captured by a real time oscilloscope (RTO) with a sample rate of 20 GS/s or greater. Broadband oscilloscope noise must be minimized in the measurement. The measured PP jitter is used (no extrapolation) for RTO measurements. Alternately - Jitter measurements may be used with a Phase Noise Analyzer (PNA) extending (flat) and integrating and folding the frequency content up to an offset from the carrier frequency of at least 200 MHz (at 300 MHz absolute frequency) below the Nyquist frequency. For PNA measurements for the 2.5 GT/s data rate the RMS jitter is converted to peak to peak jitter using a multiplication factor of 8.83. In the case where real time oscilloscope and PNA measurements have both been done and produce different results the RTO result must be used.
3. For the 16.0, 32.0, and 64.0 GT/s CC measurements SSC spurs from the fundamental and harmonics are removed up to a cutoff frequency of 2 MHz taking care to minimize removal of any non-SSC content.
4. Note that 0.7 ps RMS is to be used in channel simulations to account for additional noise in a real system.
5. Note that 0.25 ps RMS is to be used in channel simulations to account for additional noise in a real system.
6. Note that 0.15 ps RMS is to be used in channel simulations to account for additional noise in a real system.

8.6.8 Form Factor Requirements for RefClock Architectures §

Each form factor specification must include the following table (see § Table 8-20) to provide a clear summary of the clocking architecture requirements for devices that support the form factor specification. For each clocking architecture the table indicates whether that architecture is required, optional, or not allowed for this form factor. Note that this refers to the operation of the device not the underlying silicon capabilities.

A form factor must provide the CLKREQ# signal if it supports L1 PM Substates. Form factor specifications must indicate if the CLKREQ# signal is required, optional, or not allowed.

Table 8-20 Form Factor Clocking Architecture Requirements

Clock Architecture	System Board (Motherboard)	Add-in Card (Module)	Retimer
Common	**	**	**

Clock Architecture	System Board (Motherboard)	Add-in Card (Module)	Retimer
SRNS	**	**	**
SRIS	**	**	**

** Each entry in the table must be filled in with one of: Required, Optional, or Not Allowed

If the Common Reference Clock architecture is required or optional for the form factor, then there must be an additional table (see § Table 8-21) providing details for the common clock. Each entry in the table is marked required, optional, not allowed, or NA. “Clock Source” indicates the source of the common reference clock, if applicable. “SSC” indicates whether the clock source is spread.

Table 8-21 Form Factor Common Clock Architecture Details

Common Clock Details	System Board (Motherboard)	Add-in Card (Module)	Retimer
Clock Source			
SSC			

If a form factor has clocking requirements that cannot be provided in this simple one or two table form then careful consideration must be given to ensure that the form factor requirements are supported by this specification.

As an example the populated tables are shown for a hypothetical form factor that requires all components use the common clock architecture and does not allow the use of any other clocking architecture (see § Table 8-22 and § Table 8-23). The common clock source is required to be provided by the motherboard component and may optionally have SSC. L1 PM Substates are not supported and therefore CLKREQ# is not defined as a connector signal for this example form factor.

Table 8-22 Form Factor Clocking Architecture Requirements Example

Clock Architecture	System Board (Motherboard)	Add-in Card (Module)	Retimer
Common	Required	Required	Required
SRNS	Not Allowed	Not Allowed	Not Allowed
SRIS	Not Allowed	Not Allowed	Not Allowed

Table 8-23 Form Factor Common Clock Architecture Details Example

Common Clock Details	System Board (Motherboard)	Add-in Card (Module)	Retimer
Clock Source	Required	Not Allowed	Not Allowed
SSC	Optional	N/A	N/A

It is important for form factor specifications to recognize that the CLKREQ# signal is required if L1 PM Substates are to be supported, and that for L1 PM Substates the CLKREQ# signal is used even if there is no common reference clock.

If a form factor has clocking requirements that cannot be provided in this simple one or two table form then careful consideration must be given to ensure that the form factor requirements are supported by this specification.

9. Single Root I/O Virtualization and Sharing §

9.1 SR-IOV Architectural Overview §

Within the industry, significant effort has been expended to increase the effective hardware resource utilization (i.e., application execution) through the use of virtualization technology. Single Root I/O Virtualization and Sharing (SR-IOV) enables multiple System Images (SI) to share PCI hardware resources.

To illustrate how this technology can be used to increase effective resource utilization, consider the generic platform configuration illustrated in § Figure 9-1.

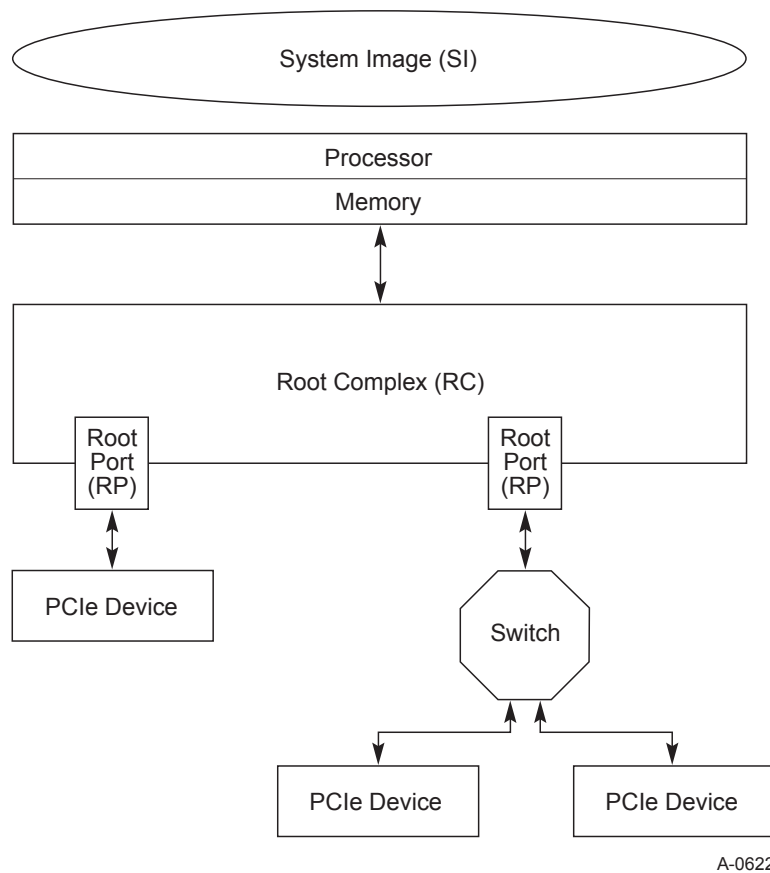


Figure 9-1 Generic Platform Configuration

The generic platform configuration is composed of the following components:

- PCIe Root Complex (RC), which includes:
 - Processor - general purpose, embedded, or specialized processing element
 - Memory - general purpose or embedded

- Root Complex Integrated Endpoints (RCiEPs)
- PCIe Root Ports (RP) - Each RP represents a separate hierarchy.
- PCIe Switch - provides I/O fan-out and connectivity
 - PCIe Device - multiple I/O device types, (e.g., network, storage, etc.)
 - System Image - software such an operating system that is used to execute applications or trusted services, (e.g., a shared or non-shared I/O device driver.)

In order to increase the effective hardware resource utilization without requiring hardware modifications, multiple SI can be executed. Software termed a Virtualization Intermediary (VI) is interposed between the hardware and the SI as illustrated in § Figure 9-2.

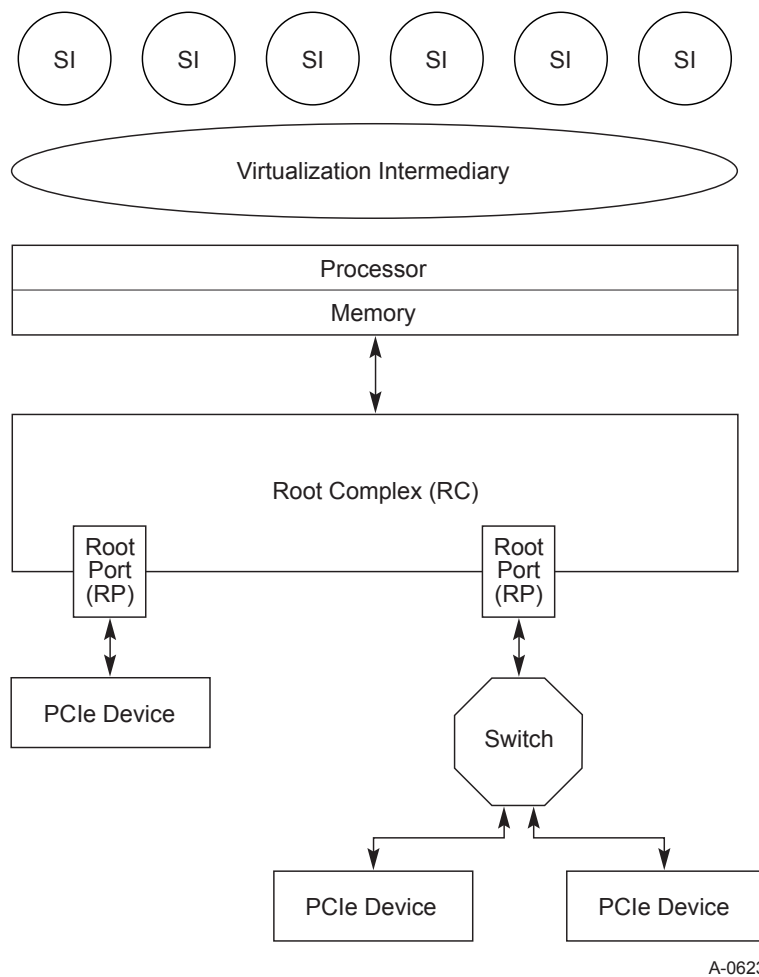


Figure 9-2 Generic Platform Configuration with a VI and Multiple SI

The VI takes sole ownership of the underlying hardware. Using a variety of methods outside of the scope of this specification, the VI abstracts the hardware to present each SI with its own virtual system. The actual hardware resources available to each SI can vary based on workload or customer-specific policies. While this approach works well for many environments, I/O intensive workloads can suffer significant performance degradation. Each I/O operation - inbound or outbound - must be intercepted and processed by the VI adding significant platform resource overhead.

SR-IOV provides tools to reduce these platform resources overheads. The benefits of SR-IOV are:

- The ability to eliminate VI involvement in main data movement actions - DMA, Memory space access, interrupt processing, etc. Elimination of VI interception and processing of each I/O operation can provide significant application and platform performance improvements.
- Standardized method to control SR-IOV resource configuration and management through Single Root PCI Manager (SR-PCIM).
 - Due to a variety of implementation options - system firmware, VI, operating system, I/O drivers, etc. - SR-PCIM implementation is outside the scope of this specification.
- The ability to reduce the hardware requirements and associated cost with provisioning potentially a significant number of I/O Functions within a device.
- The ability to integrate SR-IOV with other I/O virtualization technologies such as Address Translation Services (ATS), **Address Translation and Protection Table (ATPT)** technologies, and interrupt remapping technologies to create robust, complete I/O virtualization solutions.

§ Figure 9-3 illustrates an example SR-IOV capable platform.

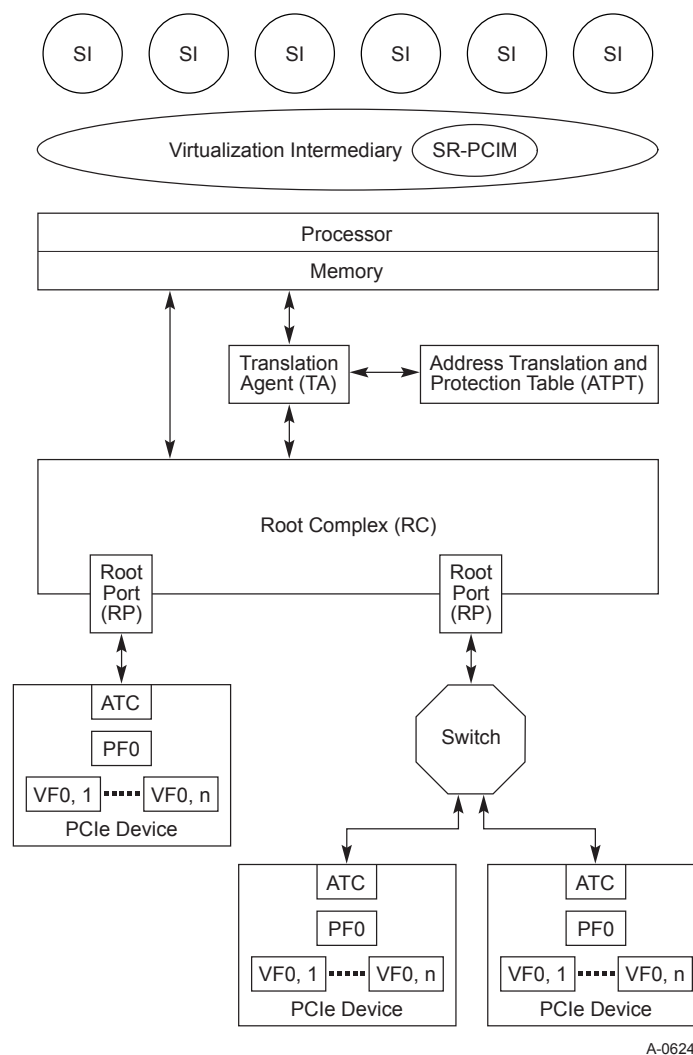


Figure 9-3 Generic Platform Configuration with SR-IOV and IOV Enablers

The SR-IOV generic platform configuration is composed of the following additional functional elements:

- SR-PCIM - Software responsible for the configuration of the SR-IOV Extended Capability, management of Physical Functions and Virtual Functions, and processing of associated error events and overall device controls such as power management and hot-plug services.
- Optional Translation Agent (TA) - A TA is hardware or a combination of hardware and software responsible for translating an address within a PCIe transaction into the associated platform physical address. A TA may contain an Address Translation Cache (ATC) to accelerate translation table access. A TA may also support Address Translation Services (ATS) which enables a PCIe Function to obtain address translations *a priori* to DMA access to the associated memory. See § Chapter 10. for more details on ATS benefits and operation.
- Optional Address Translation and Protection Table (ATPT) - An ATPT contains the set of address translations accessed by a TA to process PCIe requests - DMA Read, DMA Write, or interrupt requests. See Address Translation Services (§ Chapter 10.) for additional details.

- In PCIe, interrupts are treated as memory write operations. Through the combination of a Requester Identifier and the address contained within a PCIe transaction, an interrupt can be routed to any target (e.g., a processor core) transparent to the associated I/O Function.
- DMA Read and Write requests are translated through a combination of the Routing ID and the address contained within a PCIe transaction.
- Optional Address Translation Cache (ATC) - An ATC can exist in two locations within a platform - within the TA which can be integrated within or sit above an RC - or within a PCIe Device. Within an RC, the ATC enables accelerated translation look ups to occur. Within a Device, the ATC is populated through ATS technology. PCIe transactions that indicate they contain translated addresses may bypass the platform's ATC in order to improve performance without compromising the benefits associated with ATPT technology. See Address Translation Services (§ Chapter 10.) for additional details.
- Optional Access Control Services (ACS) - ACS defines a set of control points within a PCI Express topology to determine whether a TLP should be routed normally, blocked, or redirected. In a system that supports SR-IOV, ACS may be used to prevent device Functions assigned to the VI or different SIs from communicating with one another or a peer device. Redirection may permit a Translation Agent to translate Upstream memory TLP addresses before a peer-to-peer forwarding decision is made. Selective blocking may be provided by the optional ACS P2P Egress Control. ACS is subject to interaction with ATS. See § Section 6.12 for additional details.
- Physical Function (PF) - A PF is a PCIe Function that supports the SR-IOV Extended Capability and is accessible to an SR-PCIM, a VI, or an SI.
- Virtual Function (VF) - A VF is a “light-weight” PCIe Function that is directly accessible by an SI.
 - Minimally, resources associated with the main data movement of the Function are available to the SI. Configuration resources should be restricted to a trusted software component such as a VI or SR-PCIM.
 - A VF can be serially shared by different SI, (i.e., a VF can be assigned to one SI and then reset and assigned to another SI.)
 - A VF can be optionally migrated from one PF to another PF. The migration process itself is outside the scope of this specification but is facilitated through configuration controls defined within this specification.
- All VFs associated with a PF must be the same device type as the PF, (e.g., the same network device type or the same storage device type.)

To compare and contrast a PCIe Device with a PCIe SR-IOV capable device, examine the following set of figures. § Figure 9-4 illustrates an example PCIe-compliant Device.

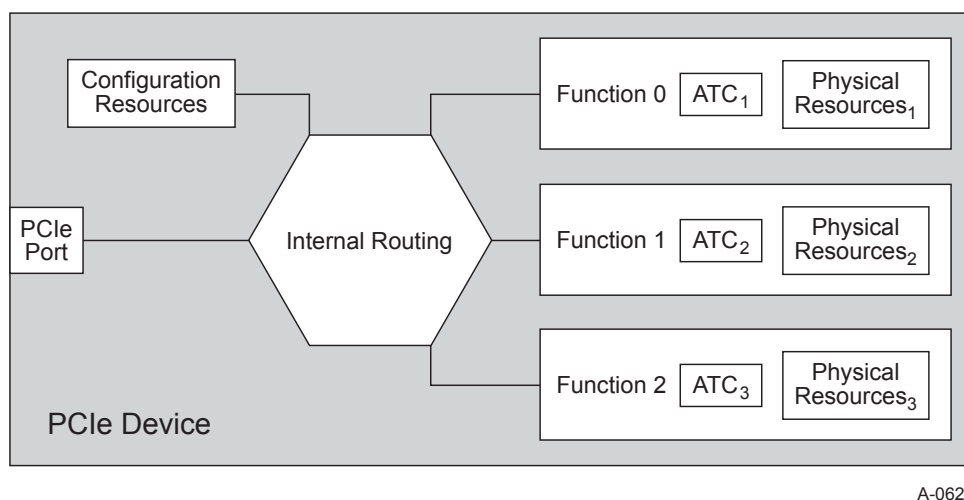
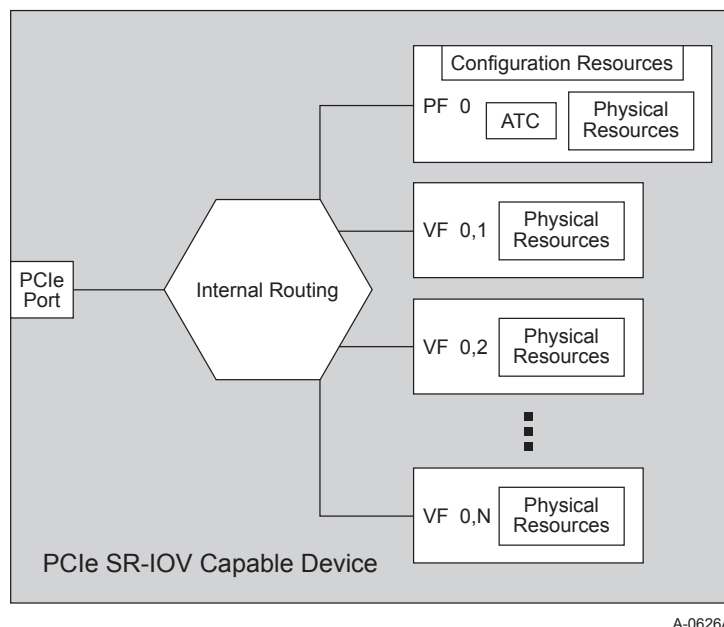


Figure 9-4 Example Multi-Function Device

This figure illustrates an example multi-Function PCIe Device with the following characteristics:

- The PCIe Device shares a common PCIe Link. The Link and PCIe functionality shared by all Functions is managed through Function 0.
 - While this figure illustrates only three Functions, with the use of the Alternative Routing Identifier (ARI) capability, a PCIe Device can support up to 256 Functions.
 - All Functions use a single Bus Number captured through the PCI enumeration process.
- In this example, each PCIe Function supports the ATS capability and therefore has an associated ATC to manage ATS obtained translated addresses.
- Each PCIe Function has a set of unique physical resources including a separate configuration space and BAR.
- Each PCIe Function can be assigned to an SI. To prevent one SI from impacting another, all PCIe configuration operations should be intercepted and processed by the VI.

As this figure illustrates, the hardware resources scale with the number of Functions provisioned. Depending upon the complexity and size of the device, the incremental cost per Function will vary. To reduce the incremental hardware cost, a device can be constructed using SR-IOV to support a single PF and multiple VFs as illustrated in § [Figure 9-5](#).



A-0626A

Figure 9-5 Example SR-IOV Single PF Capable Device

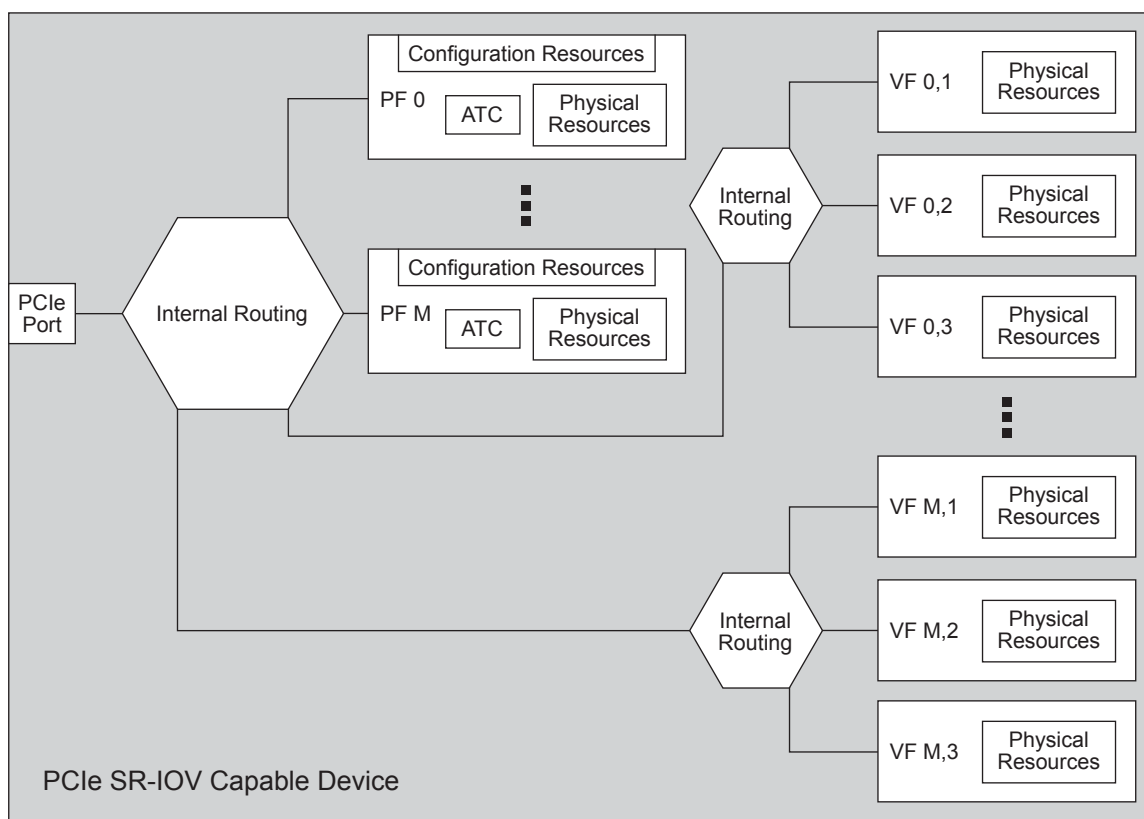
The example in § Figure 9-5 illustrates a single PF with N VFs. Key observations to note:

- The PF is a PCIe-compliant.
 - Initially and after a conventional reset, a PCIe Function that supports the SR-IOV capabilities defined in this specification shall have the SR-IOV capabilities disabled.
 - To discover the page sizes supported by a PF and its associated VF, the Supported Page Sizes configuration field is read. For additional information on how this field can be used to align PF or VF Memory space apertures on a system page boundary, see § Section 9.2.1.1.1 .
- PF nomenclature **PF M** designates the PF at Function number M.
- VF nomenclature **VF M,N** designates the Nth VF associated with PF M. VFs are numbered starting with 1 so the first VF associated with PF M is VF M,1.
- Each VF shares a number of common configuration space fields with the PF; (i.e., where the fields are applicable to all VF and controlled through a single PF. Sharing reduces the hardware resource requirements to implement each VF.)
 - A VF uses the same configuration mechanisms and header types as a PF.
 - All VFs associated with a given PF share a VF BAR set (see § Section 9.3.3.14) and share a VF Memory Space Enable (MSE) bit in the SR-IOV extended capability (see § Section 9.3.3.3.4) that controls access to the VF Memory space. That is, if the VF MSE bit is Clear, the memory mapped space allocated for all VFs is disabled.
 - The InitialVFs and TotalVFs fields (see § Section 9.3.3.5 and § Section 9.3.3.6) are used to discover the maximum number of VFs that can be associated with a PF.
 - TotalVFs and InitialVFs shall contain the same value ¹⁷¹ .

171. InitialVFs and TotalVFs could have had different values due to MR-IOV (now deprecated).

- Each Function, PF, and VF is assigned a unique Routing ID. The Routing ID for each PF is constructed as per [§ Section 2.2.4.2](#) . The Routing ID for each VF is determined using the Routing ID of its associated PF and fields in that PF's SR-IOV Extended Capability.
- All PCIe and SR-IOV configuration access is assumed to be through a trusted software component such as a VI or an SR-PCIM.
- Each VF contains a non-shared set of physical resources required to deliver Function-specific services, (e.g., resources such as work queues, data buffers, etc.) These resources can be directly accessed by an SI without requiring VI or SR-PCIM intervention.
- One or more VF may be assigned to each SI. Assignment policies are outside the scope of this specification.
- While this example illustrates a single ATC within the PF, the presence of any ATC is optional. In addition, this specification does not preclude an implementation from supporting an ATC per VF within the Device.
- Internal routing is implementation specific.
- While many potential usage models exist regarding PF operation, a common usage model is to use the PF to bootstrap the device or platform strictly under the control of a VI. Once the SR-IOV Extended Capability is configured enabling VF to be assigned to individual SI, the PF takes on a more supervisory role. For example, the PF can be used to manage device-specific functionality such as internal resource allocation to each VF, VF arbitration to shared resources such as the PCIe Link or the Function-specific Link (e.g., a network or storage Link), etc. These policy, management, and resource allocation operations are outside the scope of this specification.

Another example usage model is illustrated in [§ Figure 9-6](#). In this example, the device supports multiple PFs each with its own set of VFs.



A-0627

Figure 9-6 Example SR-IOV Multi-PF Capable Device

Key observations to note:

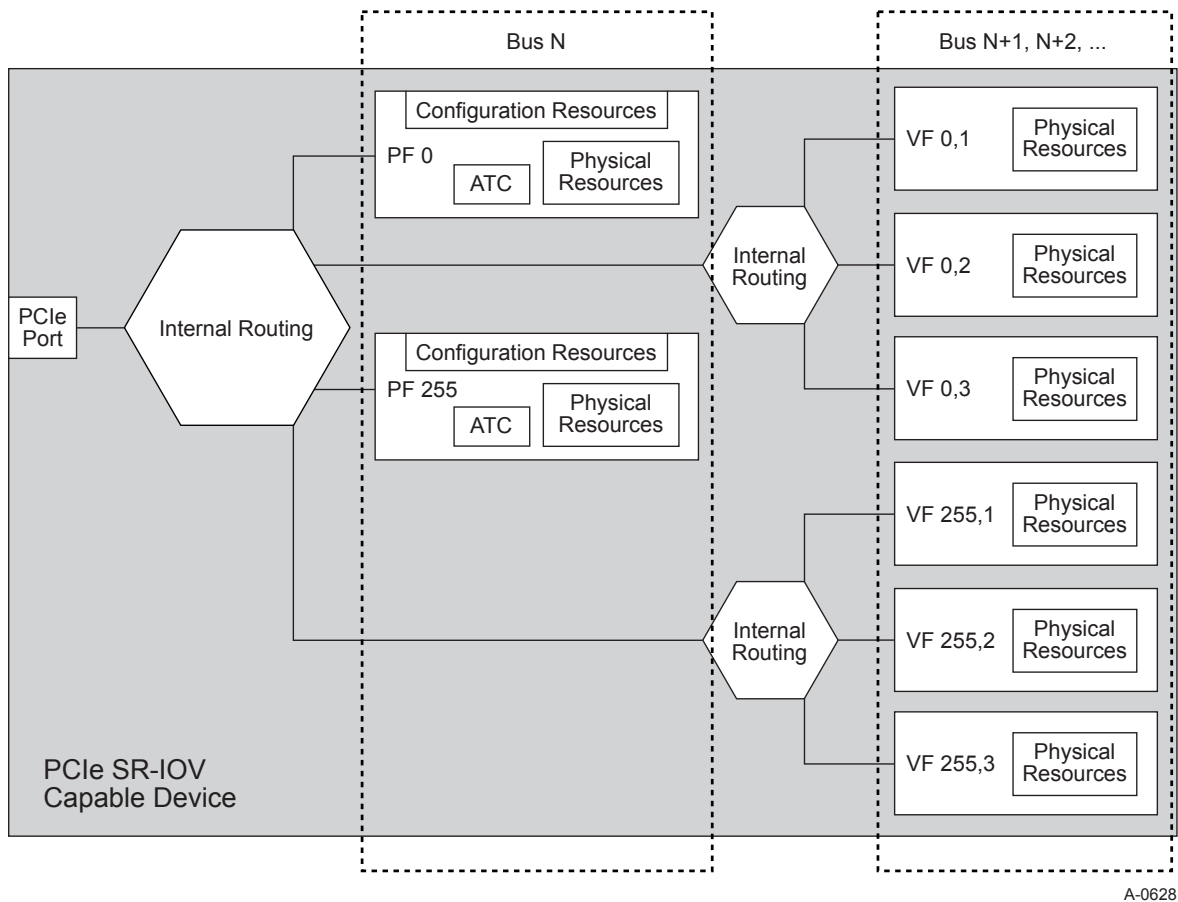
- Each PF can be assigned zero or more VFs. The number of VFs per PF is not required to be identical for all PFs within the device.
- The ARI Extended Capability enables Functions to be assigned to Function Groups and defines how Function Group arbitration can be configured. PFs and VFs can be assigned to Function Groups and take advantage of the associated arbitration capabilities. Within each Function Group, though, arbitration remains implementation specific.
- Internal routing between PFs and VFs is implementation specific.
- For some usage models, all PFs may be the same device type; (e.g., all PFs deliver the same network device or all deliver the same storage device functionality.) For other usage models, each PF may represent a different device type; (e.g., in § Figure 9-6, one PF might represent a network device while another represents an encryption device.)
 - In situations where there is a usage model dependency between device types, such as for each VF that is a network device type, each SI also requires a VF that is an encryption device type. The SR-IOV Extended Capability provides a method to indicate these dependencies. The policies used to construct these dependencies as well as assign dependent sets of VF to a given SI are outside the scope of this specification.

As seen in the prior example, the number of PF and VF can vary based on usage model requirements. To support a wide range of options, an SR-IOV Device can support the following number and mix of PF and VF:

- Using the Alternative Routing Identifier (ARI) capability, a device may support up to 256 PFs. Function Number assignment is implementation specific and may be sparse throughout the 256 Function Number space.
- A PF can only be associated with the Device's captured Bus Number as illustrated in § Figure 9-7.
- SR-IOV Devices may consume more than one Bus Number. A VF can be associated with any Bus Number within the Device's Bus Number range - the captured Bus Number plus any additional Bus Numbers configured by software. See § Section 9.2.1.2 for details.
 - Use of multiple Bus Numbers enables a device to support a very large number of VFs - up to the size of the Routing ID space minus the bits used to identify intervening busses.
 - If software does not configure sufficient additional Bus Numbers, then the VFs implemented for the additional Bus Numbers may not be visible.

IMPLEMENTATION NOTE: FUNCTION CO-LOCATION §

The ARI Extended Capability enables a Device to support up to 256 Functions - Functions, PFs, or VFs in any combination - associated with the captured Bus Number. If a usage model does not require more than 256 Functions, implementations are strongly encouraged to co-locate all Functions, PFs, and VFs within the captured Bus Number and not require additional Bus Numbers to access VFs.



A-0628

Figure 9-7 Example SR-IOV Device with Multiple Bus Numbers

In this last example, § Figure 9-8, a device implementation may mix any number of Functions, PFs, and VFs.

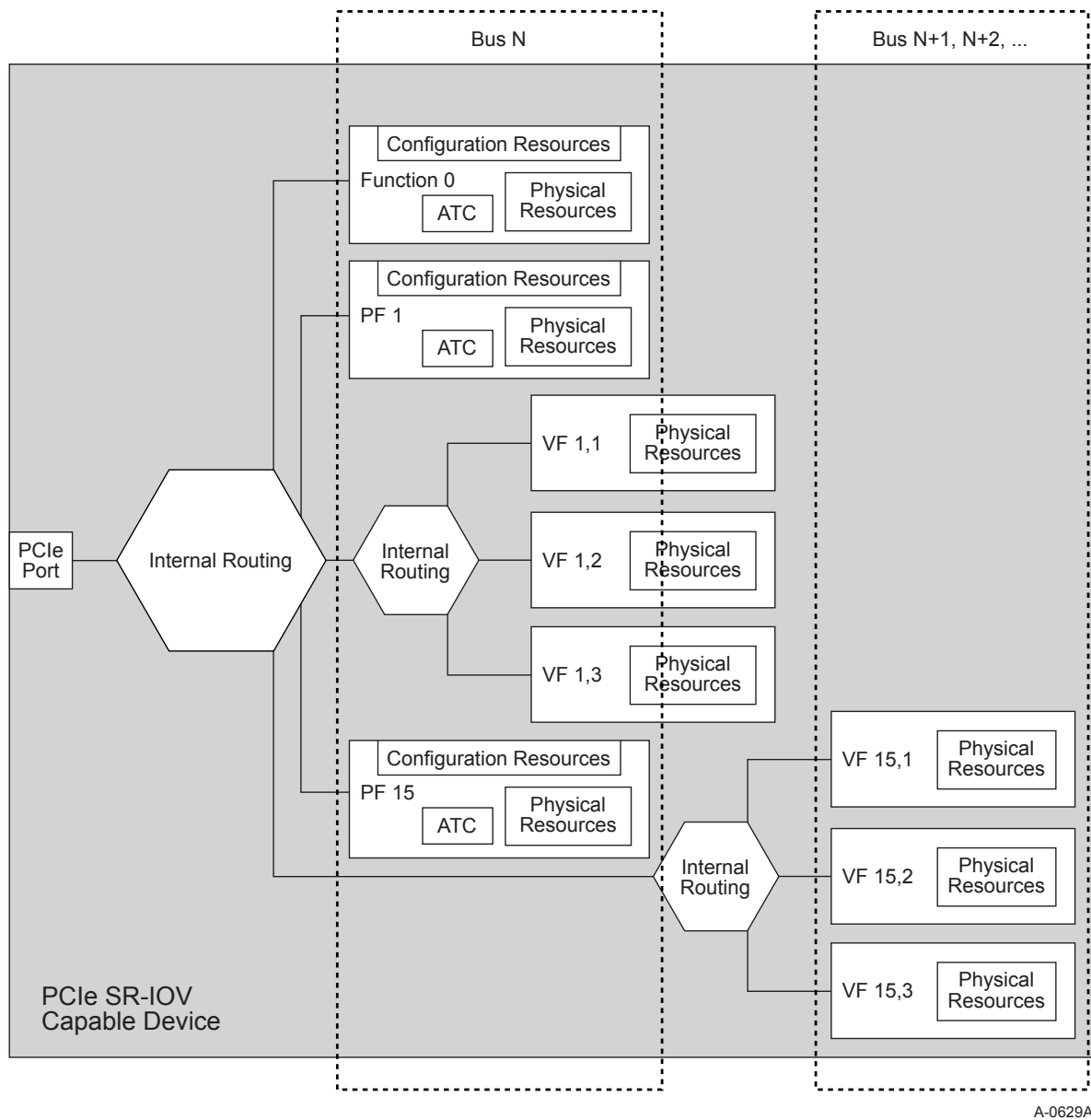


Figure 9-8 Example SR-IOV Device with a Mixture of Function Types

Key observations to note:

- Each Device must contain a Function 0. Function 0 may be a PF (i.e., it may include the SR-IOV extended capability).
- Any mix of Functions can be associated with the captured Bus Number.
 - Non-VFs can only be associated with the captured Bus Number.
- If the ARI Extended Capability is supported, Functions can be assigned to Function Groups. The assignment policy is outside the scope of this specification. If the ARI Extended Capability is not supported, Functions can still use the Function arbitration capabilities as defined in [Section 6.3.3.4](#).

9.2 SR-IOV Initialization and Resource Allocation §

9.2.1 SR-IOV Resource Discovery §

The following sections describe how software determines that a Device is SR-IOV capable and subsequently identifies VF resources through Virtual Function Configuration Space.

9.2.1.1 Configuring SR-IOV Capabilities §

This section describes the fields that must be configured before enabling a PF's IOV Capabilities. The VFs are enabled by Setting the PF's VF Enable bit (see § Section 9.3.3.3.1) in the SR-IOV extended capability.

The NumVFs field (see § Section 9.3.3.7) defines the number of VFs that are enabled when VF Enable is Set in the associated PF.

9.2.1.1.1 Configuring the VF BAR Mechanisms §

This section describes how the VF BARs are configured to map memory space. VFs do not support I/O Space and thus VF BARs shall not indicate I/O Space.

The System Page Size field (see § Section 9.3.3.13) defines the page size the system will use to map the VF's PCIe memory addresses when the PF's IOV Capabilities are enabled. The System Page Size field is used by the PF to align the Memory space aperture defined by each VF BAR to a system page boundary. The value chosen for the System Page Size must be one of the Supported Page Sizes (see § Section 9.3.3.12) in the SR-IOV extended capability.

The behavior of VF BARs is the same as the normal PCI Memory Space BARs (see § Section 7.5.1.2.1), except that a VF BAR describes the aperture for each VF, whereas a PCI BAR describes the aperture for a single Function. The attributes for some of the bits in the VF BARs are affected by the VF Resizable BAR Extended Capability (see § Section 7.8.7) if it is implemented.

- The behavior described in § Section 7.5.1.2.1 for determining the memory aperture of a Function's BAR applies to each VF BAR. That is, the size of the memory aperture required for each VF BAR can be determined by writing all "1"s and then reading the VF BAR. The results read back must be interpreted as described in § Section 7.5.1.2.1 .
- The behavior for assigning the starting memory space address of each BAR associated with the first VF is also as described in § Section 7.5.1.2.1 . That is, the address written into each VF BAR is used by the Device for the starting address of the first VF.
- The difference between VF BARs and BARs described in § Section 7.5.1.2.1 is that for each VF BAR, the memory space associated with the second and higher VFs is derived from the starting address of the first VF and the memory space aperture. For any given VF_v, the starting address of its Memory space aperture for any implemented BAR_b is calculated according to the following formula:

$$\text{BAR}_b \text{ VF}_v \text{ starting address} = \text{VF BAR}_b + (v - 1) \times (\text{VF BAR}_b \text{ aperture size})$$

where VF BAR_b aperture size is the size of VF BAR_b as determined by the usual BAR probing algorithm as described in § Section 9.3.3.14 .

VF memory space is not enabled until both VF Enable and VF MSE have been Set (see § Section 9.3.3.3.1 and § Section 9.3.3.3.4). Note that changing System Page Size (see § Section 9.3.3.13) may affect the VF BAR aperture size.

§ Figure 9-9 shows an example of the PF and VF Memory space apertures.

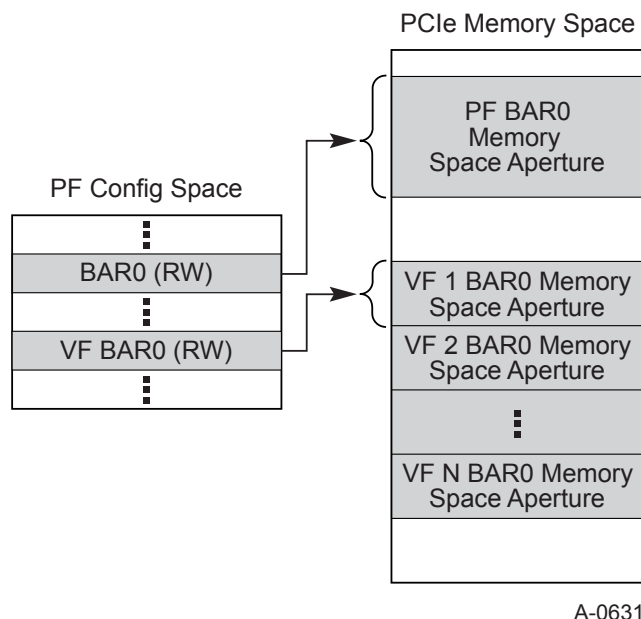


Figure 9-9 BAR Space Example for Single BAR Device

9.2.1.2 VF Discovery §

The First VF Offset and VF Stride fields in the SR-IOV extended capability are 16-bit Routing ID offsets. These offsets are used to compute the Routing IDs for the VFs with the following restrictions:

- The value in NumVFs in a PF (§ Section 9.3.3.7) may affect the values in First VF Offset (§ Section 9.3.3.9) and VF Stride (§ Section 9.3.3.10) of that PF.
- The value in ARI Capable Hierarchy (§ Section 9.3.3.3.5) in the lowest-numbered PF of the Device (for example PF₀) may affect the values in First VF Offset and VF Stride in all PFs of the Device.
- NumVFs of a PF may only be changed when VF Enable (§ Section 9.3.3.3.1) of that PF is Clear.
- ARI Capable Hierarchy (§ Section 9.3.3.3.5) may only be changed when VF Enable is Clear in all PFs of a Device.

IMPLEMENTATION NOTE: NUMVFS AND ARI CAPABLE HIERARCHY §

After configuring NumVFs and ARI Capable Hierarchy where applicable, software may read First VF Offset and VF Stride to determine how many Bus Numbers would be consumed by the PF's VFs. The additional Bus Numbers, if any, are not actually used until VF Enable is Set.

§ Table 9-1 describes the algorithm used to determine the Routing ID associated with each VF.

Table 9-1 VF Routing ID Algorithm

VF Number	VF Routing ID
VF 1	(PF Routing ID + First VF Offset) Modulo 2^{16}
VF 2	(PF Routing ID + First VF Offset + VF Stride) Modulo 2^{16}
VF 3	(PF Routing ID + First VF Offset + 2 * VF Stride) Modulo 2^{16}
...	...
VF N	(PF Routing ID + First VF Offset + (N-1) * VF Stride) Modulo 2^{16}
...	...
VF NumVFs (last one)	(PF Routing ID + First VF Offset + (NumVFs-1) * VF Stride) Modulo 2^{16}

All arithmetic used in this Routing ID computation is 16-bit unsigned dropping all carries.

All VFs and PFs must have distinct Routing IDs. The Routing ID of any PF or VF must not overlap with the Routing ID of any other PF or VF given any valid setting of NumVFs across all PFs of a device.

VF Stride and First VF Offset are constants. Except as stated earlier in this section, their values may not be affected by settings in this or other Functions of the Device.

VFs may reside on different Bus Number(s) than the associated PF. This can occur if, for example, First VF Offset has the value 0100h. A VF shall not be located on a Bus Number that is numerically smaller than its associated PF. A VF that is located on the same Bus Number as its associated PF shall not be located on a Device Number that is numerically smaller than the PF¹⁷².

VFs of an SR-IOV RCiEP Device are associated with the same Root Complex Event Collector (if any) as their PF. Such VFs are not reported in the Root Complex Event Collector Endpoint Association Extended Capability of the Root Complex Event Collector.

As per § Section 2.2.6.2, SR-IOV capable Devices that are associated with an Upstream Port capture the Bus Number from any Type 0 Configuration Write Request. SR-IOV capable Devices do not capture the Bus Number from any Type 1 Configuration Write Requests. SR-IOV capable RCiEPs use an implementation specific mechanism to assign their Bus Numbers.

Note: Bus Numbers are a constrained resource. Devices are strongly encouraged to avoid leaving “holes” in their Bus Number usage to avoid wasting Bus Numbers.

All PFs must be located on the Device’s captured Bus Number.

172. SR-IOV Devices immediately below a Downstream Port always have a Device Number of 0 and thus always satisfy this condition.

IMPLEMENTATION NOTE:

VFS SPANNING MULTIPLE BUS NUMBERS §

As an example, consider an SR-IOV Device that supports a single PF. Initially, only PF 0 is visible. Software Sets ARI Capable Hierarchy. From the SR-IOV Extended Capability it determines: InitialVFs is 600, First VF Offset is 1 and VF Stride is 1.

- If software sets NumVFs in the range [0 ... 255], then the Device uses a single Bus Number.
- If software sets NumVFs in the range [256 ... 511], then the Device uses two Bus Numbers.
- If software sets NumVFs in the range [512 ... 600], then the Device uses three Bus Numbers.

PF 0 and VF 0,1 through VF 0,255 are always on the first (captured) Bus Number. VF 0,256 through VF 0,511 are always on the second Bus Number (captured Bus Number plus 1). VF 0,512 through VF 0,600 are always on the third Bus Number (captured Bus Number plus 2).

Software should configure Switch Secondary and Subordinate Bus Number fields to route enough Bus Numbers to the Device. If sufficient Bus Numbers are not available, software should reduce a Device's Bus Number requirements by not enabling SR-IOV and/or reducing NumVFs for some or all PFs of the Device prior to enabling SR-IOV.

After VF Enable is Set in some PF n , the Device must Enable VF $n,1$ through VF n,m (inclusive) where m is the smaller of InitialVFs and NumVFs. A Device receiving a Type 0 Configuration Request targeting an Enabled VF located on the captured Bus Number must process the Request normally. A Device receiving a Type 1 Configuration Request targeting an Enabled VF not located on the captured Bus Number must process the Request normally. A Device receiving a Type 1 Configuration Request targeting the Device's captured Bus Number must follow the rules for handling Unsupported Requests. Additionally, if VF MSE is Set, each Enabled VF must respond to PCIe Memory transactions addressing the memory space associated with that VF.

Functions that are not enabled (i.e., Functions for VFs above m) do not exist in the PCI Express fabric. As per § Section 2.3.1, addressing Functions that do not exist will result in Unsupported Request (UR) being returned. This includes Functions on additional Bus Numbers.

IMPLEMENTATION NOTE:

MULTI-FUNCTION DEVICES WITH PFS AND SWITCH FUNCTIONS §

SR-IOV devices may consume multiple bus numbers. Additional bus numbers beyond the first one are consecutive and immediately follow the first bus number assigned to the device. If an SR-IOV device also contains PCI-PCI Bridges (with Type 1 Configuration Space Headers), the SR-IOV usage must be accounted for when programming the Secondary Bus Number for those Bridges. Software should determine the last Bus Number used by VFs first and then configure any co-located Bridges to use Bus Numbers above that value.

§

—

9.2.1.3 Function Dependency Lists §

PCI Devices may have vendor-specific dependencies between Functions. For example, Functions 0 and 1 might provide different mechanisms for controlling the same underlying hardware. In such situations, the Device programming model might require that these dependent Functions be assigned to SIs as a set.

Function Dependency Lists are used to describe these dependencies (or to indicate that there are no Function dependencies). Software should assign PFs and VFs to SIs such that the dependencies are satisfied.

See § Section 9.3.3.8 for details.

9.2.1.4 Interrupt Resource Allocation §

PFs and VFs support either MSI, MSI-X interrupts, or both if interrupt resources are allocated. VFs shall not implement INTx. MSI and MSI-X interrupts are described in § Section 6.1.4 .

For MSI-X interrupts, special address range isolation requirements apply for the MSI-X structures in PF and VF MMIO regions. See in § Section 7.7.2 .

9.2.2 SR-IOV Reset Mechanisms §

This section describes how reset mechanisms defined in § Section 6.6 affect Devices that support SR-IOV. It also describes the mechanisms used to reset a single VF and a single PF with its associated VFs.

9.2.2.1 SR-IOV Conventional Reset §

A Conventional Reset to a Device that supports SR-IOV shall cause all Functions (including both PFs and VFs) to be reset to their original, power-on state as per the rules in § Section 6.6.1 . § Section 9.3 describes the behavior for the fields defined.

Note: Conventional Reset clears VF Enable in the PF. Thus, VFs no longer exist after a Conventional Reset.

9.2.2.2 FLR That Targets a VF §

VFs must support Function Level Reset (FLR).

Note: Software may use FLR to reset a VF. FLR to a VF affects a VF's state but does not affect its existence in PCI Configuration Space or PCI Bus address space. The VFs BARn values (see § Section 9.3.3.14) and VF MSE (see § Section 9.3.3.3.4) in the PF's SR-IOV extended capability, and the VF Resizable BAR capability values (see § Section 7.8.7) are unaffected by FLRs issued to the VF.

9.2.2.3 FLR That Targets a PF §

PFs must support FLR.

FLR to a PF resets the PF state as well as the SR-IOV extended capability including VF Enable which means that VFs no longer exist.

9.2.3 IOV Re-initialization and Reallocation §

If VF Enable is Cleared after having been Set, all of the VFs associated with the PF no longer exist and must no longer issue PCIe transactions or respond to Configuration Space or Memory Space accesses. VFs must not retain any state after VF Enable has been Cleared (including sticky bits).

9.3 Configuration §

9.3.1 SR-IOV Configuration Overview §

This section provides SR-IOV-added requirements for implementing PFs and VFs.

PFs are discoverable in configuration space, as with all Functions. PFs contain the SR-IOV Extended Capability described in § Section 9.3.3 . PFs are used to discover, configure, and manage the VFs associated with the PF and for other things described in this specification.

9.3.2 Configuration Space §

PFs that support SR-IOV shall implement the SR-IOV Extended Capability as defined in the following sections. VFs shall implement configuration space fields and capabilities as defined in the following sections.

9.3.3 SR-IOV Extended Capability §

The SR-IOV Extended Capability defined here is a PCIe extended capability that must be implemented in each PF that supports SR-IOV. This Capability is used to describe and control a PF's SR-IOV Capabilities.

For Multi-Function Devices, each PF that supports SR-IOV shall provide the Capability structure defined in this section. This Capability structure may be present in any Function with a Type 0 Configuration Space Header. This Capability must not be present in Functions with a Type 1 Configuration Space Header.

§ Figure 9-10 shows the SR-IOV Extended Capability structure.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Byte Offset
PCI Express Extended Capability Header																																+000h
SR-IOV Capabilities Register (RO)																																+004h
SR-IOV Status Register																SR-IOV Control Register (RW)																+008h
TotalVFs (RO)																InitialVFs (RO)																+00Ch
RsvdP								Fcn Dep Link (RO)								NumVFs (RW)																+010h
VF Stride (RO)																FirstVF Offset (RO)																+014h
VF Device ID (RO)																RsvdP																+018h
Supported Page Sizes (RO)																																+01Ch
System Page Size (RW)																																+020h
VF BAR0 (RW)																																+024h
VF BAR1 (RW)																																+028h
VF BAR2 (RW)																																+02Ch
VF BAR3 (RW)																																+030h
VF BAR4 (RW)																																+034h
VF BAR5 (RW)																																+038h
VF Migration State Array Offset (RO)																																+03Ch

§ Figure 9-10 SR-IOV Extended Capability

9.3.3.1 SR-IOV Extended Capability Header (Offset 00h) §

§ Table 9-2 defines the SR-IOV Extended Capability header. The Capability ID for the SR-IOV Extended Capability is 0010h.

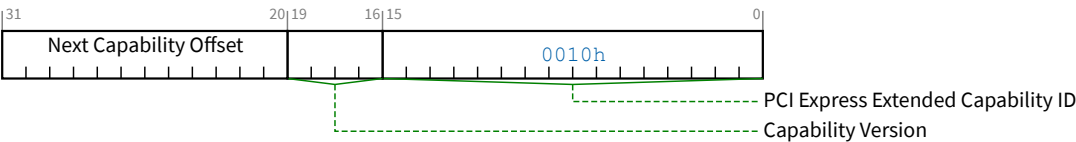


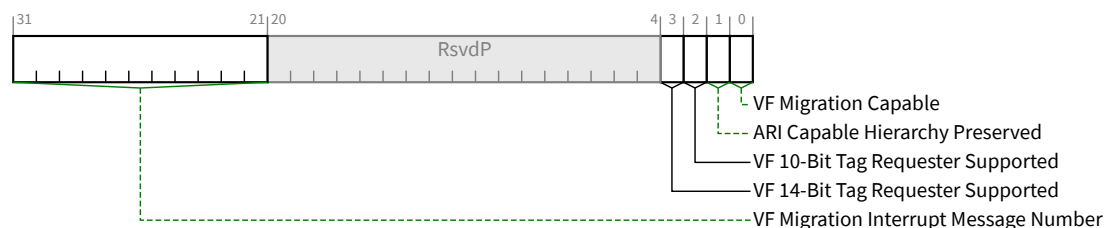
Figure 9-11 SR-IOV Extended Capability Header

Table 9-2 SR-IOV Extended Capability Header

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - This field is a PCI-SIG defined ID number that indicates the nature and format of the Extended Capability. The Extended Capability ID for the SR-IOV Extended Capability is 0010h.	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be 1h for this version of the specification.	RO
31:20	Next Capability Offset - This field contains the offset to the next PCI Express Capability structure or 000h if no other items exist in the linked list of Capabilities. For Extended Capabilities implemented in Configuration Space, this offset is relative to the beginning of PCI-compatible Configuration Space and thus must always be either 000h (for terminating list of Capabilities) or greater than 0FFh.	RO

9.3.3.2 SR-IOV Capabilities Register (04h) §

§ Table 9-3 defines the layout of the SR-IOV Capabilities field.



§Figure 9-12 SR-IOV Capabilities Register

§ Table 9-3 SR-IOV Capabilities Register

Bit Location	Register Description	Attributes
0	VF Migration Capable - If Set, the PF is Migration Capable and operating under a Migration Capable MR-PCIM. Deprecated. New designs should hardwire this bit to 0b.	RO
1	ARI Capable Hierarchy Preserved <i>PCI Express Endpoint:</i> If Set, the ARI Capable Hierarchy bit is preserved across certain power state transitions. <i>RCiEP:</i> Not applicable - this bit <i>MUST@FLIT</i> be hardwired to 0b.	RO
2	VF 10-Bit Tag Requester Supported - If Set, all VFs associated with this PF must support 10-Bit Tag Requester capability. If Clear, VFs must not support 10-Bit Tag Requester capability.	HwInit

Bit Location	Register Description	Attributes
	If the 10-Bit Tag Requester Supported bit in the PF's Device Capabilities 2 register is Clear, this bit must be Clear.	
3	VF 14-Bit Tag Requester Supported - If Set, all VFs associated with this PF must support 14-Bit Tag Requester capability. If Clear, VFs must not support 14-Bit Tag Requester capability. If the 14-Bit Tag Requester Supported bit in the PF's Device Capabilities 2 register is Clear, this bit must be Clear.	HwInit
31:21	VF Migration Interrupt Message Number - Indicates the MSI/MSI-X vector used for migration interrupts. The value in this field is undefined if VF Migration Capable is Clear. VF Migration is a feature associated with the deprecated MR-IOV. New designs should hardwire this field to 0.	RO

9.3.3.2.1 VF Migration Capable §

VF Migration Capable is Set to indicate that the PF supports VF Migration. If Clear, the PF does not support VF Migration.

VF Migration is a feature associated with the deprecated [MR-IOV]. This bit should be hardwired to 0b in new designs.

9.3.3.2.2 ARI Capable Hierarchy Preserved §

ARI Capable Hierarchy Preserved is Set to indicate that the PF preserves the ARI Capable Hierarchy bit across certain power state transitions (see § Section 9.3.3.3.5). Components must either Set this bit or Set the No_Soft_Reset bit (see § Section 5.10.2). It is recommended that components set this bit even if they also set No_Soft_Reset.

ARI Capable Hierarchy Preserved is only present in the lowest-numbered PF of a Device (for example PF0). ARI Capable Hierarchy Preserved is Read Only Zero in other PFs of a Device.

ARI Capable Hierarchy Preserved does not apply to RCiEPs, and its value is undefined (see § Section 9.3.3.3).

9.3.3.2.3 VF Larger-Tag Requester Support §

If a PF supports one or both larger-Tag Requester capabilities, its associated VFs are permitted to support the associated larger-Tag Requester capabilities as well, but this is optional. Especially for usage models where the bulk of the traffic is spread across several VFs concurrently, it may not be necessary for individual VFs to use larger Tags so they can support >256 outstanding Non-Posted Requests each.

For a given PF, it is required that either all or none of its associated VFs support larger-Tag Requester capabilities. This avoids unnecessary implementation and management complexity. See VF 14-Bit Tag Requester Supported and VF 10-Bit Tag Requester Supported.

VFs that support larger-Tag Requester capabilities have additional requirements beyond other Function types in order to simplify error handling and reduce the possibility of larger-Tag related errors with one VF impacting other traffic.

- If one of the VF larger-Tag Requester Enable bits in the SR-IOV Control Register is Set, then each VF must use the associated larger Tags for all Non-Posted Requests that it generates.
- For each outstanding larger-Tag Request, if the VF receives a Completion that matches the outstanding Request other than Tag[13:8] being 00 0000b, the VF must prevent that Request from (eventually) generating a

Completion Timeout error, and instead handle the error via a device-specific mechanism that avoids data corruption.

It is strongly recommended that software not configure Unexpected Completion errors to be handled as Uncorrectable Errors. This avoids them triggering System Errors or hardware error containment mechanisms like Downstream Port Containment (DPC).

IMPLEMENTATION NOTE:
NO VF LARGER-TAG COMPLETER SUPPORTED BITS §

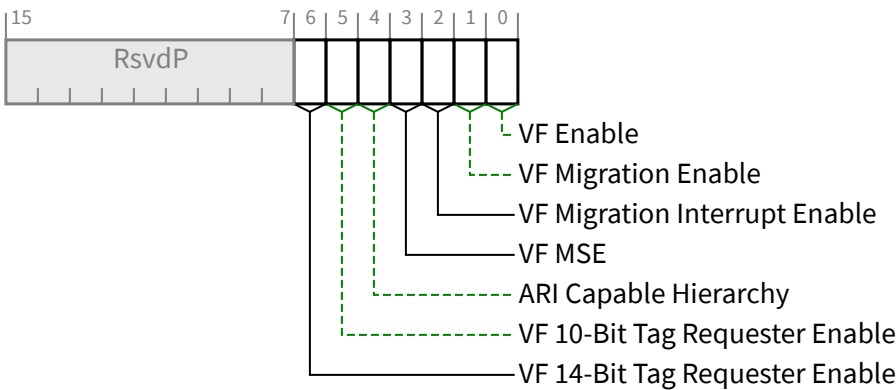
There are no VF 14-bit or VF 10-Bit Tag Completer Supported bits. If a PF supports one or both larger-Tag Completer capabilities, then all of its associated VFs are required support the associated Tag Completer capabilities as stated in 14-Bit Tag Completer Supported and 10-Bit Tag Completer Supported. This helps avoid the complexity of PCIe hierarchies where some Completers support larger-Tag capability and some do not.

9.3.3.2.4 VF Migration Interrupt Message Number §

VF Migration is associated with the now deprecated MR-IOV. New designs should hardwire this field to 0.

9.3.3.3 SR-IOV Control Register (Offset 08h) §

§ Table 9-4 defines the layout of the SR-IOV Control fields.



§ Figure 9-13 SR-IOV Control Register

§ Table 9-4 SR-IOV Control Register

Bit Location	Register Description	Attributes
0	VF Enable - Enables/Disables VFs. Default value is 0b.	RW

Bit Location	Register Description	Attributes
1	VF Migration Enable - Enables/Disables VF Migration Support. Default value is 0b. See § Section 9.3.3.3.2 .	RW or RO (see description)
2	VF Migration Interrupt Enable - Enables/Disables VF Migration State Change Interrupt. Default value is 0b.	RW
3	VF MSE - Memory Space Enable for Virtual Functions. Default value is 0b.	RW
4	ARI Capable Hierarchy - <i>PCI Express Endpoint:</i> This bit must be RW in the lowest-numbered PF of the Device and hardwired to 0b in all other PFs. If the value of this bit is 1b, the Device is permitted to locate VFs in Function Numbers 8 to 255 of the captured Bus Number. Otherwise, the Device must locate VFs as if it were a non-ARI Device. This bit is not affected by FLR of any PF or VF. Default value is 0b. <i>RCiEP:</i> Not applicable - this bit must be hardwired to 0b. Within the Root Complex, VFs are always permitted to be assigned to any Function Number allowed by First VF Offset and VF Stride rules (see § Section 9.3.3.9 and § Section 9.3.3.10).	RW or RO (see description)
5	VF 10-Bit Tag Requester Enable - If Set, all VFs must use 10-Bit Tags for all Non-Posted Requests they generate. If Clear, VFs must not use 10-Bit Tags for Non-Posted Requests they generate. See VF Larger-Tag Requester Support. This bit must not be Set if the VF 14-Bit Tag Requester Enable bit is Set; otherwise the result is undefined. If software changes the value of this bit while any VFs have outstanding Non-Posted Requests, the result is undefined. If the VF 10-Bit Tag Requester Supported bit in the SR-IOV Capabilities register is Clear, this bit is permitted to be hardwired to 0b. Default value is 0b.	RW or RO
6	VF 14-Bit Tag Requester Enable - If Set, all VFs must use 14-Bit Tags for all Non-Posted Requests they generate. If Clear, VFs must not use 14-Bit Tags for Non-Posted Requests they generate. See VF Larger-Tag Requester Support. This bit must not be Set if the VF 10-Bit Tag Requester Enable bit is Set; otherwise the result is undefined. If software changes the value of this bit while any VFs have outstanding Non-Posted Requests, the result is undefined. If the VF 14-Bit Tag Requester Supported bit in the SR-IOV Capabilities register is Clear, this bit is permitted to be hardwired to 0b. Default value is 0b.	RW or RO

9.3.3.3.1 VF Enable §

VF Enable manages the assignment of VFs to the associated PF. If VF Enable is Set, the VFs associated with the PF are accessible in the PCI Express fabric. When Set, VFs respond to and may issue PCI Express transactions following the rules for PCI Express Endpoint Functions.

If VF Enable is Clear, VFs are disabled and not visible in the PCI Express fabric; requests to these VFs shall receive UR and these VFs shall not issue PCI Express transactions.

To allow components to perform internal initialization, after changing the VF Enable bit from Cleared to Set, the system is not permitted to issue Requests to the VFs which are enabled by that VF Enable bit until one of the following is true:

- At least 100 ms has passed
- An FRS Message has been received from the PF with a Reason Code of VF Enabled
- At least VF Enable Time has passed. VF Enable Time is either (1) the Reset Time value in the Readiness Time Reporting capability associated with the VF, or (2) a value determined by system software / firmware¹⁷³.

The Root Complex and/or system software must allow at least 1.0 s after Setting the VF Enable bit, before it may determine that a VF which fails to return a Successful Completion Status for a valid Configuration Request is broken. After Setting the VF Enable bit, the VFs enabled by that VF Enable bit are permitted to return a Configuration RRS status in response to Configuration Requests up to the 1.0 s limit, if they are not ready to provide a Successful Completion Status for a valid Configuration Request. After a PF transmits an FRS Message with a Reason Code of VF Enabled, no VF associated with that PF is permitted to return Configuration RRS in response to a Configuration Request without an intervening VF disable or other valid reset condition. After returning a Successful Completion to any Request, no VF is permitted to return Configuration RRS in response to a Configuration Request without an intervening VF disable or other valid reset condition.

Since VFs don't have an MSE bit (MSE in VFs is controlled by the VF MSE bit in the SR-IOV Extended Capability in the PF), it's possible for software to issue a Memory Request before the VF is ready to handle it. Therefore, Memory Requests must not be issued to a VF until at least one of the following conditions has been met:

- The VF has responded successfully (without returning RRS) to a Configuration Request.
- After issuing an FLR to the VF, at least one of the following is true:
 - At least 1.0 s has passed since the FLR was issued.
 - The VF supports FRS and, after the FLR was issued, an FRS Message has been received from the VF with a Reason Code of FLR Completed.
 - At least FLR Time has passed since the FLR was issued. FLR Time is either (1) the FLR Time value in the Readiness Time Reporting capability associated with the VF or (2) a value determined by system software / firmware¹⁵³.
- After Setting VF Enable in a PF, at least one of the following is true:
 - At least 1.0 s has passed since VF Enable was Set.
 - The PF supports FRS and, after VF Enable was Set, an FRS Message has been received from the PF with a Reason Code of VF Enabled.
 - At least VF Enable Time has passed since VF Enable was Set. VF Enable Time is either (1) the Reset Time value in the Readiness Time Reporting capability associated with the VF or (2) a value determined by system software / firmware¹⁷⁴.

The VF is permitted to silently drop Memory Requests after an FLR has been issued to the VF or VF Enable has been Set in the associated PF's SR-IOV Extended Capability until the VF responds successfully to any Request (excluding the return of RRS in response to a Configuration Request).

Clearing VF Enable effectively destroys the VFs. Setting VF Enable effectively creates VFs. Setting VF Enable after it has previously been Cleared shall result in a new set of VFs. If the PF is in the D0 power state, the new VFs are in the D0_{uninitialized} state. If the PF is in a lower power state behavior is undefined (see Sections 9.6.1 and 9.6.2).

¹⁷³. For example, ACPI tables.

¹⁷⁴. For example, ACPI tables.

When Clearing VF Enable, a PF that supports FRS shall send an FRS Message with FRS Reason VF Disabled to indicate when this operation is complete. The PF is not permitted to send this Message if there are outstanding Non-Posted Requests issued by the PF or any of the VFs associated with the PF. The FRS Message may only be sent after these Requests have completed (or timed out).

After VF Enable is Cleared no field in the SR-IOV Extended Capability may be accessed until either:

- At least 1.0 s has elapsed after VF Enable was Cleared.
- The PF supports FRS and after VF Enable was Cleared, an FRS Message has been received from the PF with a Reason Code of VF Disabled.

§ Section 9.3.3.7 NumVFs, § Section 9.3.3.5 InitialVFs, § Section 9.3.3.6 TotalVFs, § Section 9.3.3.9 First VF Offset, § Section 9.3.3.13 System Page Size, and § Section 9.3.3.14 VF BARx describe additional semantics associated with this field.

9.3.3.3.2 VF Migration Enable §

VF Migration Enable must be Set to allow VF Migration on this PF.

VF Migration is associated with the now-deprecated [MR-IOV]. New designs should hardwire this bit to 0b.

9.3.3.3.3 VF Migration Interrupt Enable §

VF Migration is associated with the now-deprecated MR-IOV. New designs should hardwire this bit to 0b.

9.3.3.3.4 VF MSE (Memory Space Enable) §

VF MSE controls memory space enable for all Active VFs associated with this PF, as with the Memory Space Enable bit in a Function's PCI Command register. The default value for this bit is 0b.

When VF Enable is Set, VF memory space will respond only when VF MSE is Set. VFs shall follow the same error reporting rules as defined in the [PCIe] if an attempt is made to access a Virtual Function's memory space when VF Enable is Set and VF MSE is Clear.

IMPLEMENTATION NOTE: VF MSE AND VF ENABLE §

VF memory space will respond with Unsupported Request when VF Enable is Clear. Thus, VF MSE is “don't care” when VF Enable is Clear; however, software may choose to Set VF MSE after programming the VF BARn registers, prior to Setting VF Enable.

9.3.3.3.5 ARI Capable Hierarchy §

For Devices associated with an Upstream Port, ARI Capable Hierarchy is a hint to the Device that ARI has been enabled in the Root Port or Switch Downstream Port immediately above the Device. Software should set this bit to match the ARI Forwarding Enable bit in the Root Port or Switch Downstream Port immediately above the Device.

ARI Capable Hierarchy is only present in the lowest-numbered PF of a Device (for example PF₀) and affects all PFs of the Device. ARI Capable Hierarchy is Read Only Zero in other PFs of a Device.

A Device may use the setting of ARI Capable Hierarchy to determine the values for First VF Offset (see § Section 9.3.3.9) and VF Stride (see § Section 9.3.3.10). The effect of changing ARI Capable Hierarchy is undefined if VF Enable is Set in any PF. This bit must be set to its default value upon Conventional Reset. This bit is not affected by FLR of any PF or VF. If either ARI Capable Hierarchy Preserved is Set (see § Section 9.3.3.2.2) or No_Soft_Reset is Set (see § Section 5.10.2), a power state transition of this PF from D3_{Hot} to D0 does not affect the value of this bit (see § Section 5.10.2).

ARI Capable Hierarchy does not apply to RCiEPs.

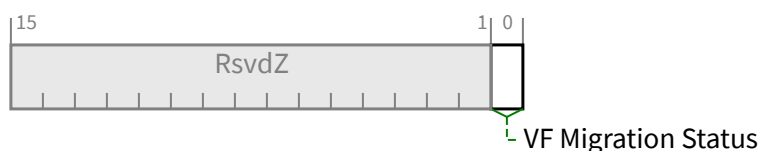
IMPLEMENTATION NOTE: ARI CAPABLE HIERARCHY §

For a Device associated with an Upstream Port, that Device has no way of knowing whether ARI has been enabled. If ARI is enabled, the Device can conserve Bus Numbers by assigning VFs to Function Numbers greater than 7 on the captured Bus Number. ARI is defined in § Section 6.13.

Since RCiEPs are not associated with an Upstream Port, ARI does not apply, and VFs may be assigned to any Function Number within the Root Complex permitted by First VF Offset and VF Stride (see § Section 9.3.3.8 and § Section 9.3.3.9).

9.3.3.4 SR-IOV Status Register (Offset 0Ah) §

§ Table 9-5: defines the layout of the SR-IOV Status field.



§

Figure 9-14 SR-IOV Status

§

Table 9-5 SR-IOV Status

Bit Location	Register Description	Attributes
0	VF Migration Status - VF Migration is associated with the now-deprecated [MR-IOV]. New designs should hardwire this bit to 0b.	RW1C

9.3.3.4.1 VF Migration Status §

VF Migration is associated with the now-deprecated [MR-IOV]. New designs should hardwire this bit to 0b.

9.3.3.5 InitialVFs (Offset 0Ch) §

VF Migration is associated with the now deprecated MR-IOV. New designs should encode this field as HwInit with a value equal to TotalVFs.

9.3.3.6 TotalVFs (Offset 0Eh) §

TotalVFs indicates the maximum number of VFs that are associated with the PF.

The minimum value of TotalVFs is 0.

This field is HwInit and must contain the same value as InitialVFs.

9.3.3.7 NumVFs (Offset 10h) §

NumVFs controls the number of VFs that are visible. SR-PCIM sets NumVFs as part of the process of creating VFs. This number of VFs shall be visible in the PCI Express fabric after both NumVFs is set to a valid value and VF Enable is Set.

The results are undefined if NumVFs is set to a value greater than TotalVFs.

NumVFs may only be written while VF Enable is Clear. If NumVFs is written when VF Enable is Set, the results are undefined.

The initial value of NumVFs is undefined.

9.3.3.8 Function Dependency Link (Offset 12h) §

The programming model for a Device may have vendor-specific dependencies between sets of Functions. The Function Dependency Link field is used to describe these dependencies.

This field describes dependencies between PFs. VF dependencies are the same as the dependencies of their associated PFs.

If a PF is independent from other PFs of a Device, this field shall contain its own Function Number.

If a PF is dependent on other PFs of a Device, this field shall contain the Function Number of the next PF in the same Function Dependency List. The last PF in a Function Dependency List shall contain the Function Number of the first PF in the Function Dependency List.

If PF_p and PF_q are in the same Function Dependency List, then any SI that is assigned $VF_{p,n}$ shall also be assigned to $VF_{q,n}$.

IMPLEMENTATION NOTE:

FUNCTION DEPENDENCY LINK EXAMPLE §

Consider the following scenario:

SR-IOV Field	PF 0	PF 1	PF 2
Function Dependency Link	1	0	2
NumVFs	4	4	6
First VF Offset	4	4	4
VF Stride	3	3	3

Function Number	Description	Independent
0	PF 0	No
1	PF 1	No
2	PF 2	Yes
3	Function not present	
4	VF 0,1 (aka PF 0 VF 1)	No
5	VF 1,1 (aka PF 1 VF 1)	No
6	VF 2,1 (aka PF 2 VF 1)	Yes
7	VF 0,2	No
8	VF 1,2	No
9	VF 2,2	Yes
10	VF 0,3	No
11	VF 1,3	No
12	VF 2,3	Yes
13	VF 0,4	No
14	VF 1,4	No
15	VF 2,4	Yes
16 to 17	Functions not present	
18	VF 2,5	Yes
19 to 20	Functions not present	
21	VF 2,6	Yes

Function Number	Description	Independent
22 to 255	Functions not present	

In this example, Functions 4 and 5 must be assigned to the same SI. Similarly, Functions 7 and 8, 10 and 11, and 13 and 14 must be assigned together. If PFs are assigned to SIs, Functions 0 and 1 must be assigned together as well. Functions 2, 6, 9, 12, 15, 18, and 21 are independent and may be assigned to SIs in any fashion.

All PFs in a Function Dependency List shall have the same values for the InitialVFs and TotalVFs fields.

SR-PCIM shall ensure that all PFs in a Function Dependency List have the same values for the NumVFs and VF Enable fields before any VF in that Function Dependency List is assigned to an SI.

VF Mapping operations occur independently for every VF. SR-PCIM shall not assign a VF to an SI until it can assign all dependent VFs.

Similarly, SR-PCIM shall not remove a VF from an SI until it can remove all dependent VFs.

9.3.3.9 First VF Offset (Offset 14h) §

First VF Offset is a constant and defines the Routing ID offset of the first VF that is associated with the PF that contains this Capability structure. The first VF's 16-bit Routing ID is calculated by adding the contents of this field to the Routing ID of the PF containing this field ignoring any carry, using unsigned, 16-bit arithmetic.

A VF shall not be located on a Bus Number that is numerically smaller than its associated PF.

This field may change value when the lowest-numbered PF's ARI Capable Hierarchy value changes or when this PF's NumVFs value changes.

Note: First VF Offset is unused if NumVFs is 0. If NumVFs is greater than 0, First VF Offset must not be zero.

9.3.3.10 VF Stride (Offset 16h) §

VF Stride defines the Routing ID offset from one VF to the next one for all VFs associated with the PF that contains this Capability structure. The next VF's 16-bit Routing ID is calculated by adding the contents of this field to the Routing ID of the current VF, ignoring any carry, using unsigned 16-bit arithmetic.

This field may change value when the lowest-numbered PF's ARI Capable Hierarchy value changes or when this PF's NumVFs value changes.

Note: VF Stride is unused if NumVFs is 0 or 1. If NumVFs is greater than 1, VF Stride must not be zero.

9.3.3.11 VF Device ID (Offset 1Ah) §

This field contains the Device ID that should be presented for every VF to the SI.

VF Device ID may be different from the PF Device ID. A VF Device ID must be managed by the vendor. The vendor must ensure that the chosen VF Device ID does not result in the use of an incompatible device driver.

9.3.3.12 Supported Page Sizes (Offset 1Ch) §

This field indicates the page sizes supported by the PF. This PF supports a page size of 2^{n+12} if bit n is Set. For example, if bit 0 is Set, the PF supports 4-KB page sizes. PFs are required to support 4-KB, 8-KB, 64-KB, 256-KB, 1-MB, and 4-MB page sizes. All other page sizes are optional.

The page size describes the minimum alignment requirements for VF BAR resources as described in § Section 9.3.3.13.

IMPLEMENTATION NOTE: NON-PREFETCHABLE ADDRESS SPACE §

Non-prefetchable address space is limited to addresses below 4 GB. Pre-fetch address space in 32-bit systems is also limited. Vendors are strongly encouraged to utilize the System Page Size feature to conserve address space while also supporting systems with larger pages.

9.3.3.13 System Page Size (Offset 20h) §

This field defines the page size the system will use to map the VFs' memory addresses. Software must set the value of the System Page Size to one of the page sizes set in the Supported Page Sizes field (see § Section 9.3.3.12). As with Supported Page Sizes, if bit n is Set in System Page Size, the VFs associated with this PF are required to support a page size of 2^{n+12} . For example, if bit 1 is Set, the system is using an 8-KB page size. The results are undefined if System Page Size is zero. The results are undefined if more than one bit is Set in System Page Size. The results are undefined if a bit is Set in System Page Size that is not Set in Supported Page Sizes.

When System Page Size is set, the VF associated with this PF is required to align all BAR resources on a System Page Size boundary. Each VF BAR n or VF BAR n pair (see § Section 9.3.3.14) shall be aligned on a System Page Size boundary. Each VF BAR n or VF BAR n pair defining a non-zero address space shall be sized to consume an integer multiple of System Page Size bytes. All data structures requiring page size alignment within a VF shall be aligned on a System Page Size boundary.

VF Enable must be zero when System Page Size is written. The results are undefined if System Page Size is written when VF Enable is Set.

Default value is 0000 0001h (i.e., 4 KB).

9.3.3.14 VF BAR0 (Offset 24h), VF BAR1 (Offset 28h), VF BAR2 (Offset 2Ch), VF BAR3 (Offset 30h), VF BAR4 (Offset 34h), VF BAR5 (Offset 38h) §

These fields must define the VF's Base Address Registers (BARs). These fields behave as normal PCI BARs, as described in § Section 7.5.1. They can be sized by writing all 1s and reading back the contents of the BARs as described in § Section 7.5.1.2.1, complying with the low order bits that define the BAR type fields.

These fields may have their attributes affected by the VF Resizable BAR Extended Capability (see § Section 7.8.7) if it is implemented.

The amount of address space decoded by each BAR shall be an integral multiple of System Page Size.

Each VF BAR n , when "sized" by writing 1s and reading back the contents, describes the amount of address space consumed and alignment required by a single Virtual Function, per BAR. When written with an actual address value, and

VF Enable and VF MSE are Set, the BAR maps NumVFs BARs. In other words, the base address is the address of the first VF BAR_{*n*} associated with this PF and all subsequent VF BAR_{*n*} address ranges follow as described below.

VF BARs shall only support 32-bit and 64-bit memory space. PCI I/O Space is not supported in VFs. Bit 0 of any implemented VF BAR_{*x*} must be RO 0b except for a VF BAR_{*x*} used to map the upper 32 bits of a 64-bit memory VF BAR pair.

The alignment requirement and size read is for a single VF, but when VF Enable is Set and VF MSE is Set, the BAR contains the base address for all (NumVFs) VF BAR_{*n*}.

The algorithm to determine the amount of address space mapped by a VF BAR_{*n*} differs from the standard BAR algorithm as follows:

1. Resize the BAR via the VF Resizable BAR Extended Capability (see § Section 7.8.7) if it is implemented.
2. After reading the low order bits to determine if the BAR is a 32-bit BAR or 64-bit BAR pair, determine the size and alignment requirements by writing all 1s to VF BAR_{*n*} (or VF BAR_{*n*} and VF BAR_{*n*+1} for a 64-bit BAR pair) and reading back the contents of the BAR or BAR pair. Convert the bit mask returned by the read(s) to a size and alignment value as described in § Section 7.5.1.2.1 . This value is the size and alignment for a single VF.
3. Multiply the value from step 2 by the value set in NumVFs to determine the total amount of space the BAR or BAR pair will map after VF Enable and VF MSE are Set.

For each VF BAR_{*n*} field, *n* corresponds to one of the VFs BAR spaces. § Table 9-8 shows the relationship between *n* and a Function's BAR.

Table 9-8 BAR Offsets

<i>n</i>	BAR Offset in a Type 0 Header
0	10h
1	14h
2	18h
3	1Ch
4	20h
5	24h

The contents of all VF BAR_{*n*} registers are indeterminate after System Page Size is changed.

9.3.3.15 VF Migration State Array Offset (Deprecated) (Offset 3Ch) §

VF Migration is associated with the now deprecated [MR-IOV]. New designs should hardwire this register to 0000 0000h.

9.3.4 PF/VF Configuration Space Header §

This section's material in previous versions of this specification has been integrated into § Section 7.5.1 .

9.3.5 PCI Express Capability Changes §

This section's material in previous versions of this specification has been integrated into § Section 7.5.3 .

9.3.6 PCI Standard Capabilities §

SR-IOV usage of PCI Standard Capabilities is described in § Table 9-9. Items marked n/a are not applicable to PFs or VFs.

Table 9-9 SR-IOV Usage of PCI Standard Capabilities

Capability ID	Description	PF Attributes	VF Attributes
00h	Null Capability	Base	Base
01h	PCI Power Management Interface	Base	Optional. See § Section 5.10 .
02h	AGP	n/a	n/a
03h	VPD	Base	Optional. See § Section 7.9.18 .
04h	Slot Identification	n/a	n/a
05h	MSI	Base	See § Section 9.2.1.4 .
06h	CompactPCI Hot Swap	n/a	n/a
07h	PCI-X	n/a	n/a
08h	HyperTransport	n/a	n/a
09h	Vendor-specific	Base	Base
0Ah	Debug Port	Base	Base
0Bh	CompactPCI Central Resource Control	n/a	n/a
0Ch	PCI Hot Plug	Base	n/a
0Dh	PCI Bridge Subsystem ID	n/a	n/a
0Eh	AGP 8x	n/a	n/a
0Fh	Secure Device	n/a	n/a
10h	PCI Express	Base	See § Section 9.3.5 .
11h	MSI-X	See § Section 9.2.1.4 .	See § Section 9.2.1.4 .
12h	Serial ATA Data/Index Configuration	Base	n/a
13h	Advanced Features	n/a	n/a
14h	Enhanced Allocation	Base	Must not implement.
15h	Flattening Portal Bridge (FPB)	n/a	n/a

9.3.7 PCI Express Extended Capabilities Changes §

SR-IOV usage of PCI Express Extended Capabilities is described in § Table 9-10. Items marked n/a are not applicable to PFs or VFs (e.g., for capabilities only present in Root Complexes, only present in Function 0, or only for managing a Port).

Table 9-10 SR-IOV Usage of PCI Express Extended Capabilities

Extended Capability ID	Description	PF Attributes	VF Attributes
0000h	Null Capability	Base	Base
0001h	Advanced Error Reporting Extended Capability (AER)	Base	See capability description
0002h	Virtual Channel Extended Capability (02h)	Base	Must not implement. See capability description
0003h	Device Serial Number Extended Capability	Base	See capability description
0004h	Power Budgeting Extended Capability	Base	Must not implement. See capability description
0005h	Root Complex Link Declaration Extended Capability	n/a	n/a
0006h	Root Complex Internal Link Control Extended Capability	n/a	n/a
0007h	Root Complex Event Collector Endpoint Association Extended Capability	n/a	n/a
0008h	Multi-Function Virtual Channel Extended Capability	Base	n/a; only present in Function 0
0009h	Virtual Channel Extended Capability (09h)	Base	Must not implement. See capability description
000Ah	RCRB Header Extended Capability	n/a	n/a
000Bh	Vendor-specific Extended Capability	Base	Base
000Ch	Deprecated; formerly used for the Configuration Access Correlation Extended Capability	n/a	n/a
000Dh	ACS Extended Capability	See capability description	See capability description
000Eh	ARI Extended Capability (ARI)	See capability description	See capability description
000Fh	ATS Extended Capability	See capability description	See capability description
0010h	SR-IOV Extended Capability	See capability description	Must not implement. See capability description
0011h	Deprecated; formerly used for the MR-IOV Extended Capability (MR-IOV)	n/a	n/a
0012h	Multicast Extended Capability	See capability description	See capability description
0013h	Page Request Extended Capability (PRI)	See capability description	See capability description
0014h	Reserved for AMD	Base	Base
0015h	Resizable BAR Extended Capability	Base	Must not implement. See capability description

Extended Capability ID	Description	PF Attributes	VF Attributes
0016h	Dynamic Power Allocation Extended Capability (DPA)	See capability description	Must not implement. See capability description
0017h	TPH Requester Extended Capability (TPH)	See capability description	See capability description
0018h	LTR Extended Capability	Base	n/a; only present in Function 0
0019h	Secondary PCI Express Extended Capability	Base	n/a; only present in Function 0
001Ah	PMUX Extended Capability	Base	n/a; only for managing a Port
001Bh	PASID Extended Capability	Base	See capability description
001Ch	Deprecated; formerly used for the LN Requester Extended Capability	n/a	n/a
001Dh	DPC Extended Capability	n/a; only for managing a Port	n/a; only for managing a Port
001Eh	L1 PM Substates Extended Capability	Base	n/a; only present in Function 0
001Fh	Precision Time Management Extended Capability (PTM)	Base	n/a; only for managing a Port
0020h	PCI Express over M-PHY Extended Capability (M-PCIe)	Base	n/a; only for managing a Port
0021h	FRS Queueing Extended Capability	n/a	n/a
0022h	Readiness Time Reporting Extended Capability	Base	See capability description
0023h	Designated vendor-specific Extended Capability	Base	Base
0024h	VF Resizable BAR Extended Capability	See capability description	Must not implement. See capability description
0025h	Data Link Feature Extended Capability	Base	n/a; only for managing a Port
0026h	Physical Layer 16.0 GT/s Extended Capability	Base	n/a; only for managing a Port
0027h	Lane Margining at the Receiver Extended Capability	Base	n/a; only for managing a Port
0028h	Hierarchy ID Extended Capability	Base	Base
0029h	Native PCIe Enclosure Management Extended Capability (NPEM)	Base	n/a; only for managing a Port
002Ah	Physical Layer 32.0 GT/s Extended Capability	Base	Must not implement
002Bh	Alternate Protocol Extended Capability	Base	Must not implement
002Ch	SFI Extended Capability (System Firmware Intermediary)	Base	Must not implement
002Dh	Shadow Functions Extended Capability	Base	Base
002Eh	Data Object Exchange Extended Capability	Base	Base
002Fh	Device 3 Extended Capability	Base	Base
0030h	IDE Extended Capability	Base	n/a; only present in Function 0

Extended Capability ID	Description	PF Attributes	VF Attributes
0031h	Physical Layer 64.0 GT/s Extended Capability	Base	Must not implement
0032h	Flit Logging Extended Capability	Base	Must not implement
0033h	Flit Performance Measurement Extended Capability	Base	Must not implement
0034h	Flit Error Injection Extended Capability	Base	Must not implement

10. Address Translation Services (ATS) §

10.1 ATS Architectural Overview §

Most contemporary system architectures make provisions for translating addresses from DMA (bus mastering) I/O Functions. In many implementations, it has been common practice to assume that the physical address space seen by the CPU and by an I/O Function is equivalent. While in others, this is not the case. The address programmed into an I/O Function is a “handle” that is processed by the Root Complex (RC). The result of this processing is often a translation to a physical memory address within the central complex. Typically, the processing includes access rights checking to insure that the DMA Function is allowed to access the referenced memory location(s).

The purposes for having DMA address translation vary and include:

- Limiting the destructiveness of a “broken” or misprogrammed DMA I/O Function
- Providing for scatter/gather
- Ability to redirect message-signaled interrupts (e.g., MSI or MSI-X) to different address ranges without requiring coordination with the underlying I/O Function
- Address space conversion (32-bit I/O Function to larger system address space)
- Virtualization support

Irrespective of the motivation, the presence of DMA address translation in the host system has certain performance implications for DMA accesses.

Depending on the implementation, DMA access time can be significantly lengthened due to the time required to resolve the actual physical address. If an implementation requires access to a main-memory-resident translation table, the access time can be significantly longer than the time for an untranslated access. Additionally, if each transaction requires multiple memory accesses (e.g., for a table walk), then the memory transaction rate (i.e., overhead) associated with DMA can be high.

To mitigate these impacts, designs often include address translation caches in the entity that performs the address translation. In a CPU, the address translation cache is most commonly referred to as a translation look-aside buffer (TLB). For an I/O TA, the term address translation cache or ATC is used to differentiate it from the translation cache used by the CPU.

While there are some similarities between TLB and ATC, there are important differences. A TLB serves the needs of a CPU that is nominally running one thread at a time. The ATC, however, is generally processing requests from multiple I/O Functions, each of which can be considered a separate thread. This difference makes sizing an ATC difficult depending upon cost models and expected technology reuse across a wide range of system configurations.

The mechanisms described in this specification allow an I/O Device to participate in the translation process and provide an ATC for its own memory accesses. The benefits of having an ATC within a Device include:

- Ability to alleviate TA resource pressure by distributing address translation caching responsibility (reduced probability of “thrashing” within the TA)
- Enable ATC Devices to have less performance dependency on a system’s ATC size
- Potential to ensure optimal access latency by sending pretranslated requests to central complex

This specification will provide the interoperability that allows PCIe Devices to be used in conjunction with a TA, but the TA and its Address Translation and Protection Table (ATPT) are treated as implementation specific and are outside the

scope of this specification. While it may be possible to implement ATS within other PCIe Components, this specification is confined to PCIe Devices and PCIe Root Complex Integrated Endpoints (RCiEPs).

§ Figure 10-1 illustrates an example platform with a TA and ATPT, along with a set of PCIe Devices and RC Integrated Endpoints with integrated ATC. A TA and an ATPT are implementation specific and can be distinct or integrated components within a given system design.

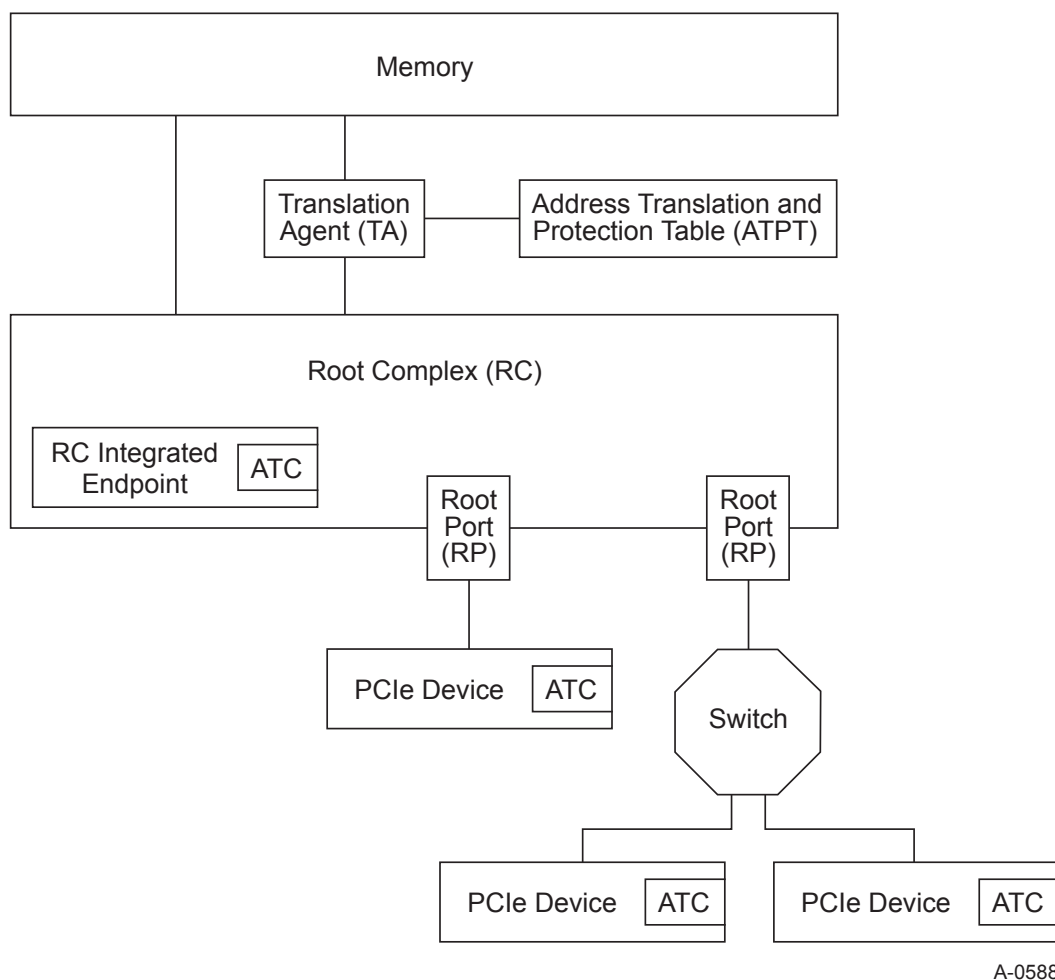


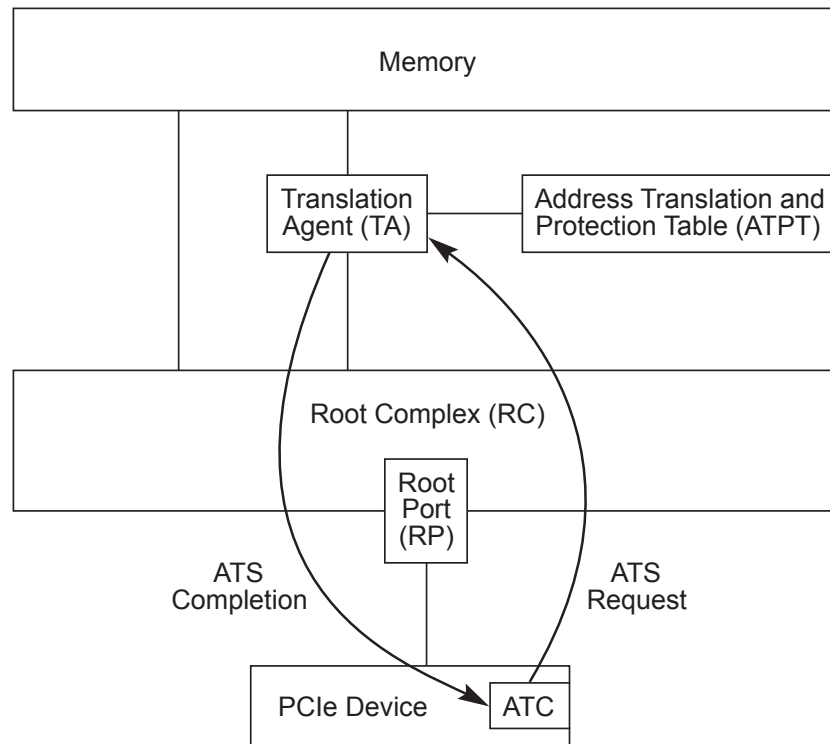
Figure 10-1 Example Illustrating a Platform with TA, ATPT, and ATC Elements

10.1.1 Address Translation Services (ATS) Overview §

The ATS chapter provides a new set of TLP and associated semantics. ATS uses a request-completion protocol between a Device¹⁷⁵ and a Root Complex (RC) to provide translation services. In addition, a new AT field is defined within the Memory Read and Memory Write TLP. The new AT field enables an RC to determine whether a given request has been translated or not via the ATS protocol.

§ Figure 10-2 illustrates the basic flow of an ATS Translation Request operation.

175. All references within this chapter to a Device apply equally to a PCIe Device or a Root Complex Integrated Endpoint. ATS does not delineate between these two types in terms of requirements, semantics, configuration, error handling, etc. From a software perspective, an ATS-capable Root Complex Integrated Endpoint must behave the same as an ATS-capable non-integrated Device.



A-0589

Figure 10-2 Example ATS Translation Request/Completion Exchange

In this example, a Function-specific work request is received by a single-Function PCIe Device. The Function determines through an implementation specific method that caching a translation within its ATC would be beneficial. There are a number of considerations a Function or software can use in making such a determination; for example:

- Memory address ranges that will be frequently accessed over an extended period of time or whose associated buffer content is subject to a significant update rate
- Memory address ranges, such as work and completion queue structures, data buffers for low-latency communications, graphics frame buffers, host memory that is used to cache Function-specific content, and so forth

Given the variability in designs and access patterns, there is no single criteria that can be applied.

The Function generates an ATS Translation Request which is sent upstream through the PCIe hierarchy to the RC which then forwards it to the TA. An ATS Translation Request uses the same routing and ordering rules as defined in this specification. Further, multiple ATS Translation Requests can be outstanding at any given time; i.e., one may pipeline multiple requests on one or more TC. Each TC represents a unique ordering domain and defines the domain that must be used by the associated ATS Translation Completion.

Upon receipt of an ATS Translation Request, the TA performs the following basic steps:

1. Validates that the Function has been configured to issue ATS Translation Requests.
2. Determines whether the Function may access the memory indicated by the ATS Translation Request and has the associated access rights.

3. Determines whether a translation can be provided to the Function. If yes, the TA issues a translation to the Function.
 - a. ATS is required to support a variety of page sizes to accommodate a range of ATPT and processor implementations.
 - i. Page sizes are required to be a power of two and naturally aligned.
 - ii. The minimum supported page size is 4096 bytes. ATS capable components are required to support this minimum page size.
 - b. A Function must be informed of the minimum translation or invalidate size it will be required to support to provide the Function an opportunity to optimize its resource utilization. The smallest minimum translation size must be 4096 bytes.
4. The TA communicates the success or failure of the request to the RC which generates an ATS Translation Completion and transmits via a Response TLP through a RP to the Function.
 - a. An RC is required to generate at least one ATS Translation Completion per ATS Translation Request; i.e., there is minimally a 1:1 correspondence independent of the success or failure of the request.
 - i. A successful translation can result in one or two ATS Translation Completion TLPs per request. The Translation Completion indicates the range of translation covered.
 - ii. An RC may pipeline multiple ATS Translation Completions; i.e., an RC may return multiple ATS Translation Completions and these ATS Translation Completions may be in any order relative to ATS Translation Requests.
 - iii. The RC is required to transmit the ATS Translation Completion using the same TC (Traffic Class) as the corresponding ATS Translation Request.
 - b. The requested address may not be valid. The RC is required to issue a Translation Completion indicating that the requested address is not accessible.

When the Function receives the ATS Translation Completion and either updates its ATC to reflect the translation or notes that a translation does not exist. The Function proceeds with processing its work request and generates subsequent requests using either a translated address or an untranslated address based on the results of the Completion.

- a. Similar to Read Completions, a Function is required to allocate resource space for each completion(s) without causing backpressure on the PCIe Link.
- b. A Function is required to discard Translation Completions that might be “stale”. Stale Translation Completions can occur for a variety of reasons.

As one can surmise, ATS Translation Request and Translation Completion processing is conceptually similar and, in many respects, identical to PCIe Read Request and Read Completion processing. This is intentional to reduce design complexity and to simplify integration of ATS into existing and new PCIe-based solutions. Keeping this in mind, ATS requires the following:

- ATS capable components must interoperate with [\[PCIe-1.1\]](#) compliant components.
- ATS is enabled through a new Capability and associated configuration structure. To enable ATS, software must detect this Capability and enable the Function to issue ATS TLP. If a Function is not enabled, the Function is required not to issue ATS Translation Requests and is required to issue all DMA Read and Write Requests with the TLP AT field set to “untranslated”.
- ATS TLPs are routed using either address-based or ID-based routing.
- ATS TLPs are required to use the same ordering rules as specified in this specification.
- ATS TLPs are required to flow unmodified through [\[PCIe-1.1\]](#) compliant Switches.
- A Function is permitted to intermix translated and untranslated requests.

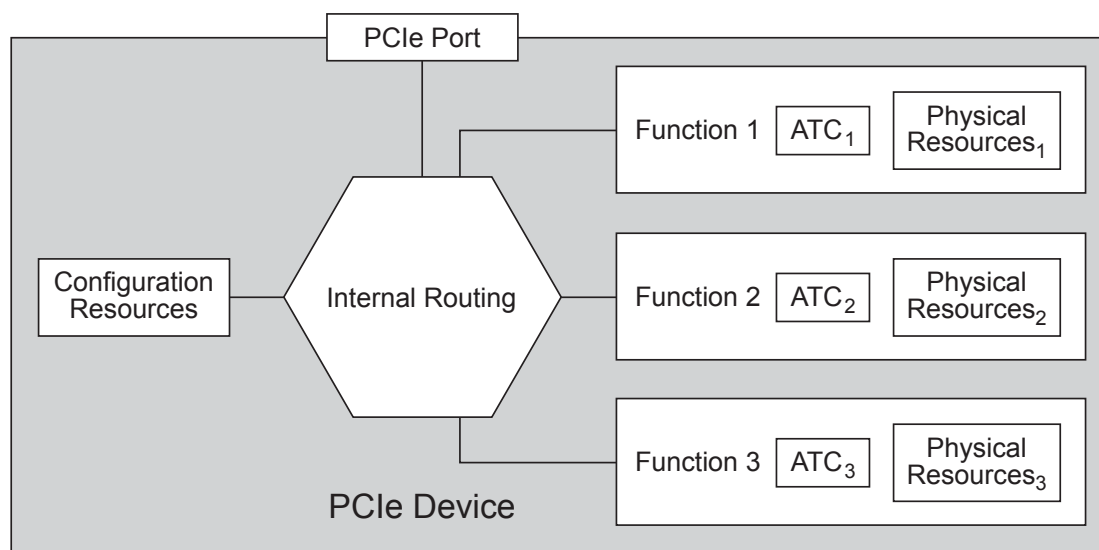
- ATS transactions are required not to rely upon the address field of a memory request to communicate additional information beyond its current use as defined by the PCI-SIG.

IMPLEMENTATION NOTE: ADDRESS RANGE OVERLAP §

It is likely that the untranslated and translated address range will overlap, perhaps in their entirety. This is not a requirement of ATS but may be an implementation constraint on the TA so that memory requests will be properly routed.

In contrast to the prior example, § Figure 10-3 illustrates an example Multi-Function Device. In this example Device, there are three Functions. Key points to note in § Figure 10-3 are:

- Each ATC is associated with a single Function. Each ATS-capable Function must be able to source and sink at least one of each ATS Translation Request or Translation Completion type.
- Each ATC is configured and accessed on a per Function basis. A Multi-Function Device is not required to implement ATS on every Function.
- If the ATC implementation shares resources among a set of Functions, then the logical behavior is required to be consistent with fully independent ATC implementations.



A-0592

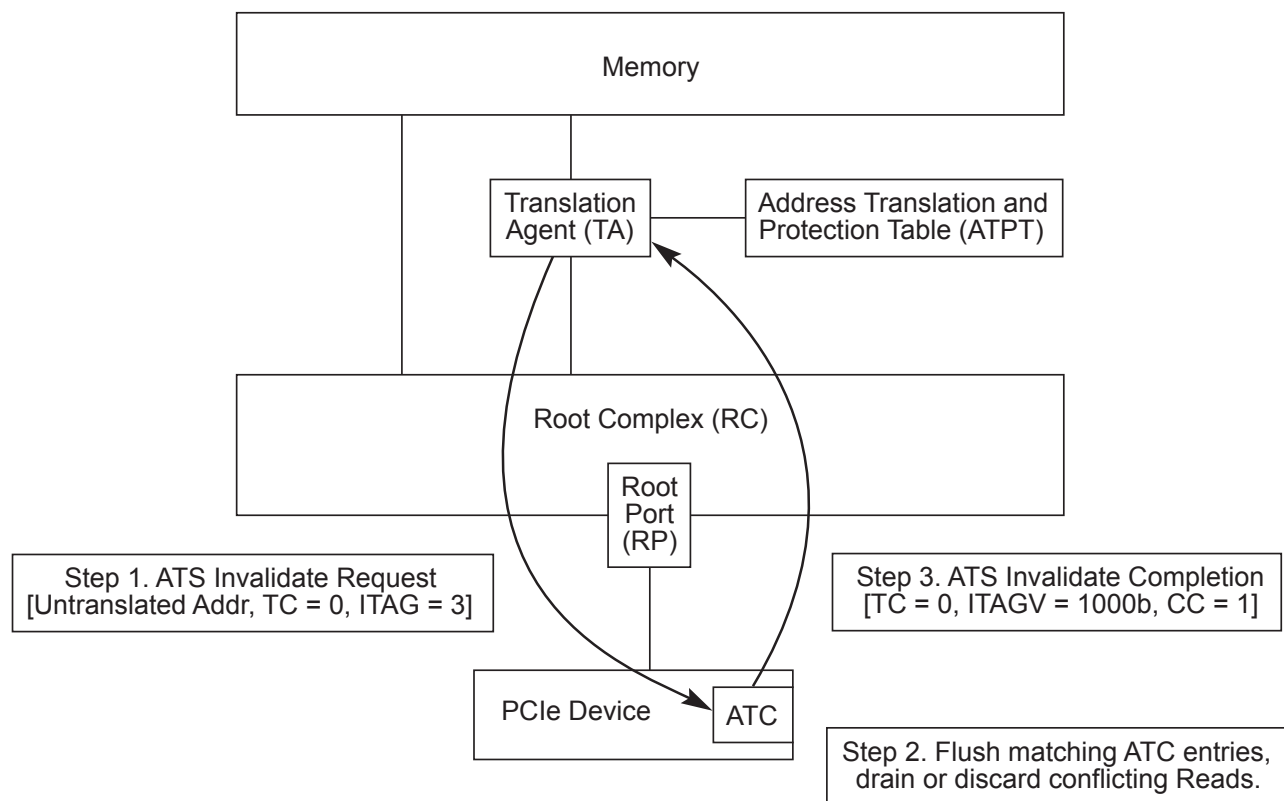
Figure 10-3 Example Multi-Function Device with ATC per Function

Independent of the number of Functions within a Device, the following are required:

- A Function is required not to issue any TLP with the AT field set unless the address within the TLP was obtained through the ATS Translation Request and Translation Completion protocol.

- Each ATC is required to only be populated using the ATS protocol; i.e., each entry within the ATC must be filled via an ATS Translation Completion in response to the Function issuing an ATS Translation Request for a given address.
- Each ATC cannot be modified except through the ATS protocol. That is:
 - Host system software cannot modify the ATC other than through the protocols defined in this specification except to invalidate one or more translations in an ATC. A Device or Function reset would be an example of an operation performed by software to change the contents of the ATC, but a reset is only allowed to invalidate entries not modify their contents.
 - It must not be possible for host system software to use software executing on the Device to modify the ATC.

When a TA determines that a Function should no longer maintain a translation within its ATC, the TA initiates the ATS invalidation protocol. The invalidation protocol consists of a single Invalidation Request and one or more Invalidate Completions.



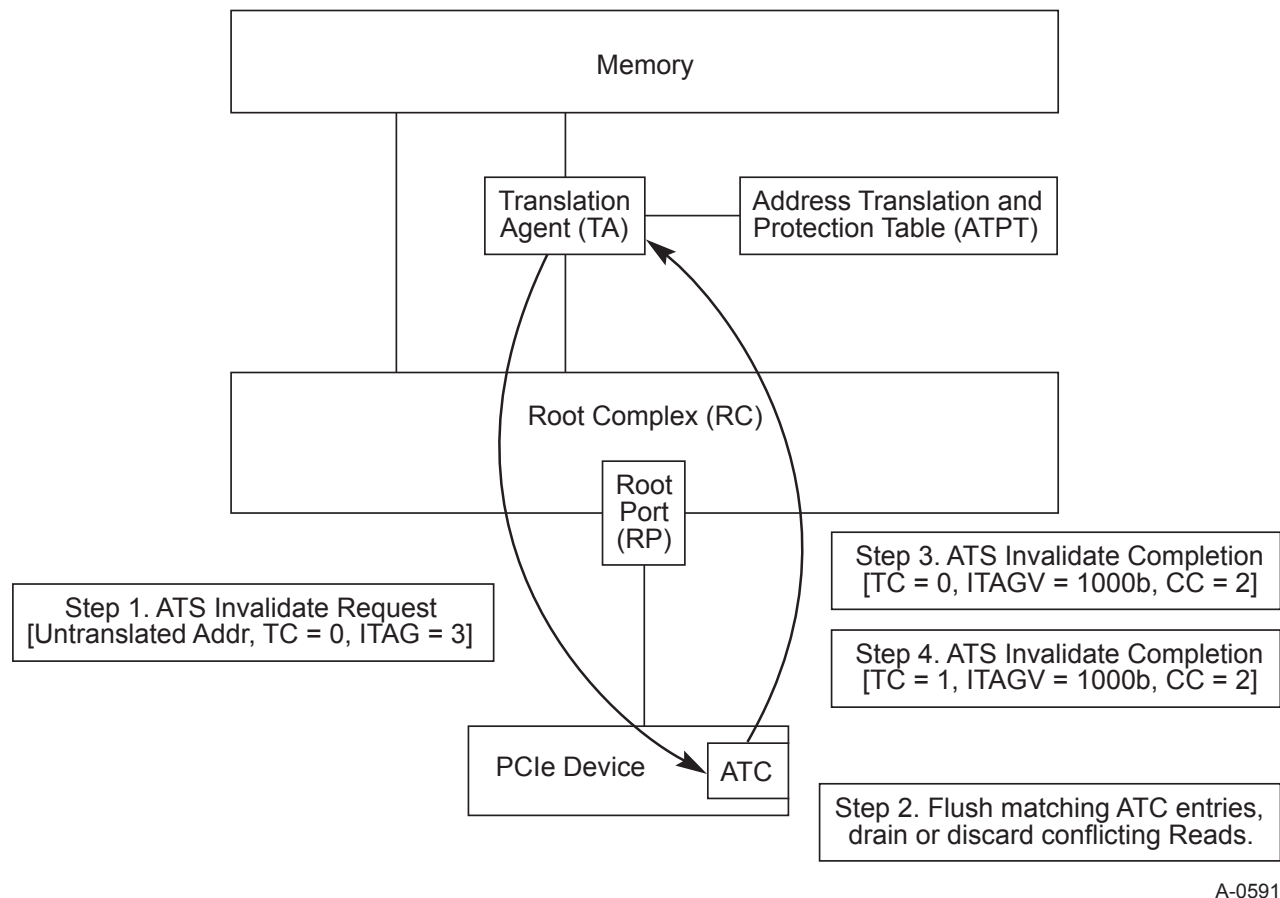
A-0590

Figure 10-4 Invalidation Protocol with a Single Invalidation Request and Completion

As § Figure 10-4 illustrates, there are essentially three steps in the ATS Invalidation protocol:

1. The system software updates an entry in the tables used by the TA. After the table is changed, the TA determines that a translation should be invalidated in an ATC and initiates an Invalidation Request TLP which is transmitted from the RP to the example single-Function Device. The Invalidate Request communicates an untranslated address range, the TC, and an RP unique tag which is used to correlate Invalidate Completions with the Invalidation Request.

2. The Function receives the Invalidate Request and invalidates all matching ATC entries. A Function is not required to immediately flush all pending requests upon receipt of an Invalidate Request. If transactions are in a queue waiting to be sent, it is not necessary for the Function to expunge requests from the queue even if those transactions use an address that is being invalidated.
 - a. A Function is required not to indicate the invalidation has completed until all outstanding Read Requests or Translation Requests that reference the associated translated address have been retired or nullified.
 - b. A Function is required to ensure that the Invalidate Completion indication to the RC will arrive at the RC after any previously posted writes that use the “stale” address.
3. When a Function has ascertained that all uses of the translated address are complete, it issues one or more ATS Invalidate Completions.
 - a. An Invalidate Completion is issued for each TC that may have referenced the range invalidated. These completions act as a flush mechanism to ensure the hierarchy is cleansed of any in-flight transactions which may contain references to the translated address.
 - i. The number of Completions required is communicated within each Invalidate Completion. A TA or RC implementation can maintain a counter to ensure that all Invalidate Completions are received before considering the translation to no longer be in use.
 - ii. If more than one Invalidation Complete is sent, the Invalidate Completion sent in each TC must be identical in the fields detailed in § Section 10.3.2.
 - b. An Invalidate Completion contains the ITAG from Invalidate Request to enable the RC to correlate Invalidate Requests and Completions.



A-0591

Figure 10-5 Single Invalidate Request with Multiple Invalidate Completions

10.1.2 Page Request Interface Extension §

ATS improves the behavior of DMA based data movement. An associated Page Request Interface (PRI) provides additional advantages by allowing DMA operations to be initiated without requiring that all the data to be moved into or out of system memory be pinned.¹⁷⁶ The overhead associated with pinning memory may be modest, but the negative impact on system performance of removing large portions of memory from the pageable pool can be significant.

PRI is functionally independent of the other aspects of ATS. That is, a device that supports ATS need not support PRI, but PRI is dependent on ATS's capabilities.

Intelligent I/O devices can be constructed to make good use of a more dynamic memory interface. Pinning will always have the best performance characteristics from a device's perspective—all the memory it wants to touch is guaranteed to be present. However, guaranteeing the residence of all the memory a device might touch can be problematic and force a sub-optimal level of device awareness on a host. Allowing a device to operate more independently (to page fault when it requires memory resources that are not present) provides a superior level of coupling between device and host.¹⁷⁷

The mechanisms used to take advantage of a Page Request Interface are very device specific. As an example of a model in which such an interface could improve overall system performance, let us examine a high-speed LAN device. Such a

¹⁷⁶. Locked in place so that it cannot be swapped out by the system's dynamic paging mechanism.

¹⁷⁷. The alternative is a private interface between a device and its driver that is used to communicate device state so that the driver can ensure the availability of pinned memory resources.

device knows its burst rate and need only have as much physical buffer space available for inbound data as it can receive within some quantum. A vector of unpinned virtual memory pages could be made available to the device, that the device then requests as needed to maintain its burst window. This minimizes the required memory footprint of the device and simplifies the interface with the host, both without negatively impacting performance.

The ability to page, begs the question of page table status flag management. Typical TAs associate flags (e.g., dirty and access indications) with each untranslated address. Without any additional hints about how to manage pages mapped to a Function, such TAs would need to conservatively assume that when they grant a Function permission to read or write a page, that Function will use the permission. Such writable pages would need to be marked as dirty before their translated addresses are made available to a Function.

This conservative dirty-on-write-permission-grant behavior is generally not a significant issue for Functions that do not support paging, where pages are pinned and the cost of saving a clean page to memory will seldom be paid. However, Functions that support the Page Request Interface could pay a significant penalty if all writable pages are treated as dirty, since such Functions operate without pinning their accessible memory footprints and may issue speculative page requests for performance. The cost of saving clean pages (instead of just discarding them) in such systems can diminish the value of otherwise attractive paging techniques. This can cause significant performance issues and risk functional issues in circumstances where the backing store is unable to be written, such as a CD-ROM.

The No Write (NW flag in Translation Requests indicates that a Function is willing to restrict its usage to only reading the page, independent of the access rights that would otherwise have been granted.

If a device chooses to request only read access by issuing a Translation Request with the NW flag Set and later determines that it needs to write to the page, then the device must issue a new Translation Request.

Upon receiving a Translation Request with the NW flag Clear, TAs are permitted to mark the associated pages dirty. Functions *MUST@FLIT* not issue such Requests unless they have been given explicit write permission. An example of write permission is where the host issues a command to a Function to load data from a storage device and write that data into memory.

10.1.3 Process Address Space ID (PASID) §

Certain TLPs can optionally be associated with a Process Address Space ID (PASID). This value is conveyed using the PASID TLP Prefix (NFM) or OHC-A1 (FM). The PASID TLP Prefix is defined in [§ Section 6.20](#).

PASID is permitted for the following types of TLPs:

- Memory Requests (including AtomicOp Requests) with Address Type (AT) of Untranslated or Translated
- Address Translation Requests (i.e., MRd with AT=01b)
- Page Request Messages
- ATS Invalidation Request Messages
- PRG Response Messages

Usage of PASID for Untranslated Memory Requests is defined in [§ Section 6.20](#). This section describes PASID for the remaining TLPs.

Each Function has an independent set of PASID values. The PASID field is 20 bits wide, although typically the number of active PASID values will be significantly lower than 2^{20} . PASID values may be allocated sparsely. If the usable width is constrained by the TA or the ATC, the unused upper bits of the PASID value must be 0b. All $2^{\text{usable_width}}$ PASID values are usable. Function hardware is not permitted to assume there are “reserved” PASID values.

An ATC may optionally support Translated Requests with PASID. The Translated Requests with PASID feature is enabled when the Translated Requests with PASID Enable bit is Set. When Translated Requests with PASID Enable is Set, an ATC is permitted to issue Translated Requests with a PASID. When enabled, if the ATC obtained a translation using a Translation

Request with PASID, the corresponding Translated Request must carry the same PASID, and also the values for the Execute Requested and Privileged Mode Requested bits must match the Execute Permitted and Privileged Mode Access bit values, respectively, returned with the translation. Similarly, if the ATC obtained a translation using a Translation Request without a PASID, the corresponding Translated Request must not carry a PASID. If these rules are not followed the resulting system behavior is undefined.

10.1.4 ATS Memory Attributes §

ATS Memory Attributes (AMAs) provide hints for performing memory operations such as cache management. The AMAs may be supplied to an Endpoint device by a Translation Completion and stored in the ATC. If the ATS Memory Attributes Enable bit is Set, the Endpoint function retrieves the AMAs from the ATC and is permitted to provide AMAs with Memory Read/Write TLPs using the TPH TLP Prefix. This serves as a performance optimization by preventing the TA from having to perform a look-up. The TA and ATC are permitted to support AMAs. If the ATC does not support AMAs but the TA does, then the TA is permitted to perform a lookup to obtain AMAs when processing Memory Read/Write TLPs received from the Endpoint device. A TA that supports AMAs is permitted to always return AMAs in all ATS Translation Completions to all Endpoints.

10.2 ATS Translation Services §

A TA does translations. An ATC can cache those translations. If an ATC is separated from the TA by PCIe, the memory request from an ATC will need to be able to indicate if the address in the transaction is translated or not. The modifications to the memory transactions are described in this section, as are the transactions that are used to communicate translations between a remote ATC and a central TA.

When Shadow Functions are enabled, the TA must treat identically all Requests from the “main” Function and its Shadows (See Section 7.9.21). This usually involves ensuring that system software configures the translation tables used by the TA provide identical answers for the “main” Function and its Shadows. Depending on the architecture of a specific TA, more work may be required (e.g., managing dirty bits, ensuring caches are consistent, etc.). Such work is outside the scope of this specification.

10.2.1 Memory Requests with Address Type §

A Function with an ATC can send Memory Requests that contain either translated or untranslated addresses. The Address Type (AT) field is used to indicate the type of address that is present in the request header (see [§ Figure 10-6](#), [§ Figure 10-7](#), [§ Figure 10-8](#), and [§ Figure 10-9](#)).

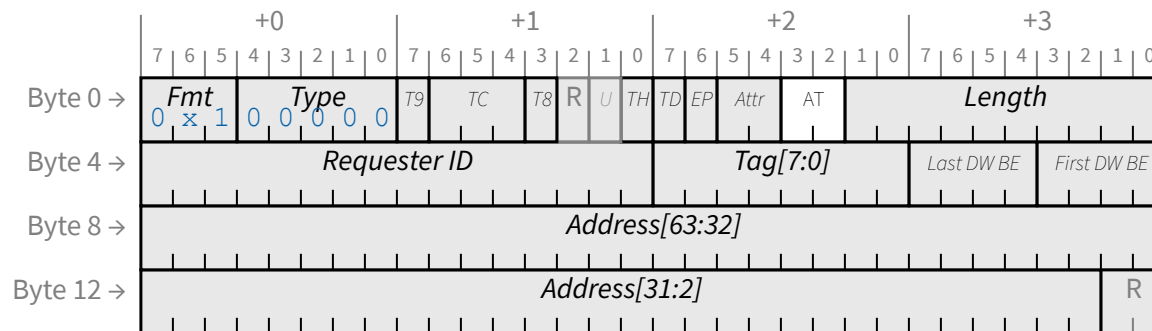


Figure 10-6 Memory Request Header with 64-bit Address Highlighting AT Field

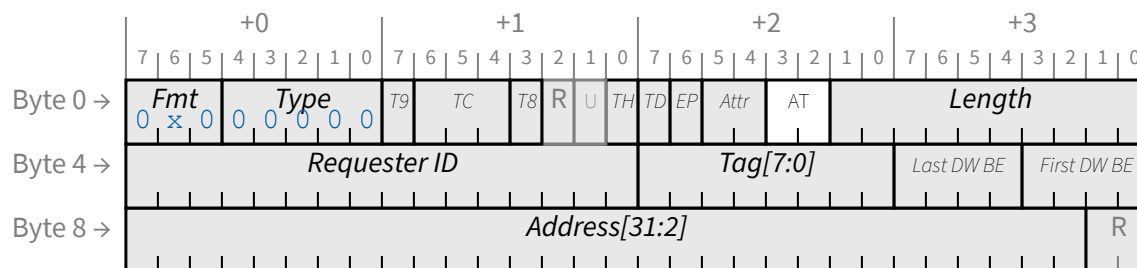


Figure 10-7 Memory Request Header with 32-bit Address Highlighting AT Field

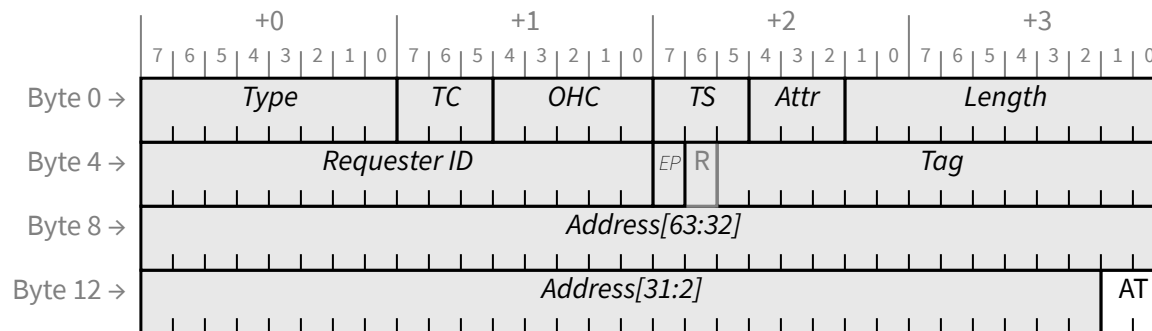


Figure 10-8 Memory Request Header with 64-bit Address Highlighting AT Field - FLIT Mode

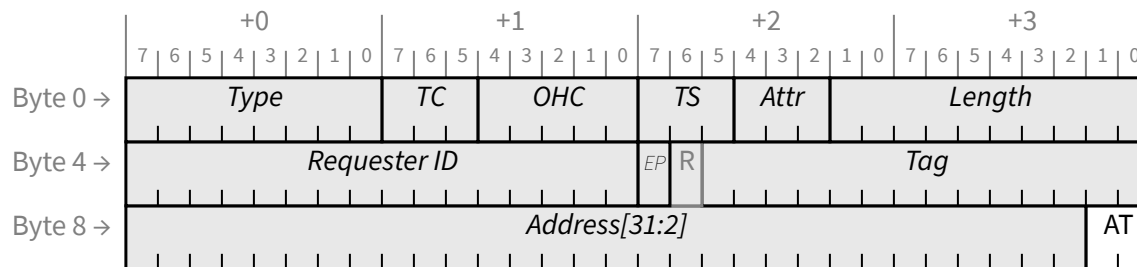


Figure 10-9 Memory Request Header with 32-bit Address Highlighting AT Field - FLIT Mode

In NFM, the AT field in the Requests is a redefinition of a reserved field in earlier version of this specification.

Functions that do not implement an ATC will continue to set the AT field to its defined reserved value (00b). Functions that implement an ATC will set the AT field as listed in § Table 10-1.

Table 10-1 Address Type (AT) Field Encodings

AT[1:0] Coding	Mnemonic	Meaning
00b	Untranslated	A TA may treat the address as either virtual or physical.
01b	Translation Request	The TA will return the translation of the address contained in the address field of the request as a read completion. This value only has meaning for an explicit Translation Request (see § Section 10.2.2). The TA will signal an Unsupported Request (UR) if it receives a TLP with the AT field set to 01b in a Memory Request other than Memory Read.
10b	Translated	The address in the transaction has been translated by an ATC. If the Function associated with the SourceID is allowed to present physical addresses to the system memory, then the TA might not translate this address. If the Function is not allowed to present physical addresses, then the TA may treat this as an UR.
11b	Reserved	The TA will signal an Unsupported Request (UR) if it receives a Memory Request TLP with the AT field set to 11b.

The AT field is only defined for Memory Requests. The field remains reserved for other TLPs.

10.2.2 Translation Requests §

A Translation Request has a format that is similar to that of a memory read. The AT field must be set to the value defined for "Translation Request" to differentiate a Translation Request from a normal Memory Read (see § Figure 10-10, § Figure 10-11, § Figure 10-12, and § Figure 10-13). In Flit Mode, OHC-A1 must be included, and the No Write (NW) flag is contained in OHC-A1.

Translation Requests have the same completion timeout intervals as Read Requests.

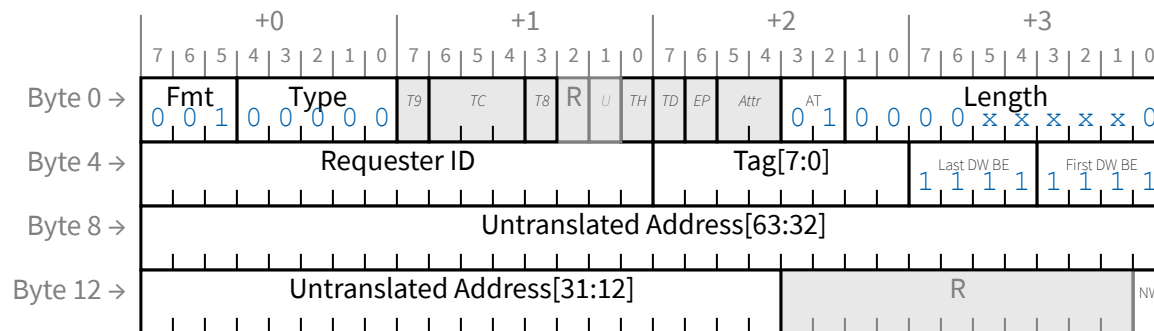


Figure 10-10 Translation Request with 64-bit Address - Non-Flit Mode

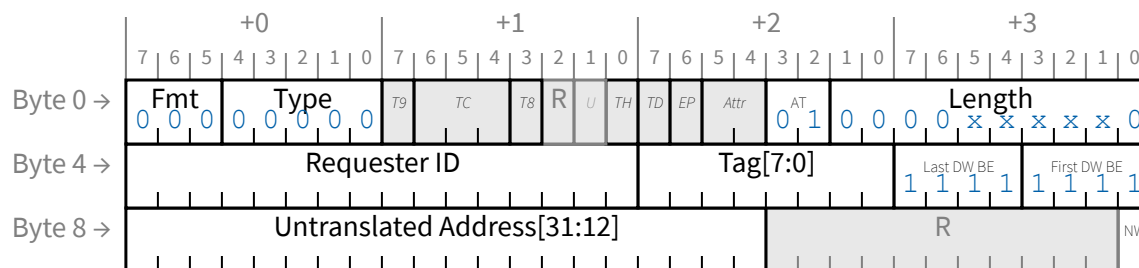
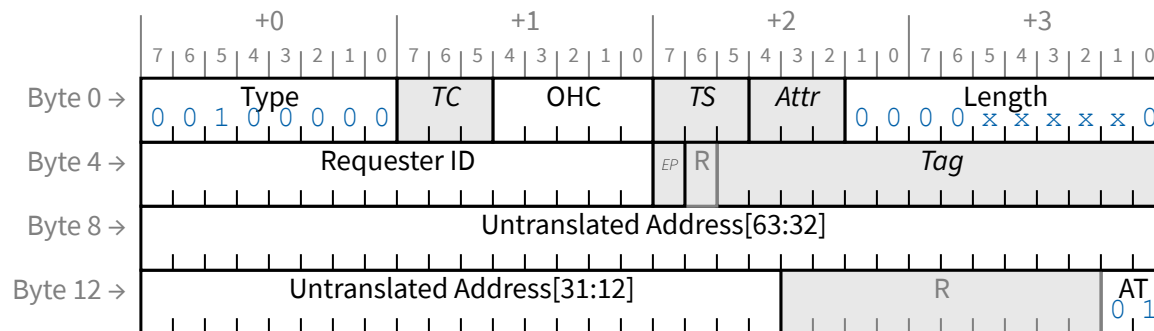


Figure 10-11 Translation Request with 32-bit Address - Non-Flit Mode

Figure 10-12 Translation Request with 64-bit Address - Flit Mode¹⁷⁸

178. The NW bit is located in OHC-A1.

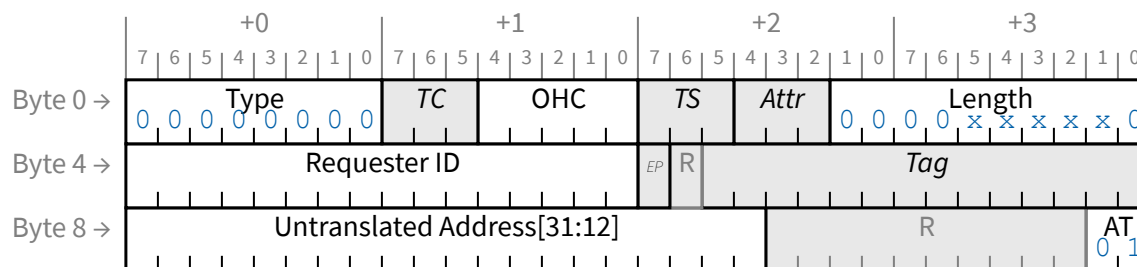


Figure 10-13 Translation Request with 32-bit Address - Flit Mode¹⁷⁹

10.2.2.1 Attribute Field §

For a Translation Request, the Relaxed Ordering (RO) bit is applicable and permitted to be Set, where it affects the ordering of its associated Translation Completions. The remainder of the Attr field is Reserved. The Requester of a Translation Request must not depend on the TA to guarantee any specific ordering relationship between Translation Completions and any other Requests or Completions. There are no ordering requirements for a Translation Request. A TA may reorder a Translation Request with respect to any other request.

IMPLEMENTATION NOTE: TRANSLATION REQUEST ORDERING §

Because no ordering can be assumed between Translation Requests and other types of Requests, a Translation Request does not make an effective flushing/ordering primitive.

10.2.2.2 Length Field §

The Length field is set to indicate how many translations may be returned in response to this request. Each translation is 8 bytes in length and represents one or more STUs (Smallest Translation Unit). The maximum setting for the Length field is the RCB of the Root Port as determined by Read Completion Boundary (RCB) in the Link Control Register. The Length field in a Translation Request must always indicate an even number of DWORDs. If Length is set to indicate a value greater than allowed, or if the least-significant bit of the Length field is non-zero, then the TA will treat the request as a Malformed Packet.

If the Length field has a value greater than two, then the Function is requesting translations for a range of memory greater than a single STU. The additional translations, if provided, are assumed to be for sequentially-increasing, equal-sized, STU-aligned regions, starting at the requested address.

10.2.2.3 Tag Field §

The Tag field has the same meaning as in a Memory Read Request.

¹⁷⁹ The NW bit is located in OHC-A1.

10.2.2.4 Untranslated Address Field §

A Translation Request includes either a 32-bit or a 64-bit Untranslated Address field. This field indicates the address to be translated. The TA will make decisions about the validity of the request, based on the address in the translation request. The TA is permitted to return fewer translations than requested, but it will not return more.

When multiple translations are requested, the TA will not return a translation if the range of that translation does not overlap the implied range of the Translation Request (this would only apply to translations after the initial value). The implied range of the Translation Request is $[2^{\text{STU}+12} * (\text{Length}/2)]$ bytes.

The Untranslated Address field in the Translation Request is any address in the range of the first STU. Address bits 11:0 are not present in the Translation Request and are implied to be zero. If a Requester has Page Aligned Request Set (see § Section 7.8.9.2), it must ensure that bits 11:2 are zero. If a Requester has Page Aligned Request Clear, it is permitted to supply any value for bits 11:2.¹⁸⁰ The TA must ignore bits 11:2 as well as any low-order bits not required to determine the translation.

For example, if using 64-bit addressing for a Function with the Page Aligned Request bit Set that is programmed with an STU of 1 (i.e., 8192-byte pages), bits 63:13 are significant, bit 12 is ignored by the TA and bits 11:0 are implied to be zero.

10.2.2.5 No Write (NW) Flag §

The No Write flag, when Set, indicates that the Function is requesting read-only access for this translation.¹⁸¹

The TA may ignore the No Write Flag, however, if the TA responds with a translation marked as read-only then the Function must not issue Memory Write transactions using that translation. In this case, the Function may issue another translation request with the No Write flag Clear, which may result in a new translation completion with or without the W (Write) bit Set.

Upon receiving a Translation Request with the NW flag Clear, TAs are permitted to mark the associated pages dirty. Functions *MUST@FLIT* not issue such Requests unless they have been given explicit write permission.

In Flit Mode, the NW bit is part of OHC-A1.

10.2.2.6 PASID on Translation Request §

If a Translation Request has a PASID, the Untranslated Address Field is an address within the process address space indicated by the PASID field.

If a Translation Request has a PASID with either the Privileged Mode Requested or Execute Requested bit Set, these may be used in constructing the Translation Completion Data Entry.

The PASID Extended Capability indicates whether a Function supports and is enabled to send and receive TLPs with the PASID.

10.2.3 Translation Completion §

A Translation Completion (either a Cpl or a CplD) is sent by a TA for each Translation Request. This specification describes the meaning of fields in Translation Completions. Fields not defined in this chapter have the same meanings proscribed for Read Completions in Chapter 2. For a Translation Completion, the Relaxed Ordering (RO) bit is applicable

180. Note: The Page Aligned Request bit was added in Revision 1.1 of the ATS Specification.

181. Note: The No Write Flag was added in Revision 1.1 of the ATS Specification.

and permitted to be Set if the corresponding Translation Request RO bit was set. The remainder of the Attr field is Reserved.

If the TA was not able to perform the requested translation, a Completion with no data (Cpl) must be returned.

IMPLEMENTATION NOTE: BYTE COUNT FIELD FOR UNSUCCESSFUL TRANSLATION COMPLETIONS WITH NO DATA §

Previous versions of this specification indicated the Byte Count and Lower Address field should be 0000 0000 0000b for Unsuccessful Translation Completions with No Data. It is strongly recommended that implementations do not depend on the Byte Count and Lower Address field being set to any particular value in Unsuccessful Translation Completions with No Data.

Note that in Flit Mode, the Byte Count and Lower Address Fields are not present.

The values and meaning for the Completion Status field are listed in § Table 10-2, where return values other than *Success* indicate an error.

Table 10-2 Translation Completion with No Data Status Codes

Value	Status	Meaning
000b	Success	This Completion Status has a nominal meaning of “success”. The TA will not return this value in a Cpl.
001b	Unsupported Request (UR)	Translation Requests from this Function are not supported by the TA. If a Function receives this Completion code, it must disable its ATC and not send requests using translated addresses until the ATC is re-enabled. For transactions the Function may internally have in flight, the Function may either terminate or complete them. The mechanism a Function receiving this code uses to report this condition is outside the scope of this specification. The TA detecting this error is a "Completer Sending a Completion with UR/CA Status" and shall behave as defined in this specification.
010b	RRS	This value is not allowed in any Completion to a Translation Request initiated by a PCI Express Function. If received by a Function, it shall be treated as a Malformed TLP.
100b	Completer Abort (CA)	The TA was not able to translate the address because of an error in the TA. This nominally causes an error to be reported to the device driver associated with the ATC. See AER in this specification.
All others	Reserved	A Translation Completion with a Reserved Completion Status value is treated as if the Completion Status was Unsupported Request (001b).

Completion header fields must be set in accordance with § Section 2.2.9 and § Section 2.3 .

Translation Completions must be sent using the same TC as the Translation Request. The Function is not required to verify that the same TC was used. A TA may optionally copy the RO bit of a Translation Request to the Translation Completion in accordance to the rule specified for the attribute field of completions in § Section 2.2.9 . It is strongly recommended that the TA copy the RO bit. However, if a TA does not copy the RO bit of a Translation Request to the Translation Completion, the TA must clear the RO bit in the Translation Completion.

A Translation Completion with RO Set must follow the ordering rules for Relaxed Ordered Completions as specified in § Section 2.4.1 . A Function that initiates a Translation Request with the RO bit Set but receives the associated Translation Completion with the RO bit Clear is permitted to order associated Translation Completions as if the RO bit is Set.

A Function that initiates a Translation Request with the RO bit Set, must not report an error if the associated Translation Completion has the RO bit Clear.

IMPLEMENTATION NOTE:
ATTRIBUTE FIELD COMPATIBILITY IN TRANSLATION
COMPLETIONS

§

Some implementations of TA may not copy the Attribute field from the request to the completion as required by § Section 2.2.9 , since the Attr field is defined as Reserved in previous versions of this specification. Therefore, the following situations may occur and are handled as follows:

- A TA that does not copy the RO bit (as is typically done for completions as indicated in § Section 2.2.9) by forcing RO to 0 is coupled with a Function conforming to this specification that allows RO to be Set in the request. The Function will accept a Translation Completion with RO Clear and not log an error.
- A TA that conforms to the Attr copy rule (in § Section 2.2.9) is coupled with a Function that does not support RO in Translation Requests. Translation Completions will return with RO Clear as the Function expects.

Therefore, the use of RO is made fully backward compatible. However, it is strongly recommended that the TA support the copy of the RO bit conforming to the rules in § Section 2.2.9 .

The Lower Address field contains a computed value that makes the packet consistent with RCB semantics. If the result is returned in a single packet, Lower Address contains RCB minus Byte Count. If the results are returned in two packets, Lower Address for the first packet contains RCB minus (Total Completion Length * 4) Lower Address for subsequent packets contains 000 0000b. See § Section 10.2.4 for additional requirements for multiple packet completions.

If the Completion Status field is 000b, then the translation was successful and a data payload will follow the header. The contents of the data payload are shown in § Figure 10-14.

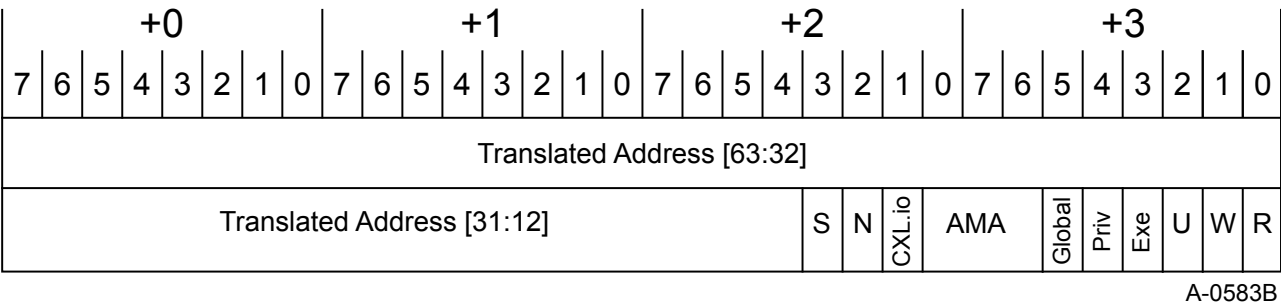


Figure 10-14 Translation Completion Data Entry

Table 10-3 Translation Completion Data Fields

Field	Meaning
S	Size of translation - This field is 0b if the translation applies to a 4096-byte range of memory. If this field is 1b, then the translation applies to a range of memory that is larger than 4096 bytes (see § Section 10.2.3.1).

Field	Meaning								
N	Non-snooped accesses - If this field is 1b, then the read and write requests that use this translation must Clear the No Snoop bit in the Attribute field. If it is 0b, then the Function may use other means to determine if No Snoop should be Set.								
CXL.io	This bit is Reserved for use by [CXL]. In non-CXL systems, this bit is Reserved.								
AMA	ATS Memory Attributes (AMAs) – If the ATS Memory Attributes Supported bit is Set, the ATC is permitted to cache this AMA value. <table> <tr> <th>AMA</th><th>ATS Memory Attributes</th></tr> <tr> <td>000b - 111b</td><td>ATS Memory Attribute values (implementation specific). The default value is 000b.</td></tr> </table>	AMA	ATS Memory Attributes	000b - 111b	ATS Memory Attribute values (implementation specific). The default value is 000b.				
AMA	ATS Memory Attributes								
000b - 111b	ATS Memory Attribute values (implementation specific). The default value is 000b.								
Global	Global Mapping - If this bit is Set, the ATC is permitted to cache this mapping entry in all PASIDs. If Clear, the ATC is permitted to cache this mapping entry only in the PASID associated with the requesting PASID. This bit may only be Set if the associated Translation Request had a PASID.								
Exe	Execute Permitted - If this bit is Set, the requesting Function is permitted to execute code contained in the associated memory range. This bit may be Set only if the associated Translation Request had the Execute Requested bit Set. If this bit is Set, R must also be Set. The Priv bit indicates the Privilege level associated with the Exe bit. If Priv is Set, the Exe bit indicates permissions associated with Privileged Mode entities in the Function. If Priv is Clear, the Exe bit indicates permissions associated with Non- Privileged Mode entities in the Function. This value may be cached if R is Set.								
Priv	Privileged Mode Access - If this bit is Set, R, W and Exe refer to permissions associated with Privileged Mode entities. If this bit is Clear, R, W and Exe refer to permissions associated with Non-Privileged Mode entities. This bit may only be Set if the associated Translation Request had the Privileged Mode Requested bit Set. This value must be cached any of the R, W or Exe values are cached.								
U	Untranslated access only - When this field is Set, the indicated range may only be accessed using untranslated addresses, and the Translated Address field of this Translation Completion Data Entry may not be used in a subsequent Read/Write Request with AT set to Translated. This value may be cached if R or W is Set.								
R,W	Read, Write - These two fields indicate the transaction types that are allowed for requests using the translation. The encodings are: <table> <tr> <td>00b</td><td>Neither read nor write transactions are allowed. This translation is considered not to be valid. The contents of the Translated Address, N, AMA, U, and Exe fields are undefined. A translation with this value must not be cached in the ATC (see § Section 10.2.3.5).</td></tr> <tr> <td>01b</td><td>Write Requests that target this range are allowed, but Read Requests are not unless they are zero-length reads.</td></tr> <tr> <td>10b</td><td>Read Requests that target this range are allowed (including zero-length reads), but Write Requests are not.</td></tr> <tr> <td>11b</td><td>Read and Write Requests that target this range are allowed.</td></tr> </table> The Priv bit indicates the Privilege level associated with R and W. If Priv is Set, R and W indicate permissions associated with Privileged Mode entities in the Function. If Priv is Clear, R and W indicate permissions associated with Non- Privileged Mode entities in the Function.	00b	Neither read nor write transactions are allowed. This translation is considered not to be valid. The contents of the Translated Address, N, AMA, U, and Exe fields are undefined. A translation with this value must not be cached in the ATC (see § Section 10.2.3.5).	01b	Write Requests that target this range are allowed, but Read Requests are not unless they are zero-length reads.	10b	Read Requests that target this range are allowed (including zero-length reads), but Write Requests are not.	11b	Read and Write Requests that target this range are allowed.
00b	Neither read nor write transactions are allowed. This translation is considered not to be valid. The contents of the Translated Address, N, AMA, U, and Exe fields are undefined. A translation with this value must not be cached in the ATC (see § Section 10.2.3.5).								
01b	Write Requests that target this range are allowed, but Read Requests are not unless they are zero-length reads.								
10b	Read Requests that target this range are allowed (including zero-length reads), but Write Requests are not.								
11b	Read and Write Requests that target this range are allowed.								

Field	Meaning
	If AMA is enabled (AMAE bit is 1b), a zero-length Read Request using a translated address must include AMAs in the TPH TLP Prefix.

10.2.3.1 Translated Address Field §

If the R and W fields are both Clear, or if U is Set, then the Translated Address field may not be used by the Function for any purpose.

If either the R or W field is Set, and the U field is Clear, then the Translated Address field contains an address that can be used by the Function in a Memory Request with the AT field set to Translated and the Function may cache the Translated Address. When cached, the R and W fields must be stored with the same value as the Translation Completion entry. The address that is cached must be a subset of the address range indicated in the Translation Completion (the subset may include the entire range).

While the Translated Address is cached in the Function's ATC, it shall not be possible for the Function to modify the entry other than to delete it. The entry must be deleted from the ATC when an Invalidation Request is received that has an indicated range that overlaps any portion of the cached address.

A Function is not allowed to make an entry into its ATC unless the entry is in a Translation Completion and the E (Enable) field within the ATS Capability is Set. Entries in an ATC cache that are written before the E field is Set must not be used in Memory Request. They must either be invalidated when the E field is Set or ignored and not used.

10.2.3.2 Translation Range Size (S) Field §

If S is Set, then the translation applies to a range that is larger than 4096 bytes. If S = 1b, then bit 12 of the Translated Address is used to indicate whether or not the range is larger than 8192 bytes. If bit 12 is 0b, then the range size is 8192 bytes, but it is larger than 8192 bytes if Set. If S = 1b and bit 12 = 1b, then bit 13 is used to determine if the range is larger than 16384 bytes or not. If bit 13 is 0b, then the range size is 16384 bytes, but it is larger than 16384 bytes if Set.

Low-order address bits are consumed in sequence to indicate the size of the range associated with the translation.

Note: This encoding method is also used to indicate the size of the memory range being invalidated.

Examples for different translation sizes are shown in § Table 10-5.

Table 10-5 Examples of Translation Size Using S Field

Address Bits																					S	Translation Range Size in Bytes	
63:32 *	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12			
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	4 K	
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	1	8 K
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	1	1	16 K	
x	x	x	x	x	x	x	x	x	x	x	x	0	1	1	1	1	1	1	1	1	1	2 M	
x	x	x	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1 G	
x	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	4 G	

Address Bits																			S	Translation Range Size in Bytes
63:32 *	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12

Note:

* Upper address bits are used to indicate the size for ranges larger than 4 GB.

The size field is set to indicate the range size in multiples of 4096 bytes regardless of the setting of STU. For example, if STU is set to indicate that the minimum translation is 8192 bytes, then S should be Set on all translation returned in a Translation Completion and in all Invalidate Requests. If STU is set to indicate a 16384-byte minimum, then S and bit 12 would both be Set in all translation and invalidate ranges.

If S is Set and bits 63:12 are all 1b, then the behavior is undefined. If S is Set and bit 63 is 0b, and bits 62:12 are all 1b, then the request is to invalidate all translations.

If a Function receives a Translation Completion with a Translation Size field smaller than the Function's programmed STU value, it shall treat the Translation Completion as if it had Completion Status UR.

10.2.3.3 Non-snooped (N) Field §

This field is Set to indicate that Read and Write Requests that target memory in the range of this translation must Clear the No Snoop Attribute bit in the Request header. When this field is 0b, the Function is allowed to Set the No Snoop Attribute bit in a Function-specific manner.

Note: When this field is Cleared, the Function is not allowed to Set No Snoop in a Memory Request if the Enable No Snoop field in the Device Control register is Cleared.

The N bit may be cached by the ATC if either R or W is Set.

When U is Set, the meaning of this field is undefined, and the TA may set this field to any value. A translation has a single value for the N field that is not affected by privilege level. An ATC is permitted to cache the N field without regard to the value of the Priv bit.

10.2.3.4 Untranslated Access Only (U) Field §

This field is Set when the Function is not allowed to access the implied range of memory using a translated address (the range is implied by the untranslated address in the Translation Request and the offset of the translation in the Translation Completion). The Function may use untranslated addresses to access the range as long as the accesses are allowed by the R and W fields. The Function may cache this translation value if either R or W is Set. If the U field is Set, the Translated Address field in the translation is not necessarily a valid memory address and the Function may not use the value in a Read or Write Request with AT set to Translated.

Note: One of the possible uses of this field is to avoid unnecessary invalidations. If a Function uses translated requests for some portions of memory, but not others, then the U field can be used on the portions for which translated requests are not used. When a translation changes if the U field is Set, then it will not necessarily be required that an Invalidate Request be sent to the Function. An example of this use is a Function with a ring buffer that is used for commands. The ring buffer may be allocated for a long period of time and have very high re-use (locality). For this reason, it is useful for the Function to use translated addresses in its memory request that target the command buffer. The same Function might access data buffers that have poor locality and low reuse. Accesses to the data buffers might best be handled by using untranslated Requests. Setting the U field for the data buffer translations ensures that the Function will not attempt to use a translated value to access the data buffer so, when the data buffer mappings are changed, no Invalidation Request is required. When U is Set and either R or W is Set, the ATC is permitted to cache U, R, W Exe, and Priv, as well as the Translation Range Size (see § Section 10.2.3.2). An Invalidation Request is required if these values change.

10.2.3.5 Read (R) and Write (W) Fields §

These fields indicate if the returned translation value may be used in a read or write memory request. The ATC is not permitted to issue a non-zero length read request using the translation value if the R field is Cleared. The ATC is not permitted to issue a write request using the translation value if the W field is Cleared. The ATC may not issue any type of request using the translation value if neither the R nor W fields are Set. If both R and W fields are Cleared, the range of the translation is still indicated, but the meaning of the other values in the translation is undefined.

Note: The range of a Translation entry is indicated even if R = W = 0b in order to allow a “hole” in the Translation Completion. For example, if the Translation Request has a Length of six DWs, then up to three translations could be included in the Translation Completion. The first and third translations may have Set R or W but the second could have R = W = 0b. To avoid ambiguity about the size of the indicated gap, the range of the gap is indicated in the Translation Completion even if R = W = 0b.

The R = 0b, W = 0b state is used to indicate that the ATC is not permitted to use address field in the translation to form a translated address value for a subsequent request.

When the host changes a translation in the TA from not valid to valid, the host is not required to send an invalidation indication to the ATC. As such the ATC will not be notified of such changes and thus the ATC is not permitted to cache a translation value of R = W = 0b.

When the host changes permissions associated with a translation in the TA, to grant additional permission (e.g., R = 1, W = 0b to R = W = 1b), the host is not required to send an invalidation indication to the ATC. As such, the ATC will not be notified of such changes. When the ATC requires permissions greater than the cached ATC entry, the ATC is permitted to request a fresh translation.

When the host changes permissions associated with a translation in the TA, to remove permission (e.g., R = W = 1b to R = 1b, W = 0b), the host is required to send an invalidation indication to the ATC. The subsequent invalidation completion tells the host that the ATC has stopped using the previously granted permissions.

If no table entry is found for the requested address, the TA must return a CplD with a single translation value with R = W = 0b.

Note: Implementations should not assume that receiving a translation response with the R or W bits Set (*independent of the value of the U bit*) implies that a subsequent read or write request with the same **untranslated** address will succeed. Although it may be possible for a device and its controlling software to ensure this property, the method for doing so is outside the scope of this specification.

When ACS Direct Translated P2P is enabled, Translated and Untranslated requests may take different paths between Requester and Completer. As such, PCI Express Ordering between Translated and Untranslated requests is not guaranteed. For transaction sequences that require ordering, Functions should avoid using a mixture of Translated and Untranslated Requests.

The Priv bit is used to qualify R and W. If Priv is Set, R and W indicate permissions granted to Privileged Mode entities in the Function. If Priv is Clear, R and W indicate permissions granted to Non-Privileged Mode entities in the Function. The R and W values for the two privilege levels are independent. The ATC must not assume any correlation between the Privileged Mode and Non-Privileged Mode permissions associated with a translation.

10.2.3.6 Execute Permitted (Exe) §

If Exe is Set, the requesting Function is permitted to execute code in the implied range of memory. If Exe is Clear, the requesting Function is not permitted to execute code in the implied range of memory.

The definition of what it means for a Function to execute code is outside the scope of this specification. Various system components may have different instruction sets. Behavior within the requesting Function when it attempts to execute code that is not permitted by this bit is outside the scope of this specification.

The Exe bit may only be Set if the TA supports Execute permissions, the associated Translation Request had an effective value of 1b for the Execute Requested bit¹⁸² and R is Set in the Translation Completion Data Entry. Otherwise, the Exe bit must be Clear.

This value may be cached if R is Set.

The Priv bit is used to qualify the Exe bit. If Priv is Set, the Exe bit indicates permission granted to Privileged Mode entities in the Function. If Priv is Clear, the Exe bit indicates permission granted to Non-Privileged Mode entities in the Function. The Exe bit values for the two privilege levels are independent. The ATC must not assume any correlation between the Privileged Mode and Non-Privileged Mode permissions associated with a translation.

Functions may optionally check that:

- If the Execute Requested bit is Clear in a Translation Request, the Exe bits in the associated Translation Completion Data Entries are also Clear.
- If Exe is Set, R is also Set.

If either optional check fails, the Function shall signal Unexpected Completion (UC). These checks are independently optional.

10.2.3.7 Privileged Mode Access (Priv) §

If Priv is Set, R, W, and Exe refer to permissions granted to entities operating in Privileged Mode in the requesting Function. If Priv is Clear, R, W, and Exe refer to permissions granted to entities operating in Non-Privileged Mode in the requesting Function.

The meaning of Privileged Mode and Non-Privileged Mode and what it means for an entity to be operating as in Privileged Mode or in Non-Privileged Mode depends on the protection model of the system and is outside the scope of this specification.

Behavior is outside the scope of this specification when an entity in the requesting Function attempts to access memory that it is not permitted to access.

The Priv bit may only be Set if the TA supports Privileged Mode and the associated Translation Request had a PASID TLP Prefix (NFM) or OHC-A1 with an effective value of 1b for the Privileged Mode Requested¹⁸³ bit. Otherwise, the Priv bit must be Clear.

The Privileged and Non-Privileged Mode versions of R, W and Exe are independent. An ATC may cache either or both versions of R, W and Exe. An ATC that receives a translation with R=W=0b for one privilege level may not assume anything about what it might receive for the other privilege level.

This value may be cached if R or W is Set. This value must be cached when the corresponding R, W, or Exe values are cached.

Note: Since the Priv bit is Set only when the requesting Function Sets the Privileged Mode Requested bit, Functions that never set that bit should always receive the Priv bit Clear and thus don't need to cache it.

Functions may optionally check that when the Privileged Mode Requested bit is Clear in a Translation Request, the Priv bits in the associated Translation Completion Data Entries are also Clear. If this optional check fails, the Function shall signal Unexpected Completion (UC).

182. The effective value of the bit is 0b unless the bit in the PASID TLP Prefix (NFM) or OHC-A1 is 1b and usage of the bit is enabled for the request.

183. The effective value of the bit is 0b unless the bit in the prefix is 1b and usage of the bit is enabled for the request.

IMPLEMENTATION NOTE: EXECUTE PERMISSION AND PRIVILEGE MODE ENFORCEMENT

§

The requesting Function determines whether a particular Memory Request needs Execute permission or is associated with a Privileged Mode or Non-Privileged Mode entity. The ATC implements the protection checks indicated by the Exe and Priv bits.

10.2.3.8 Global Mapping (Global) §

If Global is Set, the requesting Function is permitted to create a Global Mapping entry in the ATC for this translation. If Global is Clear, the requesting Function is not permitted to create a Global Mapping entry in the ATC for this translation. Global Mapping entries apply to all PASIDs of the Function. They permit the ATC to reduce the number of translation requests needed and to reduce the memory needed for caching the results.

A Function is permitted to ignore this bit and always create non-Global Mapping entries in the ATC. This could result in multiple translations being requested for the same Untranslated Address under different PASIDs.

Functions that use this bit must also have the Global Invalidate Supported bit Set (see [§ Section 10.5.1.2](#)).

IMPLEMENTATION NOTE: TLP LENGTH, ADDRESS, AND BYTE OFFSET VALUES FOR TRANSLATION REQUEST COMPLETIONS §

The intention behind the rules for Translation Completions containing multiple translations is to make the TLPs look similar to those contained in a Memory Read Completion.

For single TLP Translation Completions, the goal is to make the Completion look as though it is a Memory Read Completion that ends on an RCB. As such, the Length and Byte Count will indicate the same value, and the Lower Address will contain the value that makes the TLP appear to end on an RCB.

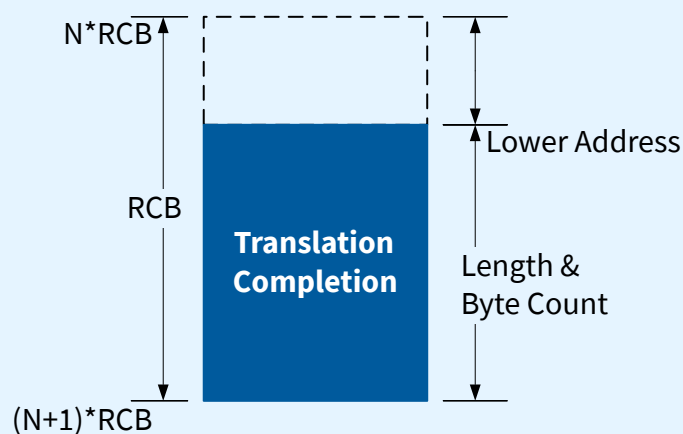


Figure 10-15 Example Translation Completion with 1 TLP

To intermediate components (which do not track the transaction) this TLP will be indistinguishable from a Memory Read Completion ending on an RCB.

For Translation Completions consisting of two TLPs, the goal is to make the Completion look as though it is a Memory Read Completion that crosses an RCB. As Such, the first Completion TLP will contain Lower Address & Length values which make the TLP appear to end on an RCB. The Byte Count of the first TLP will indicate the total length of all the Translation Completions sent in this transaction. For the second TLP, the Length and Byte Count fields will indicate the same value, and the Lower Address value will be 0.

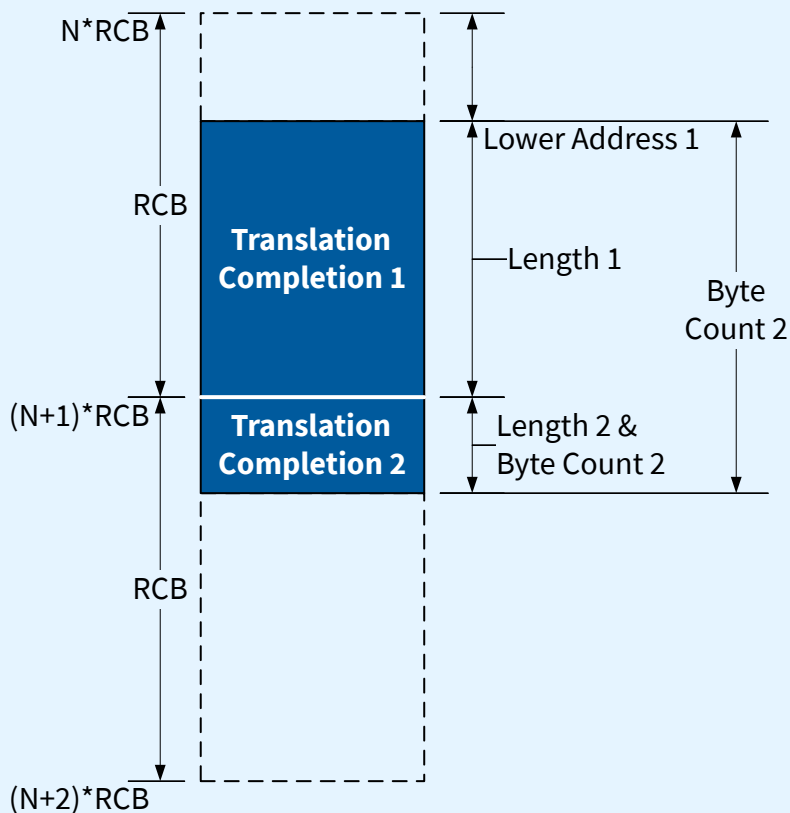


Figure 10-16 Example Translation Completion with 2 TLPs

To intermediate components (which do not track the transaction) this Completion transaction will be indistinguishable from a Memory Read Completion that crosses an RCB.

Note that the Length field is measures DWORDs, whereas the Lower Address and Byte Offset fields are measured in Bytes.

Provided the TLPs are properly formatted, A TA may choose to split the Translation Completion between any 2 Translation Completion Data Entries. Because an ATC cannot request more translations than can fit within a single RCB, the architectural maximum number of Translation Completion Data Entries can be sent in a single Completion TLP.

10.2.3.9 ATS Memory Attributes §

AMA values are associated with the Completer and are implementation specific. AMA values are opaque to the Requester, the Requester's ATC and the TA.

The method for detecting support of, and for enabling, the AMA mechanism in a TA is implementation specific. When the AMA mechanism is not enabled in both the Requester and the TA, the default AMA value is 000b. Behavior of the Completer for this case is implementation specific.

An ATC that supports AMA, caches AMA values along with other ATC information (Translated Address, Read, Write permission etc.) If the AMA value changes, for a page that could be cached in an ATC, software must issue an ATS Invalidation covering that page to ensure the ATC flushes the stale AMA value.

IMPLEMENTATION NOTE:

ATS MEMORY ATTRIBUTES §

ATS Memory Attributes (AMAs) are used with bus transactions on the Internal Coherent Bus (ICB) of devices as shown in the figure below. AMAs simplify processing at the TA. AMAs are simply passed through to the ICB avoiding a look-up at the TA.

AMAs may be supplied by a Translation Completion and stored in the ATC. The Endpoint may provide AMAs in Memory Read/Write TLPs using a TPH TLP Prefix. AMAs may also be provided using Vendor Defined Messages, however such methods are outside the scope of this specification.

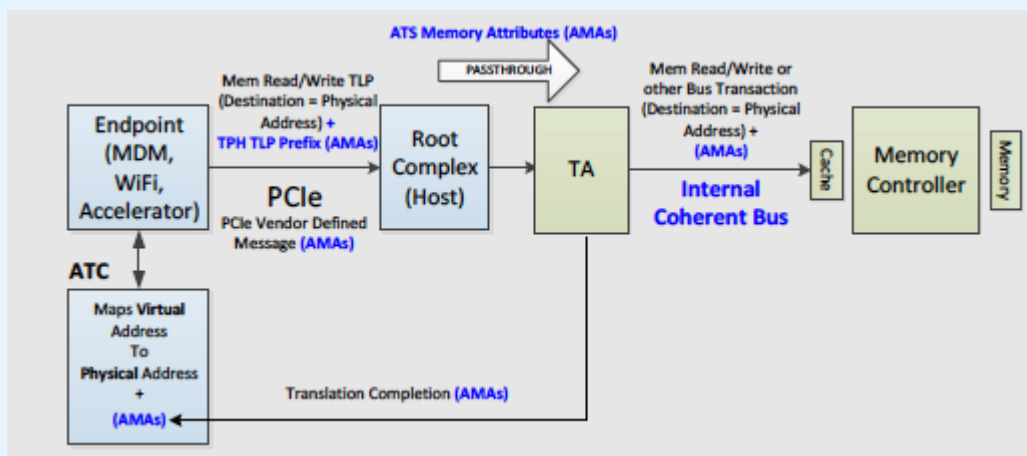


Figure 10-17 ATS Memory Attributes Example

AMAs provide hints on performing cache management as well as behavior on the ICB for a particular transaction. For example, AMAs may hint:

- Data is cacheable or non-cacheable.
- Write-hit policy. “Write back” may indicate that on a cache hit, update the cache, do not update memory. Mark the cache as dirty.
- Write-miss policy. “Write allocate” may indicate that on a cache miss, update memory, bring the data block to the cache (allocate).

AMAs are similar to TLP Processing Hints but should be considered independent. Hence all permutations of AMAs and TLP Processing Hints are permitted. However, AMAs may convey different semantics than what is provided by TLP Processing Hints. The cache behavior conveyed by AMAs and TLP Processing Hints may not align. Additionally, AMAs may convey semantics not addressed by TLP Processing Hints. For example, handling of transactions on the ICB when caching is not in use, such as, “reordering permitted/not permitted”, “early write acknowledgement permitted/not permitted.”

AMAs may be transmitted in peer-to-peer transactions. It is recommended that an application programming interface (API) be provided to enable AMAs to be configured into the Translation Agent (TA). Note: the TA might not be collocated with the Completer.

An example of memory attributes such as “strongly ordered device normal” may be found at <http://infocenter.arm.com>.

10.2.4 Completions with Multiple Translations §

An ATC is allowed to request that the TA provide translations for a virtually contiguous range of addresses. It does this by setting the Length field in the Translation Request to a value that is two times the number of requested translations as long as the request size (Total Completion Length * 4) is not larger than Read Completion Boundary (RCB) in the Link Control Register.

If multiple translations are requested, the TA may return one or more translations as long as the number of translations does not exceed the number of requested translations. It is not an error for the TA to return fewer translations than requested and no error indication is sent unless there is an error in accessing the data.

If the Translation Completion contains multiple translations, all translations must have the same indicated size. Also, successive translations must apply to the virtual address range that abuts the previous translation in the same completion.

If a translation has both R = 0b and W = 0b, the TA must still set the Size field and the lower bits of the Translated Address field used to encode the completion size to appropriate values.

Each translation in a Translation Completion will have some overlap with the implied memory range of the Translation Request (see § Section 10.2.2).

A successful Translation Completion must consist of one or two CplDs. Each CplD must contain an integral number of Translations (i.e., Length must be a multiple of 2).

The TA is permitted to choose:

- the number of translations it returns for each Translation Request (e.g., Byte Count of the first or only CplD)
- if it returns more than one translation, whether it uses one or two CplDs
- if it returns two CplDs, how many translations are returned in each CplD

The Byte Count and Length fields in each CplD is used to convey these choices to the ATC. The Lower Address field should not be needed by the ATC (its value is computed as defined in Section 10.2.3 to satisfy RCB rules but the field otherwise conveys no additional information).

- If a Translation Completion CplD has a Byte Count that is greater than four times the Length field, then this is the first of two CplDs for the transaction.
- If a Translation Completion CplD has a Byte Count that is equal to four times the Length field, then this is the second or only CplD for the request.

Note: There are multiple reasons that the TA may truncate the results of the completion. For example, the request might ask for a range of addresses, not all of which are defined. This could occur if the first translation is valid but located at the end of a page of translations. The TA, in looking up the next page of translations, may find that the page is not valid so the addresses are not valid. The range of addresses that are valid would be returned and no error indicated. When truncating a Translation Completion the TA is not allowed to pad the response with invalid entries (R = 0b, W = 0b).

Note: There are multiple reasons that the TA may break a Translation Completion into multiple TLPs. As an example, if the virtual address of the Translation Completion resolves to a table access that crosses an implementation specific address boundary, the completion to the TA may be broken into two completions. Rather than require that the TA accumulate the results, the TA is permitted to send each portion of the Translation Completion to a Function when it is received from memory.

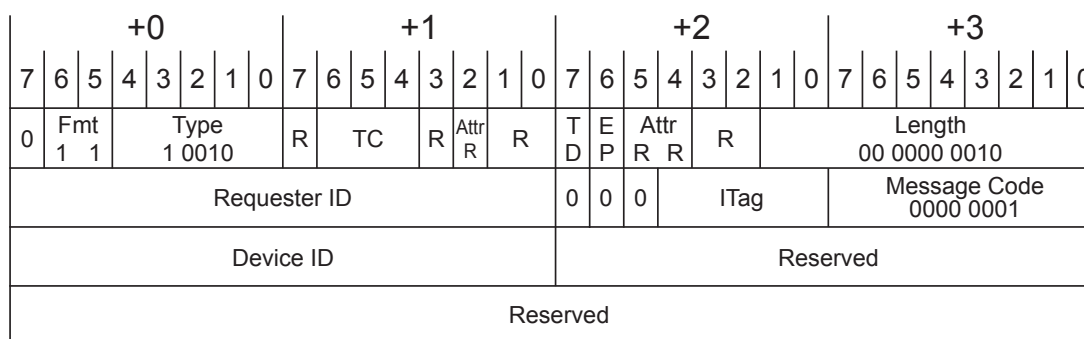
10.3 ATS Invalidation §

ATS uses the messages shown in this section to maintain consistency between the TA and the ATC. This specification assumes there is a single TA associated with each ATC. The TA (in conjunction with its associated software) must ensure that the address translations cached in the ATC are not stale by issuing Invalidate Requests.

10.3.1 Invalidate Request §

When a translation is changed in the TA and that translation might be contained within an ATC in a Function, the TA (in conjunction with its associated software) must send an Invalidate Request to the ATC to maintain proper synchronization between the ATPT and the ATC. An Invalidate Request is used to clear a specific subset of the address range from the ATC. Invalidate Requests are constrained to cover power of 2 multiple of 4096-byte pages.

The format of an Invalidate Request is shown in § Figure 10-18 and § Figure 10-19.



A-0584A

Figure 10-18 Invalidate Request Message - Non-Flit Mode

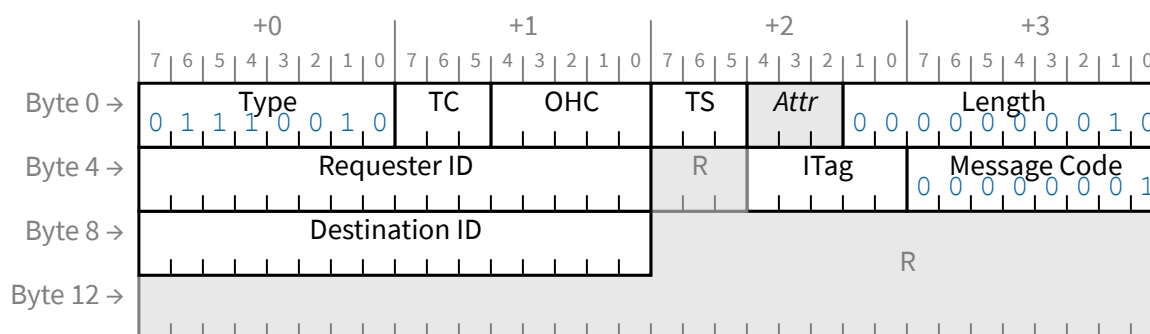


Figure 10-19 Invalidate Request Message - Flit Mode

The Invalidate Request is a MsgD transaction with 64 bits of data. Invalidate Request messages may be sent in any TC. The ITag field is constrained to the values 0 to 31 and is used by the TA to uniquely identify requests it issues. A TA must ensure that once an ITag is used, it is not reused until either released by the corresponding Invalidate Completions or by a vendor-specific timeout mechanism (see below).

The TA may have a single pool of ITag values for all invalidates that it issues or it may have a pool for each Device ID or any other combination. A Device with multiple ATCs on different Functions must manage the ITags separately for each Requester ID.

The address range specified in an Invalidate Request may span one or more STU 4096-byte pages. Invalidation ranges are required to be naturally aligned and should not be smaller than STU 4096-byte pages. Upon receiving an Invalidate Request with a range less than STU an ATC may either (1) signal an Unsupported Request (not recommended) or (2) round the range of the request up to a value greater than or equal to the STU.

IMPLEMENTATION NOTE: INVALIDATE COMPLETION TIMEOUT §

Devices should respond to Invalidate Requests within 1 minute (+50% -0%). Having a bounded time permits an ATPT to implement Invalidate Completion Timeouts and reuse the associated ITag values. ATPT designs are implementation specific. As such, Invalidate Completion Timeouts and their associated error handling are outside the scope of this specification.

The content of the payload is the untranslated address range to be invalidated. The payload format is shown in § Figure 10-20.

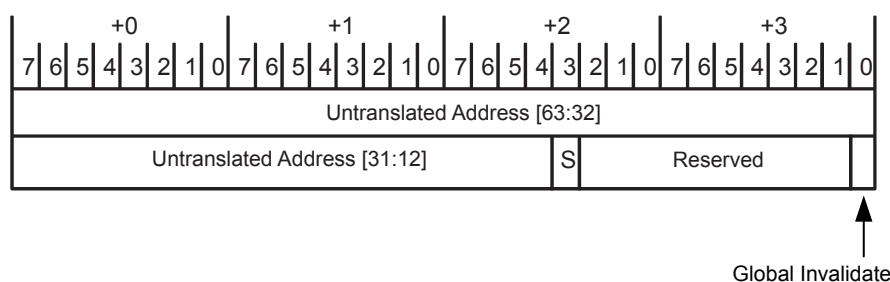


Figure 10-20 Invalidate Request Message Body

The S field is used to indicate if the range being invalidated is greater than 4096 bytes. Its meaning is the same as for the Translation Completion (see § Section 10.2.3.1 and § Section 10.2.3.2).

The Global Invalidate bit indicates that the Invalidation Request Message affects all PASID values (see § Section 10.3.8). This bit is Reserved unless the Invalidation Request has a PASID. The bit is ignored by the ATC if Global Invalidate Supported bit is Clear (see § Section 10.3.8).

IMPLEMENTATION NOTE:
INVALIDATION REQUESTS AND FUNCTION LEVEL RESET &
DEVICE POWER STATE TRANSITIONS §

Invalidation requests received while a Function is undergoing Function Level Reset, or is in (or transitioning to) non-D0 device state, may be dropped by the Function. Similarly, invalidation requests already received but pending at the time of receiving initiate FLR or D-state transition request may be dropped the Function.

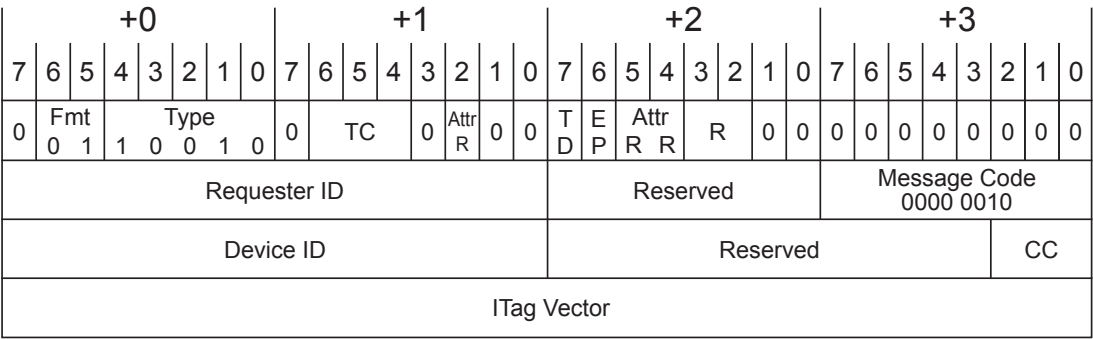
System software can avoid ATS invalidation race conditions on Function Level Reset and Device Power State transitions in a variety of ways. For example:

- 1. When disabling ATS on a Function, system software quiesces ATS invalidations to the Function (i.e., either responses are received for all invalidation requests issued to the Function, or any pending invalidation requests to the Function have timed out).
- 2. Software ensures no invalidations are issued to a Function when its ATS capability is disabled.
- 3. Before initiating the FLR (or Device power state transitions), software disables ATS as described in item 1. above.

10.3.2 Invalidate Completion §

When a Function completes an Invalidate operation, it will send one or more Invalidate Completion messages to the TA. These messages must be tagged with information extracted from the Invalidate Request to enable the TA to associate the Invalidate Completions with the Invalidate Request.

The format of the Invalidate Completion message is shown in § Figure 10-21 and § Figure 10-22.



A-0586A

Figure 10-21 Invalidate Completion Message Format - Non-Flit Mode

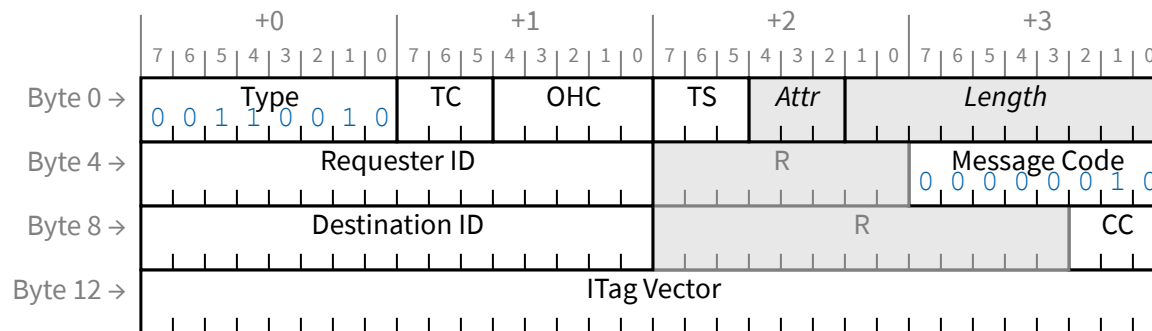


Figure 10-22 Invalidate Completion Message - Flit Mode

The Invalidate Completion message is a Msg transaction routed by ID. The Requester ID field of the Invalidate Completion message is set to the Requester ID of the Function containing the ATC. The Device ID field of the Invalidate Completion is set to the Requester ID of the TA. The ATC may derive the Requester ID of the TA from the Requester ID field of the corresponding Invalidate Request. Alternatively, since the ATC is only associated with a single TA, the ATC may sample and store the Requester ID from the first Invalidate Request following a Fundamental Reset or FLR. Subsequent Invalidate Completion messages may use this value to set the Device ID field of Invalidate Completion messages.

The Completion Count (CC) field indicates the number of individual Invalidate Completion messages that must be sent for the associated Invalidate Request. Setting the CC field to 0 indicates that eight responses must be sent. The TA is responsible for collecting all the responses associated with a given Tag before considering the corresponding Invalidate Request to be complete.

Invalidate Completion messages may be sent on any TC, independent of the TC the originating Invalidate Request was received. This enables implementations to utilize the Invalidate Completion to push outstanding transactions to the TA to guarantee the required invalidation semantics are met. Implementations that utilize a single Upstream TC are required to send a single Invalidate Completion in the utilized TC.

The ITag Vector field is used to indicate which Invalidate Request has been completed. Each of the 32 possible ITag field values from the Invalidation Request is represented by a single bit in the ITag Vector field. The least significant bit (bit 0; i.e., the right-most bit in the schematic representation of the Invalidate Completion message shown in § Figure 10-21 and § Figure 10-22) of the ITag Vector field corresponds to the ITag field value of 0. The most significant bit (bit 31) of the ITag Vector field corresponds to the ITag field value of 31. Implementations are allowed to coalesce multiple Invalidate Completions by setting multiple ITag Vector bits in a single message provided the following conditions are met:

- The Invalidate Completions flow in the same TC.
- The Invalidate Completions have the same CC value.
- All fragments of an Invalidate Completion must have identical Request ID, CC, and ITag Vector fields.

A TA that receives an Invalidation Completion for an ITag that has no outstanding Invalidation Request shall report this error using implementation specific mechanisms. One possible such mechanism is to report the Invalidation Completion as an Unexpected Completion (UC).

Functions that do not support ATS will treat an Invalidate Request as UR.

Functions supporting ATS are required to send an Invalidate Completion in response to a Invalidate Request independent of whether the Bus Master Enable bit is Set or not. Note that the above conditions must be satisfied even when Bus Master Enable is Cleared. The method for a device to achieve this is implementation dependent.

IMPLEMENTATION NOTE: BUS MASTER ENABLE CHANGE §

When Bus Master Enable changes from Set to Clear, no further memory requests should be queued. It is possible that queued write requests are present when BME is Cleared. These requests could block an Invalidate Completion. These requests must be either sent or dropped. This will ensure that all outstanding write transactions that are potentially dependent upon the outstanding invalidation are complete.

10.3.3 Invalidate Completion Semantics §

Before an ATC can return an Invalidate Completion for a given Invalidate Request, it must ensure the following conditions are satisfied:

- All new requests initiated by the Function (or its Shadows) will not utilize stale address translations.
- All outstanding read requests initiated by the Function (or its Shadows) utilizing translated address matching the invalidated range have either completed or been tagged to be discarded (method to discard is implementation specific).
- All outstanding posted writes initiated by the Function (or its Shadows) utilizing a translated address matching the invalidated range have been pushed to the TA. The ATC is required to send a copy of the Invalidate Completion message in each TC in which a posted write has been issued but not known to have been pushed to the TA. The CC field must be set to the same value in each copy of the Invalidate Completion message indicating number of copies sent. The TA is responsible for collecting all sent responses before considering the invalidation to be complete.
- Invalidate Completion Messages must contain the Requester ID used in the associated Invalidate Request.
- When Shadow Functions are enabled, it is sufficient to issue a single Invalidate Request message targeting either the Function or one of its Shadows. Since IDO is Clear, the Invalidation Completion resulting from that Invalidation Request will push to the TA any earlier Posted Writes for both the Function and its Shadows.
- Invalidation Completion behavior is independent of whether the associated Invalidate Request was issued to the Function or one of its Shadows.

IMPLEMENTATION NOTE: IMPLIED TC FLUSHING §

When making the decision as to which TC to send Invalidate Completions, an ATC may infer, in an implementation specific manner, that an issued posted write has been pushed to the TA. For example, a Function that has sent a read transaction to a destination above the TA and received its corresponding response may infer that any preceding posted writes issued in the same TC have been pushed to the TA.

10.3.4 Request Acceptance Rules §

In accord with the request acceptance rules enumerated in this section, a Function is not allowed to create a dependency in which the acceptance of a posted transaction is dependent upon the transmission of a posted

transaction. Given Invalidate Requests and Invalidate Completions both are posted transactions, Functions must not make the acceptance of an Invalidate Request dependent upon the transmission of an Invalidate Completion. The method for achieving this is implementation specific.

A Function with an ATS capability in its configuration space must be able to accept Invalidate Requests and send Invalidate Completions even if ATS is not enabled.

IMPLEMENTATION NOTE: INVALIDATE QUEUE DEPTH §

An ATC is only associated with a single TA. Each TA is limited to a total of 32 outstanding invalidations to any given ATC. This limits the number of outstanding Invalidation Requests active to a single ATC to 32. To avoid a post-to-post dependency, an ATC is required to accept up to 32 Invalidation Requests.

An ATC may choose to implement a maximally sized input queue holding Invalidate Requests. Alternatively, an ATC may choose to implement a maximally sized output queue holding Invalidate Completions. Note that queuing Invalidate Completions requires significantly less state per entry resulting in a potentially more efficient implementation than input queue buffering.

Note that the choice of whether to implement input queuing or output queuing (or a hybrid of both) has no impact on ensuring deadlock free behavior. But implementation choices with regard to queuing may have a significant impact on performance (see § Section 10.3.5).

10.3.5 Invalidate Flow Control §

Due to the variety of caching architectures and queuing strategies, implementations may vary greatly with respect to invalidation latency and throughput. It is possible that a TA may generate Invalidate Requests at a rate that exceeds the average ATC service rate. When this happens, the credit based flow control mechanisms will throttle the TA issue rate. A side effect of this is congestion spreading to other channels and Links through the credit based flow control mechanism. Depending on the frequency and duration of this congestion, performance may suffer. It is strongly recommended that TA and its associated software implement higher level flow control mechanisms.

To assist with the implementation of Invalidate Flow Control, an ATC must publish the number of Invalidate Requests it can buffer before backpressuring the Link. This field applies to all invalidations serviced by the Function, independent of the size of the invalidation. This value is communicated in the Invalidate Queue Depth field in the ATS capability structure (see § Section 7.8.9). A value of 0 0000b indicates that invalidate flow control is not necessary to this Function.

IMPLEMENTATION NOTE: INVALIDATE FLOW CONTROL §

A Function may indicate that invalidate flow control is not required when one or more of the following is true:

1. The Function can handle invalidations at the maximum arrival rate of Invalidate Requests.
2. The Function will not or very rarely cause Link backpressure (performance loss is negligible).
3. The Function can fully buffer the maximum number of incoming invalidations without backpressuring the Link.

Each Function is required to implement sufficient queuing to ensure it can hold the maximum number of outstanding Invalidation Requests from a TA (using either input or output queuing). However, with an SR-IOV device, all VFs associated with a PF share a single Invalidation Request queue in the PF. To implement Invalidation flow control, the TA must ensure that the total number of outstanding Invalidate Requests targeting the shared PF queue does not exceed the value in the PF's Invalidate Queue Depth field.

10.3.6 Invalidate Ordering Semantics §

Invalidate Requests and Translation Completions may be sent using different TC and are, therefore, unordered with respect to each other (from the Link's perspective). An ATC must ensure that the proper invalidation behavior is maintained when an Invalidate Request bypasses a Translation Completion to an overlapping region.

An ATC must “snoop” its outstanding translation request queue against all arriving Invalidate Requests. When snooping a request for a $N \times \text{STU}$ sized translation (N is a power of 2), the ATC must snoop the range of addresses starting at the STU aligned region containing the specified address and ending $(N-1)$ STU size pages later.

If an Invalidate Request overlaps the address range in an outstanding Translation Request, the Translation Request must be tagged as invalid and the results of its corresponding Translation Response must be discarded prior to transmission of the Invalidate Completion. If the Translation Response is received before the Invalidate Completion is sent, an implementation is free to issue requests utilizing the translation result provided the Invalidate Completion Semantics (see § Section 10.3.3) are satisfied.

IMPLEMENTATION NOTE:

REQUEST RANGE OVERLAP IN INVALIDATIONS §

In the description above, N is the number of STU sized translations that were requested in the Translation Request. This is equal to (Length field in Translation Request)/2.

As an example:

STU is 00 0010b indicating 16384-byte pages.

An outstanding Translation Request has a Length field of 00 0000 0100b indicating two translations covering a range of 32768 bytes.

The high-order 48 bits of the Translation Request are 0000 0FFF FFFFh.

The low-order 16 bits of the address in the request are 11xx xxxx xxxb indicating that the translation request covers a range that overlaps a 32768-byte boundary (in fact, the request crosses a 16-TB boundary).

If two translations are returned, they would cover the two STU sized regions at 0000 0FFF FFFF C000h and 0000 1000 0000 0000h.

An Invalidate Request is received with the high-order 48 bits of 0000 1000 0000h and the low-order 16 bits of 0001 1xxx xxxx xxxb.

The ATC must detect that a translation associated with a portion of the Translation Request is now invalidated and the Translation Completion associated with the invalidated region must be discarded (for simplification, the ATC is allowed to discard all of the Translation Completion).

It should be noted that, processing of the Invalidate Requests is simplified if Translation Requests do not cross alignment boundaries of the request. The Translation Request from the above example is not aligned to a 32768-byte boundary. If it were broken into two requests, it would be simpler to associate the range of the Invalidate Request with the address in the Translation Request. Breaking the Translation Requests into aligned requests is not a requirement.

10.3.7 Implicit Invalidation Events §

The following events will cause the invalidation of all ATC entries:

- Conventional Reset (all forms)
- Function Level Reset
- E field in ATS Capability changes from Clear to Set

The following events will cause the invalidation of all non-Global Mapping ATC entries that were requested using a specific PASID:

- Stopping the use of a PASID as defined in this specification.

No explicit Invalidate Completion message is sent when these implied invalidate events occur.

IMPLEMENTATION NOTE:

IMPLICIT INVALIDATION AND PASID §

Software may not change any of the PASID enable bits when the E field in the ATS Capability is Set. The invalidation that occurs when software Sets the E field also invalidates ATC entries with an associated PASID value.

10.3.8 PASID and Global Invalidate §

The requirements in this section apply to Functions that support the PASID. For Invalidation Requests that have a PASID, the ATC shall:

- Optionally signal Unsupported Request (UR) if the associated PASID value is greater than or equal to $2^{\text{Max PASID Width}}$. This error may be signaled anytime an out of range PASID value is present, even when the PASID value is ignored (see below).
- Return an Invalidation Completion if PASID Enable is Clear.
- If the Function supports Global Invalidate (see § Section 7.8.9.2):
 - If the Global Invalidate bit in the Request is Set, invalidate Global and non-Global Mapping entries in the ATC within the indicated memory range associated with any PASID value and return an Invalidation Completion. The PASID value in the PASID is ignored.
 - If the Global Invalidate bit in the Request is Clear, invalidate only non-Global Mapping entries in the ATC within the indicated memory range that were requested using the associated PASID value and return an Invalidation Completion.

Global Mapping entries in the ATC for some or all of the indicated memory range may be retained.

- If the Function does not support Global Invalidate (see § Section 7.8.9.2), invalidate entries in the ATC within the indicated memory range that were requested using the associated PASID value and return an Invalidation Completion.
- If no matching entries are present in the ATC, invalidate no ATC entries and return an Invalidation Completion.

For Invalidation Requests that do not have a PASID, the ATC shall:

- Invalidate ATC entries within the indicate memory range that were requested without a PASID value.
- Invalidate ATC entries at all addresses that were requested with any PASID value.

10.4 Page Request Services §

The general model for a page request is as follows:

1. A Function determines that it requires access to a page for which an ATS translation is not available.
2. The Function causes the associated Page Request Interface to send a Page Request Message to its RC. A Page Request Message contains a page address and a Page Request Group (PRG) index. The PRG index is used to identify the transaction and is used to match requests with responses.

3. When the RC determines its response to the request (which will typically be to make the requested page resident), it sends a PRG Response Message back to the requesting Function.
4. The Function can then employ ATS to request a translation for the requested page(s).

A Page Request Message is a PCIe Message Request that is Routed to the Root Complex with a Message Code of 4 (0000 0100b). The mechanism employed at the RC to buffer requests is implementation specific. The only requirement is that an RC not silently discard requests.

All Page Request Messages and PRG Response Messages travel in PCIe Traffic Class 0. A Page Request Message or PRG Response Message with a Traffic Class other than 0 shall be treated as Malformed TLPs by the RC or endpoint that receives the same. Intermediate routing elements (e.g., Switches) shall not detect this error.

The Relaxed Ordering and ID-Based Ordering bits in the Attr field of Page Request Messages and PRG Response messages may be used. The No Snoop bit in the Attr field is reserved.

The page request service allows grouping of page requests into Page Request Groups (PRGs). A PRG can contain one or more page requests. All pages in a PRG are responded to en mass by the host. Individual pages within a PRG are requested with independent Page Request Messages and are recognized as belonging to a common PRG by sharing the same PRG index. The last request of a PRG is marked as such within its Page Request Message. One request credit is consumed per page request (not per PRG).

A PRG Response Message is a PCIe Message Request that is Routed by ID back to the requesting Function. It is used by system software to alert a Function that the page request(s) associated with the corresponding PRG has (have) been satisfied. The page request mechanism does not guarantee any request completion order and all requests are inherently independent of all other concurrently outstanding requests. If a Function requires that a particular request be completed before another request, the initial request will need to complete before the subsequent request is issued. It is valid for a Function to speculatively request a page without ascertaining its residence state and/or to issue multiple concurrently outstanding requests for the same page.

A Page Request Interface is allocated a specific number of page request message credits. An RC (system software) can divide the available credits in any manner deemed appropriate. Any measures the host chooses to employ to ensure that credits are correctly metered by Page Request Interfaces (a Page Request Interface is not using more than its allocation) is an implementation choice. A Page Request Interface is not allowed to oversubscribe the available number of requests (doing so can result in the page request mechanism being disabled if the buffer limit is exceeded at the root). A Page Request Interface's page request allocation is static. It is determined when the Page Request Interface is enabled and can only be changed by disabling and then re-enabling the interface.

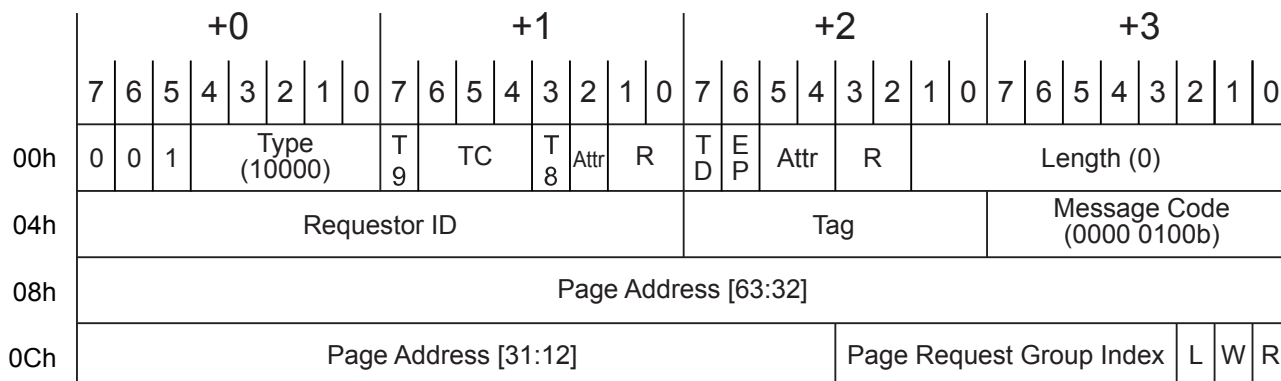
10.4.1 Page Request Message §

A Function uses a Page Request Message to send page requests to its associated host. A page request indicates a page needed by the Function. The Page Request Interface associated with a Function is given a specific Page Request allocation. A Page Request Interface shall not issue page requests that exceed its page request allocation.

A page request contains the untranslated address of the page that is needed, the access permissions needed for that page, and a PRG index. A PRG Index is a 9-bit scalar that is assigned by the Function to identify the associated page request. Multiple pages may be requested using a single PRG index. When more than a single page is to be associated with a given PRG, the Last flag in the Page Request Record is cleared in all the requests except the last request associated with a given PRG (the flag is set in the last request). Page requests are responded to en mass. No response is possible (except for a Response Failure error) until the last request of a PRG has been received by the root. The number of PRGs that a Function can have outstanding at any given time is less than or equal to the associated Page Request Interface's Outstanding Page Request Allocation. It is valid for a request group to contain multiple requests for the same page and for multiple outstanding PRGs to request the same page.

A Page Request Interface applies to the “main” Function and its enabled Shadow Functions (where the “main” is the Function that contains both the Page Request Extended Capability and the Shadow Function Extended Capability). All pages request messages of a single PRG must have the same Requester ID (of either the “main” Function or one of its Shadows).

The first two DWs of a Page Request Message contain a standard PCIe message header. The second two DWs of the message contain page request specific data fields.



A-0737A

Figure 10-23 Page Request Message - Non-Flit Mode

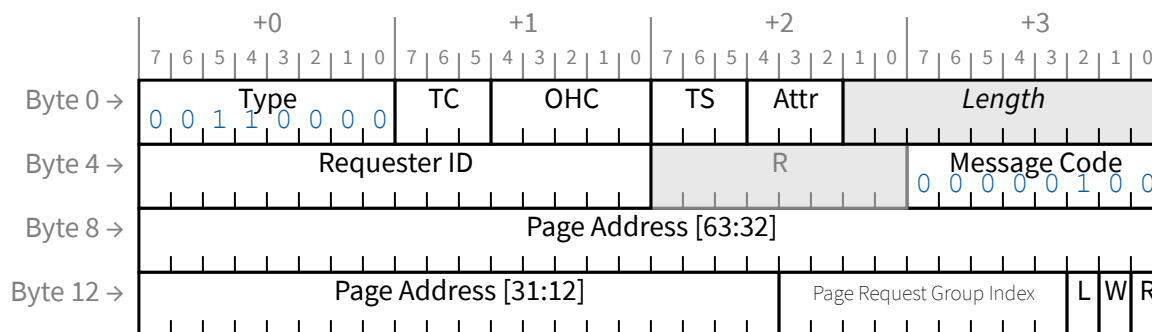


Figure 10-24 Page Request Message - Flit Mode

Table 10-6 Page Request Message Data Fields

Field	Meaning
R	Read Access Requested - This field, when Set, indicates that the requesting Function seeks read access to the associated page. When Clear, this field indicates that the requesting Function will not read the associated page. The R field must be Set for Page Requests with a PASID and that have the Execute Requested bit Set. If R and W are both Clear and L is Set, this is a Stop Marker (see § Section 10.4.1.2.1).

Field	Meaning
W	Write Access Requested - This field, when Set, indicates that the requesting Function seeks write access and/or zero-length read access to the associated page. When Clear, this field indicates that the requesting Function will not write to the associated page. Upon receiving a Page Request Message with the W field Set, the host is permitted to mark the associated page dirty. Thus, Functions must not issue such Requests unless the Function has been given explicit write permission. If R and W are both Clear and L is Set, this is a Stop Marker (see § Section 10.4.1.2.1).
L	Last Request in PRG - This field, when Set, indicates that the associated page request is the last request of the associated PRG. A PRG can have a single entry, in which case the PRG consists of a single request in which this field is Set. When Clear, this field indicates that additional page requests will be posted using this record's PRG Index. If R and W are both Clear and L is Set, this is a Stop Marker (see § Section 10.4.1.2.1).
Page Request Group Index	Page Request Group Index - This field contains a Function supplied identifier for the associated page request. A Function need not employ the entire available range of PRG index values. A host shall never respond with a PRG Index that has not been previously issued by the Function and that is not currently an outstanding request PRG Index (except when issuing a Response Failure, in which case the host need not preserve the associated request's PRG Index value in the error response).
Page Address	Page Address - This field contains the untranslated address of the page to be loaded. For pages larger than 4096 bytes, the least significant bits of this field are ignored. For example, the least significant bit of this field is ignored when an 8096-byte page is being requested.

IMPLEMENTATION NOTE:

LAST BIT AND RELAXED ORDERING §

If multiple page requests are associated with a single PRG index, the last page request of a PRG should have the Relaxed Ordering attribute bit Clear in addition to having the Last flag Set. All other page request messages may have the Relaxed Ordering attribute bit set to any value.

10.4.1.1 PASID Usage §

The PASID Extended Capability indicates whether a Function supports PASID TLP Prefixes (NFM) or OHC-A1 with PASID (FM), and whether it is enabled to send and receive them.

Functions that support PASID are permitted to send a PASID on Page Request Messages. The PASID field contains the process address space of the page being requested and the Execute Requested and Privileged Mode Requested bits indicate the access being requested.

If one Page Request Message in a PRG has a PASID, all Page Request Messages in that PRG must contain identical PASID values. Behavior is undefined when the PASID values are inconsistent.

Functions that support PASID and have the PRG Response PASID Required bit Set (see § Section 10.5.2.3), expect that PRG Response Messages will contain a PASID if the associated Page Request Message had a PASID. For such PRG Response Messages, the Execute Requested and Privileged Mode Requested bits are reserved and the PASID field contains the PASID from the associated Page Request Message.

10.4.1.2 Managing PASID Usage on PRG Requests §

There are rules for stopping and starting the use of a PASID.

This section describes additional rules that apply to Functions that have issued Page Request Messages in a PASID that is being stopped. No additional rules are required to start the usage of the Page Request Interface for a PASID.

When stopping the use of a particular PASID, a Stop Marker Message may be optionally used to avoid waiting for PRG Response Messages before the Function indicates that the stop request for a particular PASID has completed.

To stop without using a Stop Marker Message, the Function shall:

1. Stop queueing new Page Request Messages for this PASID.
2. Finish transmitting any multi-page Page Request Messages for this PASID (i.e., send the Page Request Message with the L bit Set).
3. Wait for PRG Response Messages associated any outstanding Page Request Messages for the PASID.
4. Indicate that the PASID has stopped using a device specific mechanism. This mechanism must indicate that a Stop Marker Message will not be generated.

To stop with the use of a Stop Marker Message the Function shall:

1. Stop queueing new Page Request Messages for this PASID.
2. Finish transmitting any multi-page Page Request Messages for this PASID (i.e., send the Page Request Message with the L bit Set).
3. Internally mark all outstanding Page Request Messages for this PASID as stale. PRG Response Messages associated with these requests will return Page Request Allocation credits and PRG Index values but are otherwise ignored.¹⁸⁴
4. Indicate that the PASID has stopped using a device specific mechanism. This mechanism must indicate that a Stop Marker Message will be generated.
5. Send a Stop Marker Message to indicate to the host that all subsequent Page Request Messages for this PASID are for a new use of the PASID value.

Note: Steps 4 and 5 may be performed in either order, or in parallel.

10.4.1.2.1 Stop Marker Messages §

A Stop Marker Message indicates that a Function has stopped using the Page Request Interface and has transmitted all pending Page Request Messages for a specific PASID. Stop Marker Messages are strongly ordered with respect to Page Request Messages and serve to push Page Request Messages toward the Host. When the Host receives the Stop Marker Message, this indicates that all Page Request Messages associated with the PASID being stopped have been delivered and that any subsequent Page Request Message with the same PASID value are associated with a new incarnation of that PASID value.

Stop Marker Messages do not have a response. They do not have a PRG Index and do not consume Page Request allocation (see § Section 10.5.2.5).

The Stop Marker Message bit layout is shown in § Figure 10-25.

184. Page Request Allocation is shared across all PASIDs of the Function (see § Section 10.5.2.5). If PRG Response PASID Required is Clear, PRG Index values are shared across all PASIDs of the Function (see § Section 10.5.2.3).

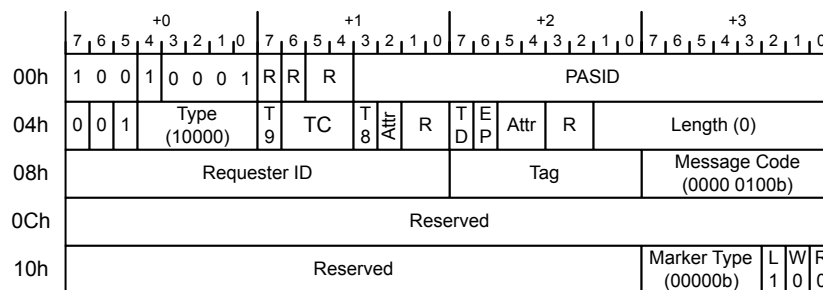


Figure 10-25 Stop Marker Message - Non-Flit Mode

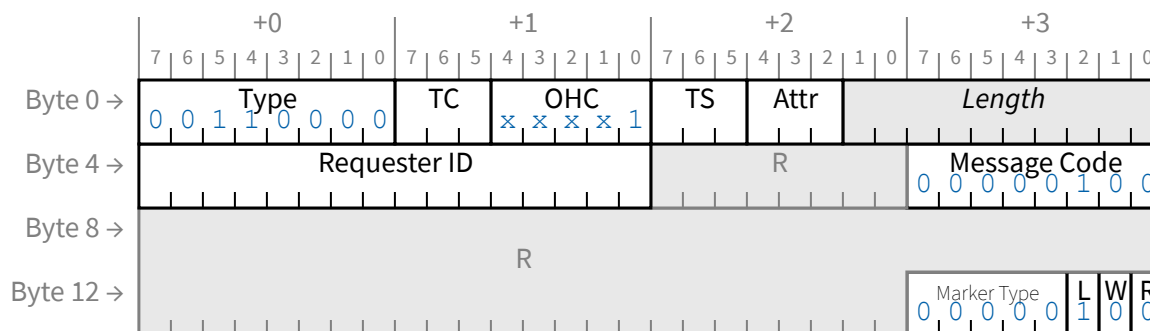


Figure 10-26 Stop Marker Message - Flit Mode

A Stop Marker Message is encoded as a Page Request Message for which:

- In NFM, includes a PASID TLP Prefix.
- In FM, includes OHC-A1 with PASID. The Execute Requested and Privileged Mode Requested bits and the Last DW BW / 1st DW BE fields in the OHC-A1 are Reserved.
- The L, W and R fields contain 1b, 0b and 0b respectively.
- The Untranslated Address field and upper bits of the PRG Index field are Reserved.
- The Marker Type field contains 0 0000b to indicate that this is a Stop Marker Message.
- The Traffic Class must be 0.
- The Relaxed Ordering attribute bit must be Clear.
- The ID-Based Ordering attribute bit may be Set.

Behavior is undefined if a Stop Marker Message is received and any of the following are true:

- Marker Type not equal to 0 0000b.
- No PASID TLP Prefix is present (NFM).
- The PASID value does not match an outstanding stop request.
- An incomplete Page Request Message for the PASID is outstanding (i.e., for some PRG Index, the most recently received Page Request Message did not have the L bit Set).

10.4.2 Page Request Group Response Message §

System hardware and/or software communicate with a Function's page request interface via PRG Response Messages. A PRG Response Message is used by a host to signal the completion of a PRG, or the catastrophic failure of the interface. A single PRG Response Message is issued in response to a PRG, independent of the number of page requests associated with the PRG. There is no mechanism for indicating a partial request completion or partial request failure. If any of the pages associated with a given PRG cannot be satisfied, then the request is considered to have failed and the reason for the failure is supplied in the PRG Response Message. The host has no obligation to partially satisfy a multi-page request. If one of the requested pages cannot be made resident, then the entire request can, but need not, be discarded. That is, the residence of pages that share a PRG with a failed page request, but that are not associated with the failure, is indeterminate from the Function's perspective.

There are four possible Page Request failures:

1. The requested page is not a valid Untranslated Address.
2. PASID support exists, the Page Request has a PASID, and either PASID usage is not enabled for this request, the PASID value is not valid, or the Execute Requested bit is Set when R is Clear.¹⁸⁵
3. The requested page does not have the requested access attributes (including Execute permission and/or Privileged Mode access when those bits are present).
4. The system is, for an unspecified reason, unable to respond to the request. This response is terminal (the host may no longer respond to any page requests and may not supply any further replies to the Function until the Function's page request interface has been reset). For example, a request that violates a Function's assigned request limit or overflows the RC's buffering capability may cause this type of failure.

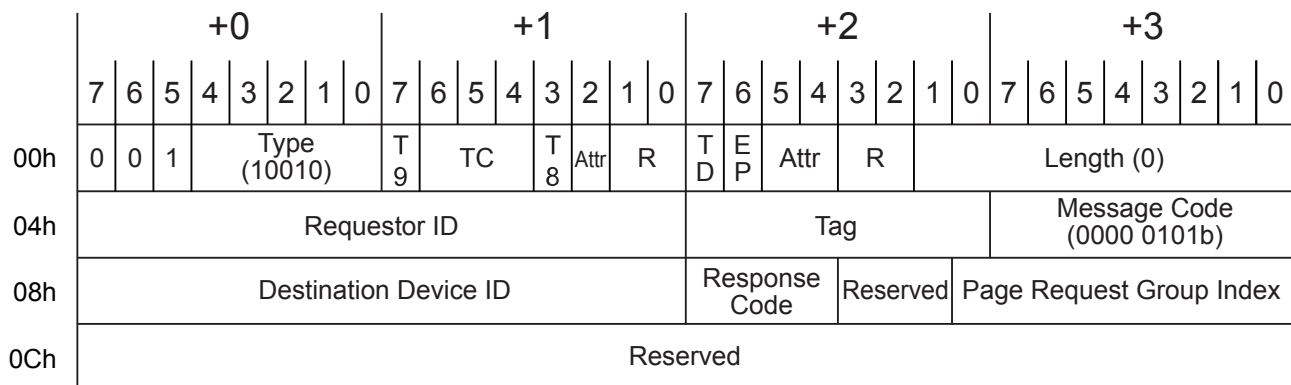
A Function's response to Page Request failure cases 1, 2, and 3 above is implementation dependent. The failure is not necessarily persistent, that is, a failed request may, in some instances succeed if re-issued. The range of possibilities precludes the precise specification of a generalized failure behavior, though on a per Function basis, the response to a failure will be an implementation dependent behavior.

All responses are sent to their associated Functions via PRG Response Messages. A Function must be capable of sinking multiple consecutive messages without losing any information. To avoid deadlock, a Function must be able to process PRG Response Messages for all of the Function's outstanding Page Request Messages without depending on the Function sending or receiving any other TLP.¹⁸⁶ A PRG Response Message is an ID routed PCIe message. The only Page Request Interface specific fields in this message are the Response Code and PRG. All other fields are standard PCIe message fields. (Note: these messages are routed based on the ID in bytes 8 and 9; with bytes 4 and 5 containing the host's Requester ID.)

Receipt of a PRG Response Message that contains a PRG Index that is not currently outstanding at a Function shall result in the UPRGI flag in the Page Request Extended Capability being Set, contingent upon the TLP otherwise being error free. Because of ambiguous language in earlier versions of this specification, it is permitted (though discouraged) to handle this case as an Unsupported Request (UR) or Unexpected Completion (UC) by the Function containing the Page Request Extended Capability, but otherwise no other error is permitted to be logged or signaled.

¹⁸⁵. Behavior when PASID support does not exist is defined in § Section 6.20.1.

¹⁸⁶. For example, processing a PRG Response Message that causes the Function to send a TLP Upstream must not block processing of subsequent downstream TLPs even if the Upstream TLP is delayed by flow control.



A-0738A

Figure 10-27 PRG Response Message - Non-Flit Mode

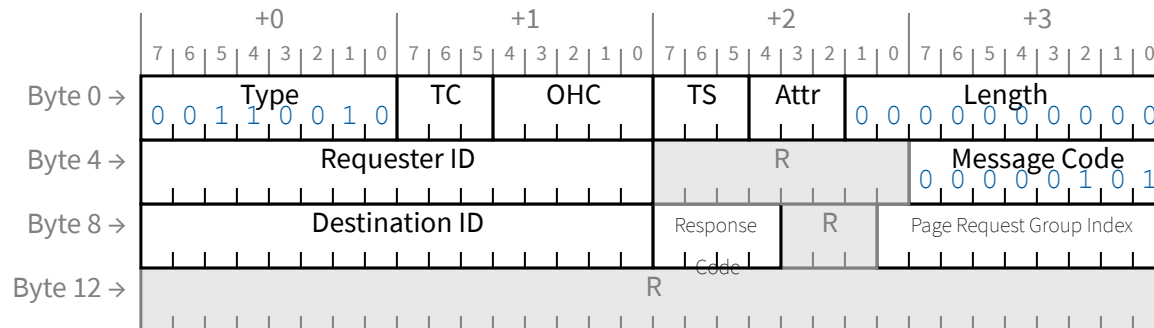


Figure 10-28 PRG Response Message - FLIT Mode

Table 10-7 PRG Response Message Data Fields

Field	Meaning
Page Request Group Index	Page Request Group Index - This field contains a Function supplied index to which the RC is responding. A given PRG Index will receive exactly one response per instance of PRG (with the possible exception of a Response Failure).
Response Code	Response Code - This field contains the response type of the associated PRG. The encodings are presented in § Section 10.4.2.1 .

10.4.2.1 Response Code Field §

The values and meaning for the Response Code field are listed in § Table 10-8.

§ Table 10-8 Response Codes

Value	Status	Meaning
0000b	Success	All pages within the associated PRG were successfully made resident.

Value	Status	Meaning
0001b	Invalid Request	One or more pages within the associated PRG do not exist or requests access privilege(s) that cannot be granted. Unless the page mapping associated with the Function is altered, re-issuance of the associated request will never result in success.
1110b:0010b	Unused	Unused Response Code values. A Function receiving such a message shall process it as if the message contained a Response Code of Response Failure.
1111b	Response Failure	One or more pages within the associated request group have encountered/caused a catastrophic error. This response disables the Page Request Interface at the Function. Any pending page requests for other PRGs will be satisfied at the convenience of the host. The Function shall ignore any subsequent PRG Response Messages, pending re-enablement of the Page Request Interface.

10.4.2.2 PASID Usage on PRG Responses §

If a Page Request has a PASID, the corresponding PRG Response Message may optionally contain one as well.

If the PRG Response PASID Required bit is Clear, PRG Response Messages do not have a PASID.

If the PRG Response PASID Required bit is Set, PRG Response Messages have a PASID if the Page Request also had one. The Function is permitted to use the PASID value from the prefix in conjunction with the PRG Index to match requests and responses.

When a PASID is attached to PRG Response Messages, the Execute Requested and Privileged Mode Requested bits are Reserved and the PASID value is copied from the PASID value of the Page Request.

10.5 ATS Configuration §

10.5.1 ATS Extended Capability §

Each Function that supports ATS (capable of generating Translation Requests) must have the ATS Extended Capability structure in its Extended Configuration Space. It is permitted to be implemented by Endpoints or RCiEPs.

ATS support is optional in SR-IOV devices. If a VF implements an ATS capability, its associated PF must implement an ATS capability. The ATS Capabilities in VFs and their associated PFs are permitted to be enabled independently.

§ Figure 10-29 details allocation of the register fields in the ATS Extended Capability structure.

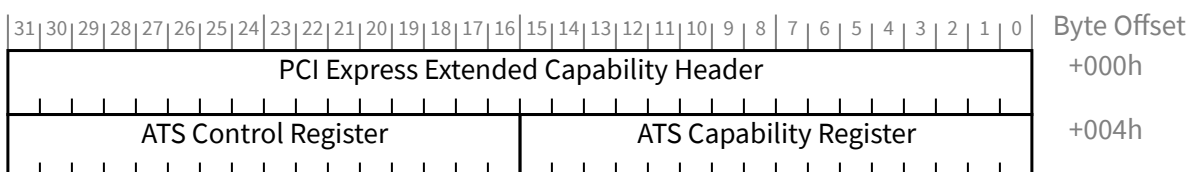


Figure 10-29 ATS Extended Capability Structure

10.5.1.1 ATS Extended Capability Header (Offset 00h) §

§ Figure 10-30 details allocation of the register fields in the ATS Extended Capability Header; § Table 10-9 provides the respective field definitions.

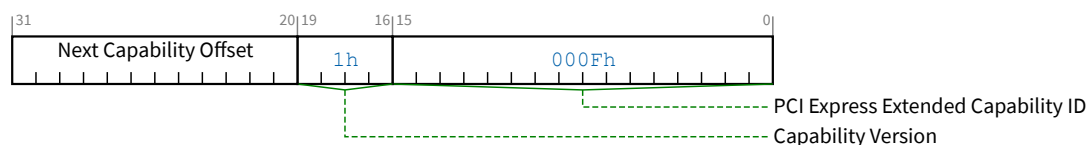


Figure 10-30 ATS Extended Capability Header

Table 10-9 ATS Extended Capability Header

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - Indicates the ATS Extended Capability structure. This field must return a Capability ID of "000Fh" indicating that this is an ATS Extended Capability structure.	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be "1h" for this version of the specification.	RO
31:20	Next Capability Offset - The offset to the next PCI Extended Capability structure or 000h if no other items exist in the linked list of capabilities.	RO

10.5.1.2 ATS Capability Register (Offset 04h) §

§ Figure 10-31 details the allocation of register fields of an ATS Capability Register; § Table 10-10 provides the respective bit definitions.

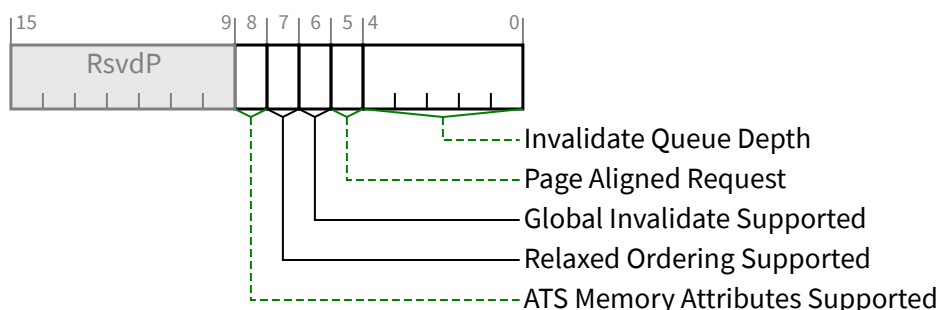


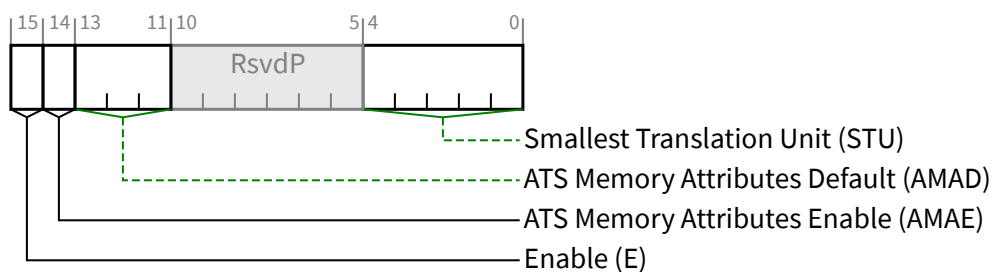
Figure 10-31 ATS Capability Register (Offset 04h)

Table 10-10 ATS Capability Register (Offset 04h)

Bit Location	Register Description	Attributes
4:0	Invalidate Queue Depth - The number of Invalidate Requests that the Function can accept before putting backpressure on the Upstream connection. A value of Zero in a non-VF Function indicates that it can accept 32 Invalidate Requests. For VFs, this field must be hardwired to Zero. The associated PF's value applies.	RO VF ROZ
5	Page Aligned Request - If Set, indicates the Untranslated Address is always aligned to a 4096 byte boundary. Setting this field is recommended. This field permits software to distinguish between implementations compatible with this specification and those compatible with an earlier version of this specification in which a Requester was permitted to supply anything in bits [11:2].	RO
6	Global Invalidate Supported - If Set, the Function supports Invalidation Requests that have the Global Invalidate bit Set. If Clear, the Function ignores the Global Invalidate bit in all Invalidate Requests (see § Section 10.3.8). This bit is 0b if the Function does not support PASID.	RO
7	Relaxed Ordering Supported - If Set, indicates this Function is permitted to Set the RO bit in Translation Requests when Enable Relaxed Ordering bit is Set.	RO
8	ATS Memory Attributes Supported - If Set, the Function supports using AMA values from ATS Translation Completions in the Function ATC. When Clear, the Function ignores the AMA field in ATS Translation Completions.	RO

10.5.1.3 ATS Control Register (Offset 06h) §

§ Figure 10-32 details the allocation of register fields of an ATS Control Register; § Table 10-11 provides the respective bit definitions.



§ Figure 10-32 ATS Control Register

§ Table 10-11 ATS Control Register

Bit Location	Register Description	Attributes
4:0	Smallest Translation Unit (STU) - This value indicates to the Function the minimum number of 4096-byte blocks that is indicated in a Translation Completions or Invalidate Requests. This is a power of 2 multiplier and the number of blocks is 2^{STU} . A value of 0 0000b indicates one block and a value of 1 1111b indicates 2^{31} blocks (or 8 TB total)	RW VF ROZ

Bit Location	Register Description	Attributes				
	For VFs, this field must be hardwired to Zero. The associated PF's value applies. Default value is 0 0000b.					
13:11	ATS Memory Attributes Default (AMAD) – When Set, as a performance optimization, and when AMAE is Set, the Requestor is permitted to provide only non-default AMA values in Requests using the TPH TLP Prefix. <table><tr><td>AMAD</td><td>ATS Memory Attributes</td></tr><tr><td>000b-111b</td><td>ATS Memory Attribute values.</td></tr></table> This field is permitted to be hardwired to 000b if ATS Memory Attributes Supported is Clear. Default value is 000b.	AMAD	ATS Memory Attributes	000b-111b	ATS Memory Attribute values.	RW/RO
AMAD	ATS Memory Attributes					
000b-111b	ATS Memory Attribute values.					
14	ATS Memory Attributes Enable (AMAE) - When Set, the Requestor is permitted to provide AMA values in Requests using the TPH TLP Prefix. This bit is permitted to be hardwired to 0b if the ATS Memory Attributes Supported bit is Clear. Default value is 0b.	RW/RO				
15	Enable (E) - When Set, the Function is enabled to cache translations. Behavior is undefined if the Endpoint supports PASID, this bit is Set and any bit in the PASID Control Register is changed (see § Section 7.8.9.3) Default value is 0b.	RW				

10.5.2 Page Request Extended Capability Structure §

A Page Request Extended Capability Structure is used to configure the Page Request Interface mechanism. A Multi-Function Device or RCiEP Device may implement a Page Request Interface and the associated capability on any Endpoint Function within the Device. For SR-IOV devices, a single Page Request Interface is permitted for the PF and is shared between the PF and its associated VFs, in which case the PF implements this capability and its VFs must not. Every Page Request Interface mechanism operates independently.

For an SR-IOV device, even though the Page Request Interface is shared between its PFs and VFs, it sends the requesting Function's ID (PF or VF) in the Requester ID field of the Page Request Message, and the requesting Function's ID must be in the Destination Device ID field of the resulting PRG Response Message.

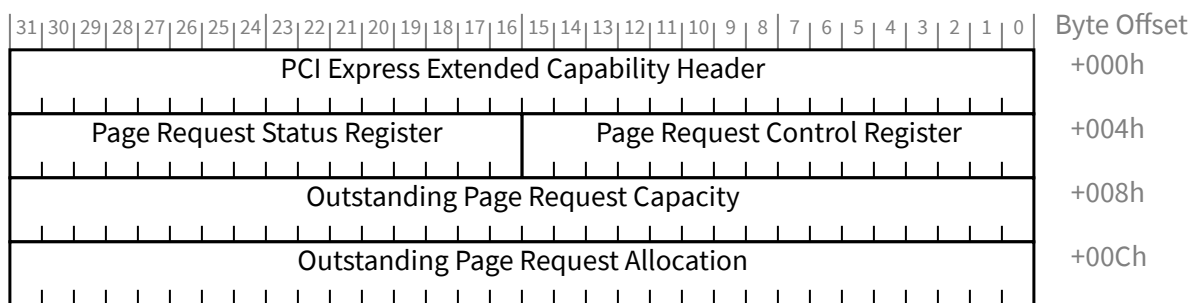


Figure 10-33 Page Request Extended Capability Structure

10.5.2.1 Page Request Extended Capability Header (Offset 00h) §

§ Figure 10-34 details allocation of the register fields in the Page Request Extended Capability Header; § Table 10-13 provides the respective field definitions.

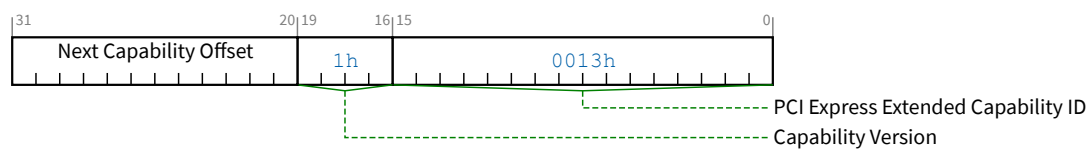


Figure 10-34 Page Request Extended Capability Header

Table 10-13 Page Request Extended Capability Header

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - Indicates that the associated extended capability structure is a Page Request Extended Capability. This field must return a Capability ID of "0013h".	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be "1h" for this version of the specification.	RO
31:20	Next Capability Offset - The offset to the next PCI Extended Capability structure or 000h if no other items exist in the linked list of capabilities.	RO

10.5.2.2 Page Request Control Register (Offset 04h) §

§ Figure 10-35 details allocation of the register fields in the Page Request Control Register; § Table 10-14 provides the respective field definitions.

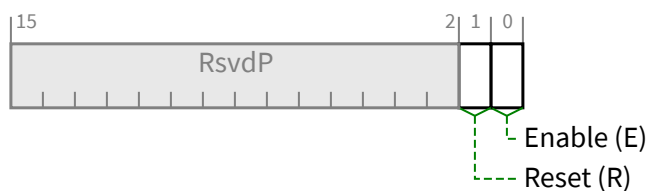


Figure 10-35 Page Request Control Register

Table 10-14 Page Request Control Register

Bit Location	Register Description	Attributes
0	Enable (E) - This field, when set, indicates that the Page Request Interface is allowed to make page requests. If this field is Clear, the Page Request Interface is not allowed to issue page requests. If both this field and the Stopped field are Clear, then the Page Request Interface will not issue new page requests, but has outstanding page requests that have been transmitted or are queued for transmission. When the Page Request Interface is transitioned from not-Enabled to Enabled, its status flags (Stopped, Response Failure, and Unexpected Page Request Group Index (UPRGI) flags) are cleared. Enabling a Page Request Interface that has not successfully Stopped has indeterminate results. Default value is 0b.	RW
1	Reset (R) - When the Enable field is clear, or is being cleared in the same register update that sets this field, writing a 1b to this field, clears the associated implementation dependent page request credit counter and pending request state for the associated Page Request Interface. No action is initiated if this field is written to 0b or if this field is written with any value while the Enable field is Set. Reads of this field return 0b	RW

10.5.2.3 Page Request Status Register (Offset 06h) §

§ Figure 10-36 details allocation of the register fields in the Page Request Error Register; § Table 10-15 provides the respective field definitions.

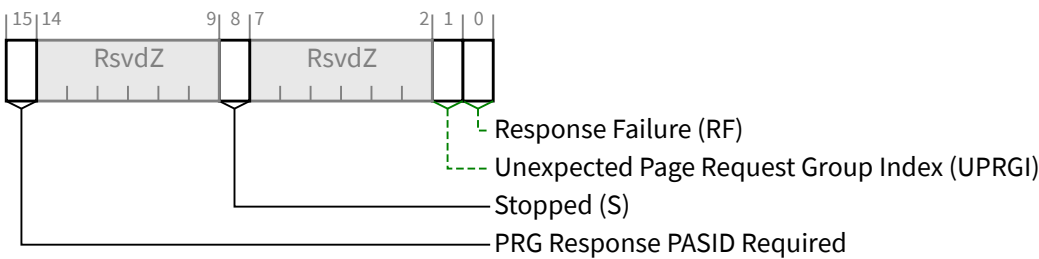


Figure 10-36 Page Request Status Register

§ Table 10-15 Page Request Status Register

Bit Location	Register Description	Attributes
0	Response Failure (RF) - This field, when Set, indicates that the Function has received a PRG Response Message indicating a Response Failure. The Function expects no further responses from the host (any received are ignored). This field is Set by the Function and Cleared when a one is written to the field. For SR-IOV, this field is Set in the PF if any associated Function (PF or VF) receives a PRG Response Message indicating Response Failure. Default value is 0b.	RW1C
1	Unexpected Page Request Group Index (UPRGI) - This field, when Set, indicates that the Function has received a PRG Response Message containing a PRG index that has no matching request. This field is Set by the Function and cleared when a one is written to the field. For SR-IOV, this field is Set in the PF if any associated Function (PF or VF) receives a PRG Response Message that does has no matching request. Default value is 0b.	RW1C
8	Stopped (S) - When this field is Set, the associated page request interface has stopped issuing additional page requests and that all previously issued Page Requests have completed. When this field is Clear the associated page request interface either has not stopped or has stopped issuing new Page Requests but has outstanding Page Requests. This field is only meaningful if Enable is Clear. If Enable is Set, this field is undefined. When the Enable field is Cleared, after having been previously Set, the interface transitions to the stopping state and Clears this field. After all page requests currently outstanding at the Function(s) have completed, this field is Set and the interface enters the disabled state. If there were no outstanding page requests, this field may be Set immediately when Enable is Cleared. Resetting the interface will cause an immediate transition to the disabled state. While in the stopping state, receipt of a Response Failure message will result in the immediate transition to the disabled state (Setting this field). For SR-IOV, this field is Set only when all associated Functions (PF and VFs) have stopped issuing page requests. Default value is 1b.	RO
15	PRG Response PASID Required - If Set, the Function expects a PASID on PRG Response Messages when the corresponding Page Requests had a PASID. If Clear, the Function does not expect PASID on any PRG Response Message. Function behavior is undefined if this bit is Clear and the Function receives a PRG Response Message with a PASID. Function behavior is undefined if this bit is Set and the Function receives a PRG Response Message with no PASID when the corresponding Page Requests had a PASID. This bit is RsvdZ if the Function does not support PASID.	RO

10.5.2.4 Outstanding Page Request Capacity (Offset 08h) §

This register contains the number of outstanding page request messages the associated Page Request Interface physically supports. This is the upper limit on the number of pages that can be usefully allocated to the Page Request Interface.

This register is Read Only.

10.5.2.5 Outstanding Page Request Allocation (Offset 0Ch) §

This register contains the number of outstanding page request messages the associated Page Request Interface is allowed to issue (have outstanding at any given instance).

The number of PRGs a Page Request Interface has outstanding is less than or equal to the number of request messages it has issued. For example, if system software allocates 1000 messages to a Page Request Interface then a single PRG could use all 1000 of the possible requests. Conversely, at one request per PRG the Page Request Interface would run out of PRG indices (of which there are only 512) before it consumes all its page request credits. A Page Request Interface must pre-allocate its request availability for any given PRG, that is, all the requests required by a given PRG must be available before any of the requests may be issued.

When Shadow functions are enabled, this allocation applies to the Function containing this capability and its enabled Shadow Functions (See Section 7.9.21).

This register is Read/Write. Behavior is undefined if this register is changed while the Enable flag is set. Behavior is undefined if this register is written with a value larger than Outstanding Page Request Capacity. Default value is 0.

When PASID is supported, the Request Allocation remains associated with the Function and is shared across the Function as well as all PASIDs of the Function.

Stopping a PASID does not affect any allocation used by that PASID. The system should continue to respond with PRG Response Messages in order to return Page Request and PRG Index resources to the Function (see § Section 10.4.2.1).

Stop Marker Messages consume buffering but are not included in this allocation (see § Section 10.4.1.2.1). Systems should provide additional buffering for Stop Marker Messages and should limit the number of outstanding Stop Marker Messages to avoid overrunning this additional buffering.

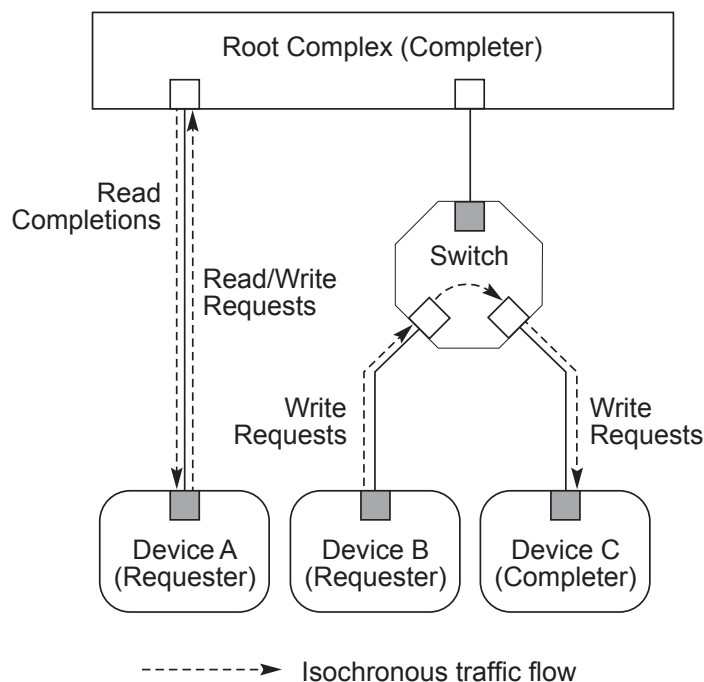
A. Isochronous Applications §

A.1 Introduction §

The design goal of isochronous mechanisms in PCI Express is to ensure that isochronous traffic receives its allocated bandwidth over a relevant time period while also preventing starvation of other non-isochronous traffic.

Furthermore, there may exist data traffic that requires a level of service falling in between what is required for bulk data traffic and isochronous data traffic. This type of traffic can be supported through the use of Port arbitration within Switches, the use of TC labels [1:7], and optional additional VC resources. Policies for assignment of TC labels and VC resources that are not isochronous-focused are outside the scope of the PCI Express specification.

Two paradigms of PCI Express communication are supported by the PCI Express isochronous mechanisms: Endpoint-to-Root-Complex communication model and peer-to-peer (Endpoint-to-Endpoint) communication model. In the Endpoint-to-Root-Complex communication model, the primary isochronous traffic is memory read and write requests to the Root Complex and read completions from the Root Complex. § Figure A-1 shows an example of a simple system with both communication models. In the figure, devices A, B, called Requesters, are PCI Express Endpoints capable of issuing isochronous request transactions, while device C and Root Complex, called Completers, are capable of being the targets of isochronous request transactions. An Endpoint-to-Root-Complex communication is established between device A and the Root Complex, and a peer-to-peer communication is established between device B and device C. In the rest of this section, Requester and Completer will be used to make reference to PCI Express elements involved in transactions. The specific aspects of each communication model will be called out explicitly.



OM14288

Figure A-1 An Example Showing Endpoint-to-Root-Complex and Peer-to-Peer Communication Models

Guaranteed bandwidth and deterministic latency require end-to-end configuration of fabric resources. If isochronous traffic is intermixed with non-isochronous traffic, it may not be possible to provide any guarantees/determinism as required by the application usage model. It is recommended that system software configure and assign fabric resources such that traffic intermix either does not occur or is such that the application usage model guarantees can be met. This can be accomplished by assigning dedicated VC resources and corresponding TC labels to the isochronous traffic flow(s) on a given path within the fabric.

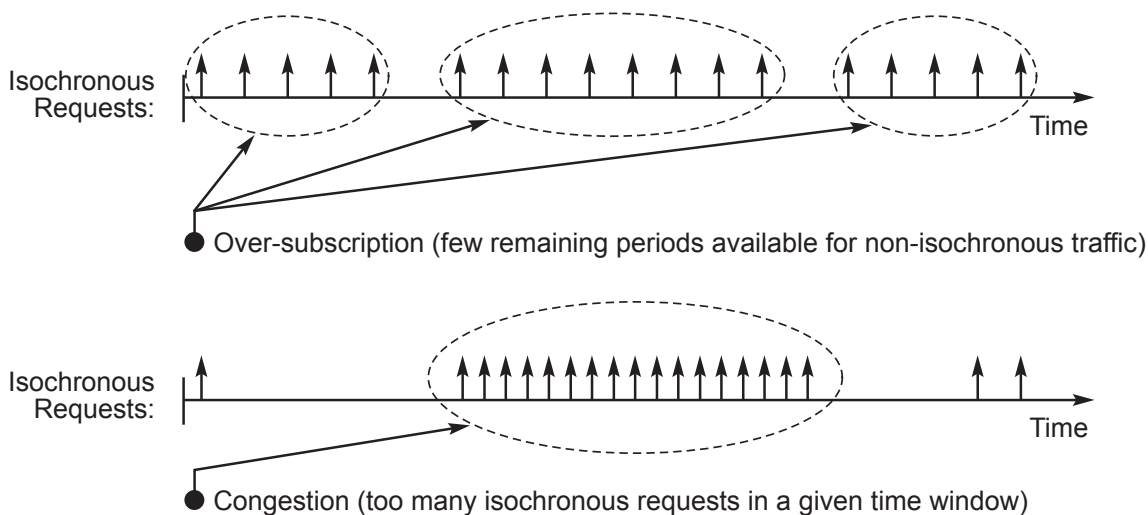
Note that there may be one or more isochronous traffic flows per VC/TC label and it is up to system software to insure that the aggregation of these flows does not exceed the requisite bandwidth and latency requirements.

It is also possible for a fabric to support multiple isochronous traffic flows separated across multiple VC (a given flow cannot span multiple VC/TC labels).

In general, as long as the device can meet the isochronous bandwidth and latency requirements, there is nothing to preclude a single VC device from supporting isochronous traffic if multiple TC labels are supported to delineate such traffic from non-isochronous traffic within the fabric.

A.2 Isochronous Contract and Contract Parameters §

In order to support isochronous data transfer with guaranteed bandwidth and deterministic latency, an isochronous contract must be established between a Requester/Completer pair and the PCI Express fabric. This contract must enforce both resource reservation and traffic regulation. Without such a contract, two basic problems, over-subscription and congestion, may occur as illustrated in § Figure A-2. When interconnect bandwidth resources are over-subscribed, the increased latency may cause failure of isochronous service and starvation of non-isochronous services. Traffic congestion occurs when flow control credits are not returned possibly due to a higher than expected/provisioned packet injection rate. This may cause excessive service latencies for both isochronous traffic and non-isochronous traffic.



OM14289A

Figure A-2 Two Basic Bandwidth Resourcing Problems: Over-Subscription and Congestion

The isochronous transfer mechanism in this specification addresses these problems with traffic regulation including admission control and service discipline. Under a software managed admission control, a Requester must not issue isochronous transactions unless the required isochronous bandwidth and resource have been allocated. Specifically, the isochronous bandwidth is given by the following formula:

$$BW = \frac{N * Y}{T}$$

§

Equation A-1 Isochronous Bandwidth

The formula defines allocated bandwidth (BW) as a function of specified number (N) of transactions of a specified payload size (Y) within a specified time period (T). Another important parameter in the isochronous contract is latency. Based on the contract, isochronous transactions are completed within a specified latency (L). Once a Requester/Completer pair is admitted for isochronous communication, the bandwidth and latency are guaranteed to the Requester (a PCI Express Endpoint) by the Completer (Root Complex for Endpoint-to-Root-Complex communication and another PCI Express Endpoint for peer-to-peer communication) and by the PCI Express fabric components (Switches).

Specific service disciplines must be implemented by isochronous-capable PCI Express components. The service disciplines are imposed to PCI Express Switches and Completers in such a manner that the service of isochronous requests is subject to a specific service interval (t). This mechanism is used to provide the method of controlling when an isochronous packet injected by a Requester is serviced. Consequently, isochronous traffic is policed in such manner that only packets that can be injected into the fabric in compliance with the isochronous contract are allowed to make immediate progress and start being serviced by the PCI Express fabric. A non-compliant Requester that tries to inject more isochronous transactions than what was being allowed by the contract is prevented from doing so by the flow-control mechanism thereby allowing compliant Requesters to correctly operate independent of non-compliant Requesters.

In the Endpoint-to-Root-Complex model, since the aggregated isochronous traffic is eventually limited by the host memory subsystem's bandwidth capabilities, isochronous read requests, and write requests (and Messages) are budgeted together. A Requester may divide the isochronous bandwidth between read requests and write requests as appropriate.

A.2.1 Isochronous Time Period and Isochronous Virtual Timeslot §

The PCI Express isochronous time period (T) is uniformly divided into units of virtual timeslots (t). To provide precise isochronous bandwidth distribution only one isochronous request packet is allowed per virtual timeslot. The virtual timeslot supported by a PCI Express component is reported through the Reference Clock field in the Virtual Channel Capability structure or the Multi-Function Virtual Channel Capability structure. When Reference Clock = 00b, duration of a virtual timeslot t is 100 ns. Duration of isochronous time period T depends on the number of phases of the supported time-based WRR Port arbitration table size. When the time-based WRR Port Arbitration Table size equals to 128, there are 128 virtual timeslots (t) in an isochronous time period (i.e., $T = 12.8 \mu\text{s}$).

Note that isochronous period T as well as virtual timeslots t do not need to be aligned and synchronized among different PCI Express isochronous devices, i.e., the notion of $\{T, t\}$ is local to each individual isochronous device.

A.2.2 Isochronous Payload Size §

The payload size (Y) for isochronous transactions must not exceed Max_Payload_Size (see § Section 7.8.4). After configuration, the Max_Payload_Size is set and fixed for each path that supports isochronous service with a value required to meet isochronous latency. The fixed Max_Payload_Size value is used for isochronous bandwidth budgeting regardless of the actual size of data payload associated with isochronous transactions. For isochronous bandwidth budgeting, we have

$Y = \text{Max_Payload_Size}$

§

Equation A-2 Isochronous Payload Size

A transaction with partial writes is treated as a normally accounted transaction. A Completer must account for partial writes as part of bandwidth assignment (for worst case servicing time).

A.2.3 Isochronous Bandwidth Allocation §

Given T , t and Y , the maximum virtual timeslots within a time period is

$$N_{\text{max}} = \frac{T}{t}$$

§

Equation A-3 N_{max}

and the maximum specifiable isochronous bandwidth is

$$BW_{\text{max}} = \frac{Y}{t}$$

§

Equation A-4 BW_{max}

The granularity with which isochronous bandwidth can be allocated is defined as:

$$BW_{\text{granularity}} = \frac{Y}{T}$$

§

Equation A-5 $BW_{\text{granularity}}$

Given T and t at 12.8 μs and 100 ns, respectively, N_{max} is 128. As shown in § Table A-1, BW_{max} and $BW_{\text{granularity}}$ are functions of the isochronous payload size Y .

Table A-1 Isochronous Bandwidth Ranges and Granularities §

Y (bytes)	128	256	512	1024
BW _{max} (MB/s)	1289	2560	5120	10240
BW _{granularity} (MB/s)	10	20	40	80

Similar to bandwidth budgeting, isochronous service disciplines including arbitration schemes are based on counting requests (not the sizes of those requests). Therefore, assigning isochronous bandwidth BW_{link} to a PCI Express Link is equivalent to assigning N_{link} virtual timeslots per isochronous period, where N_{link} is given by

$$N_{link} = \frac{BW_{link}}{BW_{granularity}} \quad \S$$

Equation A-6 N_{link}

A Switch Port serving as an Egress Port (or an RCRB serving as a “virtual” Egress Port) for an isochronous traffic, the N_{max} virtual timeslots within T are represented by the time-based WRR Port Arbitration Table in the PCI Express Virtual Channel Capability structure detailed in § Section 7.9.1 . The table consists of N_{max} entries. An entry in the table represents one virtual timeslot in the isochronous time period. When a table entry is given a value of PN, it means that the timeslot is assigned to an Ingress Port (in respect to the isochronous traffic targeting the Egress Port) designated by a Port Number of PN. Therefore, N_{link} virtual timeslots are assigned to the Ingress Port when there are N_{link} entries in the table with value of PN. The Egress Port may admit one isochronous request transaction from the Ingress Port for further service only when the table entry reached by the Egress Port's isochronous time ticker (that increments by 1 every t time and wraps around when reaching T) is set to PN. Even if there are outstanding isochronous requests ready in the Ingress Port, they will not be served until next round of time-based WRR arbitration. In this manner, the time-based Port Arbitration Table serves for both isochronous bandwidth assignment and isochronous traffic regulation.

For an Endpoint serving as a Requester or a Completer, isochronous bandwidth allocation is accomplished through negotiation between system software and device driver, which is outside of the scope of this specification.

A.2.4 Isochronous Transaction Latency §

Transaction latency is composed of the latency through the PCI Express fabric and the latency contributed by the Completer. Isochronous transaction latency is defined for each transaction and measured in units of virtual timeslot t .

- The *read latency* is defined as the round-trip latency. This is the delay from the time when the device submits a memory read request packet to its Transaction Layer (Transmit side) to the time when the corresponding read completion arrives at the device's Transaction Layer (Receive side).
- The *write latency* is defined as the delay from the time when the Requester posts a memory write request to its PCI Express Transaction Layer (Transmit side) to the time when the data write becomes globally visible within the memory subsystem of the Completer. A write to memory reaches the point of global visibility when all agents accessing that memory address get the updated data.

When the upper bound and the lower bound of isochronous transaction latency are provided, the size of isochronous data buffers in a Requester can be determined. For most of common platforms, the minimum isochronous transaction latency is much smaller than the maximum. As a conservative measure, the minimum isochronous transaction latency is assumed to be zero; only guidelines on measuring the maximum isochronous transaction latency are provided here.

For a Requester, the maximum isochronous (read or write) transaction latency (L) can be accounted as the following:

$$L = L_{Fabric} + L_{Completer} \quad \S$$

Equation A-7 Max Isochronous Transaction Latency

where L_{Fabric} is the maximum latency of the PCI Express fabric and $L_{Completer}$ is the maximum latency of the Completer.

L_{Fabric} which applies to both read and write transactions, depends on the topology, latency across each PCI Express Link, and the arbitration point in the path between the Requester to the Completer. The latency on a PCI Express Link

depends on pipeline delays, width and operational frequency of the Link, transmission of electrical signals across the medium, wake up latency from low power states, and delays caused by Data Link Layer Retry.

A restriction on the PCI Express topology may be imposed for each targeted platform in order to provide a practically meaningful guideline for L_{Fabric} . The values of L_{Fabric} should be reasonable and serve as practical upper limits under normal operating conditions.

The value of $L_{Completer}$ depends on the memory technology, memory configuration, and the arbitration policies in the Completer that comprehend PCI Express isochronous traffic. The target value for $L_{Completer}$ should provide enough headroom to allow for implementation tradeoffs.

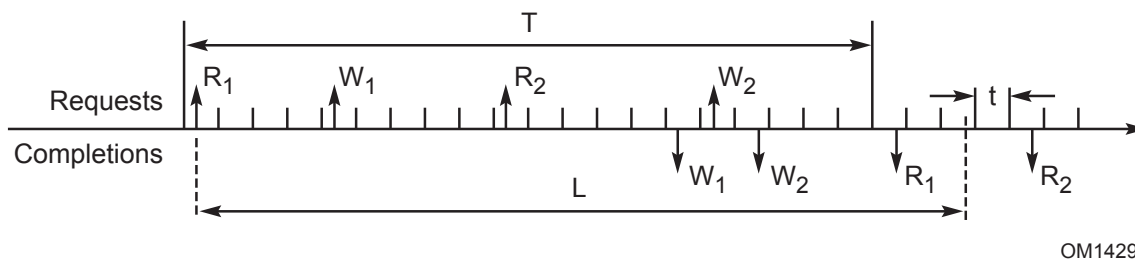
Definitions of read and write transaction latencies for a Completer are different:

- Read transaction latency for the Completer is defined as the delay from the time a memory read transaction is available at the Receiver end of a PCI Express Port in the Completer to the time the corresponding read completion transaction is posted to the transmission end of the PCI Express Port.
- Write transaction latency is defined as the delay from the time a memory write transaction is available at the Receiver end of a PCI Express Port in the Completer to the time that the transmitted data is globally visible.

All of the isochronous transaction latencies defined above are based on the assumption that the Requester injects isochronous transactions uniformly. According to an isochronous contract of $\{N, T, t\}$, the uniform traffic injection is defined such that up to N transactions are evenly distributed over the isochronous period T based on a ticker granularity of virtual timeslot t . For a Requester with non-uniform isochronous transaction injection, the Requester is responsible of accounting for any additional delay due to the deviation of its injection pattern from a uniform injection pattern.

A.2.5 An Example Illustrating Isochronous Parameters §

§ Figure A-3 illustrates the key isochronous parameters using a simplified example with $T = 20t$ and $L = 22t$. A Requester has reserved isochronous bandwidth of four transactions per T . The device shares the allocated isochronous bandwidths for both read requests and write requests. As shown, during one isochronous time period, the Requester issues two read requests and two write requests. All requests are completed within the designated transaction latency L . Also shown in the figure, there is no time dependency between the service time of write requests and the arrival time of read completions.



OM14290

Figure A-3 A Simplified Example Illustrating PCI Express Isochronous Parameters

A.3 Isochronous Transaction Rules §

Isochronous transactions follow the same rules as described in § Chapter 2. . In order to assist the Completer to meet latency requirements, the following additional rules further illustrate and clarify the proper behavior of isochronous transactions:

- The value in the Length field of requests must never exceed Max_Payload_Size.

A.4 Transaction Ordering §

In general, isochronous transactions follow the ordering rules described in § Section 2.4 . The following ordering rule further illustrates and clarifies the proper behavior of isochronous transactions:

- There are no ordering guarantees between any isochronous and non-isochronous transactions because the traffic has been segregated into distinct VC resources.
- Isochronous write requests are serviced on any PCI Express Link in strictly the same order as isochronous write requests are posted.
- Switches must allow isochronous posted requests to pass isochronous read completions.

A.5 Isochronous Data Coherency §

Cache coherency for isochronous transactions is an operating system software and Root Complex hardware issue. PCI Express provides the necessary mechanism to control Root Complex behavior in terms of enforcing hardware cache coherency on a per transaction basis.

For platforms where snoop latency in a Root Complex is either unbounded or can be excessively large, in order to meet tight maximum isochronous transaction latency $L_{Completer}$, or more precisely $L_{Root_Complex}$, all isochronous transactions should have the No Snoop Attribute bit set.

A Root Complex must report the Root Complex's capability to the system software by setting the Reject Snoop Transactions field in the VC Resource Capability register (for any VC resource capable of supporting isochronous traffic) in its RCRB. Based on whether or not a Root Complex is capable of providing hardware enforced cache coherency for isochronous traffic while still meeting isochronous latency target, system software can then inform the device driver of Endpoints to set or unset the No Snoop Attribute bit for isochronous transactions.

Note that cache coherency considerations for isochronous traffic do not apply to peer-to-peer communication.

A.6 Flow Control §

Completers and PCI Express fabric components should implement proper sizing of buffers such that under normal operating conditions, no backpressure due to flow control should be applied to isochronous traffic injected uniformly by a Requester. For Requesters that are compliant to the isochronous contract, but have bursty injection behavior, Switches and Completers may apply flow control backpressure as long as the admitted isochronous traffic is uniform and compliant to the isochronous contract. Under abnormal conditions when isochronous traffic jitter becomes significant or when isochronous traffic is oversubscribed due to excessive Data Link Layer Retry, flow control provides a natural mechanism to ensure functional correctness.

A.7 Considerations for Bandwidth Allocation §

A.7.1 Isochronous Bandwidth of PCI Express Links §

Isochronous bandwidth budgeting for PCI Express Links can be derived based on Link parameters such as isochronous payload size and the speed and width of the Link.

Isochronous bandwidth allocation for a PCI Express Link should be limited to certain percentage of the maximum effective Link bandwidth in order to leave sufficient bandwidth for non-isochronous traffic and to account for temporary Link bandwidth reduction due to retries. Link utilization is counted based on the actual cycles consumed on the physical PCI Express Link. The maximum number of virtual slots allowed per Link (N_{link}) depends on the isochronous packet payload size and the speed and width of the Link.

As isochronous bandwidth allocation on a PCI Express Link is based on number of requests N_{link} per isochronous period. There is no distinction between read requests and write requests in budgeting isochronous bandwidth on a PCI Express Link.

A.7.2 Isochronous Bandwidth of Endpoints §

For peer-to-peer communication, the device driver is responsible for reporting to system software if the device is capable of being a Completer for isochronous transactions. In addition, the driver must report the device's isochronous bandwidth capability. The specifics of the report mechanism are outside the scope of this specification.

A.7.3 Isochronous Bandwidth of Switches §

Allocation of isochronous bandwidth for a Switch must consider the capacity and utilization of PCI Express Links associated with the Ingress Port and the Egress Port of the Switch that connect the Requester and the Completer, respectively. The lowest common denominator of the two determines if a requested isochronous bandwidth can be supported.

A.7.4 Isochronous Bandwidth of Root Complex §

Isochronous bandwidth of Root Complex is reported to the software through its RCRB structure. Specifically, the Maximum Time Slots field of the VC Resource Capability register in VC Extended Capability structure indicates the total isochronous bandwidth shared by the Root Ports associated with the RCRB. Details of the platform budgeting for available isochronous bandwidth within a Root Complex are outside of the scope of this specification.

A.8 Considerations for PCI Express Components §

A.8.1 An Endpoint as a Requester §

Before an Endpoint as a Requester can start issuing isochronous request transactions, the following configuration steps must be performed by software:

- Configuration of at least one VC resource capable of supporting isochronous communication and assignment of at least one TC label.
- Enablement of this VC resource.

When the Requester uniformly injects isochronous requests, the Receive Port, either a Switch Port or a Root Port, should issue Flow Control credits back promptly such that no backpressure should be applied to the associated VC. This type of Requester may size its buffer based on the PCI Express fabric latency L_{Fabric} plus the Completer's latency $L_{Completer}$.

When isochronous transactions are injected non-uniformly, either some transactions experience longer PCI Express fabric delay or the Requester gets back-pressured on the associated VC. This type of Requester must size its buffer to account for the deviation of its injection pattern from uniformity.

A.8.2 An Endpoint as a Completer §

An Endpoint may serve as a Completer for isochronous peer-to-peer communication. Before an Endpoint starts serving isochronous transactions, system software must identify/configure a VC resource capable of supporting isochronous traffic and assigned a corresponding TC label.

An Endpoint Completer must observe the maximum isochronous transaction latency ($L_{Completer}$). An Endpoint Completer does not have to regulate isochronous request traffic if attached to a Switch since Switches implement traffic regulation. However, an Endpoint Completer must size its internal buffer such that no backpressure should be applied to the corresponding VC.

A.8.3 Switches §

A Switch may have multiple ports capable of supporting isochronous transactions. Before a Switch starts serving isochronous transactions for a Port, the software must perform the following configuration steps:

- Configuration/enablement of at least one VC resource capable of supporting isochronous communication.
- Configuration of the Port as an Ingress Port:
 - Configuration (or reconfiguration if the associated VC of the Egress Port is already enabled) of the time-based WRR Port Arbitration Table of the targeting Egress Port to include N_{link} entries set to the Ingress Port's Port Number. Here N_{link} is the isochronous allocation for the Ingress Port.
 - Enabling the targeting Egress Port to load newly programmed Port Arbitration Table.
- Configuration of the Port as an Egress Port:
 - Configuration of each VC's Port Arbitration Table with number of entries set according to the assigned isochronous bandwidth for all Ingress Ports.
 - Select proper VC Arbitration, e.g., as strict-priority based VC Arbitration.
 - If required, configuration of the Port's VC Arbitration Table with large weights assigned accordingly to each associated VC.

Each VC associated with isochronous traffic may be served as the highest priority in arbitrating for the shared PCI Express Link resource at an Egress Port. This is comprehended by a Switch's internal arbitration scheme.

In addition, a Switch Port may use “just in time” scheduling mechanism to reduce VC arbitration latency. Instead of pipelining non-isochronous Transport Layer packets to the Data Link Layer of the Egress Port in a manner that Data Link Layer transmit buffer becomes saturated, the Switch Port may hold off scheduling of a new non-isochronous packet to the Data Link Layer as long as it is possible without incurring unnecessary Link idle time.

When a VC configured to support isochronous traffic is enabled for a Switch Port (ingress) that is connected to a Requester, the Switch must enforce proper traffic regulation to ensure that isochronous traffic from the Port conforms to this specification. With such enforcement, normal isochronous transactions from compliant Requesters will not be impacted by ill behavior of any non-compliant Requester.

The above isochronous traffic regulation mechanism only applies to request transactions but not to completion transactions. When Endpoint-to-Root-Complex and peer-to-peer communications co-exist in a Switch, an Egress Port may mix isochronous write requests and read completions in the same direction. In the case of contention, the Egress Port must allow write requests to pass read completions to ensure the Switch meets latency requirement for isochronous requests.

A.8.4 Root Complex §

A Root Complex may have multiple Root Ports capable of supporting isochronous transactions. Before a Root Complex starts serving isochronous transactions for a Root Port, the Port must be configured by software to enable VC to support isochronous traffic using the following configuration steps:

- Configuration of at least one VC resource capable of supporting isochronous communication and assignment of at least one TC label.
- Configuration of the Root Port as an Ingress Port:
 - Configuration (or reconfiguration if the associated VC in RCRB is already enabled) of the time-based WRR Port Arbitration Table of the targeting RCRB to include N_{link} entries set to the Ingress Port's Port Number. Here N_{link} is the isochronous allocation for the Port.
 - Enabling the targeting RCRB to load newly programmed Port Arbitration Table.
- Configuration of the Root Port as an Egress Port:
 - If supported, configuration of the Root Port's VC Arbitration Table with large weights assigned to the associated VC.
 - If the Root Complex supports peer-to-peer traffic between Root Ports, configuration of the Root Port's Port Arbitration Table number of entries is set according to the assigned isochronous bandwidth for all Ingress Ports.

A Root Complex must observe the maximum isochronous transaction latency ($L_{Completer}$ or more precisely $L_{Root_Complex}$) that applies to all the Root Ports in the Root Complex. How a Root Complex schedules memory cycles for PCI Express isochronous transactions and other memory transactions is outside of the scope of this specification as long as $L_{Root_Complex}$ is met for PCI Express isochronous transactions.

When a VC is enabled to support isochronous traffic for a Root Port, the Root Complex must enforce proper traffic regulation to ensure that isochronous traffic from the Root Port conforms to this specification. With such enforcement, normal isochronous transactions from compliant Requesters will not be impacted by ill behavior of any non-compliant Requesters. Isochronous traffic regulation is implemented using the time-based Port Arbitration Table in RCRB.

Root Complex may perform the following operations for invalid isochronous transactions:

- Return partial completions for read requests with the value in the Length field exceeding Max_Payload_Size.

B. Symbol Encoding §

§ Table B-1 shows the byte-to-Symbol encodings for data characters. § Table B-2 shows the Symbol encodings for the Special Symbols used for TLP/DLLP Framing and for interface management. RD- and RD+ refer to the Running Disparity of the Symbol sequence on a per-Lane basis.

Table B-1 8b/10b Data Symbol Codes

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D0.0	00	000 00000	100111 0100	011000 1011
D1.0	01	000 00001	011101 0100	100010 1011
D2.0	02	000 00010	101101 0100	010010 1011
D3.0	03	000 00011	110001 1011	110001 0100
D4.0	04	000 00100	110101 0100	001010 1011
D5.0	05	000 00101	101001 1011	101001 0100
D6.0	06	000 00110	011001 1011	011001 0100
D7.0	07	000 00111	111000 1011	000111 0100
D8.0	08	000 01000	111001 0100	000110 1011
D9.0	09	000 01001	100101 1011	100101 0100
D10.0	0A	000 01010	010101 1011	010101 0100
D11.0	0B	000 01011	110100 1011	110100 0100
D12.0	0C	000 01100	001101 1011	001101 0100
D13.0	0D	000 01101	101100 1011	101100 0100
D14.0	0E	000 01110	011100 1011	011100 0100
D15.0	0F	000 01111	010111 0100	101000 1011
D16.0	10	000 10000	011011 0100	100100 1011
D17.0	11	000 10001	100011 1011	100011 0100
D18.0	12	000 10010	010011 1011	010011 0100
D19.0	13	000 10011	110010 1011	110010 0100
D20.0	14	000 10100	001011 1011	001011 0100
D21.0	15	000 10101	101010 1011	101010 0100
D22.0	16	000 10110	011010 1011	011010 0100
D23.0	17	000 10111	111010 0100	000101 1011

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D24.0	18	000 11000	110011 0100	001100 1011
D25.0	19	000 11001	100110 1011	100110 0100
D26.0	1A	000 11010	010110 1011	010110 0100
D27.0	1B	000 11011	110110 0100	001001 1011
D28.0	1C	000 11100	001110 1011	001110 0100
D29.0	1D	000 11101	101110 0100	010001 1011
D30.0	1E	000 11110	011110 0100	100001 1011
D31.0	1F	000 11111	101011 0100	010100 1011
D0.1	20	001 00000	100111 1001	011000 1001
D1.1	21	001 00001	011101 1001	100010 1001
D2.1	22	001 00010	101101 1001	010010 1001
D3.1	23	001 00011	110001 1001	110001 1001
D4.1	24	001 00100	110101 1001	001010 1001
D5.1	25	001 00101	101001 1001	101001 1001
D6.1	26	001 00110	011001 1001	011001 1001
D7.1	27	001 00111	111000 1001	000111 1001
D8.1	28	001 01000	111001 1001	000110 1001
D9.1	29	001 01001	100101 1001	100101 1001
D10.1	2A	001 01010	010101 1001	010101 1001
D11.1	2B	001 01011	110100 1001	110100 1001
D12.1	2C	001 01100	001101 1001	001101 1001
D13.1	2D	001 01101	101100 1001	101100 1001
D14.1	2E	001 01110	011100 1001	011100 1001
D15.1	2F	001 01111	010111 1001	101000 1001
D16.1	30	001 10000	011011 1001	100100 1001
D17.1	31	001 10001	100011 1001	100011 1001
D18.1	32	001 10010	010011 1001	010011 1001
D19.1	33	001 10011	110010 1001	110010 1001
D20.1	34	001 10100	001011 1001	001011 1001

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D21.1	35	001 10101	101010 1001	101010 1001
D22.1	36	001 10110	011010 1001	011010 1001
D23.1	37	001 10111	111010 1001	000101 1001
D24.1	38	001 11000	110011 1001	001100 1001
D25.1	39	001 11001	100110 1001	100110 1001
D26.1	3A	001 11010	010110 1001	010110 1001
D27.1	3B	001 11011	110110 1001	001001 1001
D28.1	3C	001 11100	001110 1001	001110 1001
D29.1	3D	001 11101	101110 1001	010001 1001
D30.1	3E	001 11110	011110 1001	100001 1001
D31.1	3F	001 11111	101011 1001	010100 1001
D0.2	40	010 00000	100111 0101	011000 0101
D1.2	41	010 00001	011101 0101	100010 0101
D2.2	42	010 00010	101101 0101	010010 0101
D3.2	43	010 00011	110001 0101	110001 0101
D4.2	44	010 00100	110101 0101	001010 0101
D5.2	45	010 00101	101001 0101	101001 0101
D6.2	46	010 00110	011001 0101	011001 0101
D7.2	47	010 00111	111000 0101	000111 0101
D8.2	48	010 01000	111001 0101	000110 0101
D9.2	49	010 01001	100101 0101	100101 0101
D10.2	4A	010 01010	010101 0101	010101 0101
D11.2	4B	010 01011	110100 0101	110100 0101
D12.2	4C	010 01100	001101 0101	001101 0101
D13.2	4D	010 01101	101100 0101	101100 0101
D14.2	4E	010 01110	011100 0101	011100 0101
D15.2	4F	010 01111	010111 0101	101000 0101
D16.2	50	010 10000	011011 0101	100100 0101
D17.2	51	010 10001	100011 0101	100011 0101

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D18.2	52	010 10010	010011 0101	010011 0101
D19.2	53	010 10011	110010 0101	110010 0101
D20.2	54	010 10100	001011 0101	001011 0101
D21.2	55	010 10101	101010 0101	101010 0101
D22.2	56	010 10110	011010 0101	011010 0101
D23.2	57	010 10111	111010 0101	000101 0101
D24.2	58	010 11000	110011 0101	001100 0101
D25.2	59	010 11001	100110 0101	100110 0101
D26.2	5A	010 11010	010110 0101	010110 0101
D27.2	5B	010 11011	110110 0101	001001 0101
D28.2	5C	010 11100	001110 0101	001110 0101
D29.2	5D	010 11101	101110 0101	010001 0101
D30.2	5E	010 11110	011110 0101	100001 0101
D31.2	5F	010 11111	101011 0101	010100 0101
D0.3	60	011 00000	100111 0011	011000 1100
D1.3	61	011 00001	011101 0011	100010 1100
D2.3	62	011 00010	101101 0011	010010 1100
D3.3	63	011 00011	110001 1100	110001 0011
D4.3	64	011 00100	110101 0011	001010 1100
D5.3	65	011 00101	101001 1100	101001 0011
D6.3	66	011 00110	011001 1100	011001 0011
D7.3	67	011 00111	111000 1100	000111 0011
D8.3	68	011 01000	111001 0011	000110 1100
D9.3	69	011 01001	100101 1100	100101 0011
D10.3	6A	011 01010	010101 1100	010101 0011
D11.3	6B	011 01011	110100 1100	110100 0011
D12.3	6C	011 01100	001101 1100	001101 0011
D13.3	6D	011 01101	101100 1100	101100 0011
D14.3	6E	011 01110	011100 1100	011100 0011

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D15.3	6F	011 01111	010111 0011	101000 1100
D16.3	70	011 10000	011011 0011	100100 1100
D17.3	71	011 10001	100011 1100	100011 0011
D18.3	72	011 10010	010011 1100	010011 0011
D19.3	73	011 10011	110010 1100	110010 0011
D20.3	74	011 10100	001011 1100	001011 0011
D21.3	75	011 10101	101010 1100	101010 0011
D22.3	76	011 10110	011010 1100	011010 0011
D23.3	77	011 10111	111010 0011	000101 1100
D24.3	78	011 11000	110011 0011	001100 1100
D25.3	79	011 11001	100110 1100	100110 0011
D26.3	7A	011 11010	010110 1100	010110 0011
D27.3	7B	011 11011	110110 0011	001001 1100
D28.3	7C	011 11100	001110 1100	001110 0011
D29.3	7D	011 11101	101110 0011	010001 1100
D30.3	7E	011 11110	011110 0011	100001 1100
D31.3	7F	011 11111	101011 0011	010100 1100
D0.4	80	100 00000	100111 0010	011000 1101
D1.4	81	100 00001	011101 0010	100010 1101
D2.4	82	100 00010	101101 0010	010010 1101
D3.4	83	100 00011	110001 1101	110001 0010
D4.4	84	100 00100	110101 0010	001010 1101
D5.4	85	100 00101	101001 1101	101001 0010
D6.4	86	100 00110	011001 1101	011001 0010
D7.4	87	100 00111	111000 1101	000111 0010
D8.4	88	100 01000	111001 0010	000110 1101
D9.4	89	100 01001	100101 1101	100101 0010
D10.4	8A	100 01010	010101 1101	010101 0010
D11.4	8B	100 01011	110100 1101	110100 0010

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D12.4	8C	100 01100	001101 1101	001101 0010
D13.4	8D	100 01101	101100 1101	101100 0010
D14.4	8E	100 01110	011100 1101	011100 0010
D15.4	8F	100 01111	010111 0010	101000 1101
D16.4	90	100 10000	011011 0010	100100 1101
D17.4	91	100 10001	100011 1101	100011 0010
D18.4	92	100 10010	010011 1101	010011 0010
D19.4	93	100 10011	110010 1101	110010 0010
D20.4	94	100 10100	001011 1101	001011 0010
D21.4	95	100 10101	101010 1101	101010 0010
D22.4	96	100 10110	011010 1101	011010 0010
D23.4	97	100 10111	111010 0010	000101 1101
D24.4	98	100 11000	110011 0010	001100 1101
D25.4	99	100 11001	100110 1101	100110 0010
D26.4	9A	100 11010	010110 1101	010110 0010
D27.4	9B	100 11011	110110 0010	001001 1101
D28.4	9C	100 11100	001110 1101	001110 0010
D29.4	9D	100 11101	101110 0010	010001 1101
D30.4	9E	100 11110	011110 0010	100001 1101
D31.4	9F	100 11111	101011 0010	010100 1101
D0.5	A0	101 00000	100111 1010	011000 1010
D1.5	A1	101 00001	011101 1010	100010 1010
D2.5	A2	101 00010	101101 1010	010010 1010
D3.5	A3	101 00011	110001 1010	110001 1010
D4.5	A4	101 00100	110101 1010	001010 1010
D5.5	A5	101 00101	101001 1010	101001 1010
D6.5	A6	101 00110	011001 1010	011001 1010
D7.5	A7	101 00111	111000 1010	000111 1010
D8.5	A8	101 01000	111001 1010	000110 1010

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D9.5	A9	101 01001	100101 1010	100101 1010
D10.5	AA	101 01010	010101 1010	010101 1010
D11.5	AB	101 01011	110100 1010	110100 1010
D12.5	AC	101 01100	001101 1010	001101 1010
D13.5	AD	101 01101	101100 1010	101100 1010
D14.5	AE	101 01110	011100 1010	011100 1010
D15.5	AF	101 01111	010111 1010	101000 1010
D16.5	B0	101 10000	011011 1010	100100 1010
D17.5	B1	101 10001	100011 1010	100011 1010
D18.5	B2	101 10010	010011 1010	010011 1010
D19.5	B3	101 10011	110010 1010	110010 1010
D20.5	B4	101 10100	001011 1010	001011 1010
D21.5	B5	101 10101	101010 1010	101010 1010
D22.5	B6	101 10110	011010 1010	011010 1010
D23.5	B7	101 10111	111010 1010	000101 1010
D24.5	B8	101 11000	110011 1010	001100 1010
D25.5	B9	101 11001	100110 1010	100110 1010
D26.5	BA	101 11010	010110 1010	010110 1010
D27.5	BB	101 11011	110110 1010	001001 1010
D28.5	BC	101 11100	001110 1010	001110 1010
D29.5	BD	101 11101	101110 1010	010001 1010
D30.5	BE	101 11110	011110 1010	100001 1010
D31.5	BF	101 11111	101011 1010	010100 1010
D0.6	C0	110 00000	100111 0110	011000 0110
D1.6	C1	110 00001	011101 0110	100010 0110
D2.6	C2	110 00010	101101 0110	010010 0110
D3.6	C3	110 00011	110001 0110	110001 0110
D4.6	C4	110 00100	110101 0110	001010 0110
D5.6	C5	110 00101	101001 0110	101001 0110

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D6.6	C6	110 00110	011001 0110	011001 0110
D7.6	C7	110 00111	111000 0110	000111 0110
D8.6	C8	110 01000	111001 0110	000110 0110
D9.6	C9	110 01001	100101 0110	100101 0110
D10.6	CA	110 01010	010101 0110	010101 0110
D11.6	CB	110 01011	110100 0110	110100 0110
D12.6	CC	110 01100	001101 0110	001101 0110
D13.6	CD	110 01101	101100 0110	101100 0110
D14.6	CE	110 01110	011100 0110	011100 0110
D15.6	CF	110 01111	010111 0110	101000 0110
D16.6	D0	110 10000	011011 0110	100100 0110
D17.6	D1	110 10001	100011 0110	100011 0110
D18.6	D2	110 10010	010011 0110	010011 0110
D19.6	D3	110 10011	110010 0110	110010 0110
D20.6	D4	110 10100	001011 0110	001011 0110
D21.6	D5	110 10101	101010 0110	101010 0110
D22.6	D6	110 10110	011010 0110	011010 0110
D23.6	D7	110 10111	111010 0110	000101 0110
D24.6	D8	110 11000	110011 0110	001100 0110
D25.6	D9	110 11001	100110 0110	100110 0110
D26.6	DA	110 11010	010110 0110	010110 0110
D27.6	DB	110 11011	110110 0110	001001 0110
D28.6	DC	110 11100	001110 0110	001110 0110
D29.6	DD	110 11101	101110 0110	010001 0110
D30.6	DE	110 11110	011110 0110	100001 0110
D31.6	DF	110 11111	101011 0110	010100 0110
D0.7	E0	111 00000	100111 0001	011000 1110
D1.7	E1	111 00001	011101 0001	100010 1110
D2.7	E2	111 00010	101101 0001	010010 1110

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D3.7	E3	111 00011	110001 1110	110001 0001
D4.7	E4	111 00100	110101 0001	001010 1110
D5.7	E5	111 00101	101001 1110	101001 0001
D6.7	E6	111 00110	011001 1110	011001 0001
D7.7	E7	111 00111	111000 1110	000111 0001
D8.7	E8	111 01000	111001 0001	000110 1110
D9.7	E9	111 01001	100101 1110	100101 0001
D10.7	EA	111 01010	010101 1110	010101 0001
D11.7	EB	111 01011	110100 1110	110100 1000
D12.7	EC	111 01100	001101 1110	001101 0001
D13.7	ED	111 01101	101100 1110	101100 1000
D14.7	EE	111 01110	011100 1110	011100 1000
D15.7	EF	111 01111	010111 0001	101000 1110
D16.7	F0	111 10000	011011 0001	100100 1110
D17.7	F1	111 10001	100011 0111	100011 0001
D18.7	F2	111 10010	010011 0111	010011 0001
D19.7	F3	111 10011	110010 1110	110010 0001
D20.7	F4	111 10100	001011 0111	001011 0001
D21.7	F5	111 10101	101010 1110	101010 0001
D22.7	F6	111 10110	011010 1110	011010 0001
D23.7	F7	111 10111	111010 0001	000101 1110
D24.7	F8	111 11000	110011 0001	001100 1110
D25.7	F9	111 11001	100110 1110	100110 0001
D26.7	FA	111 11010	010110 1110	010110 0001
D27.7	FB	111 11011	110110 0001	001001 1110
D28.7	FC	111 11100	001110 1110	001110 0001
D29.7	FD	111 11101	101110 0001	010001 1110
D30.7	FE	111 11110	011110 0001	100001 1110
D31.7	FF	111 11111	101011 0001	010100 1110

Table B-2 8b/10b Special Character Symbol Codes

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
K28.0	1C	000 11100	001111 0100	110000 1011
K28.1	3C	001 11100	001111 1001	110000 0110
K28.2	5C	010 11100	001111 0101	110000 1010
K28.3	7C	011 11100	001111 0011	110000 1100
K28.4	9C	100 11100	001111 0010	110000 1101
K28.5	BC	101 11100	001111 1010	110000 0101
K28.6	DC	110 11100	001111 0110	110000 1001
K28.7	FC	111 11100	001111 1000	110000 0111
K23.7	F7	111 10111	111010 1000	000101 0111
K27.7	FB	111 11011	110110 1000	001001 0111
K29.7	FD	111 11101	101110 1000	010001 0111
K30.7	FE	111 11110	011110 1000	100001 0111

C. Physical Layer Appendix §

C.1 8b/10b Data Scrambling Example §

The following subroutines encode and decode an 8-bit value contained in “inbyte” with the LFSR. This is presented as one example only; there are many ways to obtain the proper output. This example demonstrates how to advance the LFSR eight times in one operation and how to XOR the data in one operation. Many other implementations are possible but they must all produce the same output as that shown here.

The following algorithm uses the “C” programming language conventions, where “<<” and “>>” represent the shift left and shift right operators, “>” is the compare greater than operator, and “^” is the exclusive or operator, and “&” is the logical “AND” operator.

```

/*
  this routine implements the serial descrambling algorithm in parallel form
  for the LSFR polynomial:  $x^{16}+x^5+x^4+x^3+1$ 
  this advances the LSFR 8 bits every time it is called
  this requires fewer than 25 xor gates to implement (with a static register)
  The XOR required to advance 8 bits/clock is:
bit 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
    8  9 10 11 12 13 14 15 0  1  2  3  4  5  6  7
      8  9 10 11 12 13 14 15
        8  9 10 11 12 13 14 15
          8  9 10 11 12 13 14 15
  The serial data is just the reverse of the upper byte:
bit 0  1  2  3  4  5  6  7
    15 14 13 12 11 10 9  8
*/

```

```

int scramble_byte(int inbyte)
{
    static int scrambit[16];
    static int bit[16];
    static int bit_out[16];
    static unsigned short lfsr = 0xffff; // 16 bit short for polynomial
    int i, outbyte;

    if (inbyte == COMMA)           // if this is a comma
    {
        lfsr = 0xffff;           // reset the LFSR
        return (COMMA);          // and return the same data
    }

    if (inbyte == SKIP)           // don't advance or encode on skip
        return (SKIP);

    for (i=0; i<16;i++)           // convert LFSR to bit array for legibility
        bit[i] = (lfsr >> i) & 1;

    for (i=0; i<8; i++)           // convert byte to be scrambled for legibility
        scrambit[i] = (inbyte >> i) & 1;

    // apply the xor to the data
    if (! (inbyte & 0x100) &&      // if not a KCODE, scramble the data
        ! (TrainingSequence == TRUE)) // and if not in the middle of
    {                               // a training sequence
        scrambit[0] ^= bit[15];
        scrambit[1] ^= bit[14];
        scrambit[2] ^= bit[13];
        scrambit[3] ^= bit[12];
        scrambit[4] ^= bit[11];
        scrambit[5] ^= bit[10];
        scrambit[6] ^= bit[9];
        scrambit[7] ^= bit[8];
    }

    // Now advance the LFSR 8 serial clocks
    bit_out[ 0] = bit[ 8];
    bit_out[ 1] = bit[ 9];
    bit_out[ 2] = bit[10];
    bit_out[ 3] = bit[11] ^ bit[ 8];
    bit_out[ 4] = bit[12] ^ bit[ 9] ^ bit[ 8];
    bit_out[ 5] = bit[13] ^ bit[10] ^ bit[ 9] ^ bit[ 8];
    bit_out[ 6] = bit[14] ^ bit[11] ^ bit[10] ^ bit[ 9];
    bit_out[ 7] = bit[15] ^ bit[12] ^ bit[11] ^ bit[10];
    bit_out[ 8] = bit[ 0] ^ bit[13] ^ bit[12] ^ bit[11];
    bit_out[ 9] = bit[ 1] ^ bit[14] ^ bit[13] ^ bit[12];
    bit_out[10] = bit[ 2] ^ bit[15] ^ bit[14] ^ bit[13];
    bit_out[11] = bit[ 3] ^ bit[15] ^ bit[14];
    bit_out[12] = bit[ 4] ^ bit[15];
    bit_out[13] = bit[ 5];
    bit_out[14] = bit[ 6];
    bit_out[15] = bit[ 7];

    lfsr = 0;
    for (i=0; i <16; i++)         // convert the LFSR back to an integer
        lfsr += (bit_out[i] << i);
}

```

```
    outbyte = 0;
    for (i= 0; i<8; i++)      // convert data back to an integer
        outbyte += (scrambit[i] << i);

    return outbyte;
}
```

```

/* NOTE THAT THE DESCRAMBLE ROUTINE IS IDENTICAL TO THE SCRAMBLE ROUTINE
this routine implements the serial descrambling algorithm in parallel form
this advances the lfsr 8 bits every time it is called
this uses fewer than 25 xor gates to implement (with a static register)
The XOR tree is the same as the scrambling routine
*/

int unscramble_byte(int inbyte)
{
    static int descrambit[8];
    static int bit[16];
    static int bit_out[16];
    static unsigned short lfsr = 0xffff; // 16 bit short for polynomial
    int outbyte, i;

    if (inbyte == COMMA)    // if this is a comma
    {
        lfsr = 0xffff;      // reset the LFSR
        return (COMMA);     // and return the same data
    }
    if (inbyte == SKIP)     // don't advance or encode on skip
        return (SKIP);

    for (i=0; i<16;i++)    // convert the LFSR to bit array for legibility
        bit[i] = (lfsr >> i) & 1;

    for (i=0; i<8; i++)    // convert byte to be de-scrambled for legibility
        descrambit[i] = (inbyte >> i) & 1;

    // apply the xor to the data
    if (! (inbyte & 0x100) && // if not a KCODE, scramble the data
        ! (TrainingSequence == TRUE)) // and if not in the middle of
    {
        // a training sequence
        descrambit[0] ^= bit[15];
        descrambit[1] ^= bit[14];
        descrambit[2] ^= bit[13];
        descrambit[3] ^= bit[12];
        descrambit[4] ^= bit[11];
        descrambit[5] ^= bit[10];
        descrambit[6] ^= bit[9];
        descrambit[7] ^= bit[8];
    }

    // Now advance the LFSR 8 serial clocks
    bit_out[ 0] = bit[ 8];
    bit_out[ 1] = bit[ 9];
    bit_out[ 2] = bit[10];
    bit_out[ 3] = bit[11] ^ bit[ 8];
    bit_out[ 4] = bit[12] ^ bit[ 9] ^ bit[ 8];
    bit_out[ 5] = bit[13] ^ bit[10] ^ bit[ 9] ^ bit[ 8];
    bit_out[ 6] = bit[14] ^ bit[11] ^ bit[10] ^ bit[ 9];
    bit_out[ 7] = bit[15] ^ bit[12] ^ bit[11] ^ bit[10];
    bit_out[ 8] = bit[ 0] ^ bit[13] ^ bit[12] ^ bit[11];
    bit_out[ 9] = bit[ 1] ^ bit[14] ^ bit[13] ^ bit[12];
    bit_out[10] = bit[ 2] ^ bit[15] ^ bit[14] ^ bit[13];
    bit_out[11] = bit[ 3] ^ bit[15] ^ bit[14];
    bit_out[12] = bit[ 4] ^ bit[15];
    bit_out[13] = bit[ 5];
    bit_out[14] = bit[ 6];
    bit_out[15] = bit[ 7];

    lfsr = 0;
    for (i=0; i <16; i++)    // convert the LFSR back to an integer
        lfsr += (bit_out[i] << i);

    outbyte = 0;

    for (i= 0; i<8; i++)    // convert data back to an integer
        outbyte += (descrambit[i] << i);

    return outbyte;
}

```

The initial 16-bit values of the LFSR for the first 128 LFSR advances following a reset are listed below:

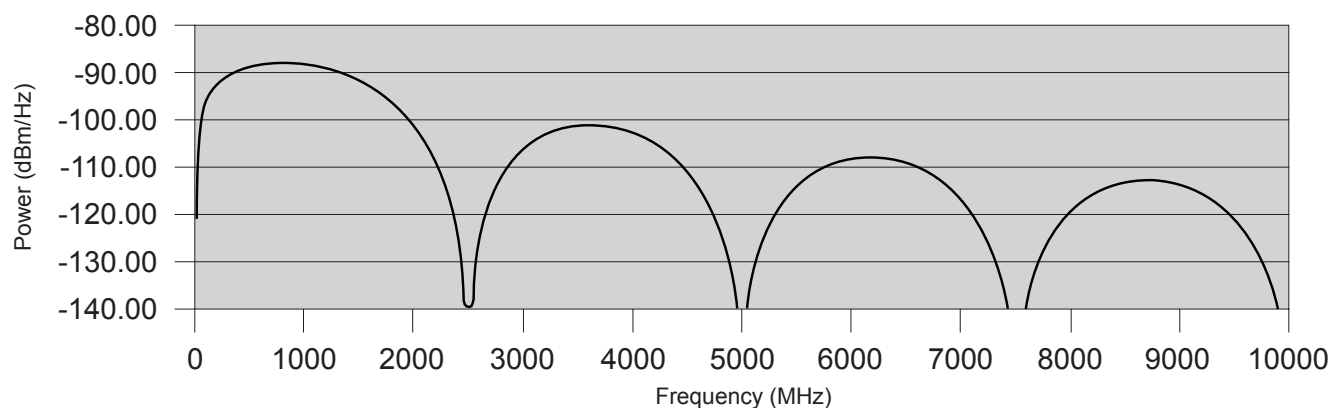
	0, 8	1, 9	2, A	3, B	4, C	5, D	6, E	7, F
00	FFFF	E817	0328	284B	4DE8	E755	404F	4140
08	4E79	761E	1466	6574	7DBD	B6E5	FDA6	B165
10	7D09	02E5	E572	673D	34CF	CB54	4743	4DEF
18	E055	40E0	EE40	54BE	B334	2C7B	7D0C	07E5
20	E5AF	BA3D	248A	8DC4	D995	85A1	BD5D	4425
28	2BA4	A2A3	B8D2	CBF8	EB43	5763	6E7F	773E
30	345F	5B54	5853	5F18	14B7	B474	6CD4	DC4C
38	5C7C	70FC	F6F0	E6E6	F376	603B	3260	64C2
40	CB84	9743	5CBF	B3FC	E47B	6E04	0C3E	3F2C
48	29D7	D1D1	C069	7BC0	CB73	6043	4A60	6FFA
50	F207	1102	01A9	A939	2351	566B	6646	4FF6
58	F927	3081	85B0	AC5D	478C	82EF	F3F2	E43B
60	2E04	027E	7E72	79AE	A501	1A7D	7F2A	2197
68	9019	0610	1096	9590	8FCD	D0E7	F650	46E6
70	E8D6	C228	3AB2	B70A	129F	9CE2	FC3C	2B5C
78	5AA3	AF6A	70C7	CDF0	E3D5	C0AB	B9C0	D9C1

An 8-bit value of 0 repeatedly encoded with the LFSR after reset produces the following consecutive 8-bit values:

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	FF	17	C0	14	B2	E7	02	82	72	6E	28	A6	BE	6D	BF	8D
10	BE	40	A7	E6	2C	D3	E2	B2	07	02	77	2A	CD	34	BE	E0
20	A7	5D	24	B1	9B	A1	BD	22	D4	45	1D	D3	D7	EA	76	EE
30	2C	DA	1A	FA	28	2D	36	3B	3A	0E	6F	67	CF	06	4C	26
40	D3	E9	3A	CD	27	76	30	FC	94	8B	03	DE	D3	06	52	F6
50	4F	88	80	95	C4	6A	66	F2	9F	0C	A1	35	E2	41	CF	27
60	74	40	7E	9E	A5	58	FE	84	09	60	08	A9	F1	0B	6F	62
70	17	43	5C	ED	48	39	3F	D4	5A	F5	0E	B3	C7	03	9D	9B
80	8B	0D	8E	5C	33	98	77	AE	2D	AC	0B	3E	DA	0B	42	7A
90	7C	D1	CF	A8	1C	12	EE	41	C2	3F	38	7A	0D	69	F4	01
A0	DA	31	72	C5	A0	D7	93	0E	DC	AF	A4	55	E7	F0	72	16
B0	68	D5	38	84	DD	00	CD	18	9E	CA	30	59	4C	75	1B	77

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
C0	31	C5	ED	CF	91	64	6E	3D	FE	E8	29	04	CF	6C	FC	C4
D0	0B	5E	DA	62	BA	5B	AB	DF	59	B7	7D	37	5E	E3	1A	C6
E0	88	14	F5	4F	8B	C8	56	CB	D3	10	42	63	04	8A	B4	F7
F0	84	01	A0	01	83	49	67	EE	3E	2A	8B	A4	76	AF	14	D5
100	4F	AC	60	B6	79	D6	62	B7	43	E7	E5	2A	40	2C	6E	7A
110	56	61	63	20	6A	97	4A	38	05	E5	DD	68	0D	78	4C	53
120	8B	D6	86	57	B2	AA	1A	80	18	DC	BA	FC	03	A3	4B	30

At 2.5 GT/s, scrambling produces the power spectrum (in the 10-bit domain) shown in § Figure C-1.



OM14292A

Figure C-1 Scrambling Spectrum at 2.5 GT/s for Data Value of 0

C.2 128b/130b Data Scrambling Example §

The following subroutines illustrate how calculate the next value of the 128b/130b scrambling LFSR and how to scramble (or descramble) an 8-bit value, both given the current value of the LFSR.

This is presented as one example only; there are many ways to obtain the proper output. This example demonstrates how to advance the LFSR eight times in one operation and how to XOR the data in one operation. Many other implementations are possible but they must all produce the same output as that shown here.

The following algorithm uses the “C” programming language conventions, where “<<” and “>>” represent the shift left and shift right operators, “^” is the exclusive or operator, and “|=” is the assignment by bitwise or operator.

```

#include <stdio.h>

// "Set Bit" – Sets bit number "bit" of "var" to the value "val". Bit "bit" of "var" must start cleared.
#define SB(var,bit,val) var |= (val & 1) << bit

// "Get Bit" – Returns the value of bit number "bit" of "var".
#define GB(var,bit) ((var >> bit) & 1)

// Function to advance the LFSR value by 8 bits, given the current LFSR value
unsigned long int calc_next_lfsr(unsigned long int lfsr) {
    unsigned long int next_lfsr = 0;
    SB(next_lfsr,22, GB(lfsr,14) ^ GB(lfsr,16) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(next_lfsr,21, GB(lfsr,13) ^ GB(lfsr,15) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,20) ^ GB(lfsr,21));
    SB(next_lfsr,20, GB(lfsr,12) ^ GB(lfsr,19) ^ GB(lfsr,21));
    SB(next_lfsr,19, GB(lfsr,11) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(next_lfsr,18, GB(lfsr,10) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,21));
    SB(next_lfsr,17, GB(lfsr,9) ^ GB(lfsr,16) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(next_lfsr,16, GB(lfsr,8) ^ GB(lfsr,15) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(next_lfsr,15, GB(lfsr,7) ^ GB(lfsr,22));
    SB(next_lfsr,14, GB(lfsr,6) ^ GB(lfsr,21));
    SB(next_lfsr,13, GB(lfsr,5) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(next_lfsr,12, GB(lfsr,4) ^ GB(lfsr,19) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(next_lfsr,11, GB(lfsr,3) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(next_lfsr,10, GB(lfsr,2) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,20) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(next_lfsr,9, GB(lfsr,1) ^ GB(lfsr,16) ^ GB(lfsr,18) ^ GB(lfsr,19) ^ GB(lfsr,20) ^ GB(lfsr,21));
    SB(next_lfsr,8, GB(lfsr,0) ^ GB(lfsr,15) ^ GB(lfsr,17) ^ GB(lfsr,18) ^ GB(lfsr,19) ^ GB(lfsr,20));
    SB(next_lfsr,7, GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,20) ^ GB(lfsr,21));
    SB(next_lfsr,6, GB(lfsr,16) ^ GB(lfsr,18) ^ GB(lfsr,19) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(next_lfsr,5, GB(lfsr,15) ^ GB(lfsr,17) ^ GB(lfsr,18) ^ GB(lfsr,19) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(next_lfsr,4, GB(lfsr,17));
    SB(next_lfsr,3, GB(lfsr,16));
    SB(next_lfsr,2, GB(lfsr,15) ^ GB(lfsr,22));
    SB(next_lfsr,1, GB(lfsr,16) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(next_lfsr,0, GB(lfsr,15) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,21) ^ GB(lfsr,22));
    return(next_lfsr);
}

// Function to scramble a byte, given the current LFSR value
unsigned char scramble_data(unsigned long lfsr, unsigned char data_in) {
    unsigned char data_out = 0;
    SB(data_out,7, GB(data_in,7) ^ GB(lfsr,15) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,21) ^ GB(lfsr,22));
    SB(data_out,6, GB(data_in,6) ^ GB(lfsr,16) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(data_out,5, GB(data_in,5) ^ GB(lfsr,17) ^ GB(lfsr,19) ^ GB(lfsr,21));
    SB(data_out,4, GB(data_in,4) ^ GB(lfsr,18) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(data_out,3, GB(data_in,3) ^ GB(lfsr,19) ^ GB(lfsr,21));
    SB(data_out,2, GB(data_in,2) ^ GB(lfsr,20) ^ GB(lfsr,22));
    SB(data_out,1, GB(data_in,1) ^ GB(lfsr,21));
    SB(data_out,0, GB(data_in,0) ^ GB(lfsr,22));
    return(data_out);
}

// Example of LFSR and scrambled data "0" sequence for Lane 0
main() {
    unsigned long lfsr = 0x1DBFBC; // Lane 0 reset LFSR value
    unsigned char unscrambled_data = 0x00;
    int i;
    printf("Iteration LFSR Next LFSR Scrambled Data\n");
    printf("- - -\n");
    for (i = 0; i < 128; i++) {
        unsigned char scrambled_data = scramble_data(lfsr,unscrambled_data);
        unsigned long next_lfsr = calc_next_lfsr(lfsr);
        printf("%3d %06X %06X %02X\n", i, lfsr, next_lfsr, scrambled_data);
        lfsr = next_lfsr;
    }
}

```

The initial 23-bit values of the LFSR for the first 128 LFSR advances for an 8-bit quantity following a reset for Lane 0 are listed below (ordered left to right, top to bottom):

	0, 8	1, 9	2, A	3, B	4, C	5, D	6, E	7, F
00	1DBFBC	498C2E	1186E9	0FC5AD	7CB75D	3D8DA2	0ECC8F	379717
08	046BDF	21D462	423FCE	5177D6	1447EF	67CBA0	794F7A	6CA2AF

	0, 8	1, 9	2, A	3, B	4, C	5, D	6, E	7, F
10	425060	3ED9D6	3DBF74	3C1A8F	7AE2E0	073E71	137D99	70B139
18	44F521	559720	1FBBA8	689D9F	376B02	597FFA	297C0E	7E450A
20	4BDE36	669B58	6BB530	78C3F9	0322C0	45C7FB	254F6A	323D89
28	07FDD2	71DFBC	68726B	799E27	1CFE8A	4AB864	42CB12	04AAF3
30	41F947	51F808	3A98CA	36A816	796895	4B4DAF	54037D	68E5C7
38	4F3302	60A51F	3AFCE3	528116	248131	1F65E6	17D2BA	34987E
40	6C0524	44DA45	7AF320	16FE71	5A5134	60B5F5	2A16E3	73AFF1
48	3D3ADA	18B5AA	4B9306	2BAB58	2D179E	5FDE68	46E1F8	644B91
50	3F78A4	7FCE1B	23CC59	7F017F	4DA97C	7EDF8B	705E13	0ADE04
58	6F1775	318CBE	70CC0C	39C021	0944ED	33F8AB	21DCBD	4AE0CE
60	1A6112	1B2F92	17ADD8	4BFA7E	42D358	1CE0F3	54C164	0BFDE2
68	0EF33F	082717	1200E1	4FCB73	39D53A	1C5FED	4ADE41	24EE12
70	7046E6	122B04	642E73	5A9AA4	0A24D0	34C250	362B24	5B5BB0
78	2833BF	73F640	648BDA	5E3281	490B97	373ECC	0CB1FA	6FE7A6

An 8-bit value of 0x00 repeatedly encoded with the LFSR after reset for Lane 0 produces the following consecutive 8-bit values (ordered left to right, top to bottom):

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	6C	BD	94	98	53	C6	D8	CE	50	6A	75	C1	04	4F	C3	07
10	75	26	C6	06	A3	B0	B4	AB	05	11	CC	57	4E	69	42	73
20	1D	0F	B7	03	E0	45	BA	5E	30	EB	D7	43	2C	5D	F5	D0
30	15	41	76	8E	C3	9D	D1	57	CD	FF	76	A1	7A	4C	64	2E
40	87	05	A3	24	89	FF	A2	4B	46	7C	1D	62	12	19	A5	2F
50	E6	B3	CA	33	ED	F3	2B	88	67	3E	AB	96	E8	9E	6A	5D
60	5C	1C	64	1D	F5	2C	51	C8	D8	A8	F4	4D	96	AC	5D	7A
70	2B	F4	2F	09	08	2E	0E	C9	02	4B	AF	D9	3D	4E	78	E7

D. Request Dependencies §

This specification does not specify the rules governing the creation of resource dependencies between TLPs using different Traffic Classes. Dependencies between packets in different Traffic Classes can create potential deadlock if devices make different assumptions about what is allowed and what is not. Dependencies can be created when a packet is forwarded (transmitted verbatim) or translated (transmitted with modification) from an input Port to an output Port.

Resource dependencies are created when received packets are forwarded/translated on the same or a different Link. Due to the fact that the forwarding/translating device has finite buffer resources this behavior creates a dependency between the ability to receive a packet and the ability to transmit a packet (in potentially a different VC or sub-channel).

The following notation is used to create a framework to enumerate the possibilities:

$X(m) \rightarrow Y(n)$

This means:

- a request in sub-channel $X(TC = m)$ is forwarded/translated in sub-channel $Y(TC = n)$.
- n and m are between 0-7.
- X and Y are either P (Posted Request), N (Non-Posted Request), or C (Completion).

The list of possible dependencies is:

$P(m) \rightarrow P(n)$
 $P(m) \rightarrow N(n)$
 $P(m) \rightarrow C(n)$
 $N(m) \rightarrow P(n)$
 $N(m) \rightarrow N(n)$
 $N(m) \rightarrow C(n)$
 $C(m) \rightarrow P(n)$
 $C(m) \rightarrow N(n)$
 $C(m) \rightarrow C(n)$

For a given system, each of these dependencies needs to be classified as legal or illegal for each of the following cases:

- Root Port forwarding to own Link.
- Root Port forwarding to different Root Port's Link.
- Endpoint or Bridge forwarding to own Link.

A Switch is not allowed to modify the TLPs that flow through it, but must ensure complete independence of resources assigned to separate VCs. System software must comprehend the system dependency rules when configuring TC/VC mapping throughout the system.

One possible legal mapping is:

	RC (Same Port)	RC (Different Port)	Endpoint
$P(m) \rightarrow P(n)$	$m \leq n$	$m \leq n$	$m < n$
$P(m) \rightarrow N(n)$	$m < n$	$m < n$	$m < n$
$P(m) \rightarrow C(n)$	illegal	illegal	illegal

	RC (Same Port)	RC (Different Port)	Endpoint
$N(m) \rightarrow P(n)$	$m < n$	$m < n$	$m < n$
$N(m) \rightarrow N(n)$	$m \leq n$	$m \leq n$	$m < n$
$N(m) \rightarrow C(n)$	$m = n$	$m = n$	$m = n$
$C(m) \rightarrow P(n)$	illegal	illegal	illegal
$C(m) \rightarrow N(n)$	illegal	illegal	illegal
$C(m) \rightarrow C(n)$	$m \geq n$	$m \geq n$	$m > n$

Note that this discussion only deals with avoiding the deadlock caused by the creation of resource dependencies. It does not deal with the additional livelock issues (or lack of forward progress) caused by the system's Virtual Channel arbitration policies.

Some of these potential dependencies are illegal or unreachable:

- $P(m) \rightarrow P(n), N(m) \rightarrow N(n)$
 - $m = n$ - This case is illegal and will lead to deadlock, except when a Request is being forwarded from one Port to another of a Switch or Root Complex.
 - $m \neq n$ - See discussion below.
- $P(m) \rightarrow N(n)$
 - $m = n$ - This case is illegal and will lead to deadlock.
 - $m \neq n$ - See discussion below.
- $N(m) \rightarrow P(n)$ - See discussion below.
- $P(m) \rightarrow C(n)$ - This case is illegal and will lead to deadlock.
- $N(m) \rightarrow C(n)$
 - $m = n$ - This case occurs during the normal servicing of a non-posted request by either a root complex or an endpoint.
 - $m \neq n$ - This case is unreachable and should never happen. Completions always use the same TC as the corresponding request.
- $C(m) \rightarrow P(n), C(m) \rightarrow N(n)$ - These cases are unreachable and should never happen due to the fact that completion buffers must be preallocated.
- $C(m) \rightarrow C(n)$
 - $m = n$ - This case is illegal and will lead to deadlock, except when a Completion is being forwarded from one Port to another of a Switch or Root Complex.
 - $m \neq n$ - This case will occur if $N(n) \rightarrow N(m)$ dependencies are allowed.

Other potential dependencies may be legal when comprehended as part of a specific usage model. For example, these cases:

$$P(m) \rightarrow P(n), N(m) \rightarrow N(n), P(m) \rightarrow N(n), N(m) \rightarrow P(n) \quad \text{where } m \neq n$$

might exist where a device services incoming requests by issuing modified versions of those requests using a different Traffic Class (for example, remapping the first requests address and generating the new request with the resulting address). In these cases, suitable rules must be applied to prevent circular dependencies that would lead to deadlock or livelock.

Examples of devices that may find the above mappings useful:

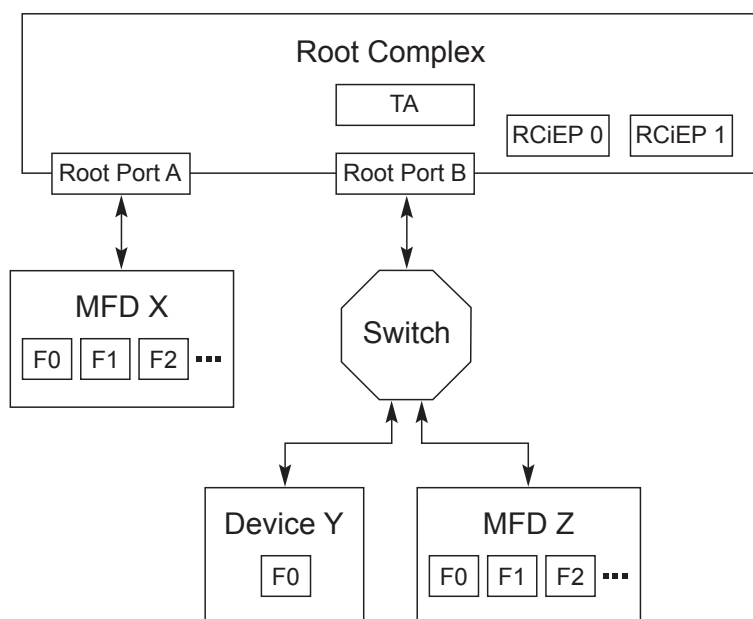
- Bridges to complex protocols that require state to be save/restored to/from host memory, i.e., PCI Express to Infiniband bridges.
- Messaging engines that must do address translation based upon page tables stored in host memory.
- UMA graphics devices that store their frame buffer in host memory.

E. ID-Based Ordering Usage §

E.1 Introduction §

ID-Based Ordering (IDO) is a mechanism that permits certain ordering restrictions to be relaxed as a means to improve performance. IDO permits certain TLPs to pass other TLPs in cases where otherwise such passing would be forbidden. The passing permitted by IDO is not required for proper operation (e.g., deadlock avoidance); it is only a means of improving performance.

For discussing IDO, it's useful to introduce the concept of a “TLP stream”, which is a set of TLPs that all have the same originator.¹⁸⁷ For several important cases where TLP passing is normally forbidden, IDO permits such passing to occur if the TLPs belong to different TLP streams.



A-0757A

Figure E-1 Reference Topology for IDO Use

§ Figure E-1 shows a reference topology. The reference topology is not intended to discourage the use of IDO with other topologies, but rather to provide specific examples for discussion.

Devices X and Z are Multi-Function Devices (MFDs); device Y is a single-Function device. The RCiEPs are Root Complex Integrated Endpoint Functions, and might or might not be part of the same Device. We will assume that one or more Functions are using the Translation Agent (TA) in the Root Complex (RC).

Referring to the ordering table and descriptions in § Section 2.4.1, having the IDO bit set in a Posted Request, Non-Posted Request, or Completion TLP permits that TLP to pass a Posted Request TLP if the two TLPs belong to different TLP streams. In the following examples, DMAR and DMAW stand for Direct Memory Access Read and Write; PIOR and PIOW stand for Programmed I/O Read and Write.

187. That is, the Requester IDs of Requests and the Completer IDs of Completions are all the same.

E.2 Potential Benefits with IDO Use §

Here are some example potential benefits that are envisioned with IDO use. Generally, IDO provides the most benefit when multiple TLP streams share a common Link and that Link becomes congested, either due to high utilization or due to temporary lack of Flow Control (FC) credit.

E.2.1 Benefits for MFD/RP Direct Connect §

Here are some examples in the context of traffic between MFD X and the RC in § Figure E-1.

- Posted Request passing another Posted Request: when a DMAW from F0 is stalled due to an TA miss, if IDO is set for a DMAW from F1, it is permitted within the RC for this DMAW to pass the stalled DMAW from F0.
- Non-Posted Request passing a Posted Request: when a DMAW from F0 is stalled due to an TA miss, if IDO is set for a DMAR Request from F1, it is permitted within the RC for this DMAR Request to pass the stalled DMAW from F0.
- Completion passing a Posted Request: when a DMAW from F0 is stalled due to an TA miss, if IDO is set for a PIOR Completion from F1, it is permitted within the RC for this PIOR Completion to pass the stalled DMAW from F0.

E.2.2 Benefits for Switched Environments §

Here are some examples in the context of traffic within the Switch in § Figure E-1.

- Non-Posted Request passing a Posted Request: when a DMAW from Device Y is stalled within the Switch due to a lack of FC credit from Root Port B, if IDO is set for a DMAR Request from MFD Z, it is permitted within the Switch for this DMAR Request to pass the stalled DMAW from Device Y. The same also holds for a DMAR Request from one Function in MFD Z passing a stalled DMAW from a different Function in MFD Z.
- Completion passing a Posted Request: when a DMAW from Device Y is stalled within the Switch due to a lack of FC credit from Root Port B, if IDO is set for a PIOR Completion from MFD Z, it is permitted within the Switch for this PIOR Completion to pass the stalled DMAW from Device Y. The same also holds for a PIOR Completion from one Function in MFD Z passing a stalled DMAW from a different Function in MFD Z.
- Posted Request passing another Posted Request: within a Switch, there is little or no envisioned benefit from having a DMAW from one TLP stream passing a DMAW from a different TLP stream. However, it is not prohibited for Switches to implement such passing as permitted by IDO.

E.2.3 Benefits for Integrated Endpoints §

Here are some examples for the Root Complex Integrated Endpoints (RCiEPs) in § Figure E-1. The benefits are basically the same as for the MFD/RP Direct Connect case.

- Posted Request passing another Posted Request: when a DMAW from RCiEP 0 is stalled due to an TA miss, if IDO is set for a DMAW from RCiEP 1, it is permitted for this DMAW to pass the stalled DMAW from RCiEP 0.
- Non-Posted Request passing a Posted Request: when a DMAW from RCiEP 0 is stalled due to an TA miss, if IDO is set for a DMAR Request from RCiEP 1, it is permitted for this DMAR Request to pass the stalled DMAW from RCiEP 0.

- Completion passing a Posted Request: when a DMAW from RCiEP 0 is stalled due to an TA miss, if IDO is set for a PIOR Completion from RCiEP 1, it is permitted for this PIOR Completion to pass the stalled DMAW from RCiEP 0.

E.2.4 IDO Use in Conjunction with RO §

IDO and RO¹⁸⁸ are orthogonal. Certain instances of passing; for example, a Posted Request passing another Posted Request, might be permitted by IDO, RO, or both at the same time. While IDO and RO have significant overlap for some cases, it is strongly recommended that both be used whenever safely possible. RO permits certain TLP passing within the same TLP stream, which is never permitted by IDO. For traffic in different TLP streams, IDO permits control traffic to pass any other traffic, and generally it is not safe to Set RO with control traffic.

E.3 When to Use IDO §

With Endpoint Functions¹⁸⁹, it is safe to Set IDO in all applicable TLPs originated by the Endpoint when the Endpoint is directly communicating with only one other entity, most commonly the RC. For the RC case, “directly communicating” specifically includes DMA traffic, PIO traffic, and interrupt traffic; communicating with RCiEPs or communicating using P2P Root Port traffic constitutes communicating with multiple entities.

With a Root Port, there are no envisioned high-benefit use models where it is safe to Set IDO in all applicable TLPs that it originates. Use models where a Root Port Sets IDO in a subset of the applicable TLPs it originates are outside the scope of this specification.

E.4 When Not to Use IDO §

E.4.1 When Not to Use IDO with Endpoints §

With Endpoint Functions, it is not always safe to Set IDO in applicable TLPs it originates if the Endpoint directly communicates with multiple entities. It may be safe to Set IDO in some TLPs and not others, but such use models are outside the scope of this specification.

For example, in § Figure E-1 if Device Y and MFD Z are communicating with P2P traffic and also communicating via host memory, it is not always safe for them to Set IDO in the TLPs they originate. As an example failure case, let’s assume that Device Y does a DMAW (to host memory) followed by a P2P Write to MFD Z. Upon observing the P2P Write, let’s assume that MFD Z then does a DMAW to the same location earlier targeted by the DMAW from Device Y. Normal ordering rules would guarantee that the DMAW from Device Y would be observed by host memory before the DMAW from MFD Z. However, if IDO is set in the DMAW from MFD Z, the RC would be permitted to have the second DMAW pass the first, causing a different end result in host memory contents.

Synchronization techniques like performing zero-length Reads might be used to avoid such communication failures when IDO is used, but specific use models are outside the scope of this specification.

188. In this Appendix, “RO” is an abbreviation for the Relaxed Ordering Attribute field.

189. Endpoint Functions include PCI Express Endpoints, Legacy PCI Express Endpoints, and Root Complex Integrated Endpoints.

E.4.2 When Not to Use IDO with Root Ports §

With Root Ports, it is not always safe to Set IDO in applicable TLPs it originates if Endpoint Functions in the hierarchy do any P2P traffic. It may be safe to Set IDO in some TLPs and not others, but such use models are outside the scope of this specification.

As an example, in § Figure E-1 if Device Y and MFD Z are communicating with P2P traffic and also communicating with host software, it is not always safe for Root Port B to Set IDO in the TLPs it originates. For example, let's assume that Device Y does a P2P Write to MFD Z followed by a DMAW (to host memory). Upon observing the DMAW, let's assume that the host does a PIOW to MFD Z. Normal ordering rules would guarantee that the P2P Write from Device Y would be observed by MFD Z before the PIOW from the host. However, if IDO is set in the PIOW from the host, the Switch would be permitted to have the PIOW pass the P2P Write, ultimately having the two Writes arrive at MFD Z out of order.

IMPLEMENTATION NOTE: REQUESTER AND COMPLETER IDS FOR RC-ORIGINATED TLPs §

With RC implementations where the Requester ID in a PIO Request does not match the Completer ID in a DMAR Completion, this enables another potential communication failure case if IDO is Set in the Completion. For this case, if a PIOW is followed by a DMAR Completion with IDO Set, a Switch below the Root Port could permit the DMAR Completion to pass the PIOW, violating the normal ordering rule that a non-RO Read Completion must not pass Posted Requests. The PIOW and DMAR Completion would appear to belong to different TLP streams, though logically they belong to the same TLP stream. Special caution is advised in setting IDO with TLPs originating from such RCs.

E.5 Software Control of IDO Use §

E.5.1 Software Control of Endpoint IDO Use §

By default, Endpoints are not enabled to Set IDO in any TLPs they originate.

IMPLEMENTATION NOTE: THE “SIMPLE” POLICY FOR IDO USE §

It is envisioned that Endpoints designed primarily to communicate directly with only one other entity (e.g., the RC) may find a “simple” policy for setting IDO to be adequate. Here's the envisioned “simple” policy. If the IDO Request Enable bit is Set, the Endpoint Sets IDO in all applicable Request TLPs that it originates. If the IDO Completion Enable bit is Set, the Endpoint Sets IDO in all Completion TLPs that it originates.

It is envisioned that a software driver associated with each Endpoint will determine when it is safe for the Endpoint to set IDO in applicable TLPs it originates. A driver should be able to determine if the Endpoint is communicating with multiple other entities, and should know the Endpoint's capabilities as far as setting IDO with all applicable TLPs when enabled, versus setting IDO selectively. If a driver determines that it is safe to enable the setting of IDO, the driver can set the IDO Request Enable and/or IDO Completion Enable bits either indirectly via OS services or directly, subject to OS policy.

If an Endpoint is designed for communication models where it is not safe to utilize the “simple” policy for IDO use, the Endpoint can implement more complex policies for determining when the Endpoint sets the IDO bit. Such implementations might utilize device-specific controls that are managed by the device driver. Such policies and device-specific control mechanisms are outside the scope of this specification.

E.5.2 Software Control of Root Port IDO Use §

Since there are no envisioned high-benefit “simple” use models for Root Ports setting the IDO bit with TLPs they originate, and there are known communication failure cases if Root Ports set the IDO bit with all applicable TLPs they originate, it is anticipated that Root Ports will rarely be enabled to set IDO in TLPs they originate. Such use models and policies for Root Ports setting IDO are outside the scope of this specification.

F. Message Code Usage §

§ Table F-1 contains a list of currently defined PCI Express Message Codes. Message codes are defined in this specification and in other specifications. This table will be updated as Messages are defined in other specifications but due to document release schedules, this table might not contain recently defined Messages.

§ *Table F-1 Message Code Usage*

Message Code	Routing r[2:0]	Type	Description
0000 0000	011	Msg	Unlock, see § Section 2.2.8.4
0000 0001	010	MsgD	Invalidate Request Message, see § Section 10.3.1
0000 0010	010	Msg	Invalidate Completion Message, see § Section 10.3.2
0000 0100	000	Msg	Page Request Message, see § Section 10.4.1
0000 0101	010	Msg	PRG Response Message, see § Section 10.4.2
0001 0000	100	Msg	Latency Tolerance Reporting (LTR) Message, see § Section 2.2.8.8
0001 0010	100	Msg	Optimized Buffer Flush/Fill (OBFF) Message, see § Section 2.2.8.9
0001 0100	100	Msg	PM_Active_State_Nak, see § Section 2.2.8.2
0001 1000	000	Msg	PM_PME, see § Section 2.2.8.2
0001 1001	011	Msg	PME_Turn_Off, see § Section 2.2.8.2
0001 1011	101	Msg	PME_TO_Ack, see § Section 2.2.8.2
0010 0000	100	Msg	Assert_INTA, see § Section 2.2.8.1
0010 0001	100	Msg	Assert_INTB, see § Section 2.2.8.1
0010 0010	100	Msg	Assert_INTC, see § Section 2.2.8.1
0010 0011	100	Msg	Assert_INTD, see § Section 2.2.8.1
0010 0100	100	Msg	Deassert_INTA, see § Section 2.2.8.1
0010 0101	100	Msg	Deassert_INTB, see § Section 2.2.8.1
0010 0110	100	Msg	Deassert_INTC, see § Section 2.2.8.1
0010 0111	100	Msg	Deassert_INTD, see § Section 2.2.8.1
0011 0000	000	Msg	ERR_COR, see § Section 2.2.8.3
0011 0001	000	Msg	ERR_NONFATAL, see § Section 2.2.8.3
0011 0011	000	Msg	ERR_FATAL, see § Section 2.2.8.3
0100 0000	100	Msg	Ignored Message, see § Section 2.2.8.7
0100 0001	100	Msg	Ignored Message, see § Section 2.2.8.7
0100 0011	100	Msg	Ignored Message, see § Section 2.2.8.7

Message Code	Routing r[2:0]	Type	Description
0100 0100	100	Msg	Ignored Message, see § Section 2.2.8.7
0100 0101	100	Msg	Ignored Message, see § Section 2.2.8.7
0100 0111	100	Msg	Ignored Message, see § Section 2.2.8.7
0100 1000	100	Msg	Ignored Message, see § Section 2.2.8.7
0101 0000	100	MsgD	Set_Slot_Power_Limit, see § Section 2.2.8.5
0101 0010	100	Msg	PTM Request, see § Section 2.2.8.10
0101 0011	100	Msg/MsgD	PTM Response/PTM ResponseD, see § Section 2.2.8.10
0111 1110	000, 010, 011, or 100	Msg/MsgD	Vendor_Defined Type 0, see § Section 2.2.8.6
0111 1111	000, 010, 011, or 100	Msg/MsgD	Vendor_Defined Type 1, see § Section 2.2.8.6
0101 0100	010 / 100	Msg	IDE Sync (see § Section 2.2.8.11)
0101 0101	010 / 100	Msg	IDE Fail (see § Section 2.2.8.11)

§ Table F-2 contains a list of currently defined Subtype codes for PCI-SIG-Defined VDMs (see § Section 2.2.8.6.1). Subtype codes are defined in this specification and in other specifications. This table will be updated as Subtype codes are defined in other specifications but due to document release schedules, this table might not contain recently defined Subtypes.

Table F-2 PCI-SIG-Defined VDM Subtype Usage

Subtype	Routing r[2:0]	Type	Description
0000 0000	010 or 011	MsgD	Deprecated; formerly used for LN Message (Lightweight Notification)
0000 0001	011	MsgD	Hierarchy ID Message, See § Section 2.2.8.6.4 and § Section 6.25
0000 1000	100	Msg	Device Readiness Status, see § Section 2.2.8.6.2
0000 1001	000	Msg	Function Readiness Status, see § Section 2.2.8.6.3

G. Protocol Multiplexing §

The Protocol Multiplexing mechanism provides a standard mechanism to transport non-PCI Express protocols across a PCI Express Link. The mechanism supports the multiplexing of PMUX Packets and TLPs onto a single PCI Express Link.

An example system topology using Protocol Multiplexing is shown in § Figure G-1. In this example, the Link may operate in two modes:

- PCI Express Link. Protocol Multiplexing is disabled.
- PMUX Link. Protocol Multiplexing is enabled. Both TLPs and PMUX Packets are used in a coordinated fashion. PMUX Packets may be used to support additional protocols efficiently.

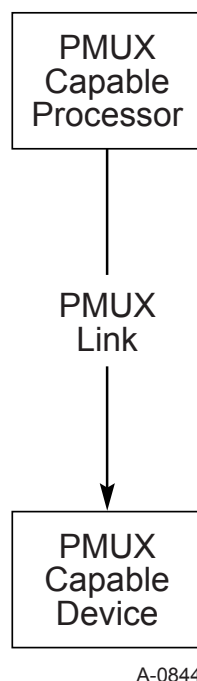
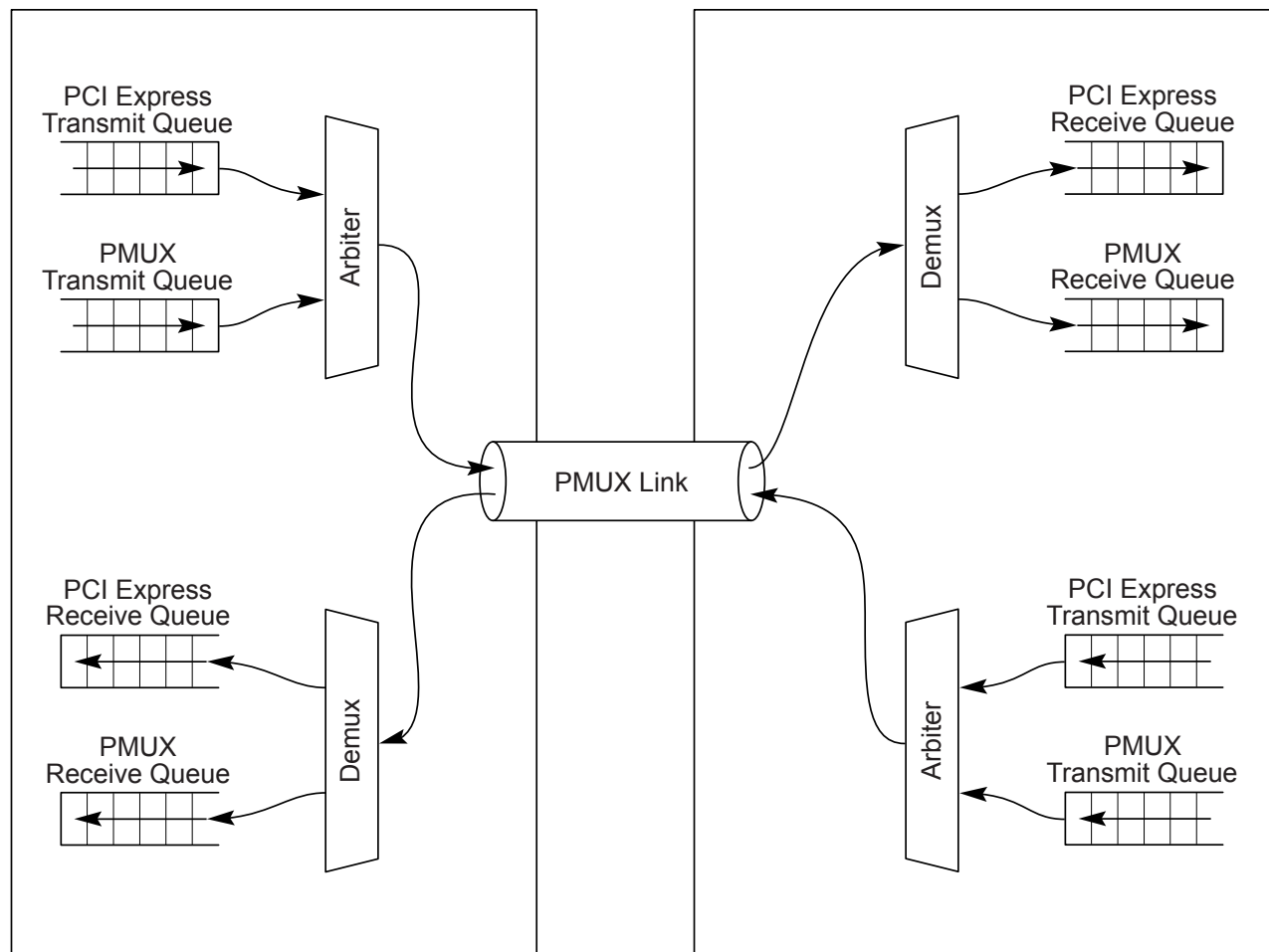


Figure G-1 Device and Processor Connected Using a PMUX Link

A PMUX Link is shown in § Figure G-2. Arbitration and encapsulation occurs between the transmit queues and the Link. Demultiplexing and decapsulation occurs between the Link and the various receive queues. Packets are sent from transmit queues to the corresponding receive queues. Packets are identified as either PMUX Packets or TLPs.



A-0845

§

Figure G-2 PMUX Link

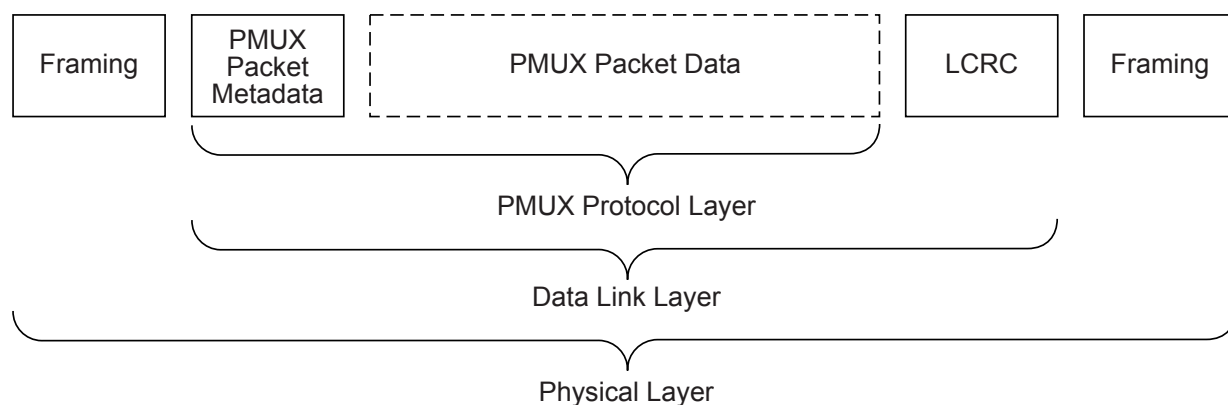
Important attributes of the Protocol Multiplexing mechanism are:

- Protocol Multiplexing support is optional normative.
- Protocol Multiplexing has no impact on PCI Express components that do not support it.
- Protocol Multiplexing has no impact on PCI Express TLPs and DLLPs, even when it is enabled.
- A Link may be used for both TLPs and PMUX Packets at the same time.
- Protocol Multiplexing does not consume or interfere with PCI Express resources (sequence numbers, credits, etc.). PMUX Packets use distinct resources associated with the specific multiplexed protocol.
- Protocol Multiplexing is disabled by default and is enabled by software. PMUX Packets must not be sent unless enabled by software. PMUX Packets received at Ports that support Protocol Multiplexing are ignored until Protocol Multiplexing is enabled by software.
- Protocol Multiplexing is selectable on a per-Link basis. Protocol Multiplexing may be used on any collection of Links in a system.
- A PMUX Link may support up to four simultaneously active PMUX Channels. Software configures the protocol used on each PMUX Channel.

- PMUX Packets contain an LCRC. This is used to provide data resiliency in a similar fashion as PCI Express TLPs.
- PMUX Packets do not use the ACK/NAK mechanism of PCI Express. Multiplexed protocol specific acknowledgement mechanisms can be used to provide reliable delivery when needed.
- PMUX Packets do not contain a TLP Sequence Number. Instead, they contain a 12 bit PMUX Packet Metadata field that is available for multiplexed protocol specific use.
- PMUX Packet transmitters must contain some arbitration/QoS mechanism for scheduling sending of PMUX Packets, TLPs and DLLPs; however, the mechanism used is outside the scope of this specification.
- The Protocol Multiplexing mechanism does not define any addressing or routing mechanism for PMUX Packets.

PMUX Packets are similar to PCI Express TLPs. The PMUX Packet Flow Through is shown in § Figure G-3. The PCI Express Packet flow through the layers is shown in § Figure 1-5. Changes from PCI Express Packet Flow Through are:

- PMUX Packets use a protocol specific PMUX Protocol Layer instead of the PCI Express Transaction Layer.
- PMUX Packets use a simplified Data Link Layer. The packet integrity portion of the Data Link Layer is mostly unchanged (LCRC computation uses a different seed value). The reliability and flow control aspects of the Data Link Layer are removed (the TLP Sequence Number field is repurposed as PMUX Packet Metadata).
- The Physical Layer is slightly modified to provide a mechanism to identify PMUX Packets.



A-0846

Figure G-3 PMUX Packet Flow Through the Layers

G.1 Protocol Multiplexing Interactions with PCI Express §

§ Table G-1 and § Table G-2 describe interactions between Protocol Multiplexing and PCI Express. § Table G-1 describes how PCI Express attributes affect Protocol Multiplexing. § Table G-2 describes how Protocol Multiplexing features affect PCI Express attributes.

Table G-1 PCI Express Attribute Impact on Protocol Multiplexing

PCI Express Attribute	Impact on Protocol Multiplexing
Link Speed	All PMUX Channels are disabled when the Current Link Speed corresponds to a speed that is not supported by Protocol Multiplexing (see § Section G.5.4). A PMUX Channel may be disabled when the Current Link Speed corresponds to a speed that is not supported by the associated protocol. ¹⁹⁰

190. The mechanism software uses to determine what Link Speeds are supported by a protocol is outside the scope of this specification.

PCI Express Attribute	Impact on Protocol Multiplexing
	Link speed can change either explicitly due to a change in the Target Link Speed field or automatically due to an autonomous Link speed change (see § Section 6.11). The PMUX Protocol Layer is permitted to influence the mechanism used by a component to determine when it requests an autonomous Link speed change. In addition, setting Hardware Autonomous Speed Disable at each end of the Link will prevent certain autonomous Link speed changes (see § Section 7.5.3.18). The PMUX Protocol Layer may be notified of the change.
Link Width	A PMUX Channel may be disabled when the Link Width corresponds to a width that is not supported by the associated protocol.¹⁹¹ The PMUX Protocol Layer is permitted to influence the mechanism used by a component to determine when it requests a Link Width change. The PMUX Protocol Layer may be notified of the change.
FLR initiated	All PMUX Channels on a Link are disabled if an FLR is directed to the Upstream Port's Function 0. No PMUX Channels are affected if an FLR is directed to any other Function.
DL_Down	All PMUX Channels on a Link are disabled.
Hot Reset	All PMUX Channels on a Link are disabled.
PERST#	All PMUX Channels on a Link are disabled.
TLP Replay	No effect on Protocol Multiplexing.
DLLP Lost	No effect on Protocol Multiplexing.
TLP Prefix	No effect on Protocol Multiplexing.
Locked Transactions	No effect on Protocol Multiplexing (including no effect on any protocol specific forwarding of PMUX Packets).
AtomicOp Transactions	No effect on Protocol Multiplexing.
Multicast Transactions	No effect on Protocol Multiplexing.
Access Control Services (ACS)	No effect on Protocol Multiplexing.
Alternative Routing ID-Interpretation (ARI)	No effect on Protocol Multiplexing.
TPH Requester	No effect on Protocol Multiplexing.
Virtual Channels	No effect on Protocol Multiplexing. PCI Express Links remain capable of supporting a full complement of VCs.
Internal Error	Corrected or Uncorrectable Internal Errors in the PMUX Protocol Layer may be reported as PCI Express Internal Errors.
L0s Link Power State	Protocol Multiplexing tracks the Link state. The PMUX Protocol Layer may request the Link transition back to L0.

191. The mechanism software uses to determine what Link Widths are supported by a protocol is outside the scope of this specification.

PCI Express Attribute	Impact on Protocol Multiplexing
L1 Link Power State	Protocol Multiplexing tracks the Link state. The PMUX Protocol Layer may request the Link transition back to L0.
Disabled LTSSM State	Disabling a Link also disables all PMUX Channels on the Link.
Loopback LTSSM State	Entering Loopback state disables all PMUX Channels on the Link.
Recovery LTSSM State	No effect on Protocol Multiplexing. The PMUX Protocol Layer may be notified.
Receiver or Framing Error	The error is reported to the PMUX Protocol Layer to indicate that data might have been lost. This can be used to initiate protocol specific error recovery mechanisms. The Error is reported to software using PCI Express Mechanisms.
Lane Reversal	No effect on Protocol Multiplexing. Support for Lane Reversal remains optional.
Polarity Inversion	No effect on Protocol Multiplexing.
Crosslink	No effect on Protocol Multiplexing. Support for Crosslink remains optional. If supported, the PMUX Protocol Layer may be notified of the outcome of the Crosslink Upstream / Downstream negotiation.
Lane assignment rules	Placement and frequency rules for STP Symbols and STP Tokens are not changed (see § Section 4.2.1.2 and § Section 4.2.2.3.2). These rules apply identically to PCI Express TLPs and PMUX Packets.
PCI Power Management Power State	All PMUX Channels on a Link are disabled if the Upstream Port's Function 0 is sent to non-D0 state.
Dynamic Power Allocation (DPA)	No effect on Protocol Multiplexing. The PMUX Protocol Layer is notified of the change and may participate in the power reduction. PCI Express power management includes any power used by the PMUX Protocol Layer.
PCI Power Management Power Consumed / Power Dissipated / Aux_Current	Power required by the PMUX Protocol Layer is included in the PCI structures.
Power Budgeting	Power required by the PMUX Protocol Layer is included in the PCI Express structures.
Slot Power Limit	Slot Power Limit includes power available to PMUX Protocol Layer.
ASPM L0s Entry Condition	The definition of Idle is extended to include: - No pending PMUX Packets to transmit over the Link. - For PMUX Channels that use protocol specific Flow Control, no credits are available to send PMUX Packets in that PMUX Channel.
ASPM L1 Entry Condition	A Link may not enter L1 if PMUX Packets are pending or scheduled to be transmitted.
ASPM L0s/L1 Exit Conditions	A Link may be directed to exit L0s or L1 if a component needs to transmit a PMUX Packet. Routing of PMUX Packets through routing elements is outside the scope of this specification; the associated L0s/L1 exit rules are also unspecified.
Bus Renumbering	No effect on Protocol Multiplexing.
Hot Plug	No direct effect on Protocol Multiplexing. Note Hot Plug events indirectly affect Data Link State which, in turn affects Protocol Multiplexing.

PCI Express Attribute	Impact on Protocol Multiplexing
TLP Sequence Number	No effect on Protocol Multiplexing. PMUX Packets do not consume TLP Sequence Numbers.
PCI Express Flow Control	No effect on Protocol Multiplexing. PMUX Packets do not consume PCI Express Flow Control credits. Flow Control Update DLLPs must be sent as required by PCI Express.
Error Reporting	No direct effect on Protocol Multiplexing. The PMUX Protocol Layer may be notified when an error is signaled or when an error message is received.
LCRC Errors in TLPs	No effect on Protocol Multiplexing.
Nullified TLPs	No effect on Protocol Multiplexing.
VC Arbitration	No effect on Protocol Multiplexing. Arbitration within PCI Express is unaffected by Protocol Multiplexing.
Port Arbitration	No effect on Protocol Multiplexing. Arbitration within PCI Express is unaffected by Protocol Multiplexing.
Electrical Idle Inference	PMUX Packets count as TLPs for the purpose of inferring Electrical Idle.
MR-IOV	Protocol Multiplexing may co-exist with MR-IOV. PMUX Packets are not part of any MR-IOV Virtual Hierarchy. Protocol Multiplexing is controlled using configuration space in the Management VH(s).

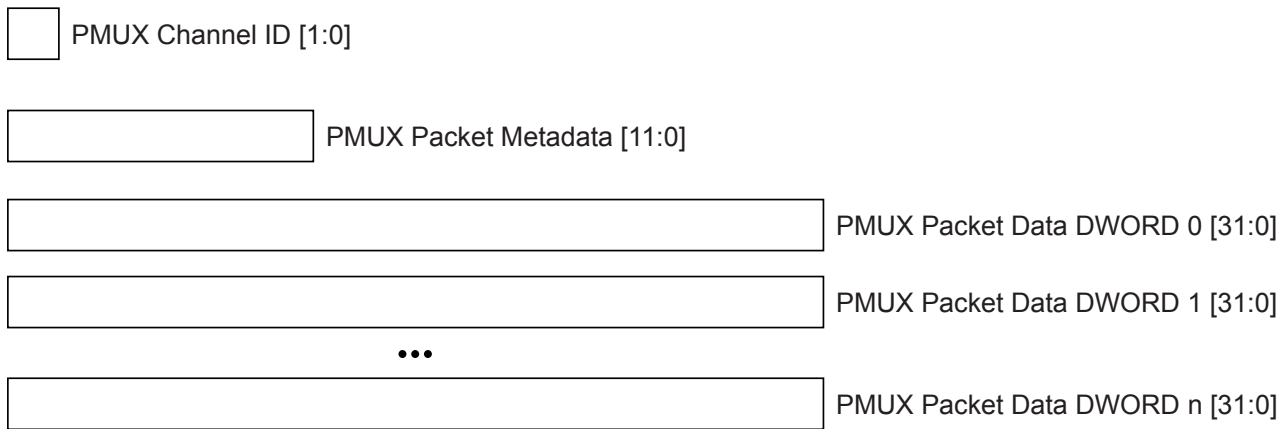
Table G-2 PMUX Attribute Impact on PCI Express

PMUX Attribute	Impact on PCI Express
PMUX Protocol Error	No effect on PCI Express.
LCRC Errors in PMUX Packets	No effect on PCI Express. PMUX Packets with LCRC errors are discarded without triggering PCI Express replay. This error is reported to the PMUX Protocol Layer and can be used to initiate protocol specific error recovery and/or error reporting mechanisms.
Link Unreliability	The PMUX Protocol Layer is permitted to influence the mechanism used by a component to determine if it requests an autonomous link speed change.
Nullified PMUX Packets	No effect on PCI Express. It is protocol specific whether PMUX Packets within a specific PMUX Channel may be nullified. If supported, PMUX Packets are nullified in the same manner as TLPs (e.g., inverting the LCRC and signaling nullification at the Physical Layer). Receiving a nullified PMUX Packet may be reported to the PMUX Protocol Layer.
Electrical Idle Inference	PMUX Packets count as TLPs for the purpose of inferring Electrical Idle.
PMUX Protocol Layer directs LTSSM to enter Recovery	Both PCI Express and the PMUX Protocol Layer are permitted to direct a transition from L0 to Recovery.
PMUX Channel Enabled / Disabled	No effect on PCI Express
PMUX Packet Receiver Buffer Overflow	No effect on PCI Express. This is a protocol problem within the PMUX Channel. The PMUX Transport Layer must continue to accept such packets dispose of them using protocol specific mechanisms.
Received PMUX Packet larger or smaller than supported by the associated protocol	No effect on PCI Express. These are protocol problems within the PMUX Channel. The PMUX Transport Layer must accept such packets and dispose of them using protocol specific mechanisms.
Received PMUX Packet that contains more than	No effect on PCI Express. This is an invalid PMUX Packet. The PMUX Transport Layer must accept such a packet and dispose of it. A protocol specific mechanism may be used to report the error.

PMUX Attribute	Impact on PCI Express
125 DWORDs of PMUX Packet Data	Note: This situation only exists for a packet encoded using 8b/10b. The TLP Length field of a packet encoded using 128b/130b cannot contain values that cause this situation.
PMUX Packet Received on disabled PMUX Channel	No effect on PCI Express. No effect on any other PMUX Channel. Receivers must silently ignore such packets regardless of packet length and regardless of whether or not the packet is nullified. PMUX Packets arriving on a disabled PMUX Channel may occur normally when software is in the process of initializing Protocol Multiplexing.
PMUX Packet Received at component that does not support Protocol Multiplexing	Software should not enable PMUX Packets unless both ends of a Link support Protocol Multiplexing. In the 128b/130b encoding, receiving a PMUX Packet by a component that does not support Protocol Multiplexing is a Framing Error (see § Section 4.2.2.3.1). In the 8b/10b encoding, the PMUX Packet LCRC is computed differently than the TLP LCRC. Receivers that do not support Protocol Multiplexing will interpret PMUX Packets as TLPs with LCRC errors and will not process them.
Large PMUX Packets when PCI Express Max_Payload_Size is small	Under certain conditions, it is possible for a large PMUX Packet to trigger a premature PCI Express replay. For example, this can occur when the time needed to transmit a PMUX Packet is larger than the REPLAY_TIMER (see § Section 3.6.2.1). To avoid this issue, implementations are permitted to not advance (hold) their REPLAY_TIMER during the reception of PMUX Packets. Note: The PCI Express REPLAY_TIMER mechanism has adequate headroom for most cases. This issue exists when (1) Max_Payload_Size is 000b, (2) PMUX Packets are larger than about 80 DWORDs, and (3) the REPLAY_TIMER is at the low end of the -0%/+100% tolerance.

G.2 PMUX Packets §

A PMUX Packet contains the information shown in § Figure G-4.



A-0847

§

Figure G-4 PMUX Packet

PMUX Channel ID is a 2 bit field that identifies which protocol is associated with a PMUX Packet. PMUX Channel ID values are between 0 and 3 (inclusive).

PMUX Packet Metadata is a 12 bit field that provides information about the PMUX Packet. Definition of this field is protocol specific and is outside the scope of this specification.

A PMUX Packet consists of between 0 and 125 DWORDs of PMUX Packet Data. Layout and usage of these DWORDs is protocol specific and is outside the scope of this specification. A PMUX Packet need not have any PMUX Packet Data and may consist only of PMUX Channel ID and PMUX Packet Metadata.

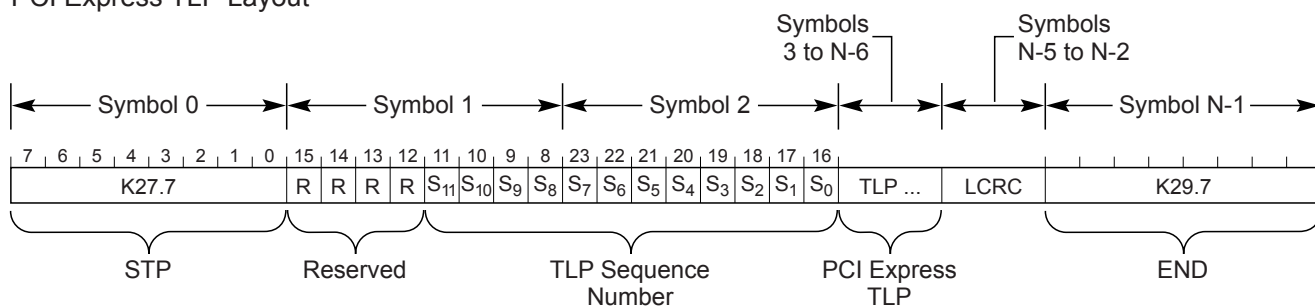
G.3 PMUX Packet Layout §

There are two layouts defined for PMUX Packets. One layout is used for 2.5 and 5.0 GT/s data rates and another layout is used for 8.0 GT/s and higher data rates. These layouts are discussed in the following sections.

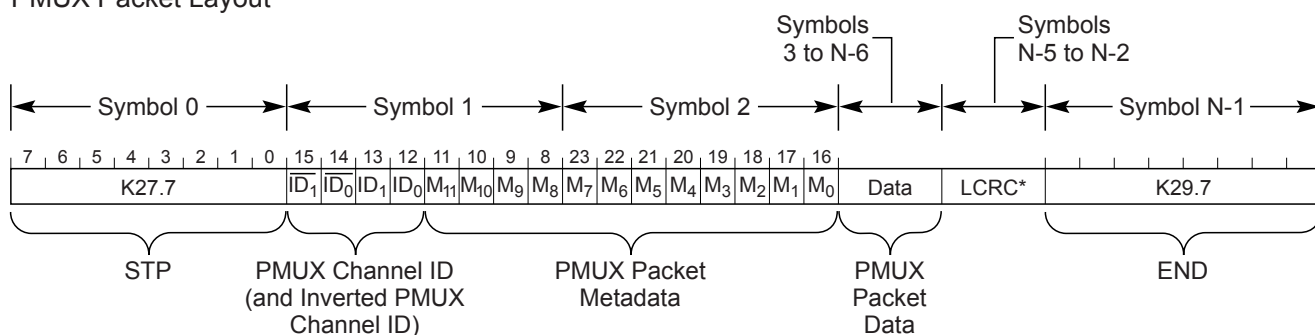
G.3.1 PMUX Packet Layout for 8b/10b Encoding §

§ Figure G-5 and § Table G-3 show the layout of PMUX Packets when using 8b/10b encoding. For reference, the 8b/10b encoding of a TLP is also shown (see § Section 4.2.1.2 for the official definition).

PCI Express TLP Layout



PMUX Packet Layout



* LCRC for PMUX Packets uses a different starting seed than that used for TLPs. This avoids aliasing problems and potential errors if software misconfigures a link so that PMUX Packets are seen by existing silicon.

A-0848

Figure G-5 TLP and PMUX Packet Framing (8b/10b Encoding)

Table G-3 PMUX Packet Layout (8b/10b Encoding)

Symbol	Field	Bit Position(s)	PMUX Packet Usage	TLP Usage
0	Start TLP Indicator	7:0	K27.7	
1	Inverted PMUX Channel ID[1:0]	7:6	Inverted (1s complement) of Symbol 1 bits 6:4	Reserved
	PMUX Channel ID[1:0]	5:4	PMUX Channel ID	Reserved
	PMUX Packet Metadata[11:8]	3:0	PMUX Packet Metadata[11:8]	TLP Sequence Number[11:8]
2	PMUX Packet Metadata[7:0]	7:0	PMUX Packet Metadata[7:0]	TLP Sequence Number[7:0]
3 to N-6	Packet	7:0	PMUX Packet	TLP
N-5 to N-2	LCRC	7:0	PMUX LCRC	PCI Express LCRC
N-1	END	7:0	K29.7	

For PMUX Packets, symbols 1 and 2 contain PMUX Packet Metadata in the same bit positions that TLPs use for TLP Sequence Number.

The PMUX LCRC algorithm is identical to the TLP LCRC algorithm as described in [§ Section 3.6.2](#) with the following modifications:

- The seed value is FB3E E248h (TLP LCRC uses FFFF FFFFh).
- The PMUX Channel ID field in Symbol 1 bits 7:4 is included in the PMUX LCRC in the same manner as the 4 reserved bits in the TLP LCRC.
- The PMUX Packet Metadata field is included in the PMUX LCRC in the same manner as the TLP Sequence Number field is included in the TLP LCRC.

IMPLEMENTATION NOTE: PMUX PACKETS AT RECEIVERS THAT DO NOT SUPPORT PROTOCOL MULTIPLEXING §

The bits used for PMUX Channel ID are reserved unless Protocol Multiplexing is supported. As such, Receivers that do not support Protocol Multiplexing must ignore the PMUX Channel ID bits. If software misconfigures Protocol Multiplexing, a component that does not support Protocol Multiplexing could receive a PMUX Packet. To prevent that component from misinterpreting such a PMUX Packet as a valid TLP, the LCRC computation is changed for PMUX Packets. The result is that a valid PMUX Packet will never be misinterpreted as a valid TLP. These LCRC “errors” may trigger PCI Express replay and may result in REPLAY_NUM Rollover correctable errors being reported.

IMPLEMENTATION NOTE: PMUX PACKET LCRC §

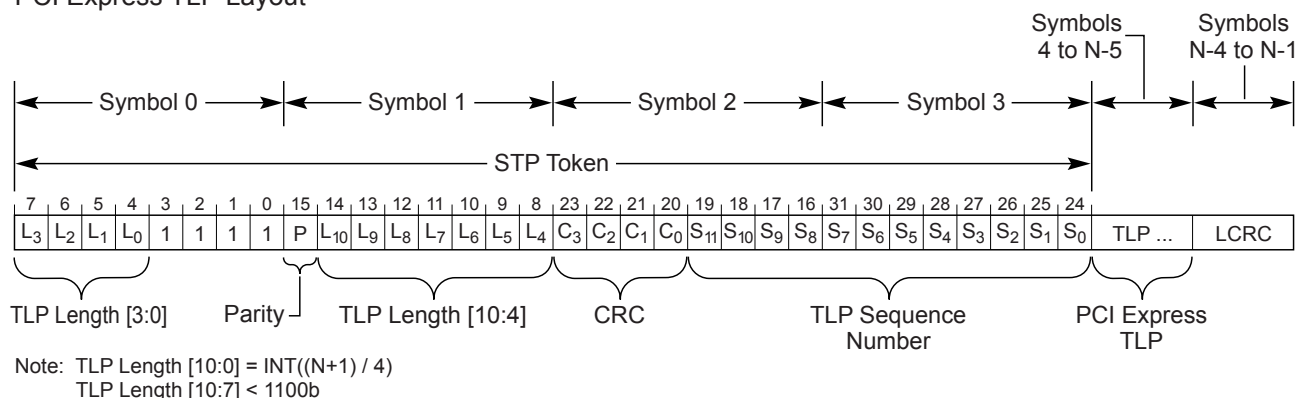
The PMUX Channel ID field is covered by the LCRC. As such, when using 8b/10b encoding, receivers must wait until the LCRC is checked to make firm decisions based on the PMUX Channel ID value. The Inverted PMUX Channel ID can be compared against the PMUX Channel ID to make tentative decisions.

Note: The value of the LCRC associated with a given PMUX Packet is independent of the encoding used to transmit the packet.

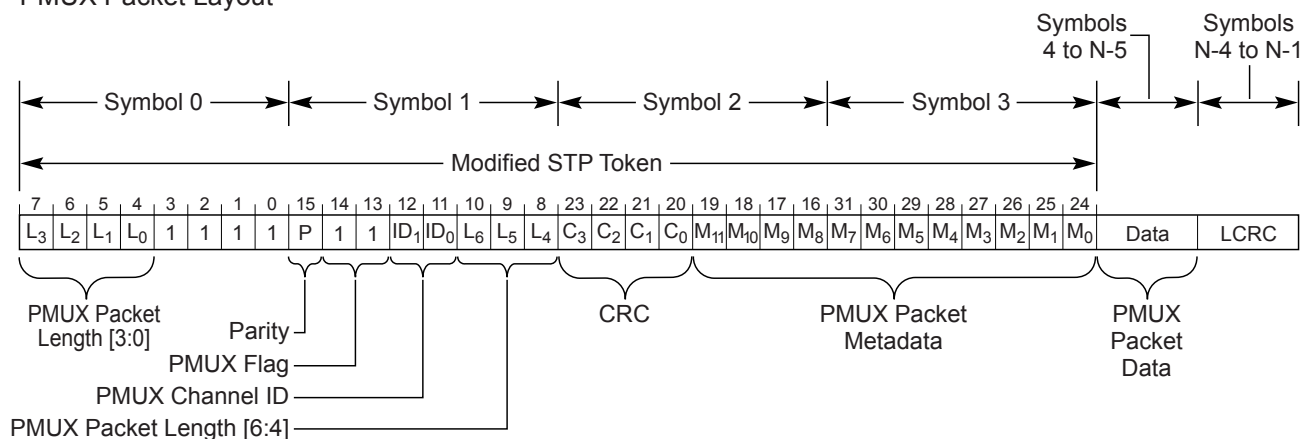
G.3.2 PMUX Packet Layout at 128b/130b Encoding §

[§ Figure G-6](#) and [§ Table G-4](#) show the layout of PMUX Packets when using 128b/130b encoding. For reference, the 128b/130b encoding of a TLP is also shown (see [§ Section 4.2.2.2](#) for the official definition).

PCI Express TLP Layout



PMUX Packet Layout



A-0849

Figure G-6 TLP and PMUX Packet Framing (128b/130b Encoding)

Table G-4 PMUX Packet Layout (128b/130b Encoding)

Symbol	Field	Bit Position(s)	PMUX Packet Usage	TLP Usage
0	Start TLP Indicator	3:0	Value of 1111b	
	PMUX Packet Length[3:0]	7:4	Bits [3:0] of the PMUX Packet Length. Bit 0 is the least significant PMUX Packet Length bit.	Bits [3:0] of the TLP Length field. Bit 0 is the least significant TLP Length bit.
1	Frame Parity (P)	7	Even parity of Symbol 0 bits [7:4], Symbol 1 bits [6:0] and Symbol 2 bits [7:4]	

Symbol	Field	Bit Position(s)	PMUX Packet Usage	TLP Usage
	PMUX Packet Indicator	6:5	Value of 11b	Bits [10:4] of the TLP Length field. Bit 10 is the most significant TLP Length bit.
	PMUX Channel ID[1:0]	4:3	PMUX Channel ID	
	PMUX Packet Length[6:4]	2:0	Bits [6:4] of PMUX Packet Length. Bit 6 is the most significant PMUX Packet Length bit.	
2	PMUX Packet Metadata[11:8]	3:0	PMUX Packet Metadata[11:8]	TLP Sequence Number[11:8]
	Frame CRC (C[3:0])	7:4	CRC of Symbol 0, bits [7:4] and Symbol 1 bits [6:0]	
3	PMUX Packet Metadata[7:0]	7:0	PMUX Packet Metadata[7:0]	TLP Sequence Number[7:0]
4 to N-5	Packet	7:0	PMUX Packet	TLP
N-4 to N-1	LCRC	7:0	PMUX LCRC	PCI Express LCRC

§ Table G-5 describes the encodings of Symbol 1 bits [6:3] in more detail. If these bits contain a value less than 1001b, the packet is a TLP and is processed as described in § Section 4.2.2. ¹⁹² If these bits contain 1001b, 1010b, or 1011b, the encoding is reserved for future standardization and is processed as described in § Section 4.2.2.3.3. If these bits contain a value greater than or equal to 1100b, the packet is a PMUX Packet is defined as specified in this appendix. ¹⁹³

§ *Table G-5 Symbol 1 Bits [6:3]*

Symbol 1 bits [6:3]	Meaning
0xxxb or 1000b	Packet is a TLP. Bits [6:3] are TLP Length [10:7].
1001b, 1010b, or 1011b	Encoding reserved for future standardization. Receivers detecting these encodings shall process them as described in § Section 4.2.2.3.3.
1100b	Packet is a PMUX Packet. PMUX Channel ID is 0.
1101b	Packet is a PMUX Packet. PMUX Channel ID is 1.
1110b	Packet is a PMUX Packet. PMUX Channel ID is 2.
1111b	Packet is a PMUX Packet. PMUX Channel ID is 3.

For PMUX Packets, the packet length in DWORDs is contained in PMUX Packet Length [6:0]. Other than being a smaller field, PMUX Packet Length is interpreted in the same manner as TLP Length. Specifically, PMUX Packet Length also includes the framing and PMUX LCRC DWORDs (see § Section 4.2.2.2).

For PMUX Packets, symbols 2 and 3 contain PMUX Packet Metadata in the same bit positions that TLPs use for TLP Sequence Number.

¹⁹² The value 1001b supports a maximum TLP Length [10:0] value of 1151 DWORDs (decimal). This will accommodate a TLP consisting of 4096 bytes of payload, 16 bytes of TLP Header, 4 bytes of TLP digest, and 480 bytes of TLP Prefix.

¹⁹³ The value 1100b was chosen to simplify distinguishing PMUX Packets from TLPs and from the reserved encodings.

The PMUX LCRC algorithm is identical to the TLP LCRC algorithm as described in § Section 3.6.2 with the following modifications:

- The seed value is FB3E E248h (TLP LCRC uses FFFF FFFFh).
- The PMUX Channel ID field in Symbol 1 bits 4:3 is used to compute a 4 bit value that is included in the PMUX LCRC in the same manner as the 4 reserved bits in the TLP LCRC. This 4 bit value contains the value that would be used, by the 8b/10b encoding, for Symbol 1 bits 7:4. Specifically, the lower 2 bits of this 4 bit value contain the PMUX Channel ID and the upper 2 bits contain the inverse (1s complement) of the PMUX Channel ID.
- The PMUX Packet Metadata field is included in the PMUX LCRC in the same manner as the TLP Sequence Number field is included in the TLP LCRC.

The Frame CRC and Frame Parity fields are computed as shown below. This is the same algorithm computed over the same bit positions as defined in § Section 4.2.2.2.

$$\begin{aligned}
 C[0] &= 1b \wedge PMUX_Channel_ID[0] \\
 &\quad \wedge L[6] \wedge L[4] \wedge L[2] \wedge L[1] \wedge L[0] \\
 C[1] &= 1b \wedge 1b \wedge PMUX_Channel_ID[0] \\
 &\quad \wedge L[5] \wedge L[4] \wedge L[3] \wedge L[2] \\
 C[2] &= 1b \wedge PMUX_Channel_ID[1] \\
 &\quad \wedge L[6] \wedge L[4] \wedge L[3] \wedge L[2] \wedge L[1] \\
 C[3] &= PMUX_Channel_ID[1] \\
 &\quad \wedge PMUX_Channel_ID[0] \\
 &\quad \wedge L[5] \wedge L[3] \wedge L[2] \wedge L[1] \wedge L[0] \\
 P &= 1b \wedge 1b \wedge PMUX_Channel_ID[1] \\
 &\quad \wedge PMUX_Channel_ID[0] \\
 &\quad \wedge L[6] \wedge L[5] \wedge L[4] \wedge L[3] \wedge L[2] \wedge L[1] \wedge L[0] \\
 &\quad \wedge C[3] \wedge C[2] \wedge C[1] \wedge C[0]
 \end{aligned}$$

IMPLEMENTATION NOTE:

PMUX CHANNEL ID AND FRAME CRC §

When using 128b/130b encoding, the PMUX Channel ID field is covered by the Frame CRC and Frame Parity fields. As such, receivers may make decisions based on the PMUX Channel ID value as soon as the Frame CRC and Frame Parity is checked and need not wait until the PMUX LCRC is checked.

Note: The PMUX Channel ID is also covered by the LCRC. The value of the LCRC associated with a given PMUX Packet is independent of the encoding used to transmit the packet.

G.4 PMUX Control §

Protocol Multiplexing is disabled by default. Each PMUX Channel must be explicitly enabled by software at each end of the associated Link. Protocol Multiplexing is disabled whenever the link drops (Data Link Layer indicates DL_Down).

A component that supports Protocol Multiplexing indicates such by the presence of the PMUX Extended Capability.

The following rules apply to components that support Protocol Multiplexing:

- PMUX Packets received in a PMUX Channel that is not enabled are silently ignored.

- PMUX Packets may not be transmitted unless the associated PMUX Channel is enabled. A PMUX Channel may also require additional, protocol specific, initialization mechanisms before PMUX Packets may be transmitted.

G.5 PMUX Extended Capability §

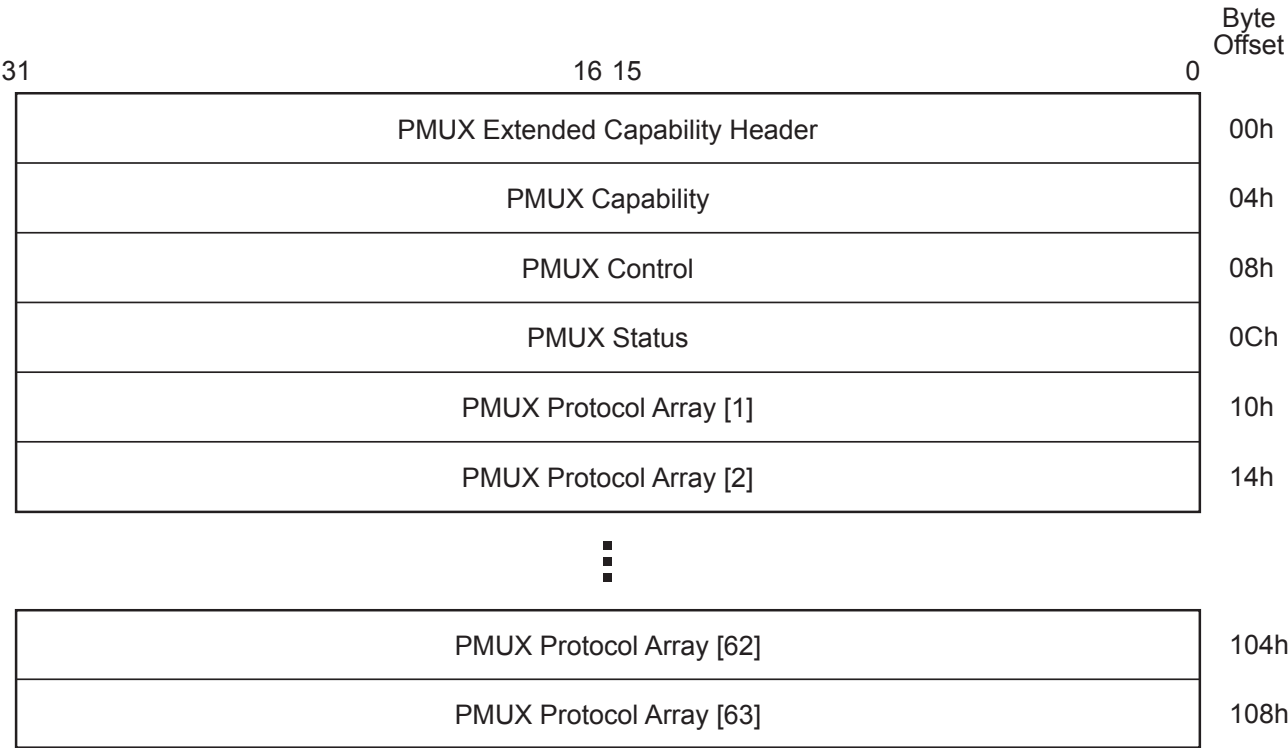
§ Figure G-7 shows the PMUX Extended Capability structure. The presence of this capability indicates that the Port supports the optional Protocol Multiplexing mechanism. This capability is optional and may be present in any Downstream Port and in Function 0 of any Upstream Port. It must not be present in non-zero Functions of Upstream Ports or in RCRBs.

The length of the PMUX Extended Capability is determined by the PMUX Protocol Array Size field (see § Section G.5.2).

This capability contains a list of the protocols supported by the Link (the PMUX Protocol Array). It also contains the mechanism software uses to enable and configure PMUX Channels. This capability must be present in both the Upstream and Downstream Ports of a Link in order for Protocol Multiplexing to be successfully enabled.

Software may enable the Upstream and Downstream Ports of a Link in either order. Software may enable multiple PMUX Channels using a single write to the PMUX Control Register.

Behavior is undefined if software enables Protocol Multiplexing in one Port and the other Port of the Link does not support Protocol Multiplexing. Behavior is also undefined if software configures a PMUX Channel inconsistently (the same PMUX Channel in the Ports on each end of a Link configured with incompatible protocols).



A-0850

§ Figure G-7 PMUX Extended Capability

G.5.1 PMUX Extended Capability Header (Offset 00h) §

§ Figure G-8 details the allocation of fields in the PMUX Extended Capability header; § Table G-6 provides the respective bit definitions.

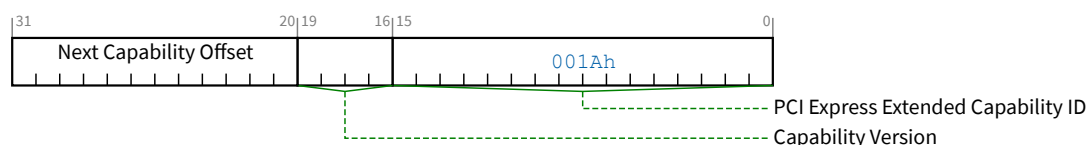


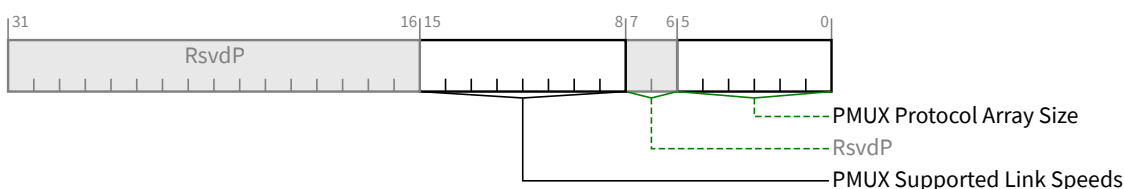
Figure G-8 PMUX Extended Capability Header

Table G-6 PMUX Extended Capability Header

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - This field is a PCI-SIG defined ID number that indicates the nature and format of the Extended Capability. The Extended Capability ID for the PMUX Extended Capability is 001Ah.	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be 1h for this version of the specification.	RO
31:20	Next Capability Offset - This field contains the offset to the next PCI Express Capability structure or 000h if no other items exist in the linked list of capabilities. This offset is relative to the beginning of PCI compatible Configuration Space and thus must always be either 000h (for terminating the list of Capabilities) or greater than 0FFh.	RO

G.5.2 PMUX Capability Register (Offset 04h) §

§ Figure G-9 details the allocation of fields in the PMUX Capability Register. § Table G-7 provides the respective bit definitions.



§ Figure G-9 PMUX Capability Register

§ *Table G-7 PMUX Capability Register*

Bit Location	Register Description	Attributes												
5:0	PMUX Protocol Array Size - Indicates the size of this Function's PMUX Protocol Array. This field may be 0 to indicate that even though no protocols are supported, the Port will ignore all received PMUX Packets.	RO												
15:8	PMUX Supported Link Speeds - This field indicates the Link speed(s) where Protocol Multiplexing is supported. Each bit corresponds to a Link speed. If a bit is Set, Protocol Multiplexing is supported at that Link speed. If a bit is Clear, Protocol Multiplexing is not supported at that Link speed. Bit definitions are: <table><tr><td>Bit 8</td><td>2.5 GT/s</td></tr><tr><td>Bit 9</td><td>5.0 GT/s</td></tr><tr><td>Bit 10</td><td>8.0 GT/s</td></tr><tr><td>Bit 11</td><td>16.0 GT/s</td></tr><tr><td>Bit 12</td><td>32.0 GT/s</td></tr><tr><td>Bits 15:13</td><td>RsvdP</td></tr></table> At least one Link speed must be supported (i.e., the field must be non-zero). A Port may support any combination of Link speeds. For example, this field could contain the value 0000 0100b indicating that Protocol Multiplexing is only supported at 8.0 GT/s. This field must not indicate support for Link speeds that are not supported by the Link (see § Section 7.5.3.18). Note that this field indicates the Link speeds supported by Protocol Multiplexing for the Link. The Link speeds that a particular protocol supports and the mechanism used to report that information are protocol specific.	Bit 8	2.5 GT/s	Bit 9	5.0 GT/s	Bit 10	8.0 GT/s	Bit 11	16.0 GT/s	Bit 12	32.0 GT/s	Bits 15:13	RsvdP	RO / RsvdP
Bit 8	2.5 GT/s													
Bit 9	5.0 GT/s													
Bit 10	8.0 GT/s													
Bit 11	16.0 GT/s													
Bit 12	32.0 GT/s													
Bits 15:13	RsvdP													

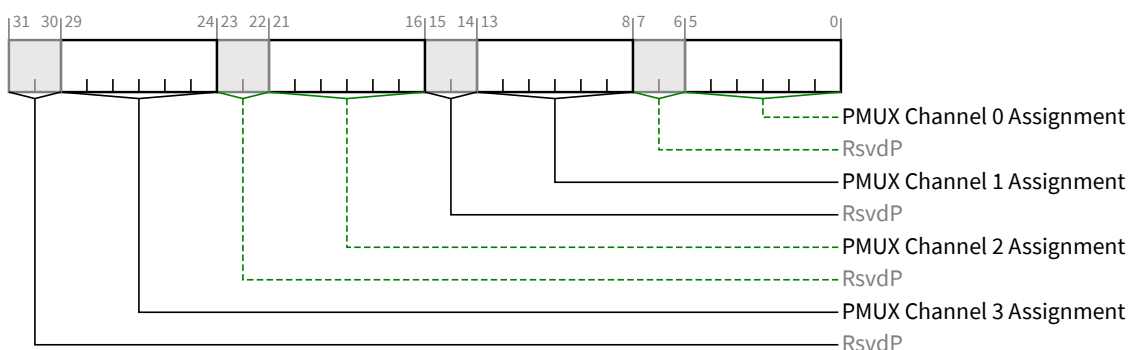
G.5.3 PMUX Control Register (Offset 08h) §

§ Figure G-10 details the allocation of fields in the PMUX Control Register. § Table G-8 provides the respective bit definitions.

Channel *n* is enabled and available for use by the PMUX Protocol Layer when all of the following are true:

- The Channel *n* Assignment field is non-zero.
- The Channel *n* Assignment field is less than or equal to PMUX Protocol Array Size.
- The Channel *n* Assignment field indicates an implemented entry in the PMUX Protocol Array (see § Section G.5.5).
- All of the PMUX Channel *n* Disabled bits are Clear (see § Section G.5.4).

Otherwise, Channel *n* is disabled.



§ Figure G-10 PMUX Control Register

§ Table G-8 PMUX Control Register

Bit Location	Register Description	Attributes
5:0	PMUX Channel 0 Assignment - This field indicates the protocol assigned to PMUX Channel 0. If the field is 0h, no protocol is assigned. If the field is non-zero, it is the index in the PMUX Protocol Array of the protocol assigned to PMUX Channel 0. If PMUX Protocol Array Size is less than 63 (see § Section G.5.2), unused upper bits of this field may be hardwired to 0b. If PMUX Protocol Array Size is 0, this entire field may be hardwired to 0. This field defaults to 0h.	RW
13:8	PMUX Channel 1 Assignment - This field indicates the protocol assigned to PMUX Channel 1. If the field is 0h, no protocol is assigned. If the field is non-zero, it is the index in the PMUX Protocol Array of the protocol assigned to PMUX Channel 1. If PMUX Protocol Array Size is less than 63 (see § Section G.5.2), unused upper bits of this field may be hardwired to 0b. If PMUX Protocol Array Size is 0, this entire field may be hardwired to 0. This field defaults to 0h.	RW
21:16	PMUX Channel 2 Assignment - This field indicates the protocol assigned to PMUX Channel 2. If the field is 0h, no protocol is assigned. If the field is non-zero, it is the index in the PMUX Protocol Array of the protocol assigned to PMUX Channel 2. If PMUX Protocol Array Size is less than 63 (see § Section G.5.2), unused upper bits of this field may be hardwired to 0b. If PMUX Protocol Array Size is 0, this entire field may be hardwired to 0. This field defaults to 0h.	RW
29:24	PMUX Channel 3 Assignment - This field indicates the protocol assigned to PMUX Channel 3. If the field is 0h, no protocol is assigned. If the field is non-zero, it is the index in the PMUX Protocol Array of the protocol assigned to PMUX Channel 3. If PMUX Protocol Array Size is less than 63 (see § Section G.5.2), unused upper bits of this field may be hardwired to 0b. If PMUX Protocol Array Size is 0, this entire field may be hardwired to 0. This field defaults to 0h.	RW

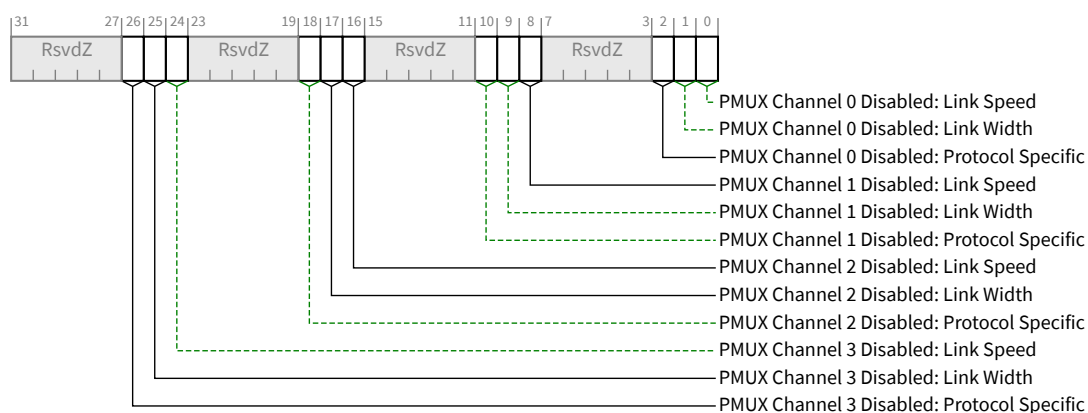
G.5.4 PMUX Status Register (Offset 0Ch) §

§ Figure G-11 details the allocation of fields in the PMUX Status Register. § Table G-9 provides the respective bit definitions.

Each channel has a set of Disabled bits. When Channel n Assignment field is non-zero, the Channel n Disabled bits reflect the error status of the channel. The following Disabled bits are defined:

- PMUX Channel n Disabled: Link Speed
- PMUX Channel n Disabled: Link Width
- PMUX Channel n Disabled: Protocol Specific

When there are multiple reasons for disabling a channel, an implementation may choose which reason(s) to report. For example, if a protocol needs bandwidth equivalent to x1 8.0 GT/s, when there is inadequate bandwidth (e.g., the Link is operating at x1 5.0 GT/s, x1 2.5 GT/s, or x2 2.5 GT/s), it could disable the PMUX Channel by indicating any or all of Disabled: Link Width, Disabled: Link Speed, or Disabled: Protocol Specific.



§ Figure G-11 PMUX Status Register

§ Table G-9 PMUX Status Register

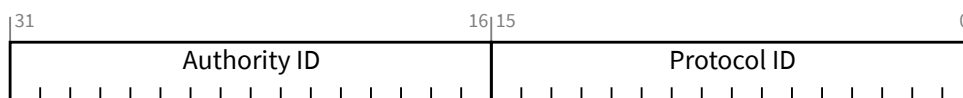
Bit Location	Register Description	Attributes
0	PMUX Channel 0 Disabled: Link Speed - If Set, Channel 0 is disabled because the Current Link Speed (§ Section 7.8.9) is not supported by Protocol Multiplexing or by the protocol assigned to Channel 0. This bit is 0 when no protocol is assigned to Channel 0 (i.e., Channel 0 Control field is 0h).	RO
1	PMUX Channel 0 Disabled: Link Width - If Set, Channel 0 is disabled because the current Link Width is not supported by the protocol assigned to Channel 0. This bit is 0 when no protocol is assigned to Channel 0 (i.e., PMUX Channel 0 Assignment field is 0h).	RO
2	PMUX Channel 0 Disabled: Protocol Specific - If Set, Channel 0 is disabled for protocol specific reasons. This bit is 0 when no protocol is assigned to Channel 0 (i.e., PMUX Channel 0 Assignment field is 0h).	RO

Bit Location	Register Description	Attributes
8	PMUX Channel 1 Disabled: Link Speed - If Set, Channel 1 is disabled because the Current Link Speed (§ Section 7.8.9) is not supported by Protocol Multiplexing or by the protocol assigned to Channel 1. This bit is 0 when no protocol is assigned to Channel 1 (i.e., PMUX Channel 1 Assignment field is 0h).	RO
9	PMUX Channel 1 Disabled: Link Width - If Set, Channel 1 is disabled because the current Link Width is not supported by the protocol assigned to Channel 1. This bit is 0 when no protocol is assigned to Channel 1 (i.e., PMUX Channel 1 Assignment field is 0h).	RO
10	PMUX Channel 1 Disabled: Protocol Specific - If Set, Channel 1 is disabled for protocol specific reasons. This bit is 0 when no protocol is assigned to Channel 1 (i.e., PMUX Channel 1 Assignment field is 0h).	RO
16	PMUX Channel 2 Disabled: Link Speed - If Set, Channel 2 is disabled because the Current Link Speed (§ Section 7.8.9) is not supported by Protocol Multiplexing or by the assigned protocol. This bit is 0 when no protocol is assigned to Channel 2 (i.e., PMUX Channel 2 Assignment field is 0h).	RO
17	PMUX Channel 2 Disabled: Link Width - If Set, Channel 2 is disabled because the current Link Width is not supported by the assigned protocol. This bit is 0 when no protocol is assigned to Channel 2 (i.e., PMUX Channel 2 Assignment field is 0h).	RO
18	PMUX Channel 2 Disabled: Protocol Specific - If Set, Channel 2 is disabled for protocol specific reasons. This bit is 0 when no protocol is assigned to Channel 2 (i.e., PMUX Channel 2 Assignment field is 0h).	RO
24	PMUX Channel 3 Disabled: Link Speed - If Set, Channel 3 is disabled because the Current Link Speed (§ Section 7.8.9) is not supported by Protocol Multiplexing or by the assigned protocol. This bit is 0 when no protocol is assigned to Channel 3 (i.e., PMUX Channel 3 Assignment field is 0h).	RO
25	PMUX Channel 3 Disabled: Link Width - If Set, Channel 3 is disabled because the current Link Width is not supported by the assigned protocol. This bit is 0 when no protocol is assigned to Channel 3 (i.e., PMUX Channel 3 Assignment field is 0h).	RO
26	PMUX Channel 3 Disabled: Protocol Specific - If Set, Channel 3 is disabled for protocol specific reasons. This bit is 0 when no protocol is assigned to Channel 3 (i.e., PMUX Channel 3 Assignment field is 0h).	RO

G.5.5 PMUX Protocol Array (Offsets 10h through 48h) §

The PMUX Protocol Array consists of up to 63 entries. The size of the PMUX Protocol Array is indicated by the PMUX Protocol Array Size field (see § Section G.5.2).

§ Figure G-12 details the allocation of fields in each PMUX Protocol Array Entry. § Table G-10 provides the respective bit definitions.



§ Figure G-12 PMUX Protocol Array Entry

§ Table G-10 PMUX Protocol Array Entry

Bit Location	Register Description	Attributes
15:0	Protocol ID - In conjunction with Authority ID designates a specific protocol and the mechanism by which that protocol is mapped onto Protocol Multiplexing. This value is assigned by the Vendor associated with the Authority ID field.	RO
31:16	Authority ID - Designates the authority controlling the values used in the Protocol ID field. The Authority ID field contains a Vendor ID as assigned by the PCI-SIG.	RO

The value 0000 0000h indicates an unimplemented PMUX Protocol Array Entry. The PMUX Protocol Array is indexed starting at 1.

PMUX Channel n is enabled and configured to support the protocol associated with PMUX Protocol Array Entry at index m when the PMUX Channel n Assignment field contains the value m (see § Section G.5.3).

Entries in the PMUX Protocol Array with the Authority ID value 1 (0001h) represent protocols that are defined by the PCI-SIG.

Duplicate Entries in the PMUX Protocol Array may be used to represent multiple instances of a particular protocol. This permits software control of the mapping between PMUX Channel ID and a specific instance of a protocol.

IMPLEMENTATION NOTE: MULTIPLE PROTOCOL INSTANCES §

A Link may have a single PMUX Protocol assigned to multiple PMUX Channels. Each PMUX Channel is assigned to a different instance of the protocol. Each instance of a protocol corresponds to an entry in the PMUX Protocol Array.

Consider a Port that supports two instances of protocol X. Two entries in the PMUX Protocol Array would indicate protocol X (indexes A and B for example). To assign instance A to PMUX Channel 0 and instance B to PMUX Channel 2, place the value A in the PMUX Channel 0 Assignment field and the value B in the PMUX Channel 2 Assignment field.

H. Flow Control Update Latency and ACK Update Latency Calculations

This appendix is informational only and should not be considered normative. Earlier revisions of this specification outlined the method used to calculate the Flow Control Update Latency and Ack Update Latency. This appendix describes the method used to derive the values and preserves the UpdateFactor and AckFactor values.

H.1 Flow Control Update Latency §

Recommended Flow Control Update Latency is described in the Implementation Note in § Section 2.6.1.2 entitled, Non-Flit Mode Flow Control Update Latency.

Flow Control Update Latency tables were simplified in the 4.0 Revision of this specification. At that time, the original tables were moved to this appendix. Note that in the 4.0 Revision of this specification, Tables 2-47 and 2-48 were distinct, but identical tables. The 5.0 Revision contains a single table that applies to 8.0 GT/s or higher.

Original Tables		Simplified Tables	
Base 3.1	Base 4.0 or later	Base 4.0	Base 5.0 or later
Table 2-42	§ Table H-1	Table 2-45	§ Table 2-48
Table 2-43	§ Table H-2	Table 2-46	§ Table 2-49
Table 2-44	§ Table H-3	Table 2-47	§ Table 2-50
n/a	n/a	Table 2-48	

The values in the Tables are measured starting from when the Receiver Transaction Layer makes additional receive buffer space available by processing a received TLP, to when the first Symbol of the corresponding UpdateFC DLLP is transmitted. The values are calculated as a function of the largest TLP payload size and Link width using the formula:

$$\frac{(\text{Rx_MPS_Limit} + \text{TLPOverhead}) \times \text{UpdateFactor}}{\text{LinkWidth}} + \text{InternalDelay}$$

§

Equation H-1 Max UpdateFC Latency

where:

Rx_MPS_Limit

The Rx_MPS_Limit value as determined in § Section 2.2.2 . For a Multi-Function Device where different Functions have different Rx_MPS_Limit values, using the smallest Rx_MPS_Limit value across all Functions is strongly recommended.

TLPOverhead

Represents the additional TLP components which consume Link bandwidth (TLP Prefix, header, LCRC, framing Symbols) and is treated here as a constant value of 28 Symbols.

UpdateFactor

Represents the number of maximum size TLPs sent during the time between UpdateFC receptions, and is used to balance Link bandwidth efficiency and receive buffer sizes - the value varies according to Rx_MPS_Limit and Link width. § Table H-1, § Table H-2, and § Table H-3 below include the UpdateFactor(UF).

LinkWidth

The operating width of the Link

InternalDelay

Represents the internal processing delays for received TLPs and transmitted DLLPs, and is treated here as a constant value of 19 Symbol Times for 2.5 GT/s mode operation, 70 Symbol Times for 5.0 GT/s mode operation, and 115 Symbol Times for 8.0 GT/s and 16.0 GT/s modes of operation.

*Table H-1 Maximum UpdateFC Transmission Latency Guidelines for 2.5 GT/s Mode
Operation by Link Width and Max Payload (Symbol Times)*

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	237 UF = 1.4	128 UF = 1.4	73 UF = 1.4	67 UF = 2.5	58 UF = 3.0	48 UF = 3.0	33 UF = 3.0
	256	416 UF = 1.4	217 UF = 1.4	118 UF = 1.4	107 UF = 2.5	90 UF = 3.0	72 UF = 3.0	45 UF = 3.0
	512	559 UF = 1.0	289 UF = 1.0	154 UF = 1.0	86 UF = 1.0	109 UF = 2.0	86 UF = 2.0	52 UF = 2.0
	1024	1071 UF = 1.0	545 UF = 1.0	282 UF = 1.0	150 UF = 1.0	194 UF = 2.0	150 UF = 2.0	84 UF = 2.0
	2048	2095 UF = 1.0	1057 UF = 1.0	538 UF = 1.0	278 UF = 1.0	365 UF = 2.0	278 UF = 2.0	148 UF = 2.0
	4096	4143 UF = 1.0	2081 UF = 1.0	1050 UF = 1.0	534 UF = 1.0	706 UF = 2.0	534 UF = 2.0	276 UF = 2.0

*Table H-2 Maximum UpdateFC Transmission Latency Guidelines for 5.0 GT/s Mode
Operation by Link Width and Max Payload (Symbol Times)*

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	288 UF = 1.4	179 UF = 1.4	124 UF = 1.4	118 UF = 2.5	109 UF = 3.0	99 UF = 3.0	84 UF = 3.0
	256	467 UF = 1.4	268 UF = 1.4	169 UF = 1.4	158 UF = 2.5	141 UF = 3.0	123 UF = 3.0	96 UF = 3.0
	512	610 UF = 1.0	340 UF = 1.0	205 UF = 1.0	137 UF = 1.0	160 UF = 2.0	137 UF = 2.0	103 UF = 2.0
	1024	1122 UF = 1.0	596 UF = 1.0	333 UF = 1.0	201 UF = 1.0	245 UF = 2.0	201 UF = 2.0	135 UF = 2.0
	2048	2146 UF = 1.0	1108 UF = 1.0	589 UF = 1.0	329 UF = 1.0	416 UF = 2.0	329 UF = 2.0	199 UF = 2.0

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
	4096	4194 UF = 1.0	2132 UF = 1.0	1101 UF = 1.0	585 UF = 1.0	757 UF = 2.0	585 UF = 2.0	327 UF = 2.0

Table H-3 Maximum UpdateFC Transmission Latency Guidelines for 8.0 GT/s Operation by Link Width and Max Payload (Symbol Times)

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	333 UF = 1.4	224 UF = 1.4	169 UF = 1.4	163 UF = 2.5	154 UF = 3.0	144 UF = 3.0	129 UF = 3.0
	256	512 UF = 1.4	313 UF = 1.4	214 UF = 1.4	203 UF = 2.5	186 UF = 3.0	168 UF = 3.0	141 UF = 3.0
	512	655 UF = 1.0	385 UF = 1.0	250 UF = 1.0	182 UF = 1.0	205 UF = 2.0	182 UF = 2.0	148 UF = 2.0
	1024	1167 UF = 1.0	641 UF = 1.0	378 UF = 1.0	246 UF = 1.0	290 UF = 2.0	246 UF = 2.0	180 UF = 2.0
	2048	2191 UF = 1.0	1153 UF = 1.0	634 UF = 1.0	374 UF = 1.0	461 UF = 2.0	374 UF = 2.0	244 UF = 2.0
	4096	4239 UF = 1.0	2177 UF = 1.0	1146 UF = 1.0	630 UF = 1.0	802 UF = 2.0	630 UF = 2.0	372 UF = 2.0

H.2 Ack Latency §

Ack Latency tables were simplified in the 4.0 Revision of this specification. At that time, the original tables were moved to this appendix. Note that in the 4.0 Revision of this specification, Tables 3-9 and 3-10 were distinct, but identical tables. The 5.0 Revision contains a single table that applies to 8.0 GT/s or higher.

Original Tables		Simplified Tables	
Base 3.1	Base 4.0 or later	Base 4.0	Base 5.0 or later
Table 3-7	§ Table H-5	Table 3-7	§ Table 3-9
Table 3-8	§ Table H-6	Table 3-8	§ Table 3-10
Table 3-9	§ Table H-7	Table 3-9	§ Table 3-11
n/a	n/a	Table 3-10	

The Maximum Ack Latency limits are calculated using the formula:

$$\frac{(\text{Rx_MPS_Limit} + \text{TLPOverhead}) \times \text{UpdateFactor}}{\text{LinkWidth}} + \text{InternalDelay}$$

§

Equation H-2 Max Ack Latency

where:

Rx_MPS_Limit

The Rx_MPS_Limit value as determined in § Section 2.2.2 . For a Multi-Function Device where different Functions have different Rx_MPS_Limit values, using the smallest Rx_MPS_Limit value across all Functions is strongly recommended.

TLPOverhead

represents the additional TLP components which consume Link bandwidth (TLP Prefix, header, LCRC, framing Symbols) and is treated here as a constant value of 28 Symbols.

AckFactor

represents the number of maximum size TLPs which can be received before an Ack is sent, and is used to balance Link bandwidth efficiency and retry buffer size - the value varies according to Rx_MPS_Limit and Link width. Table H-4, Table H-5, and Table H-6 below include the AckFactor(AF).

LinkWidth

is the operating width of the Link.

InternalDelay

represents the internal processing delays for received TLPs and transmitted DLLPs, and is treated here as a constant value of 19 Symbol Times for 2.5 GT/s mode operation, 70 Symbol Times for 5.0 GT/s operation, and 115 Symbol Times for 8.0 GT/s and 16.0 GT/s operation.

Table H-5 Maximum Ack Latency Limit and AckFactor for 2.5 GT/s (Symbol Times)

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	237 AF = 1.4	128 AF = 1.4	73 AF = 1.4	67 AF = 2.5	58 AF = 3.0	48 AF = 3.0	33 AF = 3.0
	256	416 AF = 1.4	217 AF = 1.4	118 AF = 1.4	107 AF = 2.5	90 AF = 3.0	72 AF = 3.0	45 AF = 3.0
	512	559 AF = 1.0	289 AF = 1.0	154 AF = 1.0	86 AF = 1.0	109 AF = 2.0	86 AF = 2.0	52 AF = 2.0
	1024	1071 AF = 1.0	545 AF = 1.0	282 AF = 1.0	150 AF = 1.0	194 AF = 2.0	150 AF = 2.0	84 AF = 2.0
	2048	2095 AF = 1.0	1057 AF = 1.0	538 AF = 1.0	278 AF = 1.0	365 AF = 2.0	278 AF = 2.0	148 AF = 2.0
	4096	4143 AF = 1.0	2081 AF = 1.0	1050 AF = 1.0	534 AF = 1.0	706 AF = 2.0	534 AF = 2.0	276 AF = 2.0

Table H-6 Maximum Ack Transmission Latency Limit and AckFactor for 5.0 GT/s (Symbol Times)

§

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	288 AF = 1.4	179 AF = 1.4	124 AF = 1.4	118 AF = 2.5	109 AF = 3.0	99 AF = 3.0	84 AF = 3.0
	256	467 AF = 1.4	268 AF = 1.4	169 AF = 1.4	158 AF = 2.5	141 AF = 3.0	123 AF = 3.0	96 AF = 3.0
	512	610 AF = 1.0	340 AF = 1.0	205 AF = 1.0	137 AF = 1.0	160 AF = 2.0	137 AF = 2.0	103 AF = 2.0
	1024	1122 AF = 1.0	596 AF = 1.0	333 AF = 1.0	201 AF = 1.0	245 AF = 2.0	201 AF = 2.0	135 AF = 2.0
	2048	2146 AF = 1.0	1108 AF = 1.0	589 AF = 1.0	329 AF = 1.0	416 AF = 2.0	329 AF = 2.0	199 AF = 2.0
	4096	4194 AF = 1.0	2132 AF = 1.0	1101 AF = 1.0	585 AF = 1.0	757 AF = 2.0	585 AF = 2.0	327 AF = 2.0

Table H-7 Maximum Ack Transmission Latency Limit and AckFactor for 8.0 GT/s (Symbol Times)

§

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	333 AF = 1.4	224 AF = 1.4	169 AF = 1.4	163 AF = 2.5	154 AF = 3.0	144 AF = 3.0	129 AF = 3.0
	256	512 AF = 1.4	313 AF = 1.4	214 AF = 1.4	203 AF = 2.5	186 AF = 3.0	168 AF = 3.0	141 AF = 3.0
	512	655 AF = 1.0	385 AF = 1.0	250 AF = 1.0	182 AF = 1.0	205 AF = 2.0	182 AF = 2.0	148 AF = 2.0
	1024	1167 AF = 1.0	641 AF = 1.0	378 AF = 1.0	246 AF = 1.0	290 AF = 2.0	246 AF = 2.0	180 AF = 2.0
	2048	2191 AF = 1.0	1153 AF = 1.0	634 AF = 1.0	374 AF = 1.0	461 AF = 2.0	374 AF = 2.0	244 AF = 2.0
	4096	4239 AF = 1.0	2177 AF = 1.0	1146 AF = 1.0	630 AF = 1.0	802 AF = 2.0	630 AF = 2.0	372 AF = 2.0

I. Async Hot-Plug Reference Model §

This appendix presents a recommended reference model for async hot-plug. The reference model covers three areas:

- Async hot-plug initial configuration
- Async removal configuration and interrupt handling
- Async hot-add configuration and interrupt handling

There are no normative requirements in this section. The entire reference model is contained in a series of implementation notes. The reference model documents how the various hot-plug mechanisms are envisioned be used to implement a robust asynchronous hot-plug model, but does not mandate that they be used that way.

For brevity and readability, this section uses the following acronyms:

DLL

Data Link Layer

DSP

Downstream Port

FWF

firmware first

HPS

Hot-Plug Surprise

OOB

out-of-band

OS

operating system

PD

presence detect

SFW

system firmware

The reference model covers both the HPS and DPC mechanisms. DPC is the recommended mechanism. The reference model covers FWF for DPC but not for HPS.

The reference model provides a recommended framework for DPC software to support Containment Error Recovery (CER) along with async hot-plug. The reference model does not explicitly cover CER outside of async hot-plug, but certain aspects can be leveraged for DPC support of CER when async hot-plug is not being used.

SFW may use the System Firmware Intermediary (SFI) Capability for async hot-plug, orderly removal hot-plug, or other operations. This reference model does not rely on its functionality nor cover its use.

The reference model refers to various bits or fields outlined in [§ Section 6.7.2](#) . For brevity, pointers to the associated registers are not replicated in this section.

IMPLEMENTATION NOTE: IN-BAND PRESENCE DETECT MECHANISM DEPRECATED FOR ASYNC HOT-PLUG §

Due to architectural issues, the in-band (Physical-Layer-based) portion of the PD mechanism is deprecated for use with async hot-plug. One issue is that in-band PD as architected does not detect adapter removal during certain LTSSM states, notably the L1 and Disabled States. Another issue is that when both in-band and OOB PD are being used together, the Presence Detect State bit and its associated interrupt mechanism always reflect the logical OR of the inband and OOB PD states, and with some hot-plug hardware configurations, it is important for software to detect and respond to in-band and OOB PD events independently.

If OOB PD is being used and the associated DSP supports In-Band PD Disable, it is recommended that the In-Band PD Disable bit be Set, and the Presence Detect State bit and its associated interrupt mechanism be used exclusively for OOB PD.

As a substitute for in-band PD with async hot-plug, the reference model uses either the DPC or the DLL Link Active mechanism.

The reference model assumes and covers the configurations listed below. While these cover the bulk of the envisioned use cases, many minor variations are not explicitly covered. For example, there are multiple ways to determine if a slot supports OOB PD, and the reference model does not cover them. As another example, the reference model refers to adding or removing an adapter, but some hot-pluggable modules may include a Switch and multiple Endpoint components.

- Reference model assumptions:
 - DSPs support DLL Link Active Reporting
 - DSPs support In-Band PD Disable
 - Operating systems support both HPS and DPC, using DPC if available
- Reference model covers:
 - Slots that support HPS and/or DPC (SW never enables both at same time)
 - Slots with or without OOB PD
 - RPs with or without RP Extensions for DPC

I.1 Async Hot-Plug Initial Configuration §

IMPLEMENTATION NOTE: ASYNC HOT-PLUG INITIAL CONFIGURATION §

Basic Steps	HPS	DPC
Determine capability control entity	• OS requests, and is granted control of PCIe Native Hot-Plug • If FWF, SFW retains control of AER and DPC • Else OS requests and is granted control of AER and DPC	
OS and SFW determine which async hot-plug mechanism to use; OS/SFW interactions here are outside scope of this specification	• If DPC capability then ◦ If HPS bit not Set, use DPC ◦ Else attempt to Clear HPS bit (§ Section 6.7.4.4) ▪ If successful, use DPC ▪ Else use HPS • Else if HPS bit Set, use HPS • Else async hot-plug cannot be supported by this slot • Configure DSP for selected mechanism	
OS determines if adapter present	• If OOB PD supported, use it to determine if adapter is present ◦ Set In-band PD Disable bit (SFW may have it Set by default) ◦ Read PD State bit • Else allow Link to attempt to train; use DLL Link Active to determine if adapter is present (and at least minimally functional)	
If adapter is present, OS waits for adapter to become ready for configuration	• If using Device Readiness Status (DRS), begin configuration if/after DSP receives a DRS Message • If using Configuration RRS Software Visibility, can attempt first Configuration Read (of Vendor ID field) after 100 ms • Otherwise, must wait at least 1000 ms before attempting first Configuration Read	
OS configures for appropriate case	• If adapter is present, configure system for handling async removal • Else configure system for handling async hot-add	

I.2 Async Removal Configuration and Interrupt Handling §

IMPLEMENTATION NOTE: ASYNC REMOVAL CONFIGURATION §

Basic Steps	HPS	DPC
OS/SFW configure appropriate error handling	- If OS and driver support a non-CER error recovery approach, its policies may influence some of these error settings - Configure the error handling supported by HPS	- If OS and driver support CER, its policies may influence some of these error settings - Enable uncorrectable AER errors to trigger DPC, including Surprise Down - If using RP Extensions for DPC, configure RP PIO error handling - Configure RP Completion Timeout handling per platform and OS policies
OS/SFW configure async removal interrupts	OS enables DLL State Changed interrupt	- If FWF, configure DPC for ERR_COR signaling to enable SFW handling - Else configure DPC for interrupt to enable OS handling
	If OOB PD supported, OS enables PD Changed interrupt	

IMPLEMENTATION NOTE:

ASYNCR REMOVAL INTERRUPT HANDLING §

Basic Steps	HPS	DPC
Service routine entry	If PD Changed or DLL State Change, OS is interrupted	- If PD Changed, OS is interrupted - If FWF, OS invokes SFW; preferably via OS Setting DPC Software Trigger bit (if implemented)
		- If DPC triggered - If FWF, DPC sends ERR_COR to signal SFW - Else DPC interrupts OS
Prevent further Link activity	- OS Sets Disable Link; Link goes down and stays down - OS polls DLL Link Active until Clear	DPC automatically keeps Link down
Log errors from DSP's AER/DPC		- If FWF, SFW logs DSP errors - SFW invokes OS and exits
	OS logs and then clears DSP errors	
OS notifies impacted software/driver, which cease accessing the adapter		
OS determines if adapter is still present	- Some FFs with OOB PD automatically power off slot and/or disable switched signals - If OOB PD supported, use it to determine if adapter is physically present - If adapter not determined to be absent, re-enable the Link and slot - As applicable, Clear Disable Link or release DPC - Wait until Link trains or adapter is deemed absent or non-functional - If non-functional, optionally perform a slot reset (if supported) - If still non-functional, optionally power-cycle the slot (if supported)	
If OS determined adapter to be present	- OS waits for adapter to become ready for configuration - OS enumerates and configures the adapter - If DPC and FWF, OS invokes SFW to log adapter AER errors	